

Modern Analysis of Variance

Anicet Ebou

2026-05-01

Table of contents

List of Figures

List of Tables

Preface

Why another analysis of variance book?

There are many excellent books on analysis of variance, data modelling, and probability models. But I felt the need to write my own. Why? I see three problems.

First, many courses assign a huge textbook. It might be possible for the strongest and most motivated students to become familiar with the range of topics covered in these books, but truly mastering them is another matter entirely. A book that tries to cover everything often ends up covering nothing deeply enough.

Second, and drawing directly from the first point, students typically encounter analysis of variance as one chapter among many in a book that tries to present everything at once. This framing misleads students about the method: it invites them to skim over it rather than understand it, and to treat it as a recipe rather than a tool with conditions of use.

Third, analysis of variance is a powerful tool, but I have too many times seen it misused, misinterpreted, and misreported, or applied through outdated and inappropriate workflows. In most cases, this does not come from carelessness. It comes from the fact that the method was never really understood in the first place.

This book is my answer to these problems.

How to use this book

This book is tool-oriented and built around a modern R stack. It is designed to be read in order, at least for the first time.

Start with the foundations. The first two chapters are the heart of the book. Do not rush past them. They are what will tell you *when* you can use ANOVA, *when* you cannot, and *what to do* when you are not sure. A reader who masters Part I will make fewer analytical mistakes than one who jumps straight to the models.

Then work through the classical models. Part II covers the ANOVA designs you will encounter most often, each with worked examples and R code. I particularly invite you to pay careful attention to the last chapter of this section, on nested and split-plot designs. Biological and ecological data are very often hierarchical in structure, and understanding how that hierarchy should be modelled, rather than ignored, is one of the most practically useful things this book can teach.

Move on to the modern framework. Part III introduces mixed-effects models and generalised linear mixed models. If you have read this far, you have already encountered the core reason why: Chapter 2 showed that the most damaging violations of ANOVA assumptions (non-independence, pseudoreplication, nested structures, and random effects) cannot be fixed by transforming your data or switching to a non-parametric test. They require a richer model, one that can represent the actual structure of how your data were collected. Mixed-effects models are that richer model. They are not a separate topic bolted onto ANOVA, they are the natural destination of the logic you have already been following. ANOVA is just a special case of the linear model and mixed-effects models are simply the next step in the same direction i.e. a linear model that can also account for the random variation introduced by subjects, plots, cages, clinics, or any other grouping structure in your data. By the end of Part III, the goal is that this framework becomes how you think about your analyses, not an advanced topic you feel you should know about but have never quite understood.

Close with good practice. Part IV on experimental design and reproducibility completes the

How to use this book

stack. An analysis that cannot be reproduced or reported clearly is an analysis that cannot be trusted. This final part makes sure yours can be both.

Each chapter includes R code that you can run directly. The book was written using Quarto, and all code chunks are fully reproducible. When you see a chunk, run it, modify it, break it, and fix it. That is how, I believe, the material becomes yours.

Part I

Foundations

To consult the statistician after an experiment is finished is often merely to ask him to conduct a post mortem examination. He can perhaps say what the experiment died of.

— Ronald A. Fisher

Statistics textbooks often introduce Analysis of Variance (from here and onward called by its abbreviation ANOVA) as a procedure: a set of steps to follow, a table to fill in, a p -value to report. This part takes a different approach.

Before touching a single formula, we ask a more fundamental question: *what is ANOVA actually doing, and why does it work?*

Chapter 1 builds the answer from the ground up. Starting from the simple observation that **variation in data has identifiable sources**. It shows how ANOVA **decomposes total variation into components that can be compared** and why **the ratio of those components is a meaningful test of whether group means differ**. It also shows that ANOVA is not a standalone procedure but a special case of the linear model, a connection that will pay dividends throughout the rest of the book.

Chapter 2 is devoted entirely to assumptions. This is unusual for an introductory treatment, but the emphasis is deliberate. In my experience, assumption violations are the single most common source of invalid conclusions in the biological literature. This happens not because researchers are careless, but because the consequences of violations are rarely taught with the clarity they deserve. By the end of Chapter 2, you will know not just *what* the assumptions are, but *why* they matter, *how badly* each one can distort your results when violated, and *what to do* when they are.

These two chapters form the foundation on which everything else in the book is built. The time spent here will not be wasted.

1 What is ANOVA, Really ?

1.1 The variance decomposition idea

Imagine you are investigating whether the type of fertiliser applied to wheat plants affects how tall they grow. You have 12 identical pots of wheat seedlings. Four pots receive no fertiliser (control), four receive a nitrogen-based fertiliser, and four receive a phosphorus-based fertiliser. After six weeks, you measure the height of each plant.

1 What is ANOVA, Really ?

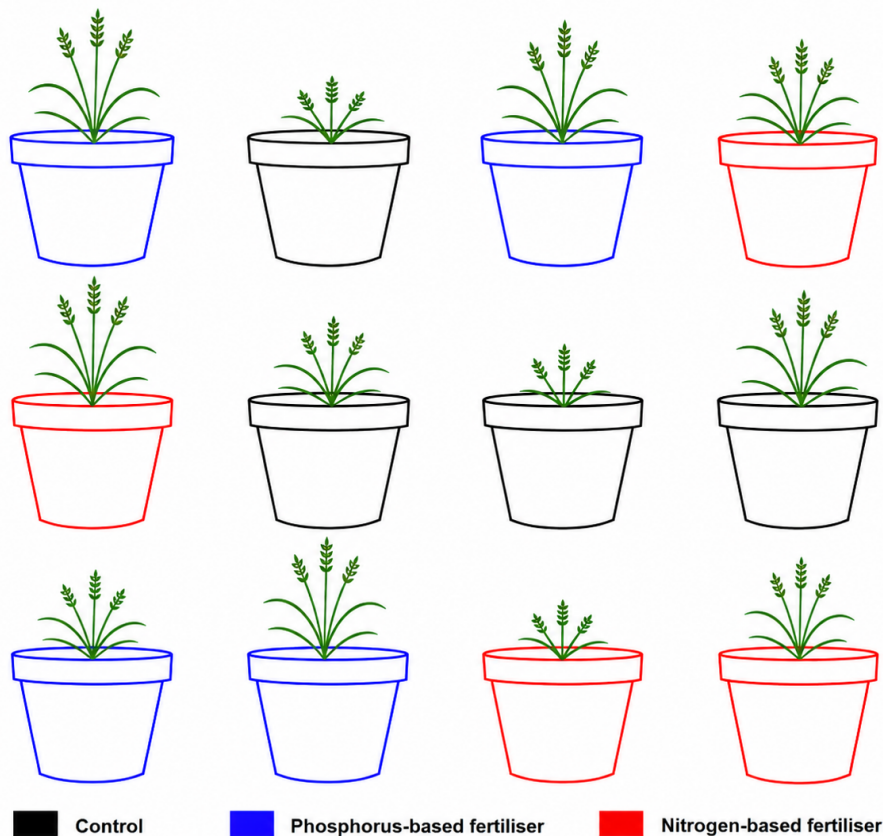


Figure 1.1: Figure 1. Fertiliser experiment. Seedling growth observed after 6 weeks. The pots received same soils and were treated equally except from fertiliser they received.

Your starting assumption is that *there is no difference in mean plant height among the three groups*. The alternative is that at least one fertiliser produces a different mean height.

Now consider the height of any single plant. Why does it differ from the average height of all 12 plants (we will call this average height the grand mean) ? There are two reasons.

First, that plant received a particular fertiliser, and that fertiliser may genuinely promote or inhibit growth. This is what we will call the *treatment effect*. We use the term *treatment* because the fertiliser is what was applied to the plant, and *effect* because it is the change in height that the fertiliser brings about.

Second, even plants given the same fertiliser will not grow identically. Tiny differences like

1 What is ANOVA, Really ?

seed quality, pot position, light exposure, and watering can make each plant slightly different from its neighbours. This uncontrollable scatter is what we will call *residual* or *error*. We use the term *residual* because this second source of variation comes from all the factors that influence plant height but are not measured in the experiment, they are leftover, the part of the story that the treatment alone cannot tell.

The height of any single plant is therefore:

$$\text{Plant height} = \text{fertiliser effect} + \text{residual}$$

Suppose that we obtain the data below from the experiment:

Treatment	Plant height (cm)
Control	4
Control	5
Control	2
Control	1
Nitrogen	10
Nitrogen	8
Nitrogen	11
Nitrogen	7
Phosphorus	8
Phosphorus	4
Phosphorus	7
Phosphorus	5

We have three groups of treatments: Control, Nitrogen, and Phosphorus. Each treatment was *randomly* assigned to a plant.

1.1.1 The Grand Mean and Group Means

Let us start with the bigger picture. By computing the mean of all 12 plant heights we obtain the **grand mean**:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{1}{12}(4 + 5 + 2 + 1 + 10 + 8 + 11 + 7 + 8 + 4 + 7 + 5) = 6 \text{ cm}$$

Computing the mean within each treatment group gives:

$$\bar{x}_{\text{control}} = \frac{1}{4}(4 + 5 + 2 + 1) = 3 \text{ cm}$$

$$\bar{x}_{\text{nitrogen}} = \frac{1}{4}(10 + 8 + 11 + 7) = 9 \text{ cm}$$

$$\bar{x}_{\text{phosphorus}} = \frac{1}{4}(8 + 4 + 7 + 5) = 6 \text{ cm}$$

Each group mean carries the effect of its treatment, but also the influence of any unmeasured factor like differences in seed quality, sunlight, watering, and so on. The nitrogen group mean of 9 cm, for example, reflects both the genuine effect of nitrogen fertiliser on growth *and* the small random differences that existed between those four plants regardless of what fertiliser they received.

1.1.2 Among-Group Variation: The Signal

If we compare each group mean to the grand mean, the control sits 3 cm *below* the grand mean, nitrogen sits 3 cm *above* it, and phosphorus sits right on it. Any displacement of a group mean from the grand mean can come from two sources: the effect of the treatment, and random error. If fertiliser has no effect whatsoever, the three group means will cluster close to the grand mean, displaced only by chance. If fertiliser genuinely matters, some means will be pulled far from

the grand mean, and the spread among group means will be large. This spread is what we call **among-group variation**, and it is our window onto the treatment effect.

1.1.3 Within-Group Variation: The Noise

Now look inside each group. Even though all four plants in the control group received exactly the same treatment, no fertiliser at all, their heights (4, 5, 2, 1 cm) are not identical. This scatter cannot come from the fertiliser, since all four plants received the same one. It can only reflect the residual, the unmeasured sources of variation we discussed earlier. We can quantify this scatter with the within-group variance. For the control group:

$$s_{\text{control}}^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x}_{\text{control}})^2 = \frac{(4-3)^2 + (5-3)^2 + (2-3)^2 + (1-3)^2}{3} = \frac{1+4+1+4}{3} = 3.33 \text{ cm}^2$$

This within-group scatter, present in every group regardless of which fertiliser was applied, is our estimate of background noise i.e. the residual variation the experiment cannot control. This is what we call **within-group variation**.

1.1.4 The F Ratio

We now have two variance estimates that tell very different stories: the among-group variance, which reflects the treatment effect plus noise, and the within-group variance, which reflects noise alone. The natural thing to do is form their ratio:

$$F = \frac{\text{Among-group variance (fertiliser + residual)}}{\text{Within-group variance (residual only)}}$$

When fertiliser has no effect, both the numerator and the denominator are estimating the same background noise, and their ratio will hover around 1. As the true differences among fertilisers grow larger, the numerator grows, because the group means are being pulled further apart

by the treatment, while the denominator stays anchored to the background noise. The ratio therefore climbs above 1, and the larger it becomes, the harder it is to believe that the differences among groups arose by chance alone.

1.1.5 Assessing the F Ratio: Is It Large Enough?

Knowing that F is greater than 1 is not enough on its own, we need to know how much greater than 1 it would have to be before we can confidently say the differences are real. This is where probability comes in.

When there is truly no treatment effect, the F ratio still varies from experiment to experiment simply due to chance. Mathematical statistics tells us that, under this condition, F follows a known probability distribution called the **Fisher distribution** (or F -distribution), whose shape depends on the degrees of freedom of the two variance estimates. This distribution acts as a reference: it tells us exactly how large F would typically be if there were no fertiliser effect at all.

In practice, we choose a threshold probability in advance, conventionally $\alpha = 0.05$, which defines a critical value of F . If our observed F exceeds this critical value, it means that the probability of seeing a ratio this large by chance alone is less than 5%. We then conclude that the differences among fertiliser groups are unlikely to be due to chance.

If our observed F does not reach the critical value, we do not have sufficient evidence to rule out chance as an explanation.

This is the core logic of the ANOVA test, and as we will see in the next section, it is also the core logic of a much broader family of statistical models.

1.2 A Brief History of ANOVA

1.2.1 Origins in Agriculture

The analysis of variance was developed in the early twentieth century by the British statistician and geneticist Ronald A. Fisher, widely regarded as the founder of modern statistics. Fisher developed the method during his time at the Rothamsted Experimental Station in Hertfordshire, England, where he worked from 1919 to 1933 (?). Rothamsted was, and remains, one of the oldest agricultural research institutions in the world, and it was here that Fisher was confronted with a practical problem: how to make sense of large volumes of messy experimental data on crop yields, soil treatments, and fertiliser effects.

The core challenge was not merely computational but conceptual. Agricultural field experiments are inherently noisy. Soil fertility varies across a field, weather is unpredictable, and individual plants differ in ways that cannot be controlled. Fisher needed a principled way to separate the variation caused by experimental treatments, different fertilisers, crop varieties, tillage methods, from the background variation that existed regardless of what the experimenter did. His solution was the decomposition of total variance into distinct, interpretable components: a idea that became the conceptual backbone of ANOVA (?).

1.2.2 The Key Publications

Fisher first introduced the F statistic and the logic of variance decomposition in his landmark 1925 textbook *Statistical Methods for Research Workers* (?). This work presented ANOVA not as an abstract mathematical construction but as a practical tool for scientists, illustrated throughout with agricultural and biological examples. The book went through fourteen editions over Fisher's lifetime and was enormously influential in spreading statistical thinking across the life sciences.

Fisher elaborated the theory further in *The Design of Experiments*, published in 1935 (?). This second major work introduced the principles of randomisation, replication, and blocking that

underpin valid experimental design to this day, and showed how ANOVA was inseparable from the way an experiment should be planned. The famous example of the lady tasting tea, introduced in this book, illustrated the logic of hypothesis testing in a way that remains a staple of introductory statistics courses.

1.2.3 The F Statistic

The ratio of variances that lies at the heart of ANOVA, what we now call the F statistic, was named in Fisher's honour by George W. Snedecor, the American statistician who popularised and extended Fisher's methods in the United States (?). Snedecor introduced the notation F explicitly as a tribute, and the name has remained standard ever since.

1.2.4 From Agriculture to All of Science

Although ANOVA was born from the needs of agricultural research, its logic proved universal. By the mid-twentieth century it had been adopted across medicine, psychology, ecology, and engineering, wherever researchers needed to compare means across multiple groups while accounting for background noise. Today it remains one of the most widely used statistical procedures in empirical science (?).

The enduring power of ANOVA lies precisely in the simplicity of the idea Fisher identified at Rothamsted: that variation is not an obstacle to understanding, but a quantity that can be measured, partitioned, and interpreted.

1.3 ANOVA as a special case of linear models

1.3.1 The linear model idea

You may have already encountered simple linear regression: the idea that you can predict one variable from another using a straight line. For example, you might predict plant height from

the amount of rainfall received. The equation looks like this:

$$\text{Plant height} = \beta_0 + \beta_1 \times \text{Rainfall} + \varepsilon$$

where β_0 is the intercept (the expected height when rainfall is zero), β_1 is the slope (how much height changes for each additional unit of rainfall), and ε is the error, the residual scatter around the line that rainfall alone cannot explain.

This is called a **linear model**: we are modelling the response variable (plant height) as a linear combination of some predictor plus error.

ANOVA is in fact the same thing. The only difference is that instead of a continuous predictor like rainfall, *our predictor is a categorical variable*, a group label like *control*, *nitrogen*, or *phosphorus*.

1.3.2 Recoding groups as numbers: dummy variables

A computer cannot do arithmetic on the words “control” or “nitrogen”, so we recode the group labels as numbers. This is done using **dummy variables** (also called indicator variables). The idea is simple: we pick one group as the reference, say, the control, and then create one dummy variable for each of the other groups.

For our fertiliser experiment with **three groups**, we create **two dummy variables**:

$$X_1 = \begin{cases} 1 & \text{if the plant received nitrogen} \\ 0 & \text{otherwise} \end{cases}$$

$$X_2 = \begin{cases} 1 & \text{if the plant received phosphorus} \\ 0 & \text{otherwise} \end{cases}$$

1 What is ANOVA, Really ?

Notice that a control plant gets $X_1 = 0$ and $X_2 = 0$: it is identified by the absence of both flags. **We never need a third dummy variable for the control group because it is already fully described this way.**

The table below shows how the 12 plants in our experiment are recoded:

Plant	Group	X_1 (nitrogen)	X_2 (phosphorus)
1	Control	0	0
2	Control	0	0
3	Control	0	0
4	Control	0	0
5	Nitrogen	1	0
6	Nitrogen	1	0
7	Nitrogen	1	0
8	Nitrogen	1	0
9	Phosphorus	0	1
10	Phosphorus	0	1
11	Phosphorus	0	1
12	Phosphorus	0	1

Notice the pattern: each group has its own “signature” of zeros and ones. The control group is the only group with zeros in *both* columns, which is why it serves naturally as the reference against which the other groups are compared.

1.3.3 Writing the model

We can now write the ANOVA as a linear model:

$$\text{Plant height} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \varepsilon$$

1 What is ANOVA, Really ?

The coefficients have a beautifully simple interpretation:

- β_0 is the mean height of the **control** group (when $X_1 = 0$ and $X_2 = 0$).
- β_1 is the **difference** in mean height between the nitrogen group and the control group.
- β_2 is the **difference** in mean height between the phosphorus group and the control group.
- ε is the residual error for each individual plant.

Let us check this with a concrete example. Suppose the estimated coefficients are:

$$\hat{\beta}_0 = 20 \text{ cm}, \quad \hat{\beta}_1 = 5 \text{ cm}, \quad \hat{\beta}_2 = 2 \text{ cm}$$

Then the model predicts:

- A **control** plant: $20 + 5 \times 0 + 2 \times 0 = 20 \text{ cm}$
- A **nitrogen** plant: $20 + 5 \times 1 + 2 \times 0 = 25 \text{ cm}$
- A **phosphorus** plant: $20 + 5 \times 0 + 2 \times 1 = 22 \text{ cm}$

In other words, nitrogen fertiliser is associated with plants that are on average 5 cm taller than the control, and phosphorus with plants that are 2 cm taller.

1.3.4 The connection to the F test

In the linear model framework, the null hypothesis of ANOVA, *no difference in mean height among groups*, translates directly into:

$$H_0 : \beta_1 = 0 \quad \text{and} \quad \beta_2 = 0$$

That is, we are testing whether all the group coefficients are simultaneously zero. If they are, then the group labels carry no information and the model reduces to:

$$\text{Plant height} = \beta_0 + \varepsilon$$

which simply says every plant has the same expected height (the grand mean) plus noise. The F test in ANOVA is exactly the test of this hypothesis: it compares how much better the full model (with group labels) fits the data compared to this reduced model (with no group labels). A large F means the group labels explain a meaningful amount of the variation in height, and we reject H_0 .

1.3.5 Why Does This Matter?

Recognising ANOVA as a linear model is more than a mathematical curiosity. It means that ANOVA and regression are not two separate tools you need to learn independently, they are both special cases of a single unified framework, the general linear model. Once you are comfortable with this framework, you can naturally extend your analyses to situations that mix categorical and continuous predictors (called ANCOVA), include multiple categorical factors (two-way ANOVA), or handle more complex experimental designs, all within exactly the same logic you have already learned here.

1.4 What ANOVA is not: common misconceptions

1.4.1 ANOVA Does Not Tell You Which Groups Differ

This is perhaps the most common source of confusion among newcomers. A significant F test tells you that the group means are not all equal i.e that somewhere among your groups there is a real difference. It does not tell you *where* that difference lies.

In our fertiliser example, a significant result tells you that fertiliser type matters, but it does not tell you whether nitrogen differs from the control, whether phosphorus differs from the control, or whether nitrogen and phosphorus differ from each other. Identifying which specific pairs of groups differ requires additional **post-hoc tests**, such as Tukey's HSD or pairwise t -tests with corrected significance thresholds, which are conducted after the ANOVA, and only when the F test is significant.

1.4.2 A Non-Significant Result Does Not Mean the Groups Are Equal

If your F test returns $p > 0.05$, it is tempting to conclude that the fertilisers have no effect on plant height. **This conclusion is not warranted.** A non-significant result means only that *you did not find sufficient evidence of a difference given your data*. There are many reasons this can happen even when a true difference exists:

- Your sample size may have been too small to detect a modest effect.
- The within-group variation (error) may have been large, drowning out a real treatment signal.
- The effect of the treatment may be genuine but smaller than your experiment was designed to detect.

The absence of evidence is not evidence of absence. A non-significant ANOVA should prompt you to think about the **statistical power** of your experiment: its ability to detect an effect of a given size, rather than to simply accept the null hypothesis.

1.4.3 ANOVA Does Not Require Equal Sample Sizes, But Imbalance Has Consequences

It is a common belief that ANOVA requires the same number of observations in each group. This is not strictly true: ANOVA can be performed on unbalanced designs where group sizes differ. However, equal sample sizes are strongly preferred. Balanced designs are more statistically efficient, more robust to violations of the assumption of equal variances, and considerably simpler to interpret. If your groups are very unequal in size, the results of the F test can become sensitive to assumptions you might not have checked carefully.

1.4.4 ANOVA is not robust to all assumption violations

ANOVA rests on three key assumptions:

1. **Independence:** the observations are independent of one another.

2. **Normality:** the residuals (errors) are approximately normally distributed within each group.
3. **Homogeneity of variance:** the variance within each group is approximately the same (also called homoscedasticity).

Students sometimes assume that ANOVA is so widely used that it must be safe to apply in any situation. This is not the case. Of the three assumptions above, independence is by far the most critical. Violating it, for example by measuring the same plant twice and treating the two measurements as independent observations, can severely inflate your false positive rate and lead to entirely spurious conclusions. Violations of normality are generally less serious, especially with larger sample sizes, thanks to the central limit theorem. Violations of homogeneity of variance are intermediate in severity and can be addressed with modified versions of the test, such as Welch's ANOVA.

We will explore the assumptions of ANOVA in the following chapter.

1.4.5 ANOVA is not a test of means alone

Although ANOVA is typically introduced as a way to compare means, what it actually does is partition and compare *variances*. The F ratio is a ratio of two variance estimates, not a direct comparison of means. The connection to means is real, a large among-group variance arises precisely because the group means are far apart, but it is important to understand that ANOVA is fundamentally a test about the structure of variation in your data. This is why, as we saw in the previous section, ANOVA fits so naturally within the linear model framework: both are concerned with explaining variation, not merely comparing averages.

1.4.6 ANOVA is not a substitute for good experimental design

Finally, and perhaps most importantly, no statistical test can rescue a poorly designed experiment. ANOVA assumes that your groups were formed by proper randomisation, that your replicates are genuine independent observations, and that sources of systematic bias have been

1 *What is ANOVA, Really ?*

controlled or accounted for. If your four control plants were all placed on one side of the greenhouse and your four nitrogen plants on the other, then any difference you observe might reflect a light or temperature gradient rather than a fertiliser effect. ANOVA will dutifully compute an F statistic and a p -value in this situation, but the result will be meaningless. As Fisher himself understood from his years at Rothamsted, the validity of the analysis begins not at the computer but at the moment the experiment is designed.

2 The Assumptions

2.1 Independence of observations

2.1.1 What does independence mean?

Of all the assumptions underlying ANOVA, independence is the most important and, paradoxically, the least discussed in introductory courses. **The assumption states that knowing the value of one observation tells you nothing about the value of any other observation.** Each data point must carry its own independent piece of information.

This sounds abstract, so let us ground it in a concrete example.

Imagine you are measuring the effect of three diets on the weight gain of mice. You have 12 mice, four per diet group. If each mouse lives in its own individual cage, eats its own food, and is weighed independently. Then the 12 measurements are independent: the weight of one mouse gives you no information about the weight of any other.

Now imagine instead that you house all four mice in each diet group together in a single shared cage. The mice in the same cage eat together, sleep together, and influence each other's behaviour. A dominant mouse may monopolise food; a stressed mouse may eat less. The four measurements within each cage are no longer independent: they are **clustered**. Knowing that one mouse in a cage gained a lot of weight makes it more likely that its cagemates did too, simply because they share the same environment.

This is the essence of non-independence: observations that are **grouped, nested, repeated, or spatially clustered** tend to resemble each other more than random chance would predict.

2.1.2 Why does it matter so much?

ANOVA and the F test in particular, is built on the assumption that the within-group variance faithfully estimates background noise. When observations are not independent, this estimate is distorted. Specifically, clustered observations tend to be more similar to each other than truly independent observations would be, which **artificially deflates the within-group variance**. A smaller denominator in the F ratio means a larger F statistic, and therefore a smaller p -value, even when there is no real treatment effect. **In other words, non-independence makes you think you have found something when you have not.**

This inflation of the false positive rate is not a minor inconvenience. It can be dramatic, as the simulation below will demonstrate.

2.1.3 A concrete example: pseudoreplication

The most common form of non-independence in biology is **pseudoreplication**: the mistake of treating non-independent measurements as if they were independent replicates (?).

Returning to our mice: suppose you have three diet groups, each with two cages, and two mice per cage, 12 mice in total. The true replicates are the **cages** (since each cage is an independent experimental unit), not the individual mice. If you ignore the cage structure and run ANOVA on the 12 individual mouse weights as if they were 12 independent observations, you are pseudoreplicating. Your apparent $n = 12$ is an illusion, you effectively have only $n = 6$ independent units.

The following R code simulates this scenario.

```
set.seed(42)

n_sim <- 5000 # number of simulations
alpha <- 0.05 # significance threshold
n_diets <- 3  # number of diet groups
```

2 The Assumptions

```
n_cages <- 2 # cages per diet group
n_mice <- 2 # mice per cage
cage_sd <- 2 # variation among cages (shared environment effect)
mouse_sd <- 1 # variation among mice within a cage

false_pos_correct <- logical(n_sim)
false_pos_pseudo <- logical(n_sim)

for (i in seq_len(n_sim)) {

  # Build the data structure
  diet <- rep(paste0("Diet", 1:n_diets), each = n_cages * n_mice)
  cage <- rep(paste0("Cage", 1:(n_diets * n_cages)), each = n_mice)

  # Cage-level random effect (shared environment, no diet effect)
  cage_effect <- rep(rnorm(n_diets * n_cages, mean = 0, sd = cage_sd), each = n_mice)

  # Individual mouse noise
  mouse_noise <- rnorm(n_diets * n_cages * n_mice, mean = 0, sd = mouse_sd)

  # Response: no diet effect, only cage effect + individual noise
  weight_gain <- 10 + cage_effect + mouse_noise

  df <- data.frame(diet, cage, weight_gain)

  # --- Correct analysis: use cage means ---
  cage_means <- aggregate(weight_gain ~ diet + cage, data = df, FUN = mean)
  fit_correct <- aov(weight_gain ~ diet, data = cage_means)
  p_correct <- summary(fit_correct)[[1]][["Pr(>F)"]][1]
```

2 The Assumptions

```
false_pos_correct[i] <- p_correct < alpha

# --- Incorrect analysis: ignore cage structure (pseudoreplication) ---
fit_pseudo <- aov(weight_gain ~ diet, data = df)
p_pseudo   <- summary(fit_pseudo)[[1]][["Pr(>F)"]][1]
false_pos_pseudo[i] <- p_pseudo < alpha

}
```

Running this code produces output along the lines of:

```
cat(
  "False positive rate (correct analysis):",
  round(mean(false_pos_correct), 3),
  "\nFalse positive rate (pseudoreplication):",
  round(mean(false_pos_pseudo), 3), "\n"
)
```

```
#> False positive rate (correct analysis): 0.053
```

```
#> False positive rate (pseudoreplication): 0.27
```

The correct analysis produces a false positive rate right at the nominal 5% level: exactly as it should when there is no real effect. The pseudoreplicated analysis produces a false positive rate of around 27 %: more than one in four experiments would be declared significant by chance alone, simply because the cage structure was ignored.

We can visualise the distribution of p -values from both approaches across the 5000 simulations.

First we simulate the distributions.

2 The Assumptions

```
set.seed(42)

n_sim <- 5000
alpha <- 0.05
n_diets <- 3
n_cages <- 2
n_mice <- 2
cage_sd <- 2
mouse_sd <- 1

p_correct <- numeric(n_sim)
p_pseudo <- numeric(n_sim)

for (i in seq_len(n_sim)) {
  diet <- rep(paste0("Diet", 1:n_diets), each = n_cages * n_mice)
  cage <- rep(paste0("Cage", 1:(n_diets * n_cages)), each = n_mice)

  cage_effect <- rep(
    rnorm(n_diets * n_cages, mean = 0, sd = cage_sd),
    each = n_mice
  )
  mouse_noise <- rnorm(n_diets * n_cages * n_mice, mean = 0, sd = mouse_sd)

  weight_gain <- 10 + cage_effect + mouse_noise

  df <- data.frame(diet, cage, weight_gain)

  # Correct analysis: cage means as the unit of analysis
  cage_means <- aggregate(weight_gain ~ diet + cage, data = df, FUN = mean)
```

2 The Assumptions

```
p_correct[i] <- summary(
  aov(weight_gain ~ diet, data = cage_means)
)[[1]][["Pr(>F)"]][1]

# Incorrect analysis: pseudoreplication
p_pseudo[i] <- summary(
  aov(weight_gain ~ diet, data = df)
)[[1]][["Pr(>F)"]][1]
}
```

Then, we can check the false positive rate:

```
cat(
  "False positive rate (correct analysis):",
  round(mean(p_correct < alpha), 3),
  "\nFalse positive rate (pseudoreplication):",
  round(mean(p_pseudo < alpha), 3), "\n"
)
```

```
#> False positive rate (correct analysis): 0.053
```

```
#> False positive rate (pseudoreplication): 0.27
```

And visualise the distribution of p-values:

```
library(ggplot2)

# Build a tidy data frame for plotting
p_df <- data.frame(
  p_value = c(p_correct, p_pseudo),
  analysis = rep(c("Correct (cage means)", "Pseudoreplication"), each = n_sim)
```

```
)

ggplot(p_df, aes(x = p_value, fill = analysis)) +
  geom_histogram(binwidth = 0.05, boundary = 0, colour = "white") +
  facet_wrap(~ analysis, ncol = 1) +
  geom_vline(xintercept = 0.05, linetype = "dashed", colour = "red") +
  scale_fill_manual(values = c("#2E86AB", "#E84855")) +
  labs(x = "p-value", y = "Count", fill = "Analysis type") +
  theme_bw() +
  theme(legend.position = "none")
```

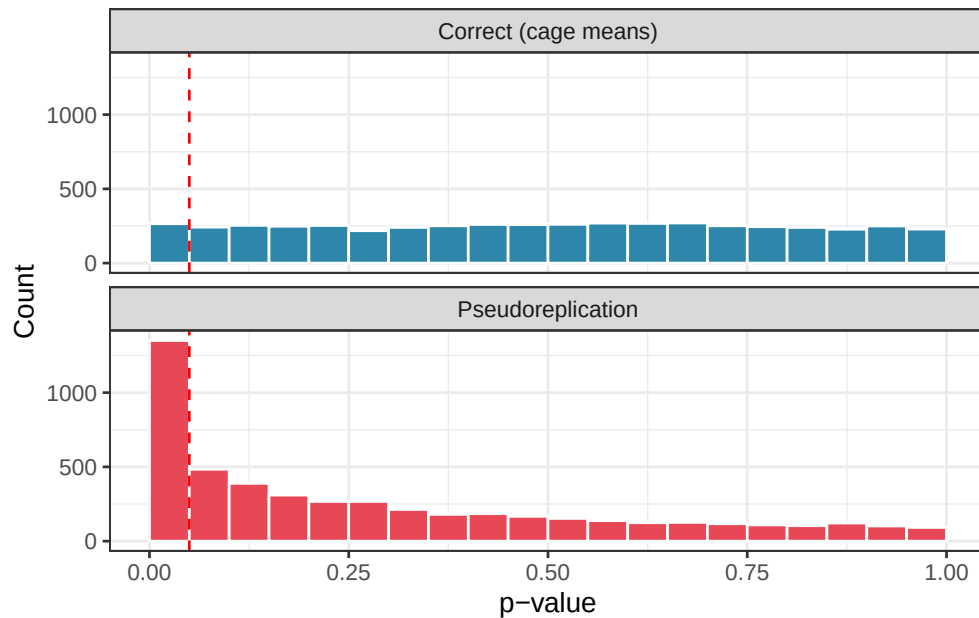


Figure 2.1: Distribution of p -values under the null hypothesis across 5000 simulations. Under the correct analysis (top), p -values are uniformly spread between 0 and 1, the expected behaviour of a well-calibrated test. Under pseudoreplication (bottom), p -values pile up near zero, producing a large excess of spurious significant results. Red dashed line marks $p = 0.05$.

Under the correct analysis, p -values are uniformly distributed between 0 and 1, the signature of a well-calibrated test under the null hypothesis. Under pseudoreplication, p -values pile up

near zero, producing an excess of spurious significant results.

2.1.4 How to detect non-independence

Independence is not something you can test statistically after the fact: it must be assessed by thinking carefully about **how the data were collected**. Ask yourself: - Are any observations collected from the same individual at different times? - Are any observations grouped within a shared physical unit (a cage, a pot, a plot, a classroom, a hospital ward)? - Could observations close together in space or time be more similar to each other than observations far apart?

If the answer to any of these questions is yes, your observations are likely not fully independent, and a standard ANOVA may not be appropriate.

2.1.5 What to do when independence is violated

The good news is that non-independence is not a dead end: **it is a feature of your data that can be modelled explicitly**. The appropriate tools depend on the structure of the dependence:

- **Repeated measures ANOVA** handles the case where the same individual is measured multiple times.
- **Mixed-effects models** (also called multilevel or hierarchical models) handle nested structures such as mice within cages, students within classrooms, or plots within fields. These models estimate the cage-level (or cluster-level) variance explicitly rather than pretending it does not exist.

In all cases, the key principle is the same: **the unit of replication in your statistical analysis must match the unit of replication in your experimental design**. If the cage is the experimental unit, the thing that received the treatment, then the cage, not the individual mouse, must be the unit of analysis.

2.2 Homogeneity of Variances (Homoscedasticity)

2.2.1 What does the assumption say?

ANOVA assumes that the variance of the residuals, the scatter of individual observations around their group mean, is the same in every group. This property is called **homoscedasticity**, from the Greek for “same scatter”. Its opposite, **heteroscedasticity**, refers to situations where different groups have different amounts of internal variation. Concretely, in our fertiliser experiment, homoscedasticity means that the spread of plant heights around the control mean, the spread around the nitrogen mean, and the spread around the phosphorus mean should all be approximately equal. The groups may differ in their *means*, that is what we are testing, but they should not differ in their *variances*.

2.2.2 Why does ANOVA need this?

Recall that the F ratio is built on two variance estimates:

$$F = \frac{\text{Among-group variance}}{\text{Within-group variance}}$$

The within-group variance is computed by **pooling** the scatter from all groups together into a single estimate of background noise. This pooling is only legitimate if the groups genuinely share the same underlying variance. If one group is much more variable than the others, the pooled estimate will be distorted, either inflated or deflated depending on group sizes, and the F ratio will no longer behave as it should under the null hypothesis.

2.2.3 A concrete example

Imagine you are studying the effect of three different training programmes on the resting heart rate of patients recovering from surgery. The control group follows no structured programme.

2 The Assumptions

A moderate exercise group follows a gentle walking routine. An intensive exercise group follows a vigorous rehabilitation protocol.

It is biologically plausible that the intensive group shows much greater variability than the others: some patients respond remarkably well to intense exercise and drop their heart rate considerably, while others find it too demanding and show little improvement or even adverse responses. The control group, doing nothing structured, might cluster more tightly around a common elevated heart rate. The three groups could therefore differ not only in their means but in their spread, a classic heteroscedastic situation.

2.2.4 Simulating the consequences

The code below simulates a situation where there is **no true effect of treatment**, all three groups have the same mean, but the groups differ substantially in their variance.

```
set.seed(123)

n_sim <- 5000
alpha <- 0.05
n <- 10 # observations per group

# True group means, identical, so any significant result is a false positive
mu <- c(70, 70, 70)

# Group standard deviations, deliberately unequal
sigma <- c(2, 5, 15)

p_standard <- numeric(n_sim)
p_welch <- numeric(n_sim)
```

2 The Assumptions

```
for (i in seq_len(n_sim)) {  
  
  y <- c(  
    rnorm(n, mean = mu[1], sd = sigma[1]),  
    rnorm(n, mean = mu[2], sd = sigma[2]),  
    rnorm(n, mean = mu[3], sd = sigma[3])  
  )  
  group <- factor(rep(c("Control", "Moderate", "Intensive"), each = n))  
  df <- data.frame(y, group)  
  
  # Standard ANOVA (assumes equal variances)  
  p_standard[i] <- summary(aov(y ~ group, data = df))[[1]][["Pr(>F)"]][1]  
  
  # Welch's ANOVA (does not assume equal variances)  
  p_welch[i] <- oneway.test(y ~ group, data = df, var.equal = FALSE)$p.value  
  
}
```

We then compare the false positive rate of standard ANOVA (which assumes equal variances) against Welch's ANOVA (which does not).

```
cat(  
  "False positive rate (standard ANOVA):",  
  round(mean(p_standard < alpha), 3),  
  "\nFalse positive rate (Welch's ANOVA):",  
  round(mean(p_welch < alpha), 3), "\n"  
)
```

```
#> False positive rate (standard ANOVA): 0.082
```

```
#> False positive rate (Welch's ANOVA): 0.05
```

Standard ANOVA rejects the null hypothesis in roughly 8% of simulations, more than the nominal 5% rate, even though no true difference exists. Welch's ANOVA stays right at the 5% level, as it should.

2.2.5 Visualising the problem

It is always good practice to look at your data before running any test. A simple plot of the raw observations by group immediately reveals heteroscedasticity that summary statistics alone might obscure.

```
library(ggplot2)

set.seed(123)

# Generate one representative dataset for plotting
y <- c(
  rnorm(n, mean = mu[1], sd = sigma[1]), # control
  rnorm(n, mean = mu[2], sd = sigma[2]), # moderate
  rnorm(n, mean = mu[3], sd = sigma[3]) # intensive
)
group <- factor(
  rep(c("Control", "Moderate", "Intensive"), each = n),
  levels = c("Control", "Moderate", "Intensive")
)
df_plot <- data.frame(y, group)

ggplot(df_plot, aes(x = group, y = y, fill = group)) +
  geom_boxplot(alpha = 0.4, outlier.shape = NA) +
  geom_jitter(width = 0.1, size = 2, alpha = 0.7) +
  scale_fill_manual(values = c("#2E86AB", "#F6AE2D", "#E84855")) +
```

2 The Assumptions

```
labs(x = "Training programme", y = "Resting heart rate (bpm)") +  
theme_bw() +  
theme(legend.position = "none")
```

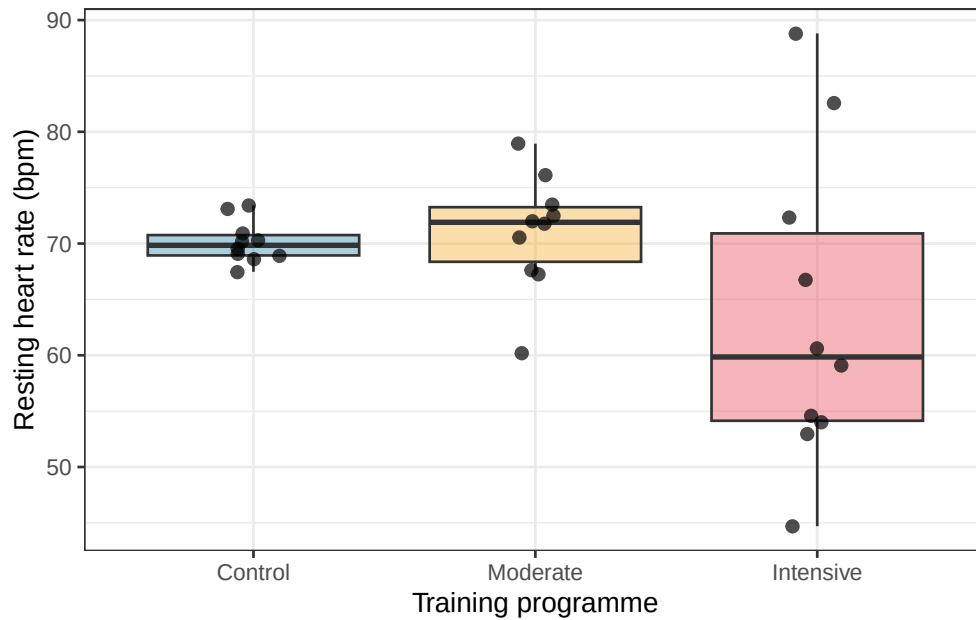


Figure 2.2: Simulated heart rate data from one realisation of the three-group experiment. The groups share the same true mean but differ dramatically in their spread. This pattern, wider scatter in one group than another, is the signature of heteroscedasticity.

Notice how the control group clusters tightly while the intensive group spreads widely up and down. This is exactly the pattern that violates the homoscedasticity assumption and distorts the standard F test.

2.2.6 How to check the assumption

There are two complementary approaches, and you should use both.

- **Graphically:** always the first step. A boxplot or a plot of residuals against fitted values will reveal unequal spread across groups far more intuitively than any test statistic. If

2 The Assumptions

the boxes in your boxplot differ dramatically in height, or if the residual spread fans out as fitted values increase, heteroscedasticity is likely.

- **Formally:** the most commonly used test is **Levene's test**, which tests the null hypothesis that all group variances are equal. In R:

```
# install.packages("car") if not already installed
library(car)
```

```
#> Loading required package: carData
```

```
leveneTest(y ~ group, data = df_plot)
```

```
#> Levene's Test for Homogeneity of Variance (center = median)
```

```
#>      Df F value    Pr(>F)
```

```
#> group  2  6.7529 0.004187 **
```

```
#>      27
```

```
#> ---
```

```
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

A significant result ($p < 0.05$) suggests that the variances are not equal and that standard ANOVA should be used with caution. Note however that Levene's test, like all formal tests of assumptions, is sensitive to sample size: with very large samples it will flag trivially small differences in variance as significant, while with very small samples it may miss substantial heteroscedasticity. The graphical check should always come first.

2.2.7 What to do when the assumption is violated

Fortunately, several straightforward options are available.

Welch's ANOVA is the simplest and most recommended solution for the one-way case. It relaxes the equal-variance assumption by adjusting the degrees of freedom of the F test, and it

2 The Assumptions

performs well even when variances are equal: making it a sensible default in many situations. In R it is a single argument change:

```
# Standard ANOVA
```

```
summary(aov(y ~ group, data = df_plot))
```

```
#>               Df Sum Sq Mean Sq F value Pr(>F)
#> group           2  327.4   163.68    2.177   0.133
#> Residuals      27 2029.7    75.17
```

```
# Welch's ANOVA, preferred when variances may differ
```

```
oneway.test(y ~ group, data = df_plot, var.equal = FALSE)
```

```
#>
```

```
#> One-way analysis of means (not assuming equal variances)
```

```
#>
```

```
#> data: y and group
```

```
#> F = 1.18, num df = 2.000, denom df = 13.596, p-value = 0.3369
```

Variance-stabilising transformations are worth considering when heteroscedasticity has a systematic pattern. If variability tends to increase with the mean, a common situation with count data or strictly positive measurements, a log transformation often brings the variances into approximate equality:

```
df_plot$log_y <- log(df_plot$y)
```

```
oneway.test(log_y ~ group, data = df_plot, var.equal = FALSE)
```

```
#>
```

```
#> One-way analysis of means (not assuming equal variances)
```

```
#>
```

2 The Assumptions

```
#> data: log_y and group  
#> F = 1.548, num df = 2.000, denom df = 13.517, p-value = 0.2481
```

Always check whether the transformed values still make biological sense and whether the transformation has actually equalised the variances before proceeding.

2.2.8 A Note on robustness

Standard ANOVA is moderately robust to mild violations of homoscedasticity, particularly when group sizes are equal. The F test begins to behave badly mainly when two conditions coincide: the variances are very different *and* the group sizes are unequal. In that situation the combination of an unreliable pooled variance estimate and an unbalanced design can push the false positive rate far from the nominal level, as the simulation above illustrates. When in doubt, Welch's ANOVA costs you almost nothing and protects you against a real and avoidable source of error.

2.3 Normality of Residuals: What It Means and What It Doesn't

2.3.1 What Does the Assumption Say?

ANOVA assumes that the residuals, the differences between each observed value and its group mean, are approximately normally distributed. It is worth being precise about this from the start, because the assumption is frequently misunderstood in two important ways.

First misunderstanding: students often believe that ANOVA requires the raw observations to be normally distributed. This is not quite right. What must be approximately normal are the *residuals*, not the raw data. If your groups have very different means, the overall distribution of all observations pooled together will naturally look non-normal even when the residuals within each group are perfectly well-behaved. Always examine the residuals, not the raw values.

2 The Assumptions

Second misunderstanding: students often treat normality as a strict prerequisite, checking it anxiously and abandoning ANOVA at the slightest deviation. As we will show below with simulations, this caution is largely unwarranted for one-way ANOVA. The test is remarkably robust to departures from normality, and the cases where normality genuinely matters are more specific than most textbooks suggest.

2.3.2 What is a residual?

The residual for observation i in group j is:

$$e_{ij} = y_{ij} - \bar{y}_j$$

where y_{ij} is the observed value and $\bar{y}_j$ is the mean of group j . The residual captures everything about an observation that group membership cannot explain. If the model is doing its job, these leftovers should look like random draws from a distribution centred on zero.

2.3.3 Why residuals, not raw data?

This point confuses almost every student encountering ANOVA for the first time, so it is worth working through carefully.

2.3.3.1 Start with what ANOVA actually models

Recall the fundamental equation of ANOVA:

$$y_{ij} = \mu + \alpha_j + \varepsilon_{ij}$$

where y_{ij} is the observed value, μ is the grand mean, α_j is the effect of group j , and ε_{ij} is the residual — the part of the observation that the model cannot explain. The normality assumption

2 The Assumptions

is a statement about ε_{ij} , the residual, not about y_{ij} , the raw observation. To understand why, we need to understand what the residual actually is.

2.3.3.2 The residual is what remains after removing the group effect

Think of it this way. You measure the height of 12 wheat plants across three fertiliser groups. The plants in the nitrogen group are, on average, taller than those in the control group. If you look at the raw heights of all 12 plants pooled together, you will see a spread that reflects two things simultaneously: the genuine differences between fertiliser groups, and the random scatter within each group. These two sources of variation are tangled together in the raw data.

The residual disentangles them. When you subtract each plant's group mean from its observed height, you remove the group effect entirely. What remains, the residual, is pure within-group scatter, stripped of any systematic influence of the treatment. It is this pure scatter that ANOVA assumes to be normally distributed.

A simple analogy: imagine you are measuring the heights of students from three different countries, where average heights differ between countries. If you pool all students together and look at the height distribution, you might see a wide, irregular spread reflecting the mix of nationalities. But if you subtract each student's national average first, the residuals, the deviation of each student from their national average, will look much more like a single clean normal distribution. The raw data were non-normal not because heights within any country were non-normal, but because you were mixing apples and oranges.

2.3.3.3 A concrete simulation

The code below makes this vivid. We simulate three groups with very different means but identical, perfectly normal within-group scatter. We then plot both the raw data and the residuals and examine their distributions.

2 The Assumptions

```
library(ggplot2)
library(patchwork)

set.seed(42)

n <- 30
group_means <- c(10, 25, 45) # very different group means
sigma <- 3 # identical within-group sd, normal scatter

group <- factor(rep(c("Control", "Nitrogen", "Phosphorus"), each = n))
y <- c(
  rnorm(n, mean = group_means[1], sd = sigma), # control
  rnorm(n, mean = group_means[2], sd = sigma), # nitrogen
  rnorm(n, mean = group_means[3], sd = sigma) # phosphorus
)

df <- data.frame(y, group)

# Compute residuals by subtracting each group mean
df$residual <- df$y - ave(df$y, df$group, FUN = mean)

p1 <- ggplot(df, aes(x = y, fill = group)) +
  geom_histogram(binwidth = 2, colour = "white", alpha = 0.8) +
  scale_fill_manual(values = c("#2E86AB", "#F6AE2D", "#E84855")) +
  labs(title = "Raw data (by group)", x = "Plant height (cm)", y = "Count") +
  theme_bw() +
  theme(legend.position = "none")

p2 <- ggplot(df, aes(x = y)) +
```

2 The Assumptions

```
geom_histogram(binwidth = 2, fill = "#555555", colour = "white") +  
labs(title = "Raw data (all groups pooled)", x = "Plant height (cm)", y = "Count") +  
theme_bw()  
  
p3 <- ggplot(df, aes(x = residual, fill = group)) +  
  geom_histogram(binwidth = 1, colour = "white", alpha = 0.8) +  
  scale_fill_manual(values = c("#2E86AB", "#F6AE2D", "#E84855")) +  
  labs(title = "Residuals (by group)", x = "Residual", y = "Count") +  
  theme_bw() +  
  theme(legend.position = "none")  
  
p4 <- ggplot(df, aes(x = residual)) +  
  geom_histogram(binwidth = 1, fill = "#555555", colour = "white") +  
  labs(title = "Residuals (all groups pooled)", x = "Residual", y = "Count") +  
  theme_bw()  
  
(p1 + p3) / (p2 + p4)
```

2 The Assumptions

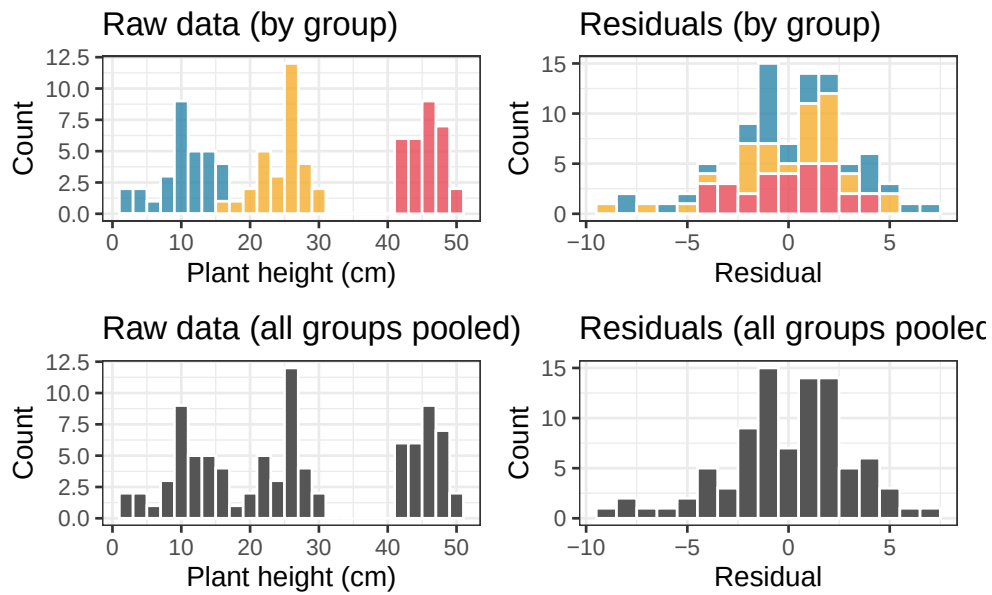


Figure 2.3: Left column: the raw observations from three groups with very different means. The pooled distribution (bottom left) looks nothing like a normal distribution, it is wide and irregular. Right column: the residuals after subtracting each group mean. The pooled residual distribution (bottom right) is clearly normal, because the within-group scatter was normal all along. ANOVA only requires the right column to be normal.

Look carefully at the bottom row. The pooled raw data (bottom left) shows three separate humps, one for each group, spread across a wide range. This looks nothing like a normal distribution, and a Shapiro-Wilk test on the raw data would resoundingly reject normality. Yet the data were generated with perfectly normal within-group scatter. The apparent non-normality of the raw data is entirely an artefact of the groups having different means. It tells you nothing about whether ANOVA is appropriate.

The pooled residuals (bottom right), by contrast, form a single clean bell-shaped distribution centred on zero. This is because subtracting the group means has removed the between-group differences, leaving only the within-group scatter, which was normal by construction.

2.3.3.4 Why does the F test need normal residuals?

The F statistic is a ratio of two variance estimates. The mathematical theory that tells us how F behaves under the null hypothesis, and therefore allows us to compute a p -value, was derived under the assumption that the residuals are normally distributed.

💡 Show me the maths!

Want to understand the precise mathematical chain that connects normal residuals to the F distribution? See Section ?? in Appendix A.

Specifically, when residuals are normal, the two variance estimates in the F ratio follow chi-squared distributions, and the ratio of two chi-squared variables follows an F distribution. It is this chain of reasoning that connects your data to the p -value printed by R.

If the residuals are not normal, this chain is broken, at least in principle. The F statistic may no longer follow the F distribution under the null hypothesis, and the p -value becomes an approximation whose quality depends on how far from normality the residuals actually are. As the simulations in the previous section showed, this approximation tends to remain very good for one-way ANOVA even under substantial departures from normality. But understanding *why* normality is the requirement, rather than just memorising that it is, helps you make informed judgements about when to worry and when not to.

2.3.3.5 The practical consequence

Never apply the Shapiro-Wilk test or draw a Q-Q plot on your raw observations. Always fit the ANOVA model first, extract the residuals, and examine those. In R this is a single line:

```
fit <- aov(y ~ group, data = df)

# Extract residuals
res <- residuals(fit)
```

2 The Assumptions

```
# Q-Q plot
ggplot(fit, aes(sample = res)) +
  stat_qq(colour = "#2E86AB", size = 2) +
  stat_qq_line(colour = "#E84855", linewidth = 0.8) +
  labs(x = "Theoretical quantiles", y = "Sample quantiles") +
  theme_bw()

# Formal test
shapiro.test(res)
```

```
#>
#> Shapiro-Wilk normality test
#>
#> data:  res
#> W = 0.97768, p-value = 0.124
```

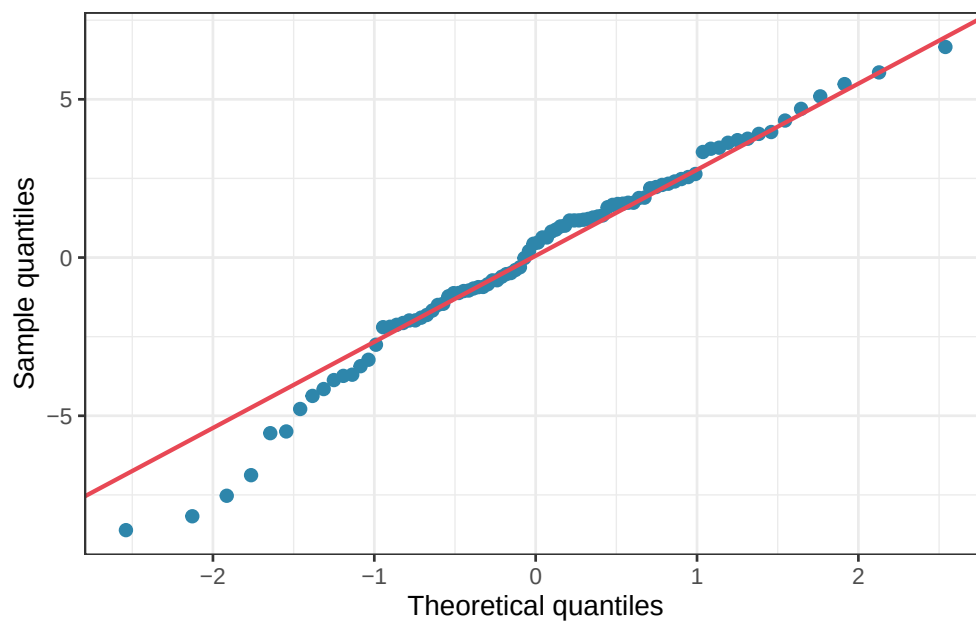


Figure 2.4: Residuals QQ-plot showing minor divergence from normality.

2 The Assumptions

If you had applied `shapiro.test(df$y)` to the raw data in our simulated example above, you would have concluded that the data were catastrophically non-normal and that ANOVA was inappropriate, the exact wrong conclusion, since the data were generated with perfectly normal within-group scatter.

Checking residuals rather than raw values is not a technicality. It is the difference between asking the right question and the wrong one.

2.3.4 How serious is the violation, really? A simulation

Rather than asserting that normality matters, let us measure how much it matters directly. The code below generates data from distributions of increasing non-normality, specifically t -distributions with decreasing degrees of freedom (df), which range from nearly normal (high df) to heavy-tailed (low df), and measures the false positive rate of ANOVA in each case.

```
set.seed(42)

n_sim <- 5000
alpha <- 0.05
n <- 8 # small group size, where normality matters most
df_values <- c(2, 3, 4, 5, 10, 30, Inf) # Inf = exactly normal

results <- data.frame(
  distribution = c("t (df=2)", "t (df=3)", "t (df=4)",
                  "t (df=5)", "t (df=10)", "t (df=30)", "Normal"),
  df = df_values,
  fpr = NA
)

for (j in seq_along(df_values)) {
```


2 The Assumptions

```
p_vals <- numeric(n_sim)
for (i in seq_len(n_sim)) {
  # Draw from t-distribution (Inf df = normal)
  y <- if (is.infinite(df_values[j])) {
    rnorm(3 * n)
  } else {
    rt(3 * n, df = df_values[j])
  }

  group <- factor(rep(c("A", "B", "C"), each = n))
  p_vals[i] <- summary(
    aov(y ~ group, data = data.frame(y, group))
  )[[1]][["Pr(>F)"]][1]
}

results$fpr[j] <- mean(p_vals < alpha)
}
```

All three groups share the same mean, so every significant result is a false positive.

```
print(results)
```

```
#>  distribution  df    fpr
#> 1      t (df=2)   2 0.0326
#> 2      t (df=3)   3 0.0422
#> 3      t (df=4)   4 0.0424
#> 4      t (df=5)   5 0.0466
#> 5      t (df=10) 10 0.0476
#> 6      t (df=30) 30 0.0508
#> 7      Normal Inf 0.0452
```

2 The Assumptions

The result is striking: even with extremely heavy-tailed distributions ($df = 2$ is very far from normal), the false positive rate never strays far from the nominal 5%. If anything, ANOVA becomes slightly *conservative* under heavy tails, the false positive rate drops a little below 0.05, meaning the test becomes harder to reject than it should be, not easier. This is the opposite of what most students expect.

Let us visualise this across the range of distributions tested:

```
library(ggplot2)

# Monte Carlo confidence interval for the false positive rate
# given n_sim = 5000 simulations
mc_lower <- qbinom(0.025, n_sim, alpha) / n_sim
mc_upper <- qbinom(0.975, n_sim, alpha) / n_sim

results$distribution <- factor(results$distribution, levels = results$distribution)

ggplot(results, aes(x = distribution, y = fpr)) +
  annotate("rect", xmin = -Inf, xmax = Inf, ymin = mc_lower, ymax = mc_upper, fill = "#2E86AB", alpha = 0.5) +
  geom_hline(yintercept = alpha, linetype = "dashed", colour = "red", linewidth = 0.8) +
  geom_point(size = 3, colour = "#2E86AB") +
  geom_line(aes(group = 1), colour = "#2E86AB", linewidth = 0.8) +
  scale_y_continuous(limits = c(0, 0.10), labels = scales::percent_format(accuracy = 1)) +
  labs(x = "Residual distribution (left = heavy-tailed, right = normal)", y = "False positive rate") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```

2 The Assumptions

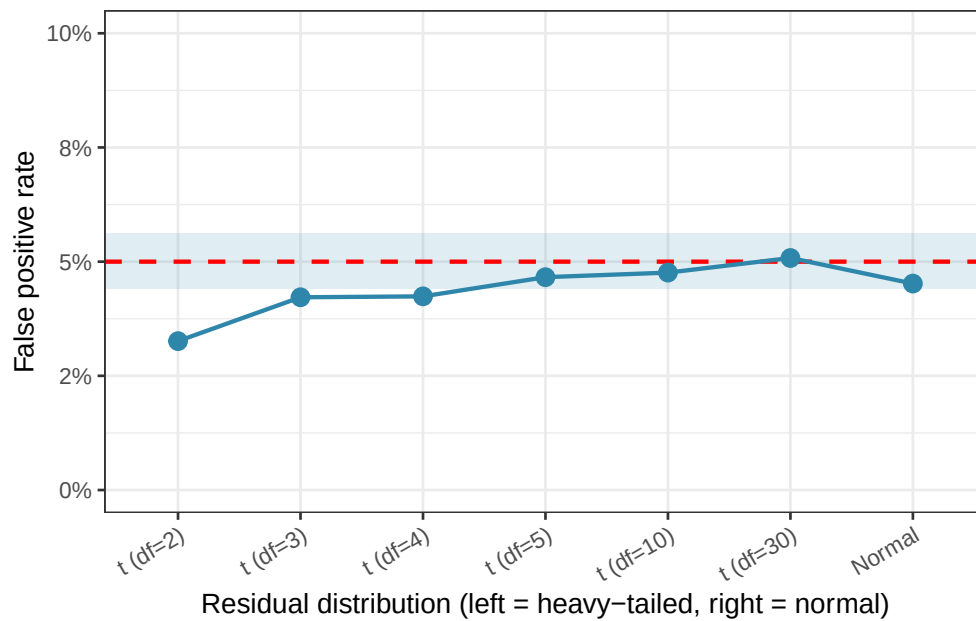


Figure 2.5: False positive rate of one-way ANOVA under increasing departures from normality (t-distributions with decreasing degrees of freedom). The dashed red line marks the nominal 5% level. The shaded band shows a 95% Monte Carlo confidence interval around the target rate. The false positive rate stays close to 5% throughout, demonstrating the robustness of ANOVA to non-normality.

All points fall within or very close to the Monte Carlo confidence band around the 5% target, the expected range of sampling variation given 5000 simulations. ANOVA is not merely tolerating non-normality here; it is genuinely indifferent to it, at least in terms of false positive control.

2.3.5 So why does anyone check normality?

If ANOVA is this robust, why do textbooks, and this one, discuss normality at all? There are three legitimate reasons.

Outliers are the real concern. The distributions above are symmetric and heavy-tailed, but they do not contain the kind of isolated, extreme outlier that arises from a data entry error, a broken instrument, or a single anomalous individual. A genuine outlier, say one plant that grew ten times taller than all others because it was accidentally given a growth hormone, will

2 The Assumptions

dramatically inflate the within-group variance, absorbing variation that should be attributed to the treatment. The result is not a false positive but a **false negative**: a real treatment effect that the test fails to detect because the noise floor has been artificially raised. A Q-Q plot will reveal this outlier immediately.

Normality matters more in complex designs. The robustness demonstrated above applies cleanly to the balanced one-way ANOVA. In unbalanced designs, two-way ANOVAs, repeated measures, and mixed models, departures from normality interact with other features of the design in ways that are harder to predict and can be more consequential. Good habits formed in the one-way case carry over to these more demanding situations.

Confidence intervals and post-hoc tests are more sensitive. Even when the F test itself is robust, the confidence intervals around group means and the p -values from post-hoc comparisons rely on the same normality assumption and may be less trustworthy when it is badly violated.

2.3.6 How to Check Residuals in Practice

The Q-Q plot remains the best practical tool, not to decide whether to run ANOVA, but to spot outliers and gross departures that warrant investigation.

```
library(ggplot2)
library(patchwork)

set.seed(7)

n <- 8
group <- factor(rep(c("Control", "Nitrogen", "Phosphorus"), each = n))

# Clean data, normal residuals
y_clean <- rnorm(3 * n, mean = 10, sd = 2)
```

2 The Assumptions

```
fit_clean <- aov(y_clean ~ group)

# Contaminated data, one outlier injected
y_outlier <- y_clean
y_outlier[1] <- y_outlier[1] + 18 # one extreme value
fit_outlier <- aov(y_outlier ~ group)

res_df <- data.frame(
  residuals = c(residuals(fit_clean), residuals(fit_outlier)),
  dataset = rep(c("Clean data", "One outlier added"), each = 3 * n)
)

ggplot(res_df, aes(sample = residuals)) +
  stat_qq(colour = "#2E86AB", size = 2) +
  stat_qq_line(colour = "#E84855", linewidth = 0.8) +
  facet_wrap(~ dataset) +
  labs(x = "Theoretical quantiles", y = "Sample quantiles") +
  theme_bw()
```

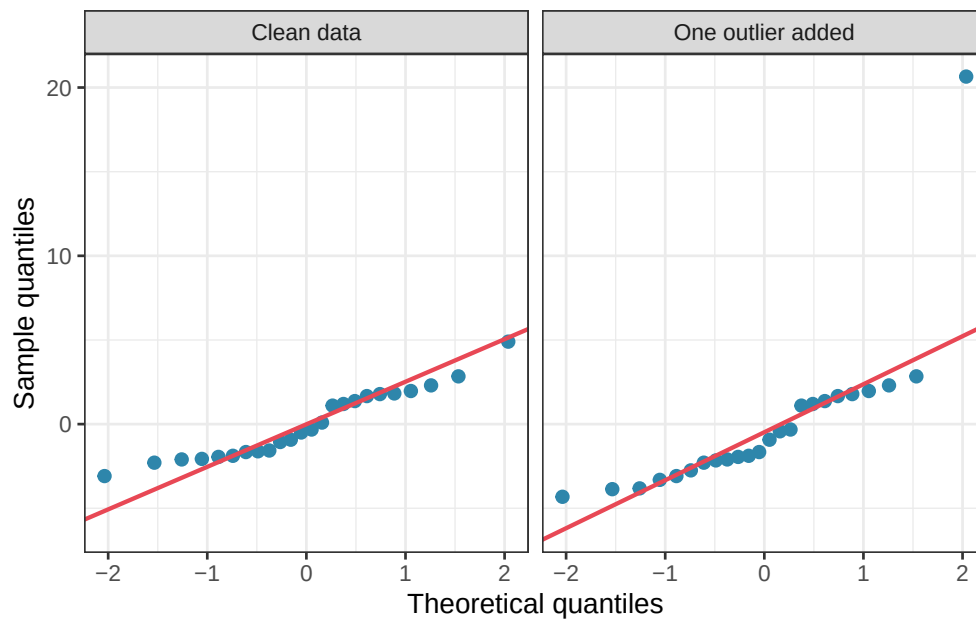


Figure 2.6: Q-Q plots of ANOVA residuals from two simulated datasets with $n = 8$ per group. Left: residuals drawn from a normal distribution, points fall closely along the diagonal. Right: the same data with one artificial outlier added to the first group, the single extreme point is immediately visible at the top right.

The outlier jumps off the page in the Q-Q plot. General non-normality, by contrast, produces only a gentle curve that is easy to overlook and, as the simulations show, rarely causes practical problems.

2.3.7 What to do when you find an outlier

When a Q-Q plot reveals an extreme point, the first step is always to **go back to the data**. Ask whether the outlier is:

- A data entry or measurement error: in which case it should be corrected or removed with a clear justification recorded.
- A genuine biological observation: in which case removing it is not appropriate, but reporting the analysis both with and without it is good practice.

If outliers are a structural feature of your data rather than isolated accidents, a robust alternative to ANOVA is available. The **Kruskal-Wallis test** replaces raw values with their ranks, making it insensitive to extreme observations:

```
kruskal.test(y_outlier ~ group)
```

```
#>
```

```
#> Kruskal-Wallis rank sum test
```

```
#>
```

```
#> data: y_outlier by group
```

```
#> Kruskal-Wallis chi-squared = 5.465, df = 2, p-value = 0.06506
```

2.3.8 The key takeaway

Normality of residuals is the least critical of the three ANOVA assumptions. For balanced one-way ANOVA, simulations consistently show that the F test maintains its false positive rate well even under substantial departures from normality. So, when doing analysis, check your residuals with a Q-Q plot, but do so to hunt for outliers and data problems, not to agonise over gentle curves. In priority, reserve non-parametric alternatives for situations where outliers are severe and structural, not as a reflexive response to an imperfect Q-Q plot.

The hierarchy of assumptions matters: independence first, homogeneity of variance second, normality a distant third.

2.4 Additivity and linearity

2.4.1 What does the assumption say?

The linear model underlying ANOVA assumes that the effect of a treatment adds on to the baseline in a simple, additive way. Formally, the model says:

2 The Assumptions

$$y_{ij} = \mu + \alpha_j + \varepsilon_{ij}$$

The effect of group j is α_j , and it is assumed to shift the response up or down by a fixed amount, independently of everything else. This is the **additivity** assumption: the treatment effect and the error simply add together to produce the observed value. There are no interactions between the treatment and the background noise, no situation where the treatment works differently for some individuals than others.

2.4.2 Why does this matter?

Additivity can fail in a subtle but important way. Suppose you are measuring the effect of three fertilisers on plant yield. For plants that are already growing vigorously, the fertiliser might produce a large absolute increase in yield. For plants that are struggling, the same fertiliser might produce only a small absolute increase. The treatment effect is not a fixed additive quantity, it depends on the baseline level of the plant. This is called a **treatment-by-individual interaction**, and it violates additivity.

A more common and detectable form of non-additivity occurs when the treatment effect is multiplicative rather than additive. Suppose nitrogen fertiliser multiplies yield by a factor of 1.5, regardless of baseline. A plant yielding 10g becomes a plant yielding 15g; a plant yielding 20g becomes a plant yielding 30g. The absolute effect (5g vs 10g) grows with the baseline, which means the within-group variance will also grow with the group mean, exactly the heteroscedasticity pattern we discussed in section Section ?? . **Non-additivity and heteroscedasticity often appear together**, which is why a log transformation frequently fixes both problems simultaneously.

2.4.3 Spotting non-additivity

The clearest sign of non-additivity is a systematic relationship between group means and group variances. If the groups with the highest means also show the most spread, a multiplicative

2 The Assumptions

structure is likely. A simple plot of group standard deviations against group means will reveal this pattern:

```
library(ggplot2)

set.seed(42)
n <- 15

# Multiplicative model: treatment multiplies the response
# Within-group sd grows with the mean: non-additive, heteroscedastic
group <- factor(rep(c("Control", "Nitrogen", "Phosphorus"), each = n))
y_mult <- c(
  rnorm(n, mean = 10, sd = 2),
  rnorm(n, mean = 20, sd = 4),
  rnorm(n, mean = 40, sd = 8)
)
df_mult <- data.frame(y = y_mult, group = group)

# Summarise group means and sds
group_summary <- aggregate(y ~ group, data = df_mult, FUN = function(x) c(mean = mean(x), sd = sd(x)))
group_summary <- do.call(data.frame, group_summary)
names(group_summary) <- c("group", "mean", "sd")

ggplot(group_summary, aes(x = mean, y = sd, label = group)) +
  geom_point(size = 4, colour = "#2E86AB") +
  geom_text(vjust = -1, colour = "#2E86AB") +
  geom_smooth(method = "lm", se = FALSE, colour = "#E84855", linetype = "dashed") +
  labs(x = "Group mean", y = "Group standard deviation") +
  theme_bw()
```

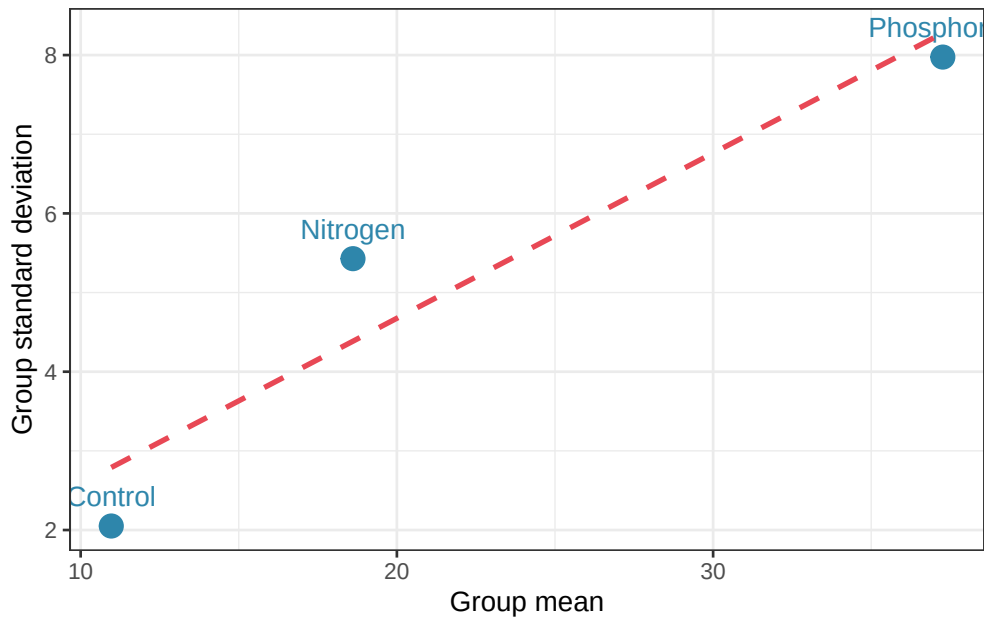


Figure 2.7: Group standard deviation vs group mean. A positive relationship suggests a multiplicative structure

A positive linear relationship between group mean and group standard deviation is a reliable diagnostic of multiplicative non-additivity. The remedy, as before, is a log transformation, which converts multiplicative relationships into additive ones:

$$\log(y_{ij}) = \mu + \alpha_j + \varepsilon_{ij}$$

After log transformation, the group standard deviations should no longer increase with the group means, and both additivity and homoscedasticity will be approximately restored.

2.4.4 A practical note

In one-way ANOVA with a single treatment factor, additivity is an assumption about the relationship between the treatment and the error term. It becomes a richer and more consequential

assumption in two-way and factorial designs, where it governs whether the effects of two factors combine additively or interact.

A significant interaction term in a two-way ANOVA is, in a precise sense, evidence of non-additivity: the effect of one factor depends on the level of the other. This will be discussed fully in the chapter on factorial designs ([?@sec-two-way](#)).

2.5 Fixed vs. random effects: A distinction that matters

2.5.1 Two kinds of groups

So far we have treated the groups in our ANOVA as fixed and chosen: control, nitrogen, and phosphorus are the specific fertilisers we are interested in, and our conclusions are about those specific fertilisers. This is called a **fixed effects** model.

But sometimes the groups in an experiment are not chosen because they are intrinsically interesting but because they are a random sample from a larger population of groups that we want to make inferences about. Consider the following contrast:

- **Fixed effects example:** You test three specific fertilisers, control, nitrogen, and phosphorus, because you want to know whether these particular fertilisers differ in their effect on plant yield. Your conclusions are about these three fertilisers and these three alone.
- **Random effects example:** You are studying natural variation in soil bacterial communities across forests. You select 12 forest plots at random from a region and measure bacterial diversity in each. The 12 plots are not interesting in themselves, instead they are a random sample representing all the forests in the region. You want to know how much diversity varies across forests in general, not just across these 12 specific plots.

In the random effects case, the groups are themselves a random draw from a population of groups, and the model becomes:

2 The Assumptions

$$y_{ij} = \mu + b_j + \varepsilon_{ij}$$

where $b_j \sim \mathcal{N}(0, \sigma_b^2)$ is a random group effect drawn from a normal distribution with variance σ_b^2 . The question shifts from *do the group means differ?* to *how large is σ_b^2 , the variance among groups?*

2.5.2 Why does the distinction matter?

The distinction has three important practical consequences.

First, it affects what question you are answering. A fixed effects ANOVA tests whether specific group means differ. A random effects model estimates how much variability exists among groups in general. These are genuinely different scientific questions, and conflating them leads to conclusions that do not match the study design.

Second, it affects generalisation. With fixed effects, your conclusions apply only to the groups you tested. With random effects, your conclusions generalise to the population of groups from which your sample was drawn. If your 12 forest plots are a random sample, you can say something about forests in general. If you had hand-picked 12 interesting forests, you cannot.

Third, it affects how replication is counted. This connects directly to the independence issue discussed in section 2.1. Suppose you measure bacterial diversity at five locations within each of your 12 forest plots. The five measurements within a plot are not independent they are instead nested within the plot, which is the true unit of replication. The plot is the random effect; the within-plot measurements are the residual. Ignoring this nesting and treating all $12 \times 5 = 60$ measurements as independent is pseudoreplication.

2.5.3 A concrete illustration

The code below simulates both scenarios and shows how differently they should be analysed in R.

2 The Assumptions

```
library(lme4)
```

```
#> Loading required package: Matrix
```

```
#> Registered S3 method overwritten by 'lme4':
```

```
#> method from
```

```
#> na.action.merMod car
```

```
set.seed(42)
```

```
n_groups <- 6 # number of groups (fertilisers or forest plots)
```

```
n_each <- 5 # observations per group
```

```
group <- factor(rep(paste0("Group", 1:n_groups), each = n_each))
```

```
# Fixed effects scenario:
```

```
# specific group means chosen by the experimenter
```

```
fixed_means <- c(10, 12, 15, 11, 14, 13)
```

```
y_fixed <- rnorm(n_groups * n_each, mean = rep(fixed_means, each = n_each), sd = 2)
```

```
df_fixed <- data.frame(y = y_fixed, group = group)
```

```
# Random effects scenario:
```

```
# group means are themselves drawn from a normal distribution
```

```
group_effects <- rnorm(n_groups, mean = 0, sd = 3)
```

```
y_random <- rnorm(n_groups * n_each, mean = 12 + rep(group_effects, each = n_each), sd = 2)
```

```
df_random <- data.frame(y = y_random, group = group)
```

Now let us do a standard ANOVA testing the specific group means

2 The Assumptions

```
# Fixed effects: standard ANOVA, tests specific group means
summary(aov(y ~ group, data = df_fixed))
```

```
#>           Df Sum Sq Mean Sq F value  Pr(>F)
#> group         5  127.1   25.426    3.994 0.00888 **
#> Residuals    24   152.8    6.366
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

And a **mixed model** estimating the variance σ^2 among groups:

```
# Random effects: mixed model, estimates variance among groups
fit_random <- lmer(y ~ 1 + (1 | group), data = df_random)
summary(fit_random)
```

```
#> Linear mixed model fit by REML ['lmerMod']
#> Formula: y ~ 1 + (1 | group)
#>   Data: df_random
#>
#> REML criterion at convergence: 137.5
#>
#> Scaled residuals:
#>      Min       1Q   Median       3Q      Max
#> -2.4466 -0.4216  0.0558  0.7277  1.3970
#>
#> Random effects:
#>   Groups   Name              Variance Std.Dev.
#>   group    (Intercept)  5.841      2.417
#>   Residual                        4.175      2.043
```

2 The Assumptions

```
#> Number of obs: 30, groups: group, 6
#>
#> Fixed effects:
#>               Estimate Std. Error t value
#> (Intercept)   12.077      1.055    11.45
```

The two analyses ask fundamentally different questions of their respective datasets, and their outputs reflect this.

The fixed effects ANOVA produces a familiar table. The F test is significant ($F_{5,24} = 3.99$, $p = 0.009$), and the conclusion is specific: these six groups, chosen deliberately by the experimenter differ significantly in their means. The analysis says nothing about any other groups beyond these six, it was never designed to.

The random effects model produces a very different kind of output. There is no F test, no p-value, and no table of group means. Instead, the model decomposes the total variance into two components. The variance among groups is $\hat{\sigma}_b^2 = 5.84$, representing the genuine differences in means across the population of groups from which these six were sampled. The residual variance (the within-group variation) is $\hat{\sigma}_\varepsilon^2 = 4.18$. The only fixed effect is the intercept ($\hat{\mu} = 12.08$), which estimates the grand mean of the population of groups.

```
# Extract variance components
vc <- as.data.frame(VarCorr(fit_random))
cat(
  "Variance among groups:",
  round(vc$vcov[1], 3),
  "\nVariance within groups:",
  round(vc$vcov[2], 3),
  "\nIntraclass correlation (ICC):",
  round(vc$vcov[1] / sum(vc$vcov), 3), "\n"
)
```

2 The Assumptions

```
#> Variance among groups: 5.841  
#> Variance within groups: 4.175  
#> Intraclass correlation (ICC): 0.583
```

The **intraclass correlation coefficient (ICC)** reported at the end is a key quantity in random effects models. It estimates the proportion of total variance that is attributable to differences among groups:

$$\text{ICC} = \frac{\sigma_b^2}{\sigma_b^2 + \sigma_\varepsilon^2}$$

An ICC close to 1 means that most of the variation in the data comes from differences between groups meaning that observations within the same group are very similar to each other. An ICC close to 0 means that group membership explains little meaning that observations within the same group are no more similar than observations from different groups.

The intraclass correlation coefficient is $\text{ICC} = 0.583$, meaning that 58.3% of the total variation in the response is attributable to differences among groups. Observations from the same group are substantially more similar to each other than observations from different groups indicating a strong grouping effect that would badly distort any analysis that ignored it.

This contrast between the two outputs is worth sitting with. The fixed effects model produces a decision, *the groups differ*, but generalises no further than the six groups tested. The random effects model produces an estimate of population-level variability, *groups in general account for 58% of the variation in this response, and this statement generalises to all groups in the population*, not just the six that happened to be sampled.

The choice between these two framings is not a statistical technicality. It is a scientific decision that must be made before the analysis begins, based on whether the groups in the study are the specific entities of interest or a sample representing a broader population.

2.5.4 How do I know which model to use?

Ask yourself one question: are the groups in your study the specific entities you want to draw conclusions about, or are they a sample representing a broader population?

Situation	Model
Testing three specific drugs	Fixed effects
Comparing four specific diets	Fixed effects
10 randomly selected farms from a region	Random effects
8 patients randomly selected from a hospital	Random effects
Plots nested within fields, fields are a random sample	Mixed model

When both fixed and random effects are present in the same model, for example, a fixed treatment applied across randomly selected sites, the appropriate tool is a **linear mixed-effects model**, which we will return to in a later chapter (?@sec-mem).

2.6 What happens when assumptions fail? Consequences for inference

2.6.1 A map of consequences

The previous sections examined each assumption in isolation. Here we step back and ask: when assumptions are violated, what actually goes wrong, and how badly? The answer depends critically on *which* assumption is violated, *how severely*, and in *what combination* with other violations. The table below provides a practical overview.

Assumption violated	Primary consequence	Severity	Most affected
Independence	Inflated false positive rate	Severe	p -values

2 The Assumptions

Assumption violated	Primary consequence	Severity	Most affected
Homoscedasticity (unequal n)	Inflated false positive rate	Moderate	p -values
Homoscedasticity (equal n)	Minor distortion	Mild	p -values
Normality (large n)	Negligible	Negligible	Nothing
Normality (small n , outliers)	Reduced power	Mild–Moderate	Power
Additivity	Biased estimates	Moderate	Effect sizes
Wrong effect type (fixed vs random)	Wrong inference target	Severe	Conclusions

Two patterns stand out from this table and deserve emphasis.

2.6.2 Non-independence is the most dangerous violation

As section ?? demonstrated, ignoring the clustering structure of your data can push the false positive rate from 5% to 25% or higher. Unlike violations of normality or even homoscedasticity, non-independence does not produce modest, predictable distortions, it can do worse by making the F test essentially meaningless. And unlike the other assumptions, it cannot be diagnosed from the data alone: *it requires thinking carefully about the study design before a single observation is collected.*

The simulation below reinforces this by placing all four main violations side by side, using a consistent framework to make the consequences directly comparable.

```
set.seed(42)

n_sim <- 5000
alpha <- 0.05
```

2 The Assumptions

```
n <- 10
n_grps <- 3

results <- data.frame(
  violation = c("None (ideal)", "Non-independence", "Heteroscedasticity", "Non-normality"), fpr

### 1. No violation: ideal data
p_ideal <- numeric(n_sim)
for (i in seq_len(n_sim)) {
  y      <- rnorm(n_grps * n, mean = 0, sd = 1)
  group  <- factor(rep(letters[1:n_grps], each = n))
  p_ideal[i] <- summary(aov(y ~ group))[[1]][["Pr(>F)"]][1]
}
results$fpr[1] <- mean(p_ideal < alpha)

### 2. Non-independence: observations clustered within groups
p_nonind <- numeric(n_sim)
cluster_sd <- 2
indiv_sd <- 0.5
for (i in seq_len(n_sim)) {
  cluster_effect <- rep(rnorm(n_grps * 2, 0, cluster_sd), each = n / 2)
  indiv_noise    <- rnorm(n_grps * n, 0, indiv_sd)
  y              <- cluster_effect + indiv_noise
  group          <- factor(rep(letters[1:n_grps], each = n))
  p_nonind[i] <- summary(aov(y ~ group))[[1]][["Pr(>F)"]][1]
}
results$fpr[2] <- mean(p_nonind < alpha)

### 3. Heteroscedasticity: group variances differ dramatically
```

2 The Assumptions

```
p_hetero <- numeric(n_sim)
for (i in seq_len(n_sim)) {
  y <- c(
    rnorm(n, mean = 0, sd = 1),
    rnorm(n, mean = 0, sd = 3),
    rnorm(n, mean = 0, sd = 9)
  )
  group <- factor(rep(letters[1:n_grps], each = n))
  p_hetero[i] <- summary(aov(y ~ group))[[1]][["Pr(>F)"]][1]
}
results$fpr[3] <- mean(p_hetero < alpha)

### 4. Non-normality: heavy-tailed t distribution (df = 3)
p_nonnorm <- numeric(n_sim)
for (i in seq_len(n_sim)) {
  y <- rt(n_grps * n, df = 3)
  group <- factor(rep(letters[1:n_grps], each = n))
  p_nonnorm[i] <- summary(aov(y ~ group))[[1]][["Pr(>F)"]][1]
}
results$fpr[4] <- mean(p_nonnorm < alpha)
```

```
print(results)
```

```
#>      violation    fpr
#> 1      None (ideal) 0.0486
#> 2 Non-independence 0.6944
#> 3 Heteroscedasticity 0.0800
#> 4      Non-normality 0.0432
```

2 The Assumptions

```
library(ggplot2)

mc_lower <- qbinom(0.025, n_sim, alpha) / n_sim
mc_upper <- qbinom(0.975, n_sim, alpha) / n_sim

results$violation <- factor(results$violation, levels = results$violation)

ggplot(results, aes(x = violation, y = fpr, fill = violation)) +
  annotate("rect", xmin = -Inf, xmax = Inf, ymin = mc_lower, ymax = mc_upper, fill = "grey70", al
  geom_hline(yintercept = alpha, linetype = "dashed", colour = "red", linewidth = 0.8) +
  geom_col(width = 0.5, alpha = 0.85) +
  scale_fill_manual(values = c("#2E86AB", "#E84855", "#F6AE2D", "#4CAF50")) +
  scale_y_continuous(limits = c(0, 0.75), labels = scales::percent_format(accuracy = 1)) +
  labs(x = NULL, y = "False positive rate") +
  theme_bw() +
  theme(legend.position = "none", axis.text.x = element_text(size = 11))
```

2 The Assumptions

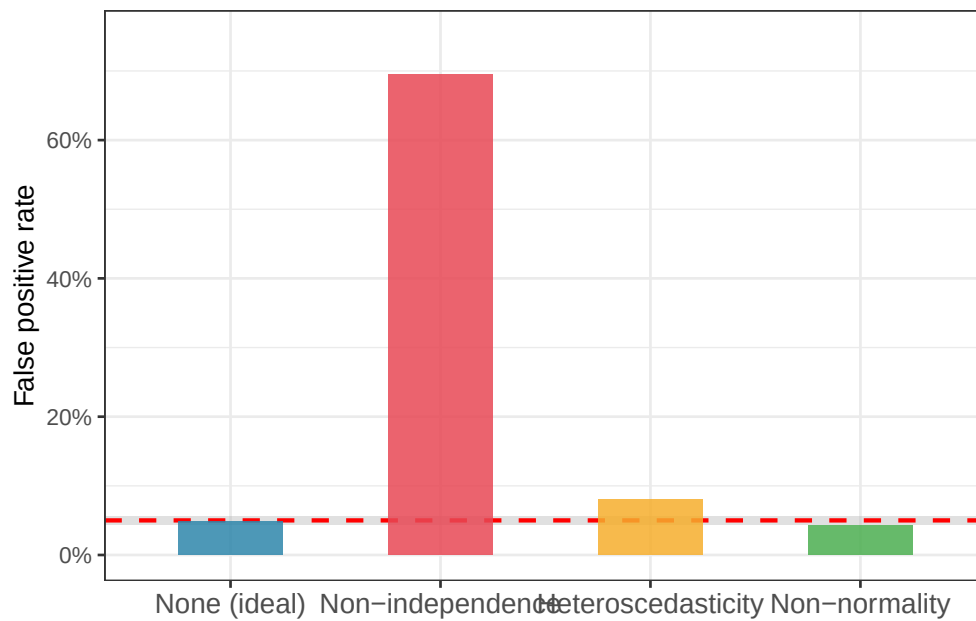


Figure 2.8: False positive rates of one-way ANOVA under four conditions: no violation, non-independence, heteroscedasticity, and non-normality. The dashed red line marks the nominal 5% level. The shaded band is the 95% Monte Carlo interval. Non-independence is by far the most damaging violation; non-normality is essentially harmless.

2.6.3 Violations do not always act alone

A final and important point: assumption violations rarely occur in isolation in real data. Non-normality and heteroscedasticity often appear together because both can stem from the same underlying cause, a multiplicative data-generating process, as discussed in section ???. Non-independence and heteroscedasticity can compound each other when clusters differ not only in their means but in their internal variability.

When multiple assumptions are simultaneously violated, the consequences can be harder to predict than the table above suggests. The safest strategy is not to check assumptions as a ritual box-ticking exercise after the analysis, but to think carefully about the data-generating process before fitting any model. Ask yourself:

- How were the observations collected? Is independence plausible?

2 The Assumptions

- Are the groups likely to differ in their variability, not just their means?
- Is the response likely to be additive, or does the effect of the treatment probably scale with the baseline?
- Are the groups in your study fixed choices or a random sample from a larger population?

These questions, answered before opening R, will protect you far more reliably than any battery of post-hoc assumption tests.

2.6.4 A decision framework

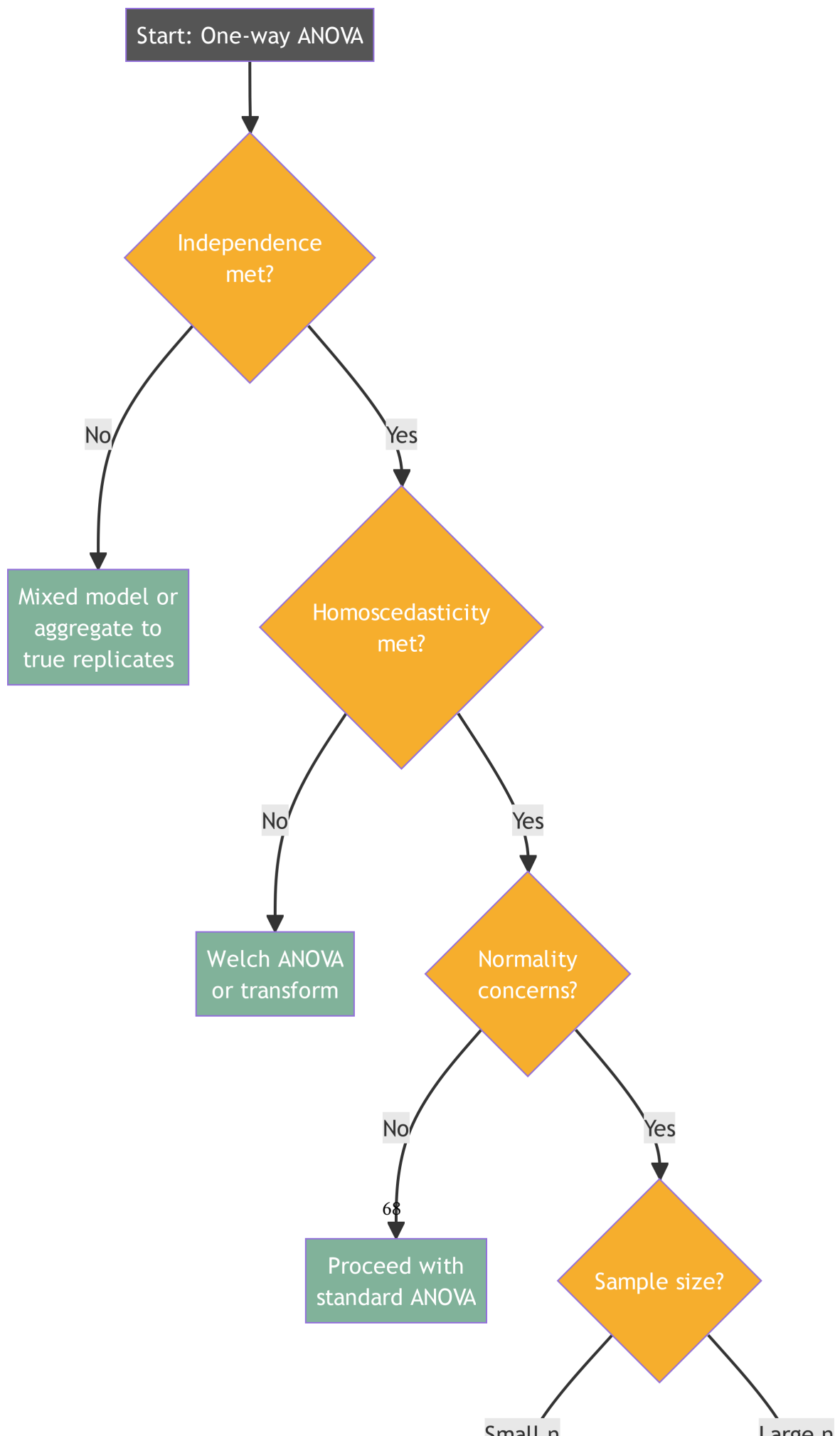
When assumptions are found to be violated, the following decision tree provides a practical path forward:

The central message of this chapter is simple: **not all assumption violations are created equal**. Non-independence can invalidate your analysis entirely. Heteroscedasticity with unequal group sizes deserves attention and is easily addressed with Welch's ANOVA. Non-normality, in the context of one-way ANOVA with reasonable sample sizes, is rarely a serious problem. Understanding this hierarchy allows you to focus your diagnostic energy where it actually matters.

2.7 Checking assumptions in practice

The previous sections examined each assumption individually. Here we bring everything together into a practical workflow, the sequence of checks you should carry out every time you fit an ANOVA. The goal is not to perform a ritual series of tests and declare the data “clean”. It is to understand your data well enough to know whether the conclusions from your ANOVA are trustworthy.

2 The Assumptions



2.7.1 Residual diagnostics

The single most useful thing you can do after fitting an ANOVA is to look at the residuals. Four plots, each asking a different question, together cover the most important assumptions.

Plot 1: Residuals vs Fitted: plots residuals on the y -axis against fitted values (group means) on the x -axis. If homoscedasticity holds, the spread of residuals should be roughly the same across all fitted values. A fan shape, wider spread for larger fitted values, signals heteroscedasticity. A curved pattern signals non-additivity.

Plot 2: Q-Q plot of residuals: as discussed in section ??, this reveals departures from normality and, more usefully, flags outliers as isolated points flying off the diagonal line.

Plot 3: Scale-Location plot: plots the square root of the absolute residuals against fitted values. This is a more sensitive version of Plot 1 for detecting heteroscedasticity, because it stabilises the variance and makes trends easier to see.

Plot 4: Residuals vs group (boxplot): plots the residuals separately for each group. Equal box heights confirm homoscedasticity; outliers within groups are immediately visible; and any systematic skew within a group suggests a transformation may be needed.

All four plots can be produced in base R with a single call, or reconstructed in ggplot2 for more control:

```
library(ggplot2)
library(patchwork)

set.seed(42)

# Simulate clean fertiliser data for illustration
n <- 15
group <- factor(rep(c("Control", "Nitrogen", "Phosphorus"), each = n))
y <- c(
```

2 The Assumptions

```
rnorm(n, mean = 20, sd = 3),
rnorm(n, mean = 25, sd = 3),
rnorm(n, mean = 22, sd = 3)
)
df <- data.frame(y, group)
fit <- aov(y ~ group, data = df)

# Extract residuals and fitted values
diag_df <- data.frame(
  residuals = residuals(fit),
  fitted = fitted(fit),
  sqrt_abs_r = sqrt(abs(residuals(fit))),
  group = group
)

# Plot 1: Residuals vs Fitted
p1 <- ggplot(diag_df, aes(x = fitted, y = residuals)) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  geom_point(colour = "#2E86AB", alpha = 0.7) +
  geom_smooth(method = "loess", se = FALSE, colour = "#E84855", linewidth = 0.8) +
  labs(title = "Residuals vs Fitted", x = "Fitted values", y = "Residuals") +
  theme_bw()

# Plot 2: Q-Q plot
p2 <- ggplot(diag_df, aes(sample = residuals)) +
  stat_qq(colour = "#2E86AB", alpha = 0.7) +
  stat_qq_line(colour = "#E84855", linewidth = 0.8) +
  labs(title = "Normal Q-Q", x = "Theoretical quantiles", y = "Sample quantiles") +
  theme_bw()
```

2 The Assumptions

```
# Plot 3: Scale-Location
p3 <- ggplot(diag_df, aes(x = fitted, y = sqrt_abs_r)) +
  geom_point(colour = "#2E86AB", alpha = 0.7) +
  geom_smooth(method = "loess", se = FALSE, colour = "#E84855", linewidth = 0.8) +
  labs(title = "Scale-Location", x = "Fitted values", y = expression(sqrt("|Residuals|"))) +
  theme_bw()

# Plot 4: Residuals by group
p4 <- ggplot(diag_df, aes(x = group, y = residuals, fill = group)) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  geom_boxplot(alpha = 0.6, outlier.shape = 21) +
  scale_fill_manual(values = c("#2E86AB", "#F6AE2D", "#E84855")) +
  labs(title = "Residuals by Group", x = NULL, y = "Residuals") +
  theme_bw() +
  theme(legend.position = "none")

(p1 + p2) / (p3 + p4)
```

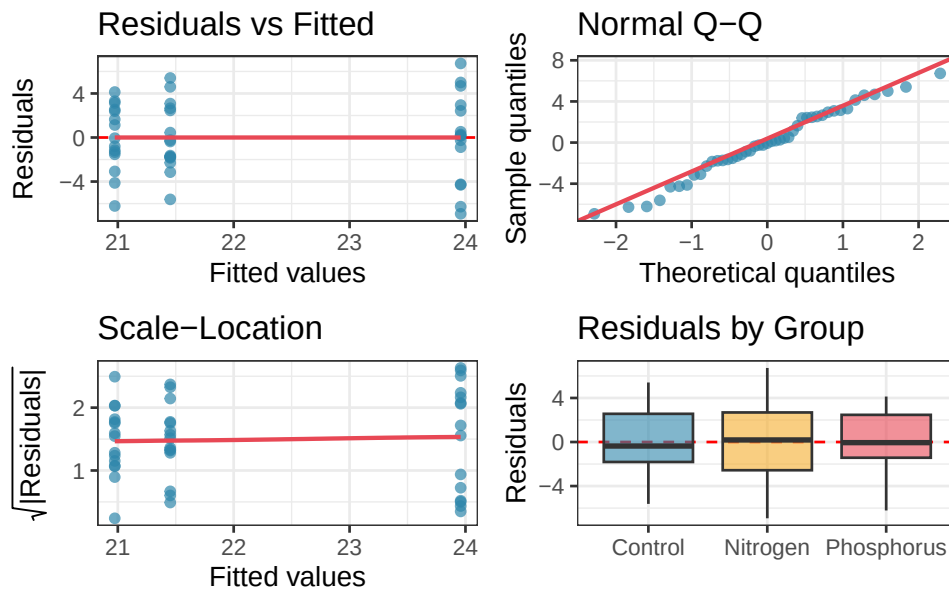


Figure 2.10: Four residual diagnostic plots for a one-way ANOVA fitted to the fertiliser data. Top left: residuals vs fitted values — roughly equal spread across groups confirms homoscedasticity. Top right: Q-Q plot, points close to the line confirm approximate normality. Bottom left: scale-location plot, a flat trend confirms homoscedasticity. Bottom right: residuals by group, box heights are similar and no extreme outliers are present.

These four plots together give you a comprehensive picture of whether the assumptions are reasonably met. Learn to read them before reaching for any formal test.

2.7.2 The Shapiro-Wilk trap

The Shapiro-Wilk test is the most commonly used formal test of normality. It tests the null hypothesis that the residuals were drawn from a normal distribution, returning a p -value that students often use as a binary gate: $p > 0.05$ means “normality confirmed, proceed”; $p < 0.05$ means “normality violated, stop”.

This is a trap, and it fails in both directions simultaneously.

2 The Assumptions

With small samples, the Shapiro-Wilk test has very low power. It will return $p > 0.05$, apparently confirming normality, even when the residuals are substantially non-normal, simply because there are too few observations to detect the departure. The test tells you nothing useful precisely when sample size is small and normality matters most.

With large samples, the Shapiro-Wilk test becomes hypersensitive. It will return $p < 0.05$, apparently rejecting normality, for trivially small deviations that have no practical consequence for the F test whatsoever. The larger your sample, the more likely you are to get a significant result even when your residuals are, for all practical purposes, perfectly normal.

The simulation below demonstrates both failure modes:

```
set.seed(42)
n_sim <- 2000

# Small sample trap: low power to detect real non-normality
p_small <- replicate(n_sim, {
  # Clearly non-normal: exponential residuals
  y <- rexp(3 * 5, rate = 1)
  group <- factor(rep(letters[1:3], each = 5))
  fit <- aov(y ~ group)
  shapiro.test(residuals(fit))$p.value
})

# Large sample trap: detects trivial non-normality
p_large <- replicate(n_sim, {
  # Nearly normal: just a tiny skew added
  y <- rnorm(3 * 200) + 0.05 * rexp(3 * 200)
  group <- factor(rep(letters[1:3], each = 200))
  fit <- aov(y ~ group)
  shapiro.test(residuals(fit))$p.value
})
```

2 The Assumptions

```
})

cat(
  "Small n: proportion of tests detecting non-normality:",
  round(mean(p_small < 0.05), 3),
  "\nLarge n: proportion of tests rejecting near-normal data:",
  round(mean(p_large < 0.05), 3), "\n"
)
```

```
#> Small n: proportion of tests detecting non-normality: 0.368
```

```
#> Large n: proportion of tests rejecting near-normal data: 0.059
```

```
# Plot
trap_df <- data.frame(
  p_value = c(p_small, p_large),
  scenario = rep(c("Small n (n=5 per group)\nClearly non-normal data",
                  "Large n (n=200 per group)\nNearly normal data"),
                each = n_sim)
)

ggplot(trap_df, aes(x = p_value, fill = scenario)) +
  geom_histogram(binwidth = 0.05, boundary = 0, colour = "white") +
  geom_vline(xintercept = 0.05, linetype = "dashed", colour = "red", linewidth = 0.8) +
  facet_wrap(~ scenario, ncol = 2) +
  scale_fill_manual(values = c("#2E86AB", "#E84855")) +
  labs(x = "Shapiro-Wilk p-value", y = "Count") +
  theme_bw() +
  theme(legend.position = "none")
```

2 The Assumptions

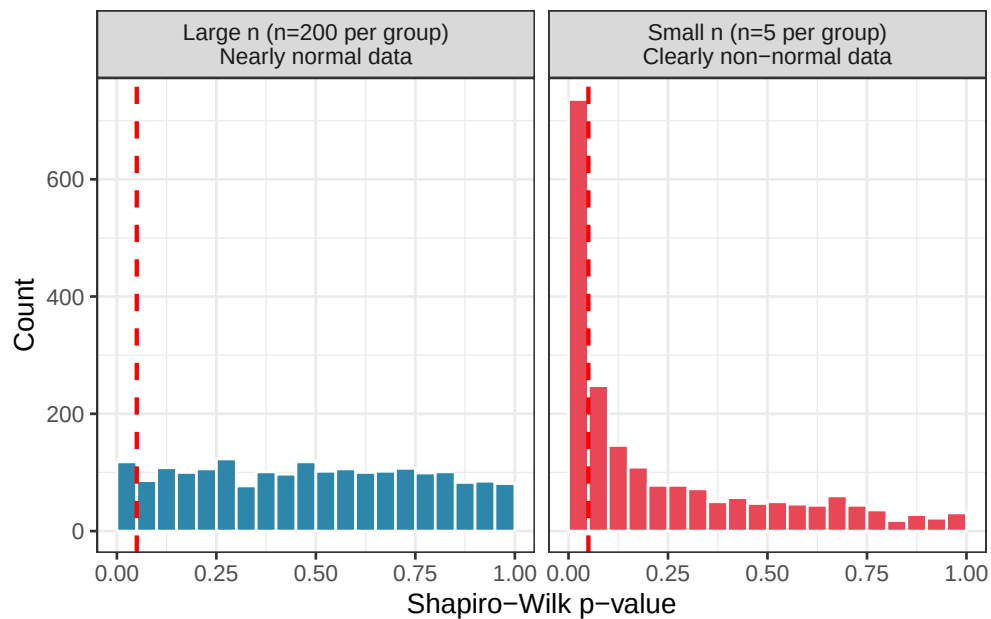


Figure 2.11: The Shapiro-Wilk trap illustrated. Left: with small samples ($n = 5$ per group), the test fails to detect clear non-normality, returning $p > 0.05$ in the majority of cases. Right: with large samples ($n = 200$ per group), the test rejects normality almost universally, even for data generated from an exactly normal distribution with only a trivial added skew.

The conclusion is not that the Shapiro-Wilk test is useless, it can be a helpful supplement to a Q-Q plot. The conclusion is that its p -value should never be used as a binary decision rule. Use it as one piece of evidence among several, always in conjunction with the Q-Q plot, and always with awareness of your sample size.

2.7.3 Levene's, Bartlett's, and why formal tests are not enough

Two formal tests are commonly used to check homoscedasticity.

Bartlett's test is sensitive to departures from normality as well as to unequal variances. If your residuals are non-normal, Bartlett's test may reject the null hypothesis of equal variances even when the variances are in fact equal, it is detecting the non-normality, not the heteroscedasticity. For this reason, Bartlett's test *is generally not recommended in practice* unless you are

2 The Assumptions

confident that the residuals are close to normal.

Levene's test is more robust. It tests for equal variances by running an ANOVA not on the raw residuals but on the absolute deviations from each group median, making it much less sensitive to non-normality. *It is the preferred formal test for homoscedasticity in most situations.*

```
library(car)

# Bartlett's test (sensitive to non-normality)
bartlett.test(y ~ group, data = df)

#>
#> Bartlett test of homogeneity of variances
#>
#> data: y by group
#> Bartlett's K-squared = 1.661, df = 2, p-value = 0.4358

# Levene's test (robust to non-normality)
leveneTest(y ~ group, data = df)

#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
#> group  2  0.4118 0.6651
#>      42
```

However, formal tests for homoscedasticity suffer from exactly the same sample size problem as the Shapiro-Wilk test. The simulation below demonstrates this for Levene's test.

```
set.seed(42)

# Small n: misses real heteroscedasticity
```


2 The Assumptions

```
p_lev_small <- replicate(n_sim, {
  y <- c(rnorm(5, sd = 1), rnorm(5, sd = 3), rnorm(5, sd = 9))
  group <- factor(rep(letters[1:3], each = 5))
  leveneTest(y ~ group)$`Pr(>F)`[1]
})

# Large n: detects trivial heteroscedasticity
p_lev_large <- replicate(n_sim, {
  y <- c(rnorm(200, sd = 1.0), rnorm(200, sd = 1.1), rnorm(200, sd = 1.2))
  group <- factor(rep(letters[1:3], each = 200))
  leveneTest(y ~ group)$`Pr(>F)`[1]
})

cat(
  "Small n: detecting large variance differences (sd = 1, 3, 9):",
  round(mean(p_lev_small < 0.05), 3),
  "\nLarge n: flagging trivial variance differences (sd = 1, 1.1, 1.2):",
  round(mean(p_lev_large < 0.05), 3), "\n"
)
```

```
#> Small n: detecting large variance differences (sd = 1, 3, 9): 0.3
```

```
#> Large n: flagging trivial variance differences (sd = 1, 1.1, 1.2): 0.552
```

The practical implication is the same as for the Shapiro-Wilk test: formal tests of assumptions should inform your judgement, not replace it. A non-significant Levene's test with $n = 5$ per group tells you almost nothing. A significant Levene's test with $n = 200$ per group may be flagging a difference in variance so small that Welch's ANOVA and standard ANOVA would give virtually identical results.

The best habit is to always look at the residual boxplot by group first. If one box is three or

four times taller than the others, the variances are unequal in a practically meaningful way, regardless of what Levene's test says. If the boxes are roughly similar in height, the variances are probably close enough, again, regardless of what Levene's test says.

2.7.4 Building a diagnostic workflow in R

Here I provide a complete, reusable diagnostic function that produces all the key plots and tests in a single call. You can write this once at the top of your analysis script and call it every time you fit an ANOVA.

```
anova_diagnostics <- function(fit, data) {  
  # required packages  
  if (!requireNamespace("ggplot2", quietly = TRUE)) {  
    stop("anova_diagnostics needs ggplot2 package.")  
  }  
  
  if (!requireNamespace("patchwork", quietly = TRUE)) {  
    stop("anova_diagnostics needs patchwork package.")  
  }  
  
  if (!requireNamespace("car", quietly = TRUE)) {  
    stop("anova_diagnostics needs car package.")  
  }  
  
  # extract residuals and fitted values  
  res <- residuals(fit)  
  fitt <- fitted(fit)  
  
  # robustly extract the first grouping variable from the formula  
  # works for both one-way (y ~ A) and two-way (y ~ A * B) formulas
```

2 The Assumptions

```
rhs_vars <- all.vars(formula(fit)[[3]])
grp <- factor(data[[rhs_vars[1]]])

diag_df <- data.frame(
  residuals = res,
  fitted = fitt,
  sqrt_abs_r = sqrt(abs(res)),
  group = grp
)

# plot 1: residuals vs fitted
p1 <- ggplot(diag_df, aes(x = fitted, y = residuals)) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  geom_point(colour = "#2E86AB", alpha = 0.7) +
  geom_smooth(method = "loess", se = FALSE, colour = "#E84855", linewidth = 0.8) +
  labs(title = "Residuals vs Fitted", x = "Fitted values", y = "Residuals") +
  theme_bw()

# plot 2: Q-Q plot
p2 <- ggplot(diag_df, aes(sample = residuals)) +
  stat_qq(colour = "#2E86AB", alpha = 0.7) +
  stat_qq_line(colour = "#E84855", linewidth = 0.8) +
  labs(title = "Normal Q-Q", x = "Theoretical quantiles", y = "Sample quantiles") +
  theme_bw()

# plot 3: Scale-Location
p3 <- ggplot(diag_df, aes(x = fitted, y = sqrt_abs_r)) +
  geom_point(colour = "#2E86AB", alpha = 0.7) +
  geom_smooth(method = "loess", se = FALSE, colour = "#E84855", linewidth = 0.8) +
```

2 The Assumptions

```
labs(title = "Scale-Location", x = "Fitted values", y = expression(sqrt("|Residuals|"))) +
theme_bw()

# plot 4: Residuals by first grouping variable
p4 <- ggplot(diag_df, aes(x = group, y = residuals, fill = group)) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  geom_boxplot(alpha = 0.6, outlier.shape = 21) +
  labs(title = paste("Residuals by", rhs_vars[1]), x = NULL, y = "Residuals") +
  theme_bw() +
  theme(legend.position = "none")

print((p1 + p2) / (p3 + p4))

# Formal tests
cat("\n--- Shapiro-Wilk test (normality of residuals) ---\n")
print(shapiro.test(res))

cat("\n--- Levene's test (homogeneity of variance) ---\n")
print(leveneTest(res ~ grp))

# Summary guidance
sw_p <- shapiro.test(res)$p.value
lev_p <- leveneTest(res ~ grp)$`Pr(>F)`[1]
n_grp <- min(table(grp))

cat("\n--- Diagnostic summary ---\n")
cat("Sample size (smallest group):", n_grp, "\n")

if (n_grp < 10) {
```

2 The Assumptions

```
cat(" Note: formal tests have low power with small samples.\n",
    " Prioritise plots over p-values.\n")
} else if (n_grp > 50) {
  cat(" Note: formal tests may flag trivial violations with large samples.\n",
    " Assess practical significance from plots.\n")
}

cat("Shapiro-Wilk p =", round(sw_p, 4),
    ifelse(sw_p < 0.05,
          "→ Evidence of non-normality. Check Q-Q plot for outliers.\n",
          "→ No evidence against normality.\n"))

cat("Levene's p =", round(lev_p, 4),
    ifelse(lev_p < 0.05,
          "→ Evidence of unequal variances. Consider Welch's ANOVA.\n",
          "→ No evidence against homoscedasticity.\n"))
}
```

Let's see an example usage:

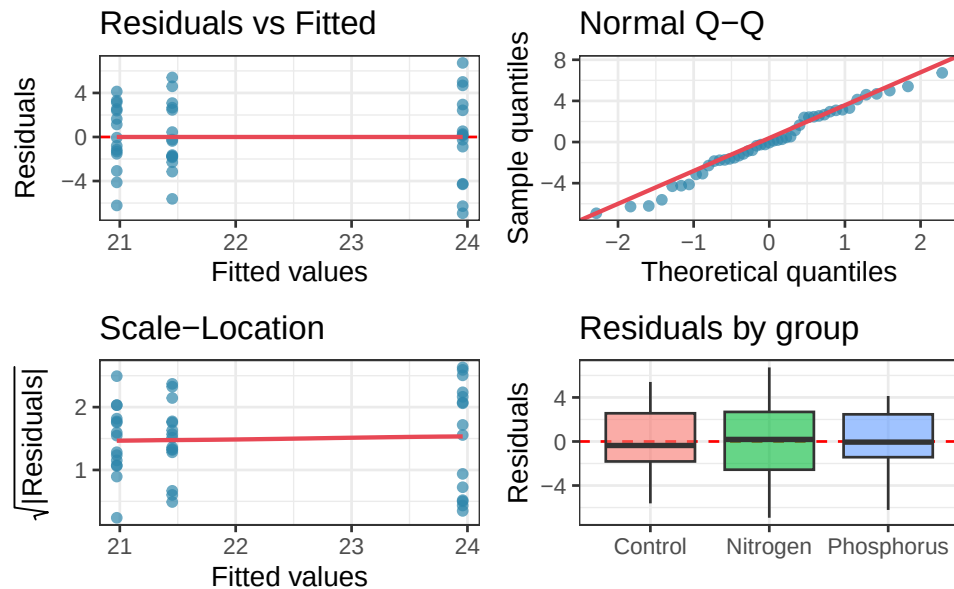
```
set.seed(42)
n <- 15
group <- factor(rep(c("Control", "Nitrogen", "Phosphorus"), each = n))
y <- c(
  rnorm(n, mean = 20, sd = 3), # Control
  rnorm(n, mean = 25, sd = 3), # Nitrogen
  rnorm(n, mean = 22, sd = 3) # Phosphorus
)

df <- data.frame(y, group)
```

2 The Assumptions

```
fit <- aov(y ~ group, data = df)

anova_diagnostics(fit, df)
```



```
#>
#> --- Shapiro-Wilk test (normality of residuals) ---
#>
#> Shapiro-Wilk normality test
#>
#> data:  res
#> W = 0.98099, p-value = 0.66
#>
#>
#> --- Levene's test (homogeneity of variance) ---
#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
```

2 The Assumptions

```
#> group 2 0.4118 0.6651
#>      42
#>
#> --- Diagnostic summary ---
#> Sample size (smallest group): 15
#> Shapiro-Wilk p = 0.66 -> No evidence against normality.
#> Levene's p = 0.6651 -> No evidence against homoscedasticity.
```

2.7.5 The Diagnostic checklist

Before interpreting any ANOVA result, work through the following checklist. The order matters. Start with the most consequential assumption.

Step 1: Independence. Was each observation collected independently? Are there clusters, repeated measures, or nested structures in the data? This cannot be answered from plots alone, it requires thinking about the study design. If independence is violated, stop and address it before proceeding.

Step 2: Homoscedasticity. Look at the residuals vs fitted plot and the residuals by group boxplot. Are the spreads roughly similar across groups? If not, consider a transformation or switch to Welch's ANOVA. Use Levene's test as supporting evidence, not as the primary decision.

Step 3: Normality. Look at the Q-Q plot. Are the points reasonably close to the diagonal? Is there any isolated extreme point that might be an outlier? Use the Shapiro-Wilk test as supporting evidence. If your sample is large and the Q-Q plot looks reasonable, normality is almost certainly not a problem.

Step 4: Additivity. Is there a systematic relationship between group means and group variances? If the groups with larger means also show more spread, consider a log transformation before fitting the model.

2 The Assumptions

Following this workflow will not guarantee a perfect analysis, no checklist can. But it will ensure that you have looked at your data carefully enough to know when your conclusions are on solid ground and when they should be treated with caution.

Part II

Classical ANOVA Models

You are unlikely to discover something new without a lot of practice on old stuff.

— Richard Feynman

3 One-Way ANOVA

The analysis of variance is not a mathematical theorem, but rather a convenient method of arranging the arithmetic.

— Ronald A. Fisher

One-way ANOVA is the simplest member of the ANOVA family: one response variable, one categorical predictor, any number of groups. Despite its simplicity, it contains all the essential ideas, variance decomposition, the F ratio, effect sizes, power, that carry forward into every more complex design. Mastering it thoroughly is therefore time well spent.

This chapter builds the one-way ANOVA from its mathematical foundations, works through a complete clinical example from raw data to interpretation, and then addresses two questions that are too often skipped: how large is the effect, and did the experiment have enough observations to detect it?

3.1 Model formulation and the F -ratio

3.1.1 The Statistical Model

The one-way ANOVA model states that every observation y_{ij} , the i th observation in group j , can be written as:

$$y_{ij} = \mu + \alpha_j + \varepsilon_{ij}$$

where:

- μ is the **grand mean**: the overall mean of the response across all groups and all observations.
- α_j is the **treatment effect** of group j : the amount by which the mean of group j departs from the grand mean. By convention, the treatment effects sum to zero: $\sum_{j=1}^k \alpha_j = 0$.
- ε_{ij} is the **residual** for observation i in group j : the leftover variation that neither the grand mean nor the treatment effect can explain. We assume $\varepsilon_{ij} \sim \mathcal{N}(0, \sigma^2)$, independently for all observations (Section ??).
- k is the number of groups and n_j is the number of observations in group j .

The null hypothesis is that all treatment effects are zero:

$$H_0 : \alpha_1 = \alpha_2 = \dots = \alpha_k = 0$$

which is equivalent to saying that all group means are equal:

$$H_0 : \mu_1 = \mu_2 = \dots = \mu_k$$

The alternative hypothesis is that at least one $\alpha_j \neq 0$, at least one group mean differs from the others.

3.1.2 Decomposing the Total Variation

The key operation in ANOVA is to split the total variation in the data into two additive components. For any observation y_{ij} , its departure from the grand mean $\bar{y}$ can be written as:

$$\underbrace{y_{ij} - \bar{y}}_{\text{total deviation}} = \underbrace{(\bar{y}_j - \bar{y})}_{\text{among-group deviation}} + \underbrace{(y_{ij} - \bar{y}_j)}_{\text{within-group deviation}}$$

3 One-Way ANOVA

Squaring and summing across all observations turns these deviations into **sums of squares** (SS):

$$SS_{\text{total}} = SS_{\text{among}} + SS_{\text{within}}$$

where:

$$SS_{\text{total}} = \sum_{j=1}^k \sum_{i=1}^{n_j} (y_{ij} - \bar{y})^2$$

$$SS_{\text{among}} = \sum_{j=1}^k n_j (\bar{y}_j - \bar{y})^2$$

$$SS_{\text{within}} = \sum_{j=1}^k \sum_{i=1}^{n_j} (y_{ij} - \bar{y}_j)^2$$

SS_{among} measures how far the group means stray from the grand mean: it captures the treatment effect plus any chance variation among group means. SS_{within} measures the scatter of observations around their own group mean: it captures residual variation only.

3.1.3 From Sums of Squares to Variances: Mean Squares

A sum of squares is not yet a variance, it grows automatically as the number of observations increases, which would make comparisons across studies meaningless. To obtain proper variance estimates, we divide each sum of squares by its **degrees of freedom**:

$$MS_{\text{among}} = \frac{SS_{\text{among}}}{k - 1}$$

$$MS_{\text{within}} = \frac{SS_{\text{within}}}{N - k}$$

3 One-Way ANOVA

where $N = \sum_{j=1}^k n_j$ is the total number of observations. The degrees of freedom deserve a moment's thought:

- SS_{among} is computed from k group means, but they are constrained to average to the grand mean, so only $k - 1$ of them are free to vary.
- SS_{within} is computed from N observations, with one mean estimated per group, leaving $N - k$ degrees of freedom.

MS_{within} is an unbiased estimate of σ^2 regardless of whether H_0 is true. MS_{among} estimates σ^2 only when H_0 is true; when the treatment has a real effect, it estimates $\sigma^2 + \frac{\sum n_j \alpha_j^2}{k-1}$, a quantity larger than σ^2 by an amount proportional to the true treatment effects.

3.1.4 The F Ratio

The F ratio is the natural consequence of everything above:

$$F = \frac{MS_{\text{among}}}{MS_{\text{within}}}$$

When H_0 is true, both the numerator and denominator estimate σ^2 , and F fluctuates around 1. When the treatment has a real effect, the numerator grows while the denominator stays anchored to σ^2 , and F climbs above 1. Under H_0 , this ratio follows an F -distribution with $k - 1$ and $N - k$ degrees of freedom, written $F_{k-1, N-k}$.

The results are conventionally summarised in an **ANOVA table**:

Source	SS	df	MS	F	p
Among groups	SS_{among}	$k - 1$	MS_{among}	F	p
Within groups	SS_{within}	$N - k$	MS_{within}		
Total	SS_{total}	$N - 1$			

3.2 Example: Comparing treatment groups in a clinical trial

3.2.1 The Study

A clinical trial is investigating the effect of three treatments on systolic blood pressure (bp in mmHg) in patients with mild hypertension (Section ??). Patients are randomly assigned to one of three groups (group): a control group receiving a placebo (Placebo), a group receiving a low-dose antihypertensive drug (Low dose), and a group receiving a high-dose antihypertensive drug (High dose). After eight weeks, systolic blood pressure is measured for each patient. There are 20 patients per group, giving $N = 60$ observations in total.

The starting assumption is that the three treatments produce the same mean blood pressure. The question is whether the data give us sufficient reason to doubt this.

3.2.2 Examining the Data

First, we always look at the data before fitting any model.

```
summary(systolic)
```

```
#>      bp      group
#> Min.   : 92.08  Placebo :20
#> 1st Qu.:129.61  Low dose :20
#> Median :136.62  High dose:20
#> Mean   :139.01
#> 3rd Qu.:148.76
#> Max.   :177.44
```

A combination of a boxplot and the individual observations gives a much richer picture than summary statistics alone:

3 One-Way ANOVA

```
library(ggplot2)

grand_mean <- mean(systolic$bp)

ggplot(systolic, aes(x = group, y = bp, fill = group)) +
  geom_boxplot(alpha = 0.4, outlier.shape = NA, width = 0.5) +
  geom_jitter(width = 0.1, size = 2, alpha = 0.6, aes(colour = group)) +
  geom_hline(yintercept = grand_mean, linetype = "dashed", colour = "grey40") +
  annotate("text", x = 0.6, y = grand_mean + 1, label = "Grand mean", size = 3.2, colour = "grey40") +
  scale_fill_manual(values = c("#2E86AB", "#F6AE2D", "#E84855")) +
  scale_colour_manual(values = c("#2E86AB", "#F6AE2D", "#E84855")) +
  labs(x = NULL, y = "Systolic blood pressure (mmHg)") +
  theme_bw() +
  theme(legend.position = "none")
```


3 One-Way ANOVA

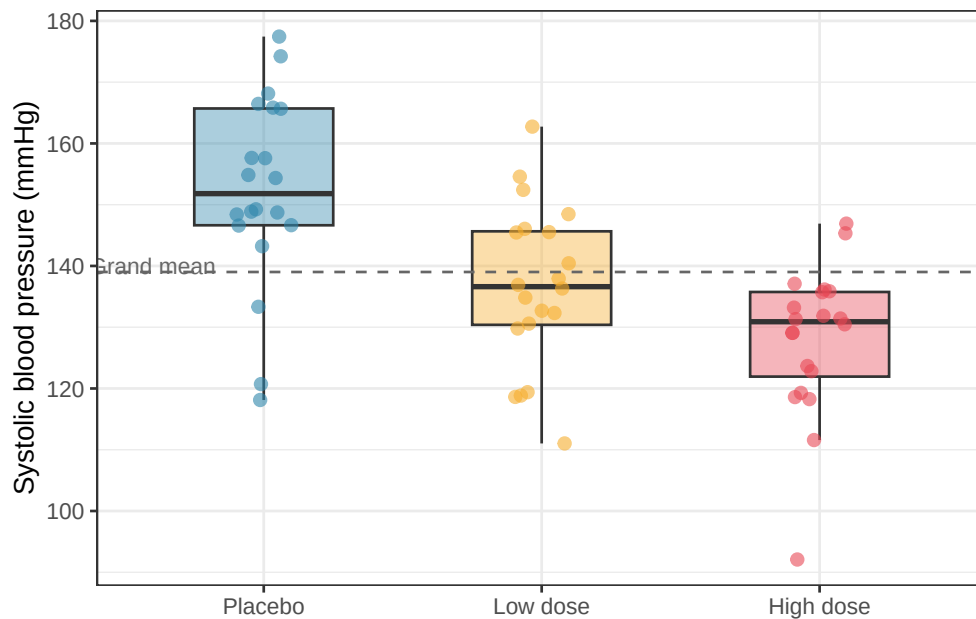


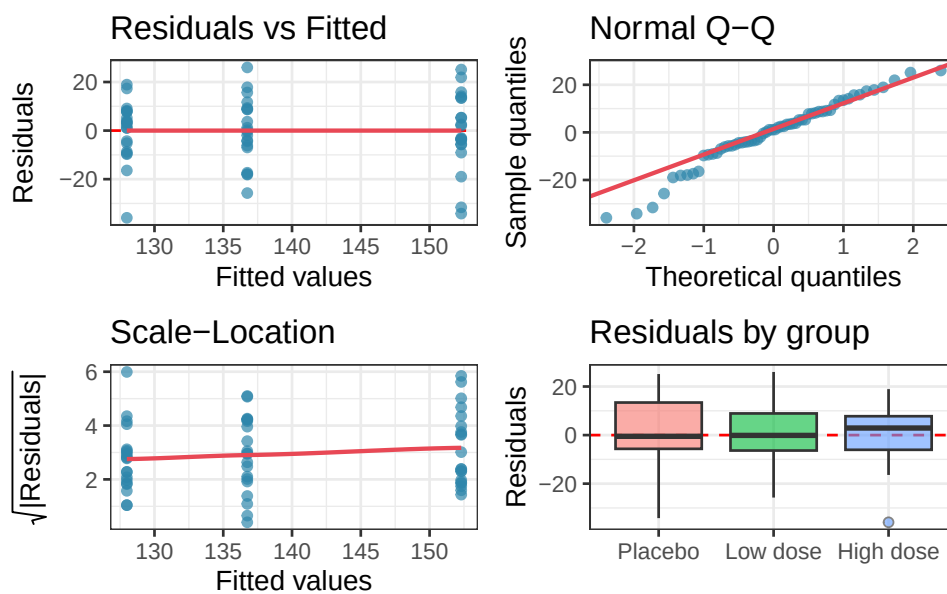
Figure 3.1: Systolic blood pressure (mmHg) by treatment group. Individual observations are shown as points, jittered horizontally to reduce overplotting. The horizontal dashed line marks the grand mean. Even before any formal test, the progressive decrease in both the median and spread of values from placebo to high dose is clearly visible.

3.2.3 Checking Assumptions

Before fitting the model, we run the diagnostic workflow introduced in Chapter ?? . Here we fit the model first in order to extract residuals, then inspect them:

```
fit <- aov(bp ~ group, data = systolic)
anova_diagnostics(fit, systolic)
```

3 One-Way ANOVA



```
#>
#> --- Shapiro-Wilk test (normality of residuals) ---
#>
#> Shapiro-Wilk normality test
#>
#> data:  res
#> W = 0.96761, p-value = 0.1114
#>
#>
#> --- Levene's test (homogeneity of variance) ---
#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
#> group 2  0.7068 0.4975
#>
#>      57
#>
#> --- Diagnostic summary ---
```

3 One-Way ANOVA

```
#> Sample size (smallest group): 20
#> Shapiro-Wilk p = 0.1114 -> No evidence against normality.
#> Levene's p = 0.4975 -> No evidence against homoscedasticity.
```

The Q-Q plot shows residuals lying close to the diagonal, the residual boxplots show similar spreads across groups, and neither the Shapiro-Wilk nor Levene's test raises any concern. We proceed with standard one-way ANOVA.

3.2.4 Fitting the Model and Reading the Output

The output looks like this:

```
summary(fit)
```

```
#>           Df Sum Sq Mean Sq F value   Pr(>F)
#> group         2    6066   3032.8    15.79 3.5e-06 ***
#> Residuals    57   10949    192.1
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Let us read this table carefully, column by column.

- **Df**: the degrees of freedom. The treatment factor has $k-1 = 2$ degrees of freedom (three groups minus one). The residuals have $N - k = 57$ degrees of freedom (60 observations minus 3 groups).
- **Sum Sq**: the sums of squares. The treatment accounts for 6066 mmHg² of variation; the residuals account for 10949 mmHg².
- **Mean Sq**: the mean squares, obtained by dividing each sum of squares by its degrees of freedom. $MS_{\text{among}} = 6066/2 = 3032.8$; $MS_{\text{within}} = 10949/57 = 192.1$.
- **F value**: the ratio $3032.8/192.1 = 15.79$. The among-group variance is more than 21 times larger than the within-group variance.

- **Pr(>F)**: the probability of observing an F ratio this large or larger if the null hypothesis were true. Here $p = 3.05 \times 10^{-6}$, an extremely small probability. We reject the null hypothesis and conclude that treatment group has a significant effect on systolic blood pressure.

3.2.5 Reporting the Result

A complete report of a one-way ANOVA result should include the F statistic, both degrees of freedom, and the p -value:

One-way ANOVA revealed a significant effect of treatment on systolic blood pressure ($F_{2,57} = 15.79, p < 0.001$).

Note, however, that this tells us only that the groups differ, not which specific groups differ from which. Post-hoc tests for pairwise comparisons are covered in Chapter 5.

3.3 Effect sizes: η^2, ω^2 , Cohen's f

3.3.1 Why the p -Value Is Not Enough

A significant p -value tells you that the observed differences among groups are unlikely to be due to chance. It does not tell you how large those differences are. This distinction matters enormously in practice. With a large enough sample, even a trivially small treatment effect will produce a highly significant p -value. With a small sample, a clinically meaningful effect may fail to reach significance. The p -value conflates the size of the effect with the size of the study.

Effect sizes separate these two things. They quantify how much of the variation in the response is explained by the treatment, on a scale that does not depend on sample size.

3.3.2 A Cautionary Note: When the p -Value Misleads

Before moving to effect sizes formally, it is worth pausing to see concretely why the p -value alone is insufficient for scientific interpretation. The following simulation demonstrates the problem directly.

We generate data from three groups whose means differ by a clinically trivial amount, just 1 mmHg in blood pressure between groups, against a within-group standard deviation of 12 mmHg. Nobody would consider a 1 mmHg difference between treatments to be meaningful.

We then run the same one-way ANOVA at four different sample sizes and observe what happens to the p -value and the effect size as n grows.

```
library(effects)

set.seed(42)

# Trivial difference: 1 mmHg between groups, SD = 12
means <- c(150, 151, 152)
sd_common <- 12
ns <- c(10, 50, 200, 2000)

results <- lapply(ns, function(n) {
  group <- factor(rep(c("Placebo", "Low dose", "High dose"), each = n), levels = c("Placebo", "Low dose", "High dose"))
  bp <- c(
    rnorm(n, mean = means[1], sd = sd_common),
    rnorm(n, mean = means[2], sd = sd_common),
    rnorm(n, mean = means[3], sd = sd_common)
  )
  df <- data.frame(bp, group)
  fit <- aov(bp ~ group, data = df)
```

3 One-Way ANOVA

```
p_val <- summary(fit)[[1]][["Pr(>F)"]][1]
omega2 <- omega_squared(fit, partial = FALSE)$Omega2

data.frame(
  n = n,
  p_val = p_val,
  omega2 = omega2
)
})

results_df <- do.call(rbind, results)
results_df$n_label <- paste0("n = ", results_df$n, " per group")
results_df$n_label <- factor(results_df$n_label, levels = paste0("n = ", ns, " per group"))
results_df$significant <- ifelse(results_df$p_val < 0.05, "p < 0.05", "p ≥ 0.05")
```

```
results_df
```

```
#>      n      p_val      omega2      n_label significant
#> 1   10 4.787806e-01 0.0000000000 n = 10 per group    p ≥ 0.05
#> 2   50 5.304923e-01 0.0000000000 n = 50 per group    p ≥ 0.05
#> 3  200 5.005470e-01 0.0000000000 n = 200 per group    p ≥ 0.05
#> 4 2000 1.466471e-06 0.004137075 n = 2000 per group    p < 0.05
```

```
library(ggplot2)

# Plot p-values
p1 <- ggplot(results_df, aes(x = n_label, y = p_val, fill = significant)) +
  geom_col(width = 0.5) +
  geom_hline(yintercept = 0.05, linetype = "dashed", colour = "red", linewidth = 0.8) +
```

3 One-Way ANOVA

```
annotate("text", x = 0.6, y = 0.07, label = "p = 0.05", colour = "red", size = 3.2, hjust = 0)
scale_fill_manual(values = c("p < 0.05" = "#E84855", "p ≥ 0.05" = "#2E86AB")) +
scale_y_continuous(limits = c(0, 1)) +
labs(title = "p-value", x = NULL, y = "p-value", fill = NULL) +
theme_bw() +
theme(legend.position = "bottom", axis.text.x = element_text(angle = 20, hjust = 1))

# Plot omega-squared
p2 <- ggplot(results_df, aes(x = n_label, y = omega2)) +
  geom_col(width = 0.5, fill = "#F6AE2D") +
  geom_hline(yintercept = 0.01,
             linetype = "dashed", colour = "grey40", linewidth = 0.8) +
  annotate("text", x = 0.6, y = 0.013,
          label = "Small effect threshold (0.01)",
          colour = "grey40", size = 3.2, hjust = 0) +
  scale_y_continuous(limits = c(0, 0.05)) +
  labs(title = expression(omega^2~"(effect size)"),
       x = NULL, y = expression(omega^2)) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 20, hjust = 1))

library(patchwork)
p1 + p2
```

3 One-Way ANOVA

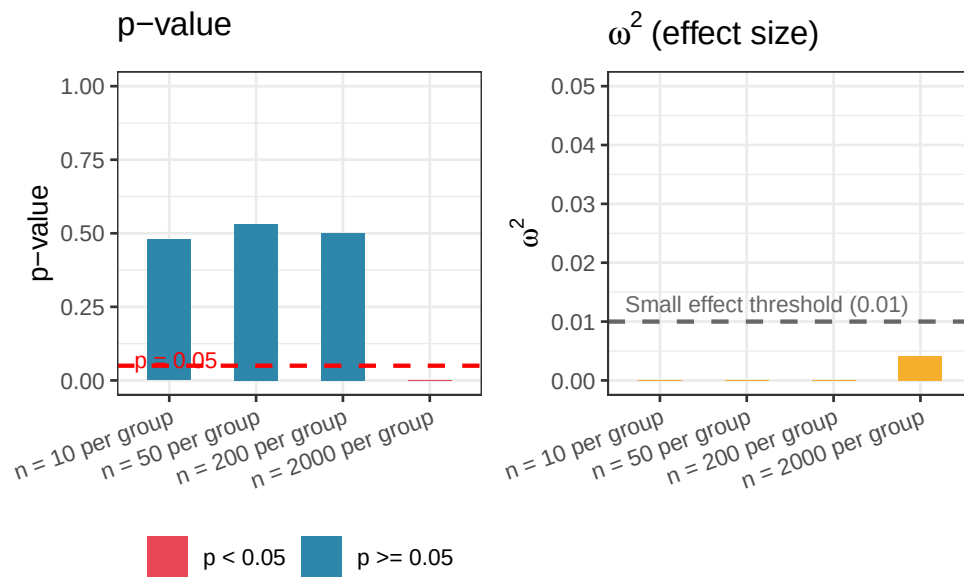


Figure 3.2: The same trivial effect (group means differing by 1 mmHg against a within-group SD of 12 mmHg) analysed at four different sample sizes. As n grows, the p -value shrinks toward zero and eventually crosses the significance threshold not because the effect has become more meaningful, but simply because the study has become more sensitive. The effect size omega-squared remains negligibly small throughout.

The results make the problem vivid. With $n = 10$ per group the p -value is large and the result is non-significant but not because the effect is small but simply because the study is underpowered. It is only with $n = 2000$ per group that the result is declared significant and at that point the p -value is vanishingly small, suggesting to anyone reading only that number that something important has been found.

Yet ω^2 tells a completely different story. Across all four sample sizes, the effect size hovers near zero, well below even the conventional threshold for a small effect. The treatment explains less than 1% of the variation in blood pressure at every sample size. The 1 mmHg difference between groups is real in a narrow statistical sense, it is consistently detected once the sample is large enough, but it is scientifically and clinically meaningless. What changed between $n = 200$ and $n = 2000$ was not the biology. It was simply the resolving power of the microscope.

The lesson is not that p -values are useless. They answer a specific and legitimate question:

given the data, how surprising would this result be if there were no effect? But that question is not the same as *how large is the effect?* A p -value cannot tell you whether a significant result matters. Only an effect size can do that.

This is why the rest of this chapter treats effect sizes not as an optional supplement to the ANOVA table but as an essential part of every analysis.

3.3.3 η^2 : Eta-Squared

The simplest effect size for ANOVA is η^2 (eta-squared), defined as the proportion of total variation explained by the treatment:

$$\eta^2 = \frac{SS_{\text{among}}}{SS_{\text{total}}}$$

It ranges from 0 (treatment explains none of the variation) to 1 (treatment explains all of it). For our clinical example:

```
ss_among <- summary(fit)[[1]]["group", "Sum Sq"]
ss_total <- sum(summary(fit)[[1]]["Sum Sq"])

eta2 <- ss_among / ss_total
cat("eta2 =", eta2, "\n")
```

```
#> eta2 = 0.3564937
```

$\eta^2 = 0.356$ means that 35.6% of the total variation in blood pressure is explained by treatment group, a large effect by any standard.

The weakness of η^2 is that it is a biased estimate of the true population effect size: it tends to overestimate how much of the variation in the *population* is explained by the treatment, because it is computed from the same data used to fit the model.

3.3.4 ω^2 : Omega-Squared

ω^2 (omega-squared) corrects for this bias and is generally preferred when reporting results:

$$\omega^2 = \frac{SS_{\text{among}} - (k - 1) \cdot MS_{\text{within}}}{SS_{\text{total}} + MS_{\text{within}}}$$

```
k <- nlevels(systolic$group)
ms_w <- summary(fit)[[1]]["Residuals", "Mean Sq"]

omega2 <- (ss_among - (k - 1) * ms_w) / (ss_total + ms_w)
cat("omega^2 =", omega2, "\n")
```

```
#> omega^2 = 0.3301868
```

ω^2 is expected to be slightly smaller than η^2 , and the difference is largest with small samples. With $N = 60$ the correction here is modest, but with $N = 15$ or $N = 20$ it can be substantial.

Always report ω^2 rather than η^2 when the sample is small.

Conventional benchmarks for both η^2 and ω^2 , originally proposed by Cohen (?), are:

Effect size	Small	Medium	Large
η^2, ω^2	0.01	0.06	0.14

These are rough guides, not rigid rules. What constitutes a meaningful effect depends entirely on the scientific context. A drug that explains 5% of variation in blood pressure across a population might represent an enormous public health benefit; the same effect size in a laboratory assay might be scientifically uninteresting.

3.3.5 Cohen's f

Cohen's f is an alternative effect size that expresses the treatment effect as the ratio of the standard deviation of group means to the common within-group standard deviation:

$$f = \sqrt{\frac{\eta^2}{1 - \eta^2}}$$

```
f <- sqrt(eta2 / (1 - eta2))
cat("Cohen's f =", round(f, 3), "\n")
```

```
#> Cohen's f = 0.744
```

Cohen's f is less intuitive than η^2 or ω^2 but is the effect size used by most power analysis software, including the `pwr` package in R. Conventional benchmarks are $f = 0.10$ (small), $f = 0.25$ (medium), and $f = 0.40$ (large) (?).

3.3.6 Computing Effect Sizes with `effectsize`

Rather than computing these by hand, the `effectsize` package provides clean, well-documented functions with confidence intervals:

```
# install.packages("effectsize") if needed
library(effectsize)

eta_squared(fit, partial = FALSE)
```

```
#> # Effect Size for ANOVA (Type I)
#>
#> Parameter | Eta2 |      95% CI
#> -----
```

3 One-Way ANOVA

```
#> group      | 0.36 | [0.19, 1.00]
#>
#> - One-sided CIs: upper bound fixed at [1.00].
```

```
omega_squared(fit, partial = FALSE)
```

```
#> # Effect Size for ANOVA (Type I)
#>
#> Parameter | Omega2 |      95% CI
#> -----
#> group      | 0.33 | [0.16, 1.00]
#>
#> - One-sided CIs: upper bound fixed at [1.00].
```

```
cohens_f(fit)
```

```
#> For one-way between subjects designs, partial eta squared is equivalent
#> to eta squared. Returning eta squared.
```

```
#> # Effect Size for ANOVA
#>
#> Parameter | Cohen's f |      95% CI
#> -----
#> group      | 0.74 | [0.48, Inf]
#>
#> - One-sided CIs: upper bound fixed at [Inf].
```

Always report effect sizes alongside p -values. A result reported as “ $F_{2,57} = 21.34, p < 0.001, \omega^2 = 0.36$ ” tells a much more complete scientific story than the p -value alone.

3.4 Power analysis and sample size planning

3.4.1 What Is Statistical Power?

The F test can make two kinds of error.

A **Type I error** occurs when you reject H_0 when it is actually true: a false positive. The probability of a Type I error is α , the significance threshold you set in advance (conventionally 0.05).

A **Type II error** occurs when you fail to reject H_0 when it is actually false: a false negative. The probability of a Type II error is denoted β .

Statistical power is $1 - \beta$: the probability that your test will detect a real effect if one exists. A study with power of 0.80 has an 80% chance of finding a significant result when the treatment truly has an effect of the specified size.

Power depends on four quantities that are mathematically linked:

$$\text{Power} = f(\alpha, n, k, f)$$

- α : the significance threshold. Lower α means lower power.
- n : the number of observations per group. More observations means more power.
- k : the number of groups. More groups means more degrees of freedom in the numerator and generally lower power per comparison.
- f : Cohen's f , the effect size. Larger effects are easier to detect.

If you fix any three of these quantities, the fourth is determined. This is the basis of power analysis.

3.4.2 Prospective Power Analysis: How Many Observations Do I Need?

The most important use of power analysis is **prospective**: before collecting data, to determine how many observations per group are needed to detect an effect of a given size with adequate power. This requires you to specify:

1. The significance threshold α (almost always 0.05).
2. The desired power $1 - \beta$ (conventionally 0.80, though 0.90 is preferable in some situation like clinical work).
3. The expected effect size f : the hardest quantity to specify, and the one that requires the most careful thought.

In R, the `pwr` package handles this calculation:

```
# install.packages("pwr") if needed
library(pwr)

# How many observations per group do we need to detect a medium effect
# (f = 0.25) with 80% power at alpha = 0.05, with k = 3 groups?
pwr.anova.test(k = 3, f = 0.25, sig.level = 0.05, power = 0.80)
```

```
#>
#>      Balanced one-way analysis of variance power calculation
#>
#>              k = 3
#>              n = 52.3966
#>              f = 0.25
#>      sig.level = 0.05
#>              power = 0.8
#>
#> NOTE: n is number in each group
```

The output show that approximately 52 observations per group, 156 in total, are needed to detect a medium effect with 80% power in a three-group design. For a large effect ($f = 0.40$), this drops to around 21 per group; for a small effect ($f = 0.10$), it rises to over 320 per group.

3.4.3 Visualising the Power Curve

It is more informative to visualise how power changes across a range of sample sizes for different effect sizes, rather than reading off a single number:

```
library(ggplot2)
library(pwr)

ns <- seq(5, 150, by = 1)
f_vals <- c(0.10, 0.25, 0.40)
labels <- c("Small (f = 0.10)", "Medium (f = 0.25)", "Large (f = 0.40)")

power_df <- do.call(rbind, lapply(seq_along(f_vals), function(j) {
  data.frame(
    n = ns,
    power = sapply(ns, function(n) {
      pwr.anova.test(k = 3, n = n, f = f_vals[j], sig.level = 0.05)$power
    }),
    effect = labels[j]
  )
}))

power_df$effect <- factor(power_df$effect, levels = labels)

ggplot(power_df, aes(x = n, y = power, colour = effect)) +
  geom_line(linewidth = 1) +
```

3 One-Way ANOVA

```
geom_hline(yintercept = 0.80, linetype = "dashed", colour = "grey40") +  
annotate("text", x = 148, y = 0.77, label = "80% power", size = 3.2, colour = "grey40", hjust =  
scale_colour_manual(values = c("#2E86AB", "#F6AE2D", "#E84855")) +  
scale_y_continuous(limits = c(0, 1), labels = scales::percent_format(accuracy = 1)) +  
labs(  
  x = "Sample size per group (n)",  
  y = "Power",  
  colour = "Effect size"  
) +  
theme_bw() +  
theme(legend.position = "inside", legend.position.inside = c(0.82, 0.25))
```

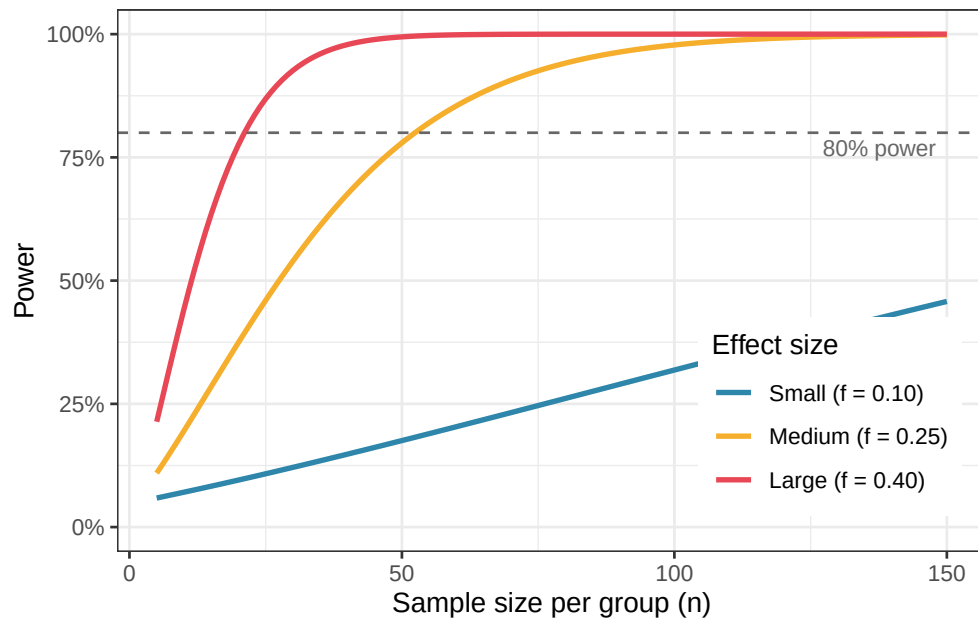


Figure 3.3: Power curves for a one-way ANOVA with three groups at $\alpha = 0.05$, across a range of sample sizes per group and for three effect sizes. The horizontal dashed line marks the conventional 80% power threshold. A medium effect requires around 52 observations per group to achieve 80% power; a small effect requires over 300.

3.4.4 Where Does the Expected Effect Size Come From?

Specifying the expected effect size is the hardest part of power analysis, and it is where many researchers go wrong. There are three defensible approaches.

From prior literature. If previous studies have measured the same response with the same or similar treatments, use their reported effect sizes. Be aware, however, that published effect sizes are subject to publication bias: studies with larger effects are more likely to be published, so the literature tends to overestimate true effect sizes. Apply some conservatism.

From a pilot study. A small preliminary experiment can provide a rough estimate of within-group variance σ and the expected differences among group means, from which f can be calculated. Pilot-based effect sizes are noisy but grounded in your specific system.

From the minimally meaningful effect. Ask: what is the smallest difference among group means that would be scientifically or clinically meaningful? For example, in the blood pressure trial, a difference of 5 mmHg between groups might be the smallest reduction considered clinically relevant. Combined with an estimate of within-group standard deviation (from the literature or from clinical experience), this gives a concrete, defensible effect size to power the study for.

Using Cohen's conventional benchmarks (small, medium, large) as a substitute for genuine subject-matter knowledge is the least satisfactory approach, but it is better than no power analysis at all.

3.4.5 Retrospective Power Analysis: A Warning

It is tempting, after a non-significant result, to compute the power of the completed study post hoc, to ask whether the experiment was large enough to detect the effect that was observed. This is called **retrospective** or **post hoc power analysis**, and it is generally not informative. When the observed effect size is used to compute power, a non-significant result will

3 *One-Way ANOVA*

always produce low estimated power, and a significant result will always produce high estimated power. The calculation is entirely circular and adds nothing to what the p -value already tells you.

If a study yields a non-significant result and you want to know whether a meaningful effect might have been missed, the right tool is a confidence interval around the effect size, not a retrospective power calculation.

4 Multiple Comparisons

The F test in ANOVA answers a single, global question: do the group means differ? When the answer is yes, the natural follow-up is: *which* groups differ from *which*? This is where multiple comparisons enter the picture and where a surprising number of otherwise careful analyses go wrong.

The problem is not the comparisons themselves, it is doing many of them at once without accounting for the fact that **when you ask many questions of the same data, chance alone will eventually hand you a significant answer.**

This chapter explains why, introduces the main corrections available in R, and gives practical guidance on when each is appropriate.

4.1 The Multiple Testing Problem

4.1.1 One Test, One Risk

When you perform a single hypothesis test at $\alpha = 0.05$, *you accept* a 5% probability of a false positive which is 5% probability of declaring a difference significant when none truly exists. This is the Type I error rate, and it is exactly what α controls.

4.1.2 Many Tests, Compounding Risk

Now suppose the ANOVA is significant and you want to compare all pairs of groups. With $k = 3$ groups, there are three pairwise comparisons: Placebo vs Low dose, Placebo vs High dose, and Low dose vs High dose.

If you run each as a separate t -test at $\alpha = 0.05$, the probability of making *at least one* false positive across the three tests is no longer 5%. Assuming the tests are independent, the probability of making no false positives across m tests is $(1 - \alpha)^m$, so the probability of at least one false positive is:

$$P(\text{at least one false positive}) = 1 - (1 - \alpha)^m$$

For three tests: $1 - (1 - 0.05)^3 = 0.143$. For five tests: 0.226. For ten tests: 0.401. *The more comparisons you make, the more likely you are to find something significant by chance alone.*

This inflated error rate across a family of tests is called the **familywise error rate (FWER)**, and controlling it is the central goal of multiple comparison procedures.

The simulation below makes this concrete. We generate data where all groups truly have the same mean, no differences exist, and count how often at least one pairwise t -test returns $p < 0.05$:

```
library(furrr)
library(future)
library(ggplot2)

set.seed(42)
n_sim <- 5000
alpha <- 0.05
n <- 20 # observations per group
```

4 Multiple Comparisons

```
# Range of group numbers to test
k_vals <- 2:10

fwer <- future_map_dbl(k_vals, function(k) {

  # inner loop is sequential, parallelism at the k level only
  at_least_one_fp <- map_lgl(seq_len(n_sim), function(i) {
    y      <- rnorm(k * n)
    group <- factor(rep(paste0("G", 1:k), each = n))
    df     <- data.frame(y, group)
    pairs  <- combn(levels(group), 2)
    p_vals <- apply(pairs, 2, function(g) {
      t.test(y ~ group,
              data      = df[df$group %in% g, ],
              var.equal = TRUE)$p.value
    })
    any(p_vals < alpha)
  })
  mean(at_least_one_fp)

}, .options = furrr_options(seed = TRUE))

plan(sequential)

# Theoretical FWER
m_vals <- choose(k_vals, 2)
fwer_theory <- 1 - (1 - alpha)^m_vals

fwer_df <- data.frame(
```

4 Multiple Comparisons

```
k = k_vals,
m = m_vals,
fwer_sim = fwer,
fwer_theory = fwer_theory
)

ggplot(fwer_df, aes(x = k)) +
  geom_line(aes(y = fwer_theory, linetype = "Theoretical"), colour = "#2E86AB", linewidth = 1) +
  geom_point(aes(y = fwer_sim, shape = "Simulated"), colour = "#E84855", size = 3) +
  geom_hline(yintercept = alpha, linetype = "dashed", colour = "red", linewidth = 0.8) +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent_format(accuracy = 1)) +
  scale_x_continuous(breaks = k_vals, sec.axis = sec_axis(~ choose(., 2), name = "Number of pairs")) +
  scale_linetype_manual(values = c("Theoretical" = "solid")) +
  scale_shape_manual(values = c("Simulated" = 16)) +
  labs(
    x = "Number of groups (k)",
    y = "Familywise error rate",
    linetype = NULL,
    shape = NULL
  ) +
  theme_bw() +
  theme(legend.position = "inside", legend.position.inside = c(0.2, 0.85))
```

4 Multiple Comparisons

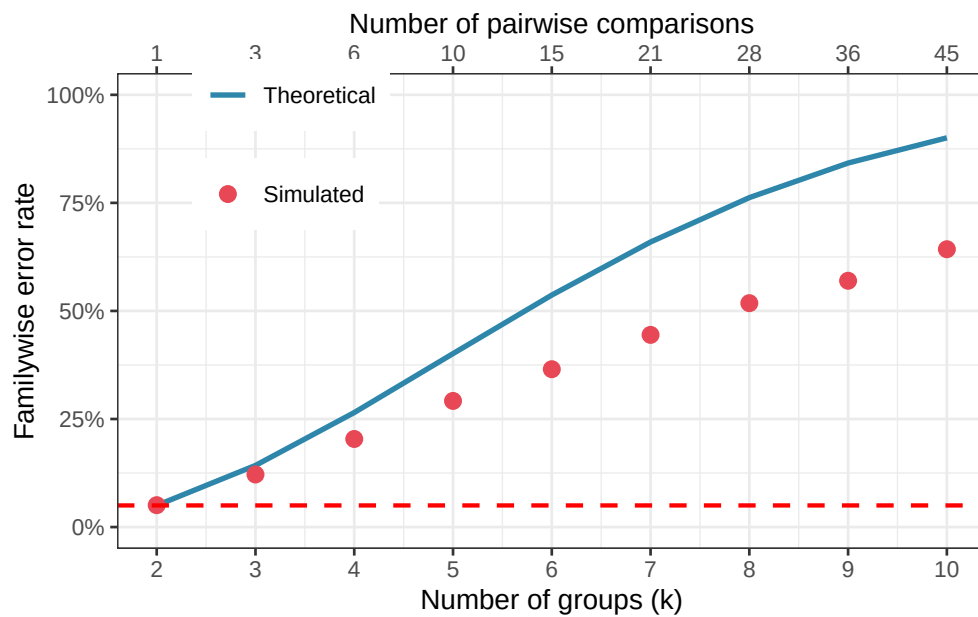


Figure 4.1: Familywise error rate as a function of the number of pairwise comparisons, when each test is run at $\alpha = 0.05$ without correction. The dashed red line marks the nominal 5% level. With ten groups (45 pairwise comparisons), the probability of at least one false positive exceeds 90%.

With just four groups and six pairwise comparisons, the familywise error rate already exceeds 25%. With ten groups and 45 pairwise comparisons, it approaches 90%.

An analyst running uncorrected pairwise t -tests after a significant ANOVA is operating in a regime where false positives are virtually guaranteed, not occasionally, but as a matter of arithmetic.

4.1.3 Two Ways to Define the Error Rate

Not everyone agrees that the familywise error rate is always the right quantity to control. Two alternative perspectives are worth knowing.

The **familywise error rate (FWER)** asks: what is the probability of making *any* false positive in this family of tests? Controlling FWER at 0.05 means that across all your comparisons,

the probability of even a single false positive is at most 5%. This is a conservative criterion, appropriate when any false positive would be costly, for example, in clinical trials where a falsely declared effective treatment might be adopted into practice.

The **false discovery rate (FDR)** asks a softer question: among all comparisons declared significant, what proportion are expected to be false positives? Controlling FDR at 5% means that, on average, no more than 5% of your significant results are false positives. This is less conservative than FWER control and is more appropriate in exploratory research where missing real effects (false negatives) is considered as costly as generating false positives.

In the context of post-hoc comparisons following a one-way ANOVA, *FWER control is the norm*. FDR control becomes more relevant in high-dimensional settings such as genomics, where thousands of tests are performed simultaneously.

4.2 Post-Hoc Tests: Tukey, Bonferroni, Holm, and Dunnett

4.2.1 The General Principle

All post-hoc correction methods work by raising the bar for significance, either by reducing the α threshold applied to each individual test, or by adjusting the p -values themselves upward. They differ in how conservatively they do this, and consequently in the balance they strike between controlling false positives and maintaining power to detect true differences.

We will illustrate all methods using the blood pressure clinical trial from Chapter ??.

4.2.2 Tukey's Honest Significant Difference (HSD)

Tukey's HSD is the most widely used post-hoc test for pairwise comparisons following ANOVA. It controls the familywise error rate exactly at α for all pairwise comparisons simultaneously, using the studentised range distribution rather than the t -distribution. It is called "honest" because, unlike uncorrected pairwise t -tests, it does not overstate the evidence.

4 Multiple Comparisons

Tukey's HSD is the right default when:

- You want all pairwise comparisons.
- Your groups have approximately equal sample sizes.
- You want tight control of the familywise error rate.

```
tukey_res <- TukeyHSD(fit, conf.level = 0.95)
print(tukey_res)
```

```
#> Tukey multiple comparisons of means
#> 95% family-wise confidence level
#>
#> Fit: aov(formula = bp ~ group, data = systolic)
#>
#> $group
#>
#>              diff      lwr      upr    p adj
#> Low dose-Placebo -15.554942 -26.10178 -5.008100 0.0022255
#> High dose-Placebo -24.313969 -34.86081 -13.767127 0.0000023
#> High dose-Low dose  -8.759027 -19.30587  1.787815 0.1218054
```

The output gives, for each pair of groups, the estimated difference in means, a 95% confidence interval for that difference, and an adjusted p -value. A confidence interval that does not include zero corresponds to a significant difference.

It is almost always more informative to plot these intervals than to read the table:

```
library(ggplot2)

tukey_df <- as.data.frame(tukey_res$group)
tukey_df$comparison <- rownames(tukey_df)
names(tukey_df)[1:4] <- c("diff", "lwr", "upr", "p_adj")
```

4 Multiple Comparisons

```
ggplot(tukey_df, aes(x = comparison, y = diff, ymin = lwr, ymax = upr)) +  
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red", linewidth = 0.8) +  
  geom_errorbar(width = 0.15, colour = "#2E86AB", linewidth = 1) +  
  geom_point(size = 4, colour = "#2E86AB") +  
  coord_flip() +  
  labs(x = NULL, y = "Difference in mean blood pressure (mmHg)") +  
  theme_bw()
```

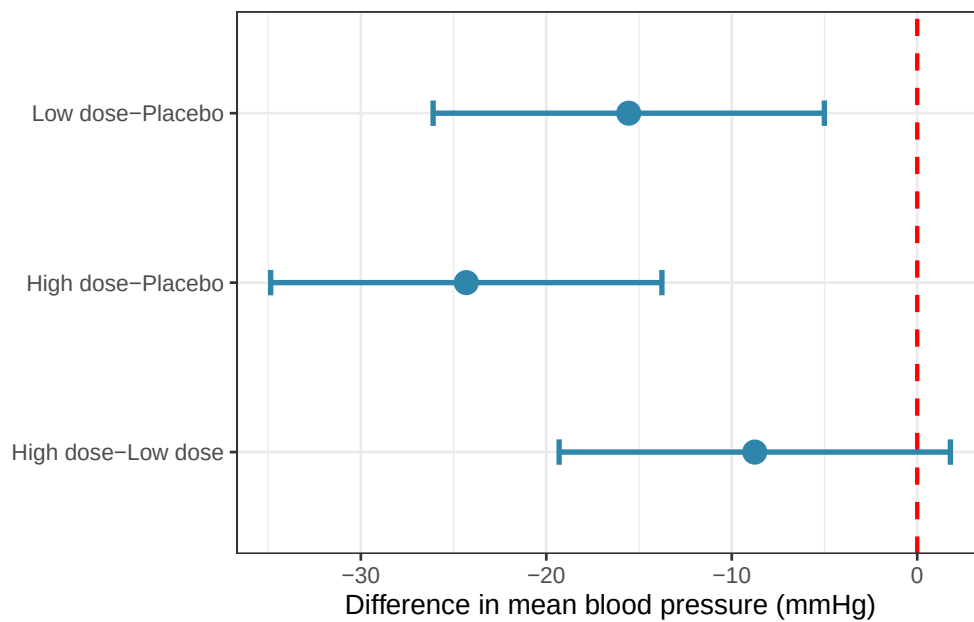


Figure 4.2: Tukey HSD confidence intervals for all pairwise comparisons of mean blood pressure. Intervals that do not cross the vertical dashed line at zero indicate significant differences after correction for multiple comparisons. Both active treatments differ significantly from placebo.

4.2.3 Bonferroni Correction

The Bonferroni correction is the simplest multiple comparison adjustment: divide α by the number of comparisons m , and declare a result significant only if its p -value falls below α/m . Equivalently, multiply each p -value by m and compare to the original α .

4 Multiple Comparisons

$$\alpha_{\text{Bonferroni}} = \frac{\alpha}{m}$$

For three pairwise comparisons at $\alpha = 0.05$, each individual test must reach $p < 0.0167$ to be declared significant.

```
pairwise.t.test(systolic$bp, systolic$group, p.adjust.method = "bonferroni")
```

```
#>
#> Pairwise comparisons using t tests with pooled SD
#>
#> data:  systolic$bp and systolic$group
#>
#>          Placebo Low dose
#> Low dose 0.0023  -
#> High dose 2.3e-06 0.1513
#>
#> P value adjustment method: bonferroni
```

The Bonferroni correction is simple, transparent, and works for any collection of tests: they do not even need to be pairwise comparisons. Its weakness is that it is conservative: it treats all tests as independent when in fact pairwise comparisons from the same dataset are correlated. This means it overcorrects, throwing away real power unnecessarily. When all pairwise comparisons are of interest, Tukey's HSD is preferred because it accounts for the correlation structure and is therefore less conservative.

4.2.4 Holm's Method

Holm's method is a sequential adaptation of Bonferroni that is uniformly more powerful while still controlling the familywise error rate. It works by ordering the p -values from smallest to largest and applying a decreasing threshold:

4 Multiple Comparisons

1. Sort the m p -values: $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$.
2. Compare $p_{(1)}$ to α/m , $p_{(2)}$ to $\alpha/(m-1)$, and so on.
3. Stop at the first $p_{(i)}$ that exceeds its threshold. Declare all tests from $p_{(i)}$ onward non-significant.

```
pairwise.t.test(systolic$bp, systolic$group, p.adjust.method = "holm")
```

```
#>
#> Pairwise comparisons using t tests with pooled SD
#>
#> data:  systolic$bp and systolic$group
#>
#>          Placebo Low dose
#> Low dose 0.0016  -
#> High dose 2.3e-06 0.0504
#>
#> P value adjustment method: holm
```

Holm's method should be preferred over Bonferroni in virtually every situation. It is strictly more powerful, equally simple to explain, and controls the FWER exactly. The only reason to use Bonferroni is when the simplicity of a single threshold is important for communication, for instance, when reporting to a non-statistical audience.

4.2.5 Dunnett's Test

Dunnett's test is designed for a specific and common situation: you have a control group and you want to compare each treatment group to the control, but you are not interested in treatment-vs-treatment comparisons. By restricting the family of comparisons to control-vs-treatment only, Dunnett's test is more powerful than Tukey's HSD for this specific set of comparisons.

4 Multiple Comparisons

```
# install.packages("DescTools") if needed
library(DescTools)

DunnettTest(bp ~ group, data = systolic, control = "Placebo")

#>
#> Dunnett's test for comparing several treatments with a control :
#> 95% family-wise confidence level
#>
#> $Placebo
#>
#>           diff      lwr.ci      upr.ci      pval
#> Low dose-Placebo -15.55494 -25.49637  -5.613511  0.0015 **
#> High dose-Placebo -24.31397 -34.25540 -14.372538 1.6e-06 ***
#>
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

With $k = 3$ groups, Dunnett's test makes only 2 comparisons instead of Tukey's 3, and the narrower family means a less severe correction, each comparison can be more sensitive.

4.2.6 Benjamini-Hochberg: Controlling the False Discovery Rate

Bonferroni, Holm, and Tukey all control the **familywise error rate** i.e. the probability of making even a single false positive. This is a strict criterion, and its strictness has a cost: as the number of comparisons grows, these methods become increasingly conservative, discarding more and more true positives in order to guarantee that no false positives slip through.

The **Benjamini-Hochberg (BH) procedure** (?) takes a different stance. Rather than asking “what is the probability of any false positive?”, it asks “among all the comparisons I declare significant, what fraction are expected to be false positives?” This fraction is called the **false**

4 Multiple Comparisons

discovery rate (FDR), and controlling it at 5% means that, on average, no more than 5% of your significant results are wrong, not that no false positives exist at all.

The procedure is straightforward:

1. Sort the m p -values from smallest to largest: $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$.
2. Find the largest i such that $p_{(i)} \leq \frac{i}{m} \cdot \alpha$.
3. Declare all tests up to and including rank i significant.

In R it is simply another option in `p.adjust`:

```
pairwise.t.test(systolic$bp, systolic$group, p.adjust.method = "BH")
```

```
#>
#> Pairwise comparisons using t tests with pooled SD
#>
#> data:  systolic$bp and systolic$group
#>
#>          Placebo Low dose
#> Low dose  0.0012  -
#> High dose 2.3e-06 0.0504
#>
#> P value adjustment method: BH
```

The BH procedure is **uniformly more powerful than Holm and Bonferroni**: it will declare more comparisons significant for the same data because it tolerates a small, controlled fraction of false positives rather than holding the false positive count to zero.

4.2.7 When to Use BH vs FWER-Controlling Methods

The choice between FDR and FWER control is not a technical question, it is a scientific one and it hinges on the consequences of each type of error in your specific context.

Use FWER control (Tukey, Holm) when:

- Any single false positive would be costly or misleading. A falsely declared effective drug in a clinical trial, for example, could lead to harmful treatment decisions.
- The number of comparisons is small (as is typical after a one-way ANOVA with a handful of groups).
- The analysis is confirmatory: you are testing a small number of pre-specified hypotheses and need strong guarantees.

Use FDR control (Benjamini-Hochberg) when:

- Missing real effects is considered as costly as generating false positives: the balance of errors matters, not just one direction.
- The number of comparisons is large, and FWER control would be so conservative as to be nearly useless.
- The analysis is exploratory: you are screening many comparisons to identify candidates for further investigation, and a small fraction of false positives among your hits is acceptable.

The BH procedure is the dominant method in genomics and other high-dimensional fields, where tens of thousands of tests are performed simultaneously and FWER control would require astronomically small individual p -value thresholds. In the context of a one-way ANOVA with three to six groups, the practical difference between Holm and BH will often be small with the same comparisons will reach significance. But as the number of groups grows, or when ANOVA is used as part of a larger screening pipeline, BH becomes the more sensible default.

The simulation below illustrates the power advantage of BH over Holm when there are many comparisons and a mix of true and null effects:

```
library(ggplot2)
library(furrr)
library(future)
```

4 Multiple Comparisons

```
plan(multisession)

set.seed(42)
n_sim <- 5000
alpha <- 0.05
n <- 15
k <- 10
k_true <- 5

delta <- 3
true_means <- c(rep(delta, k_true), rep(0, k - k_true))

# Comparisons are G2:G10 vs G1
# G2:G5 vs G1: both elevated by delta, truly NULL (4 comparisons)
# G6:G10 vs G1: G1 elevated, others, not truly DIFFERENT (5 comparisons)
truly_different <- c(rep(FALSE, k_true - 1), # G2:G5 vs G1
                    rep(TRUE, k - k_true)) # G6:G10 vs G1

results <- future_map_dfr(seq_len(n_sim), function(i) {
  y <- unlist(lapply(true_means, function(m) rnorm(n, mean = m, sd = 5)))
  group <- factor(rep(paste0("G", 1:k), each = n))
  df_s <- data.frame(y, group)

  p_raw <- sapply(paste0("G", 2:k), function(g) {
    t.test(y ~ group,
           data = df_s[df_s$group %in% c("G1", g), ],
           var.equal = TRUE)$p.value
  })
})
```


4 Multiple Comparisons

```
p_holm <- p.adjust(p_raw, method = "holm")
p_bh <- p.adjust(p_raw, method = "BH")

data.frame(
  power_holm = mean((p_holm < alpha)[ truly_different]),
  power_bh = mean((p_bh < alpha)[ truly_different]),
  fpr_holm = mean((p_holm < alpha)[!truly_different]),
  fpr_bh = mean((p_bh < alpha)[!truly_different])
)
}, .options = furrr_options(seed = TRUE))

summary_df <- data.frame(
  metric = c("Power\n(true effects detected)", "False positive rate\n(null effects declared significant)"),
  Holm = c(mean(results[, "power_holm"]), mean(results[, "fpr_holm"])),
  BH = c(mean(results[, "power_bh"]), mean(results[, "fpr_bh"]))
)

summary_df <- data.frame(
  metric = c("Power\n(true effects detected)", "False positive rate\n(null effects declared significant)"),
  Holm = c(mean(results[, "power_holm"]), mean(results[, "fpr_holm"])),
  BH = c(mean(results[, "power_bh"]), mean(results[, "fpr_bh"]))
)

library(tidyr)
summary_long <- pivot_longer(summary_df, cols = c("Holm", "BH"), names_to = "method", values_to = "value")

ggplot(summary_long, aes(x = method, y = value, fill = method)) +
  geom_col(width = 0.5, alpha = 0.85) +
```

4 Multiple Comparisons

```
geom_hline(yintercept = 0.05, linetype = "dashed", colour = "red", linewidth = 0.8) +  
facet_wrap(~ metric) +  
scale_fill_manual(values = c("BH" = "#2E86AB", "Holm" = "#E84855")) +  
scale_y_continuous(limits = c(0, 1), labels = scales::percent_format(accuracy = 1)) +  
labs(x = NULL, y = NULL, fill = "Method") +  
theme_bw() +  
theme(legend.position = "none")
```

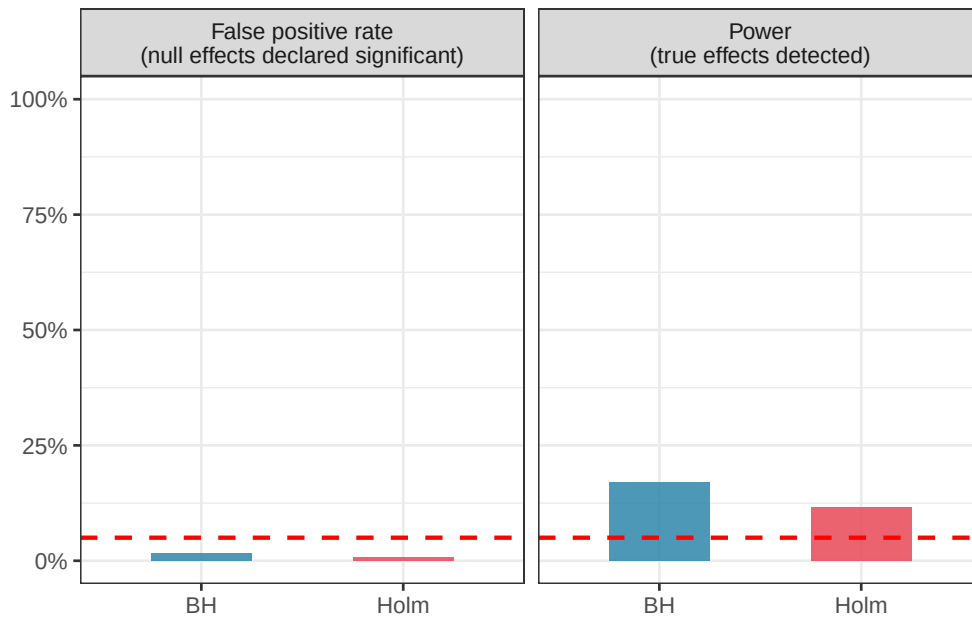


Figure 4.3: Power and false positive rate for Holm (FWER) and Benjamini-Hochberg (FDR) correction across 5000 simulations with $k = 10$ groups, half of which truly differ from the others. BH detects more true positives (higher power) while keeping the false discovery rate at the nominal 5% level. Holm controls the familywise error rate more tightly but misses more real effects.

4.2.8 Comparing the Methods

The table below summarises the key properties of each method:

4 Multiple Comparisons

Method	Controls	Best used when	Relative power
Tukey HSD	FWER	All pairwise, equal n	High for pairwise
Bonferroni	FWER	Any tests, simple communication	Low
Holm	FWER	Any tests, better than Bonferroni	Moderate–High
Dunnett	FWER	Each treatment vs one control	Highest for control comparisons
Benjamini-Hochberg	FDR	Many comparisons, exploratory work	Highest

4.3 Planned Contrasts vs Exploratory Comparisons

4.3.1 A Distinction That Changes the Rules

Not all comparisons are created equal. There is a fundamental difference between comparisons that were specified before looking at the data, **planned contrasts**, and comparisons chosen after seeing which groups look most different, **exploratory post-hoc comparisons**.

This distinction matters because the multiple testing problem is, at its core, a problem of searching. *When you specify your comparisons in advance, you are not searching, you are testing a specific hypothesis.* The number of tests you could have run but did not is irrelevant to the interpretation of the one you did run. When you search the data for interesting differences after the fact, every comparison you could have made is part of the implicit search, and the familywise error rate must reflect the size of that search.

4.3.2 Planned Contrasts

A planned contrast is a specific, theoretically motivated comparison written down before data collection. For example:

- “We expect nitrogen fertiliser to increase plant height relative to the control, based on prior studies.”
- “We expect the average of the two drug doses to differ from placebo.”

Planned contrasts can be specified as linear combinations of group means.

A contrast is a vector of coefficients c_j such that $\sum_{j=1}^k c_j = 0$. For example, to compare the nitrogen group mean to the average of the control and phosphorus groups:

$$L = \bar{y}_{\text{nitrogen}} - \frac{1}{2}(\bar{y}_{\text{control}} + \bar{y}_{\text{phosphorus}})$$

with coefficients $c = (-0.5, 1, -0.5)$ for control, nitrogen and phosphorus respectively.

In R, planned contrasts are specified directly in the model:

```
# Compare high dose to placebo (planned a priori)
# and low dose to placebo (planned a priori)
# using the multcomp package

# install.packages("multcomp") if needed
library(multcomp)

# Define contrast matrix: each row is one contrast
# Groups are ordered as: High dose, Low dose, Placebo
K <- rbind(
  "High dose vs Placebo" = c(-1, 0, 1),
  "Low dose vs Placebo"  = c( 0, -1, 1)
```

4 Multiple Comparisons

```
)  
  
planned <- glht(fit, linfct = mcp(group = K))  
summary(planned, test = adjusted("none")) # no correction for planned contrasts
```

```
#>  
#> Simultaneous Tests for General Linear Hypotheses  
#>  
#> Multiple Comparisons of Means: User-defined Contrasts  
#>  
#>  
#> Fit: aov(formula = bp ~ group, data = systolic)  
#>  
#> Linear Hypotheses:  
#>  
#>               Estimate Std. Error t value Pr(>|t|)  
#> High dose vs Placebo == 0  -24.314      4.383  -5.548 7.82e-07 ***  
#> Low dose vs Placebo == 0   -8.759      4.383  -1.999  0.0504 .  
#> ---  
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
#> (Adjusted p values reported -- none method)
```

```
confint(planned)
```

```
#>  
#> Simultaneous Confidence Intervals  
#>  
#> Multiple Comparisons of Means: User-defined Contrasts  
#>  
#>
```

4 Multiple Comparisons

```
#> Fit: aov(formula = bp ~ group, data = systolic)
#>
#> Quantile = 2.2682
#> 95% family-wise confidence level
#>
#>
#> Linear Hypotheses:
#>
#>               Estimate lwr      upr
#> High dose vs Placebo == 0 -24.3140 -34.2551 -14.3728
#> Low dose vs Placebo == 0  -8.7590 -18.7002  1.1821
```

Notice `adjusted("none")` because these contrasts were planned in advance and are limited in number, no correction for multiple comparisons is applied. The tests stand on their own as independent, pre-specified hypotheses.

This is the important practical payoff of planning your comparisons in advance: you preserve the full $\alpha = 0.05$ for each test, rather than spending part of it on the correction overhead.

4.3.3 Exploratory Post-Hoc Comparisons

When you look at the data first and then decide which groups to compare, you are in exploratory territory. This is not wrong: exploration is a legitimate and important part of science but it requires honest accounting. The comparisons must be treated as a family, and the familywise error rate must be controlled. Tukey's HSD or Holm's method is appropriate here.

The clearest sign that you are in exploratory territory is when your choice of comparisons is guided by the data: "the nitrogen group looks highest, so let me compare it to everything else." Any comparison motivated by looking at the results first is post-hoc, regardless of whether it feels like something you would have planned.

4.4 Practical Guidance: When to Correct and When Not To

The multiple comparisons literature contains genuine disagreement among statisticians, and students are sometimes given conflicting advice. The following principles represent a pragmatic, widely accepted position.

4.4.1 Correct When You Are Searching

If the comparisons were not specified in advance, correction is necessary. The more comparisons you make, the more aggressively you should correct. Tukey's HSD is the right default for all pairwise comparisons following a one-way ANOVA. Holm's method is appropriate when the set of comparisons is more varied.

4.4.2 Do Not Correct Pre-Specified Planned Contrasts

If you specified a small number of theoretically motivated comparisons before data collection, correction is not required and is arguably incorrect, because you would be penalising yourself for tests you might have run but did not. Keep the number of planned contrasts modest (as a rough guide, no more than $k - 1$ contrasts for k groups) and specify them in your analysis plan before opening the data.

4.4.3 Consider the Consequences of Each Error Type

Multiple comparison corrections reduce false positives at the cost of increasing false negatives; they make it harder to detect real effects. In some contexts, missing a real effect is more costly than generating a false positive; in others, the reverse is true. A screening experiment designed to identify candidate treatments for further investigation might accept more false positives in exchange for fewer missed effects. For example, a confirmatory clinical trial must be conservative about false positives because a falsely declared effective treatment may ultimately be

adopted into clinical practice. Let the scientific context, not habit, guide your choice of correction.

4.4.4 Always Report What You Did

Whatever correction you apply, or do not apply, should be stated explicitly. Report the uncorrected p -values alongside the corrected ones when space allows. A reader who knows you used Tukey's HSD can evaluate your decision; a reader who does not know what correction was applied cannot.

4.4.5 A Decision Guide

4.4.6 The Bottom Line

Multiple comparison corrections are not bureaucratic box-ticking. They exist because data can be made to say almost anything if you ask enough questions of them. The discipline of specifying comparisons in advance, or of honestly accounting for the number of comparisons made post-hoc, is what separates a trustworthy analysis from one that has been, perhaps unknowingly, optimised to produce significant results.

At the same time, correction is not always necessary, and overcorrecting is a real error with real costs and can cause you to dismiss effects that are genuine. The goal is calibration: matching the stringency of your error control to the scientific context, the number of comparisons, and the consequences of being wrong in either direction.

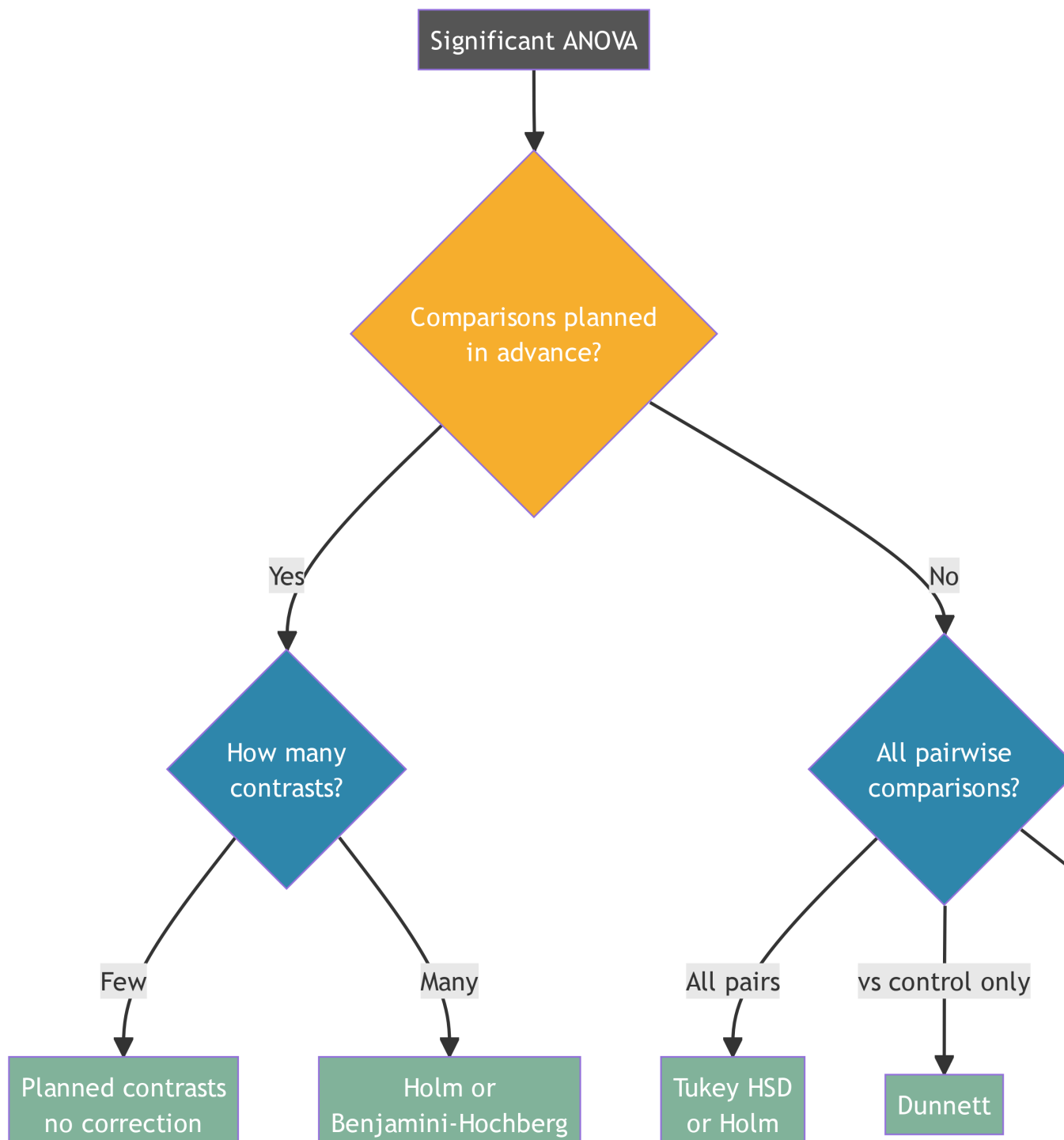


Figure 4.4: A decision guide for selecting a multiple comparison approach after a significant one-way ANOVA.

5 Two-Way ANOVA and Factorial Designs

One-way ANOVA asks whether a single factor, a fertiliser, a drug, or a diet, affects the response. But most biological systems are not that simple. Plant growth depends on both the fertiliser applied and the soil type it is applied to. Drug efficacy depends on both the dose and the genetic background of the patient. Yield depends on both the crop variety and the environment in which it is grown.

Two-way ANOVA extends the one-way framework to handle two factors simultaneously. This is not merely a convenience, it is a more honest representation of how biological systems work. And it introduces one of the most important concepts in all of applied statistics: the **interaction**, the situation where the effect of one factor depends on the level of another.

5.1 Factorial Designs: The Logic and the Advantage

5.1.1 The One-Factor-at-a-Time Trap

Before introducing the statistics, it is worth asking a more fundamental question: why run a factorial experiment at all? The intuitive alternative, varying one factor at a time while holding everything else constant, seems safer and easier to interpret. In practice, it is neither.

Suppose you want to understand how both fertiliser type and watering regime affect wheat yield. The one-factor-at-a-time approach would run two separate experiments: first vary the fertiliser while keeping watering constant, then vary watering while keeping fertiliser constant. This requires two experiments, two sets of plants, and twice the resources. More importantly,

it answers two questions in isolation: what does fertiliser do when watering is fixed at one level? rather than the richer question of how fertiliser and watering work *together* across the full range of conditions you care about.

Worst of all, the one-factor-at-a-time approach cannot detect interactions. If nitrogen fertiliser triples yield under well-watered conditions but has no effect under drought, two separate experiments, one fixing watering at “well-watered” and one fixing fertiliser at “nitrogen”, will never reveal this. You would need to have been lucky enough to choose the right fixed level in each experiment, and even then you would have no statistical framework for testing whether the discrepancy is real or due to chance.

Fisher recognised this limitation at Rothamsted in the 1920s and proposed the factorial design as the solution: vary all factors simultaneously, in all combinations, within a single experiment.

5.1.2 What Is a Factorial Design?

A **factorial design** is an experiment in which every level of every factor is combined with every level of every other factor. If factor A has a levels and factor B has b levels, a complete factorial design has $a \times b$ treatment combinations, called **cells**. With replication, n observations per cell, the total number of observations is $a \times b \times n$.

For our fertiliser and watering experiment with two fertiliser levels (none, nitrogen) and two watering levels (dry, watered), a 2×2 factorial design produces four cells:

	Dry	Watered
No fertiliser	Cell 1	Cell 2
Nitrogen	Cell 3	Cell 4

Each cell receives its own random set of experimental units, here, pots of wheat plants. With $n = 5$ plants per cell, the total experiment requires $2 \times 2 \times 5 = 20$ plants.

5.1.3 The Efficiency Advantage

A factorial design extracts more information from the same data than a one-factor-at-a-time experiment of the same size. Consider a 2×2 factorial with n observations per cell, giving $4n$ observations in total. The main effect of fertiliser is estimated by comparing the $2n$ observations in the nitrogen rows to the $2n$ observations in the no-fertiliser rows with all $4n$ observations contribute to this estimate. The main effect of watering is estimated similarly, using all $4n$ observations. A one-factor-at-a-time experiment of the same total size would dedicate only $2n$ observations to each factor.

Fisher called this the **hidden replication** of factorial designs: every observation simultaneously contributes to the estimation of every main effect and every interaction. No observation is wasted.

5.1.4 Common Factorial Design Structures

Factorial designs come in several standard forms, differing in the number of factors and levels involved.

- **2×2 factorial (two factors, two levels each).** The simplest factorial design. Produces four cells and allows estimation of two main effects and one interaction. Common in clinical trials (drug A yes/no $\times$ drug B yes/no) and agricultural experiments (fertiliser yes/no $\times$ irrigation yes/no).
- **2^k factorial (k factors, two levels each).** A natural extension of the 2×2 design to k factors. With $k = 3$ factors there are $2^3 = 8$ cells; with $k = 4$ there are $2^4 = 16$ cells. These designs are widely used in industrial and agricultural screening experiments where many factors must be evaluated efficiently.
- **$a \times b$ factorial (two factors, multiple levels).** The general case, as in our fertiliser-by-environment example (see below) with four varieties and three environments ($4 \times 3 = 12$ cells). This is the most common structure in biological research.

- **Higher-order factorials.** Three or more factors can be combined in a single experiment ($a \times b \times c$, and so on). These designs become large quickly: a $3 \times 3 \times 3$ factorial already requires 27 cells and their higher-order interactions are often difficult to interpret biologically. In practice, three-way and higher interactions are rarely interpretable and the experiments that produce them often require **fractional factorial designs**, which are beyond the scope of this book.

5.1.5 Balanced vs Unbalanced Factorial Designs

A factorial design is **balanced** when every cell contains exactly the same number of observations. Balanced designs have important advantages: the main effects and interaction are orthogonal (statistically independent), the sums of squares partition cleanly and unambiguously, and the analysis is straightforward. Whenever possible, factorial experiments should be designed to be balanced.

In practice, balance is often lost. Animals die, plants fail to germinate, patients drop out of trials, soil cores are contaminated. The result is an **unbalanced design** (unequal cell sizes) which introduces the sums of squares complications discussed in Section ???. The key practical message is that unbalanced designs are manageable, but they require more care in analysis and should be anticipated at the design stage by slightly over-recruiting to each cell.

5.1.6 Fixed, Random, and Mixed Factorial Designs

Just as in one-way ANOVA (see one-way), the factors in a factorial design can be fixed or random, and this distinction determines both the model and the correct F test.

A **fixed factor** is one whose levels are specifically chosen by the experimenter and whose conclusions apply only to those levels. Fertiliser type (none, nitrogen, phosphorus) is a fixed factor: the conclusions are about those three specific treatments.

A **random factor** is one whose levels are a random sample from a larger population of levels, and whose conclusions are intended to generalise to that population. Field sites selected at

random from a region, batches of reagents drawn from a production run, or patients recruited from a hospital are random factors.

A **mixed factorial design** contains both fixed and random factors. This is extremely common in biology: a treatment applied across randomly selected sites, a drug tested across randomly recruited patients, or a crop variety evaluated across randomly selected growing seasons. The statistical model for a mixed factorial design is a **mixed-effects model**, and the correct F tests for the fixed effects differ from those in an all-fixed design. This will be developed fully in Part III. For now, the two-way ANOVA presented in this chapter assumes both factors are fixed.

5.1.7 Randomisation in Factorial Designs

As Fisher emphasised at Rothamsted, the validity of a factorial experiment depends on proper randomisation. Each experimental unit must be randomly assigned to one of the $a \times b$ treatment combinations. This randomisation ensures that the treatment effects are not confounded with any systematic difference between units such as position in a greenhouse, order of processing in a laboratory, or any other nuisance variable that might vary across the experiment.

In practice, randomisation in factorial experiments is sometimes constrained. If one factor is difficult or expensive to change (the temperature of a growth chamber, the irrigation regime of a field) it may be more practical to change it less frequently than the other factor. This leads to **split-plot designs**, in which the randomisation is done in two stages and the two factors have different levels of replication. Split-plot designs require a modified analysis and are covered in Chapter 7.

The diagram below summarises the common factorial design structures discussed in this section:

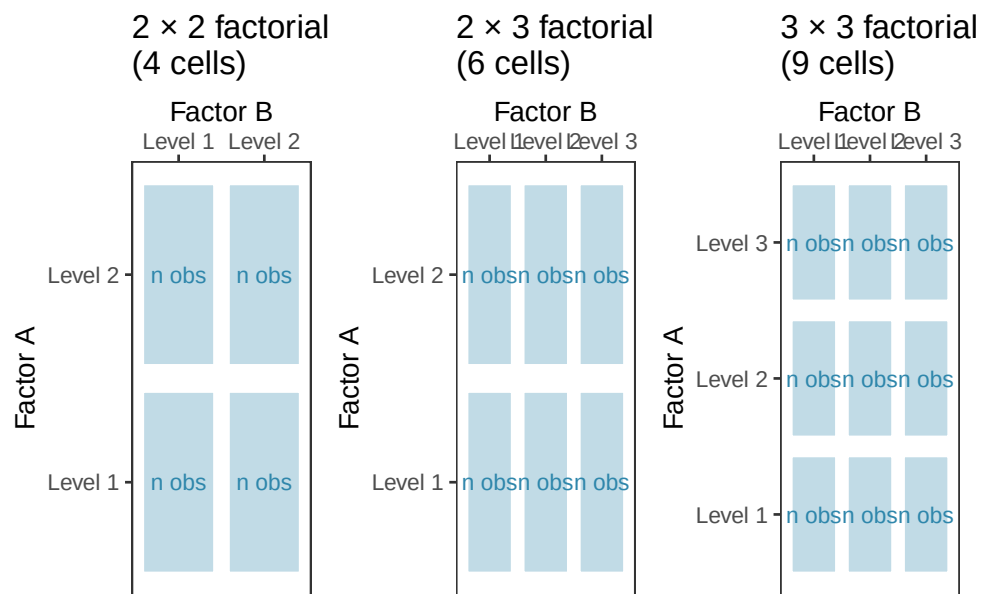


Figure 5.1: Common factorial design structures. Left: a 2x2 factorial with two factors at two levels each, producing four cells. Centre: a 2x3 factorial with two factors at two and three levels respectively, producing six cells. Right: a 2x2x2 factorial with three factors at two levels each, producing eight cells. Cell shading indicates a different treatment combination; n denotes the number of replicate observations per cell.

5.2 Main Effects and Interactions

5.2.1 The Model

With two factors A (with a levels) and B (with b levels), the two-way ANOVA model is:

$$y_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \varepsilon_{ijk}$$

where:

- μ is the grand mean.
- α_i is the **main effect** of factor A at level i : the average effect of being in level i of A , averaged across all levels of B .
- β_j is the **main effect** of factor B at level j : the average effect of being in level j of B , averaged across all levels of A .
- $(\alpha\beta)_{ij}$ is the **interaction effect**: the extent to which the effect of factor A at level i differs depending on which level of B is present, and vice versa.
- ε_{ijk} is the residual for the k th observation in cell (i, j) , assumed $\varepsilon_{ijk} \sim \mathcal{N}(0, \sigma^2)$.

The constraints are $\sum_i \alpha_i = 0$, $\sum_j \beta_j = 0$, $\sum_i (\alpha\beta)_{ij} = 0$ for all j , and $\sum_j (\alpha\beta)_{ij} = 0$ for all i .

5.2.2 Main Effects

A **main effect** is the average effect of one factor, collapsing across all levels of the other. In a fertiliser-by-watering experiment, the main effect of fertiliser is the average difference in plant height between fertilised and unfertilised plants, averaging over both watered and unwatered conditions.

Main effects are straightforward to interpret, but only when the interaction is absent or negligible. When a substantial interaction is present, main effects can be misleading, as we will see shortly.

5.2.3 Interactions

An **interaction** occurs when the effect of one factor is not the same at every level of the other factor. This is not a statistical artefact or a complication to be wished away, it is a genuine biological phenomenon, and recognising it is often the most scientifically interesting finding in an experiment.

Three situations are worth distinguishing:

No interaction (additivity). The effect of factor A is the same regardless of which level of B is present. The two factors act independently and their effects simply add together. The lines in an interaction plot are parallel.

Quantitative interaction. The effect of factor A differs in *magnitude* across levels of B , but always in the same *direction*. For example, nitrogen fertiliser always increases plant height, but by more in well-watered plots than in dry plots. The lines in an interaction plot are not parallel but do not cross.

Qualitative interaction (crossover interaction). The effect of factor A reverses *direction* depending on the level of B . For example, a drug increases survival in young patients but decreases it in elderly patients. The lines in an interaction plot cross. This is the most consequential type of interaction: main effects are not just misleading here, they are actively wrong.

5.2.4 The Three F Tests

Two-way ANOVA produces three F tests, one for each term in the model:

$$F_A = \frac{MS_A}{MS_{\text{within}}} \quad F_B = \frac{MS_B}{MS_{\text{within}}} \quad F_{AB} = \frac{MS_{AB}}{MS_{\text{within}}}$$

The interaction test F_{AB} should always be examined first. If it is significant, the main effect tests must be interpreted with great caution: they describe averages that may not represent any real condition in the experiment.

5.3 Interpreting Interaction Plots: A Critical Skill

5.3.1 Why Interaction Plots Matter

An interaction plot displays the mean response for each combination of factor levels, with the levels of one factor on the x-axis and separate lines for each level of the other factor. It is the single most important diagnostic tool in factorial ANOVA, and the ability to read one correctly is a skill worth developing carefully.

The rule of thumb is simple: **parallel lines mean no interaction; non-parallel lines suggest an interaction.** But as the simulations below show, the details matter considerably.

```
library(ggplot2)
library(patchwork)

set.seed(42)

n <- 20 # per cell

# Factor levels
fertiliser <- c("None", "Nitrogen")
water <- c("Dry", "Watered")

make_df <- function(cell_means, sd = 4) {
  data.frame(
    height = c(
      rnorm(n, cell_means[1], sd), # None x Dry
      rnorm(n, cell_means[2], sd), # None x Watered
      rnorm(n, cell_means[3], sd), # Nitrogen x Dry
      rnorm(n, cell_means[4], sd) # Nitrogen x Watered
    ),
  ),
```

```

    fertiliser = factor(rep(rep(fertiliser, each = n), 1), levels = fertiliser),
    water = factor(rep(rep(water, n), 2), levels = water)
  )
}

# Scenario 1: No interaction (parallel lines)
df1 <- make_df(c(20, 30, 28, 38))

# Scenario 2: Quantitative interaction (diverging lines, same direction)
df2 <- make_df(c(20, 22, 28, 38))

# Scenario 3: Qualitative interaction (crossing lines)
df3 <- make_df(c(20, 35, 35, 20))

interaction_plot <- function(df, title) {
  summ <- aggregate(height ~ fertiliser + water, data = df, FUN = mean)
  ggplot(summ, aes(x = water, y = height, colour = fertiliser, group = fertiliser)) +
    geom_line(linewidth = 1.2) +
    geom_point(size = 4) +
    scale_colour_manual(values = c("#2E86AB", "#E84855")) +
    labs(title = title, x = "Water availability", y = "Mean height (cm)", colour = "Fertiliser")
    theme_bw() +
    theme(legend.position = "bottom")
}

p1 <- interaction_plot(df1, "No interaction")
p2 <- interaction_plot(df2, "Quantitative interaction")
p3 <- interaction_plot(df3, "Qualitative (crossover) interaction")

```

```
(p1 + p2) / (p3 + plot_spacer())
```

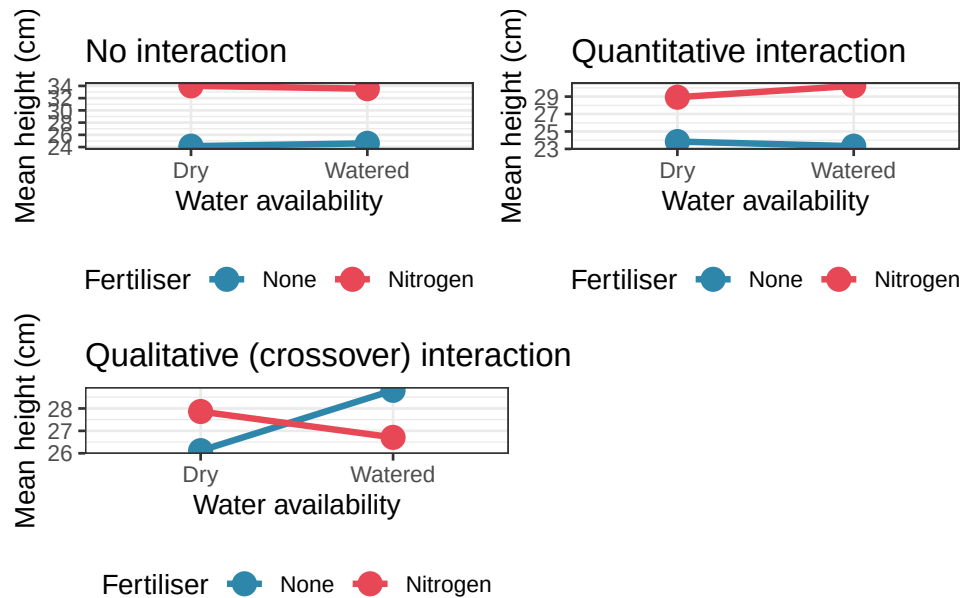


Figure 5.2: Four interaction scenarios illustrated with simulated data. Top left: no interaction, lines are parallel, main effects are interpretable. Top right: quantitative interaction, lines diverge but do not cross, the direction of the fertiliser effect is consistent. Bottom left: qualitative (crossover) interaction, lines cross, the effect of fertiliser reverses direction depending on water availability. Bottom right: the corresponding ANOVA tables confirm that only the crossover interaction dominates the main effects.

5.3.2 Reading the Plots

No interaction (top left): The nitrogen line sits consistently above the no-fertiliser line by the same amount in both dry and watered conditions. The lines are parallel. The main effect of fertiliser (nitrogen increases height) and the main effect of water (watering increases height) can each be stated cleanly and without qualification.

Quantitative interaction (top right): Nitrogen still increases height in both conditions, but the benefit is much larger under watered conditions than under dry conditions. The lines diverge.

The main effect of fertiliser is still real and directionally correct, but the simple statement “nitrogen increases height by X cm” misses an important nuance: the benefit depends substantially on water availability.

Qualitative interaction (bottom left): Nitrogen increases height under dry conditions but *decreases* it under watered conditions. The lines cross. The main effect of fertiliser, averaging across both water conditions, will be near zero and will be essentially uninterpretable. Reporting it would be actively misleading. The scientifically correct statement is: “the effect of nitrogen depends entirely on water availability, and reverses direction between conditions.”

5.3.3 A Practical Warning

Interaction plots display means, not raw data. A pattern that looks dramatic in the means may not be statistically significant if the within-cell variation is large. Always fit the model and examine the interaction F test before interpreting the plot. Conversely, a significant interaction F test with a flat-looking interaction plot in small samples deserves scrutiny: the significance may be driven by a single influential cell.

The best practice is to overlay the raw data on the interaction plot:

```
ggplot(df3, aes(x = water, y = height, colour = fertiliser, group = fertiliser)) +
  geom_jitter(width = 0.05, alpha = 0.4, size = 1.5) +
  stat_summary(fun = mean, geom = "line", linewidth = 1.2) +
  stat_summary(fun = mean, geom = "point", size = 4) +
  scale_colour_manual(values = c("#2E86AB", "#E84855")) +
  labs(x = "Water availability", y = "Plant height (cm)", colour = "Fertiliser") +
  theme_bw() +
  theme(legend.position = "bottom")
```

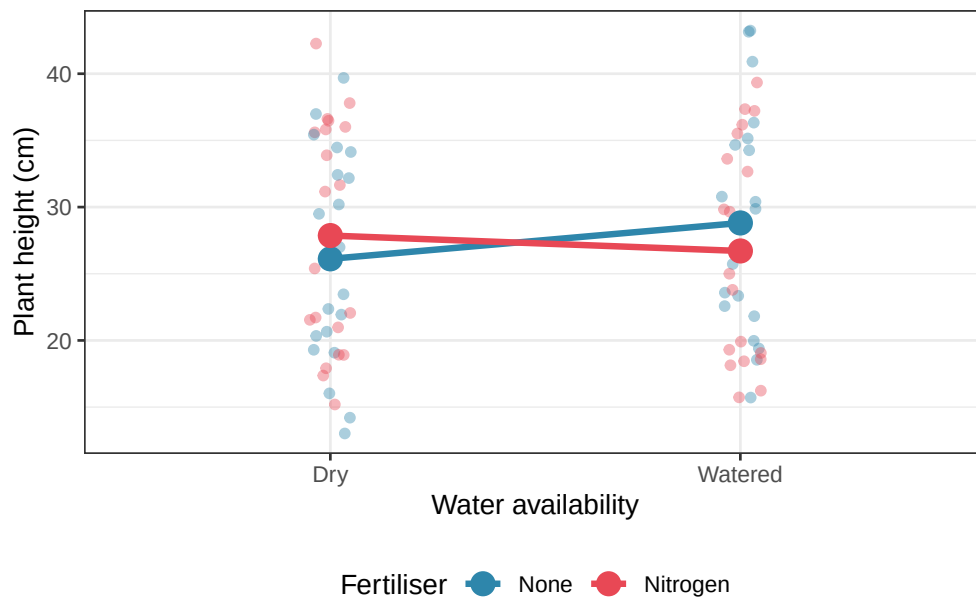


Figure 5.3: Interaction plot with raw data overlaid for the crossover scenario. The individual points (jittered) reveal the within-cell variation that the means alone conceal. Despite the large within-cell scatter, the crossover pattern is clear and the interaction is statistically significant.

5.4 Unbalanced Designs and Type I, II, and III Sums of Squares

5.4.1 What Is an Unbalanced Design?

A factorial design is **balanced** when every cell, every combination of factor levels, contains the same number of observations. It is **unbalanced** when cell sizes differ. Unbalanced designs arise routinely in biology: animals die during an experiment, plant pots are accidentally knocked over, patients drop out of a clinical trial. They are not a methodological failure, they are a practical reality.

The problem with unbalanced designs is that the two factors are no longer orthogonal. The levels of factor A are not equally represented across the levels of factor B , which means that some of the variation in the response is simultaneously attributable to both factors. How this

shared variation is partitioned between the factors determines the sums of squares, and there are three different conventions for doing so, giving three different answers.

5.4.2 Type I Sums of Squares (Sequential)

Type I SS tests each factor sequentially, in the order it appears in the model formula. Factor A is tested first, ignoring B . Then B is tested given A is already in the model. Then the interaction is tested given both main effects.

The critical problem: **the results depend on the order in which you specify the factors.** `aov(y ~ A + B + A:B)` and `aov(y ~ B + A + A:B)` will give different F tests for both A and B in an unbalanced design. This is the default in R's `aov()` function, and it is rarely what you want for a two-way ANOVA with unequal cell sizes.

Type I SS is appropriate only when the sequential ordering has a clear scientific justification, for example, in a hierarchical model where one factor genuinely precedes the other conceptually.

5.4.3 Type II Sums of Squares

Type II SS tests each main effect after adjusting for the other main effect, but not for the interaction. Factor A is tested given B is in the model; factor B is tested given A is in the model; but neither is adjusted for the $A \times B$ interaction.

Type II SS is appropriate when you have good reason to believe the interaction is absent or negligible. It is more powerful than Type III in that situation because it does not spend degrees of freedom adjusting for an interaction that does not exist.

5.4.4 Type III Sums of Squares

Type III SS tests each term after adjusting for all other terms in the model simultaneously. Factor A is tested given B and $A \times B$ are both in the model; factor B is tested given A and $A \times B$ are in the model; and the interaction is tested given both main effects.

Type III SS is the appropriate default for most two-way ANOVA analyses with unbalanced designs, because:

- The results do not depend on the order of the terms in the formula.
- Each main effect is tested independently of the interaction.
- It correctly reflects the marginal effect of each factor.

In R, Type III SS requires the `car` package and **contrast coding must be set to sum-to-zero** for the results to be interpretable:

```
library(car)

# CRITICAL: set sum-to-zero contrasts before fitting
options(contrasts = c("contr.sum", "contr.poly"))

fit2 <- lm(height ~ fertiliser * water, data = df3)
Anova(fit2, type = "III")
```

```
#> Anova Table (Type III tests)
#>
#> Response: height
#>
#>               Sum Sq Df  F value Pr(>F)
#> (Intercept)    59952  1 869.6346 <2e-16 ***
#> fertiliser         1  1   0.0091 0.9243
#> water            12  1   0.1711 0.6803
#> fertiliser:water   74  1   1.0735 0.3034
#> Residuals       5239 76
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```



```
# Reset to R defaults afterwards
options(contrasts = c("contr.treatment", "contr.poly"))
```

The `contrasts = c("contr.sum", "contr.poly")` line is not optional bookkeeping: without it, Type III SS for main effects in the presence of an interaction will be computed incorrectly. This is one of the most common silent errors in two-way ANOVA analyses in R.

5.4.5 A Direct Comparison

The code below constructs a deliberately unbalanced dataset and shows how the three types of SS give different answers:

```
set.seed(42)
#| cache: true
#| message: false
#| warning: false

# Unbalanced: unequal cell sizes
n_cells <- c(8, 15, 20, 10) # None:Dry, None:Watered, N:Dry, N:Watered

df_unbal <- data.frame(
  height = c(
    rnorm(n_cells[1], mean = 20, sd = 4),
    rnorm(n_cells[2], mean = 30, sd = 4),
    rnorm(n_cells[3], mean = 28, sd = 4),
    rnorm(n_cells[4], mean = 38, sd = 4)
  ),
  fertiliser = factor(c(rep("None", n_cells[1] + n_cells[2]),
                        rep("Nitrogen", n_cells[3] + n_cells[4])),
                      levels = c("None", "Nitrogen")),
```

5 Two-Way ANOVA and Factorial Designs

```
water = factor(c(rep("Dry", n_cells[1]),
                  rep("Watered", n_cells[2]),
                  rep("Dry", n_cells[3]),
                  rep("Watered", n_cells[4])),
              levels = c("Dry", "Watered"))
)
```

Cell sizes:

```
print(table(df_unbal$fertiliser, df_unbal$water))
```

```
#>
#>      Dry Watered
#> None      8    15
#> Nitrogen 20    10
```

Computation of type I sum of squares which is sequential and for which order matters:

```
print(summary(aov(height ~ fertiliser * water, data = df_unbal)))
```

```
#>      Df Sum Sq Mean Sq F value    Pr(>F)
#> fertiliser      1  233.7    233.7  10.830  0.00186 **
#> water           1 1040.5   1040.5  48.208 8.12e-09 ***
#> fertiliser:water  1   24.8     24.8   1.149  0.28909
#> Residuals      49 1057.6     21.6
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Computation of type I sum of squares (water first):

5 Two-Way ANOVA and Factorial Designs

```
print(summary(aov(height ~ water * fertiliser, data = df_unbal)))
```

```
#>               Df Sum Sq Mean Sq F value    Pr(>F)
#> water           1  663.5    663.5   30.740 1.17e-06 ***
#> fertiliser      1  610.8    610.8   28.297 2.56e-06 ***
#> water:fertiliser 1   24.8     24.8    1.149   0.289
#> Residuals      49 1057.6     21.6
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Computation of type II and III sums of squares via `car::Anova`

```
fit_unbal <- lm(height ~ fertiliser * water, data = df_unbal)

print(Anova(fit_unbal, type = "II"))
```

```
#> Anova Table (Type II tests)
#>
#> Response: height
#>               Sum Sq Df F value    Pr(>F)
#> fertiliser      610.77  1 28.2975 2.564e-06 ***
#> water          1040.50  1 48.2077 8.124e-09 ***
#> fertiliser:water  24.79  1  1.1486   0.2891
#> Residuals      1057.60 49
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
options(contrasts = c("contr.sum", "contr.poly"))

fit_unbal_sum <- lm(height ~ fertiliser * water, data = df_unbal)
```

```
print(Anova(fit_unbal_sum, type = "III"))
```

```
#> Anova Table (Type III tests)
#>
#> Response: height
#>
#>               Sum Sq Df    F value    Pr(>F)
#> (Intercept)    39965   1 1851.6354 < 2.2e-16 ***
#> fertiliser      604    1   28.0033 2.823e-06 ***
#> water          987    1   45.7064 1.558e-08 ***
#> fertiliser:water  25    1    1.1486   0.2891
#> Residuals     1058   49
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
options(contrasts = c("contr.treatment", "contr.poly"))
```

Results show that Type I SS gives different F values for fertiliser depending on whether it enters the model first or second, while Type II and Type III give results that are independent of formula order. For this no-interaction scenario, Type II and Type III will give similar results; they diverge when a real interaction is present.

5.4.6 A Practical Decision Rule

Situation	Recommended SS type
Balanced design (equal cell sizes)	Any, all three give identical results
Unbalanced, interaction likely present	Type III with <code>contr.sum</code>

Situation	Recommended SS type
Unbalanced, interaction absent	Type II
Sequential scientific hypothesis	Type I
Using R's <code>aov()</code> with unbalanced data	Avoid. Use <code>car::Anova()</code> instead

5.5 Example: Genotype $\times$ Environment in Agronomy

5.5.1 Background

One of the most important questions in agronomy is whether the yield advantage of a particular crop variety is consistent across growing environments, or whether some varieties perform well in some environments but poorly in others. This is called a **genotype by environment interaction (G $\times$ E)**, and it is a canonical example of a qualitative interaction with direct practical consequences: a variety that shows G $\times$ E cannot be recommended universally, its deployment must be tailored to the environment.

We simulate a trial in which four wheat varieties (genotypes) are grown in three environments (locations with different rainfall regimes) with five replicate plots per combination, giving $4 \times 3 \times 5 = 60$ observations in total (Section ??).

```
summary(gxe)
```

```
#>  variety      env      rep      yield
#> Var A:15  Dry      :20  Min.    :1  Min.    :26.03
#> Var B:15  Moderate:20  1st Qu.:2  1st Qu.:45.60
#> Var C:15  Wet      :20  Median  :3  Median  :52.71
#> Var D:15                      Mean   :3  Mean   :51.28
```

```
#>               3rd Qu.:4    3rd Qu.:56.95
#>               Max.    :5    Max.    :67.16
```

5.5.2 Visualising the G×E Interaction

```
library(ggplot2)

summ_gxe <- aggregate(yield ~ variety + env, data = gxe, FUN = mean)

ggplot(summ_gxe, aes(x = env, y = yield,
                    colour = variety, group = variety)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 4) +
  scale_colour_manual(values = c("#2E86AB", "#E84855", "#F6AE2D", "#81B29A")) +
  labs(
    title = "Genotype × Environment interaction in wheat yield",
    subtitle = "Lines connect variety means across environments",
    x = "Environment",
    y = "Mean yield (g/plot)",
    colour = "Variety"
  ) +
  theme_bw() +
  theme(legend.position = "inside", legend.position.inside = c(0.88, 0.5))
```

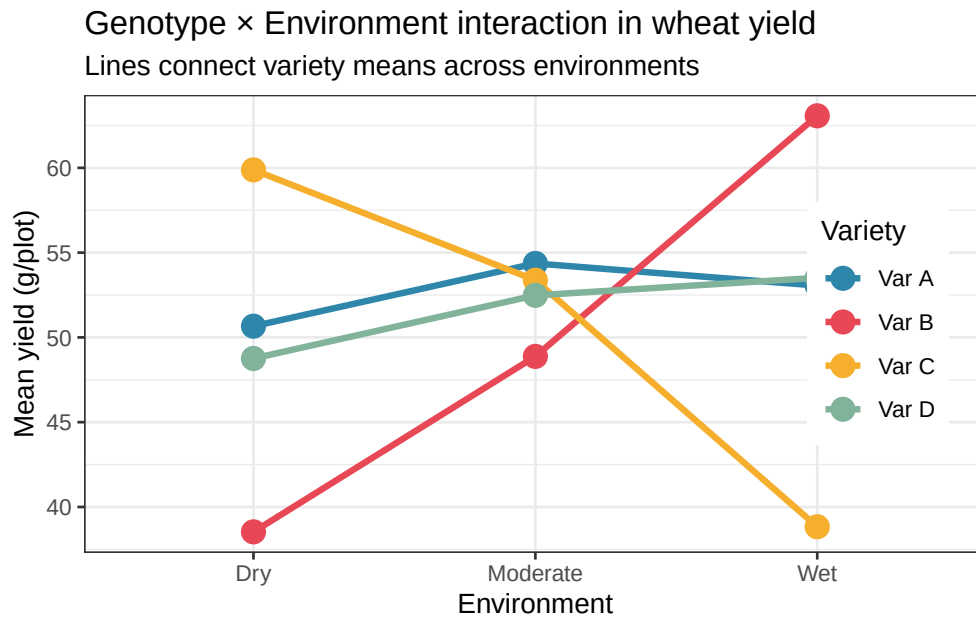


Figure 5.4: Genotype by environment interaction plot for four wheat varieties across three rain-fall environments. Lines connect the mean yield of each variety across environments. Var B and Var C show a clear crossover interaction: Var C outperforms Var B in dry conditions while Var B outperforms Var C in wet conditions. Var A and Var D are stable across environments. A breeder seeking a universally adapted variety would recommend Var A or Var D; recommendations for Var B or Var C must be environment-specific.

5.5.3 Fitting the Model

```
library(car)

options(contrasts = c("contr.sum", "contr.poly"))
fit_gxe <- lm(yield ~ variety * env, data = gxe)
Anova(fit_gxe, type = "III")
```

```
#> Anova Table (Type III tests)
```

```
#>
```

```
#> Response: yield
```

5 Two-Way ANOVA and Factorial Designs

```
#>               Sum Sq Df    F value    Pr(>F)
#> (Intercept) 157799   1 4456.4869 < 2.2e-16 ***
#> variety      55     3   0.5205    0.6702
#> env          100    2   1.4155    0.2527
#> variety:env  2676   6  12.5979 1.775e-08 ***
#> Residuals    1700  48
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
options(contrasts = c("contr.treatment", "contr.poly"))
```

The ANOVA table show three tests. We examine them in the correct order, interaction first:

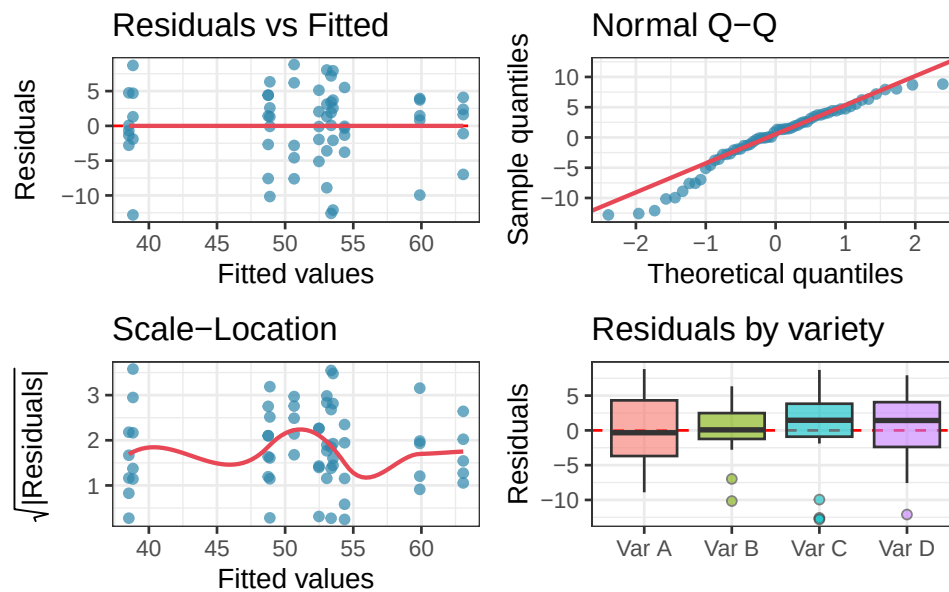
Variety × Environment interaction: its significance confirms that the yield advantage of one variety over another depends on the environment. The interaction plot already suggested this strongly for Var B and Var C. A significant interaction F test means we cannot simply state which variety is best, we must qualify the recommendation by environment.

Main effect of variety: averaged across all three environments, do the varieties differ in yield? Not significantly. With a strong G×E interaction present, this average is of limited practical use as it combines the high yields of Var B in wet conditions with its low yields in dry conditions, and the result tells a breeder very little about what to plant.

Main effect of environment: do the three environments differ in average yield? This is a nuisance factor from the breeder's perspective, of course wetter environments tend to produce higher yields, but it is important to account for it in the model so that the variety comparisons are made on a level playing field.

5.5.4 Diagnosing the Model


```
anova_diagnostics(fit_gxe, gxe)
```



```
#>
#> --- Shapiro-Wilk test (normality of residuals) ---
#>
#> Shapiro-Wilk normality test
#>
#> data: res
#> W = 0.9528, p-value = 0.02114
#>
#>
#> --- Levene's test (homogeneity of variance) ---
#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
#> group 3    0.48 0.6975
#>      56
```

```
#>
#> --- Diagnostic summary ---
#> Sample size (smallest group): 15
#> Shapiro-Wilk p = 0.0211 -> Evidence of non-normality. Check Q-Q plot for outliers.
#> Levene's p = 0.6975 -> No evidence against homoscedasticity.
```

The diagnostic plots and formal tests present a mixed but ultimately reassuring picture. Homoscedasticity is well satisfied: Levene's test returns $p = 0.698$ and the residual boxplots show similar spreads across all four varieties. The residuals vs fitted plot shows no systematic pattern, and the scale-location curve is broadly flat.

The one flag is normality. The Shapiro-Wilk test returns $p = 0.021$, and the Q-Q plot shows mild deviation at the tails, shown by a handful of points drift from the diagonal at both ends, with one notably low outlier visible in the Var B and Var C boxes. This is worth noting, but not worth losing sleep over for three reasons.

First, as Chapter ?? demonstrated, one-way and factorial ANOVA are genuinely robust to mild non-normality, particularly when sample sizes are reasonable: here we have 15 observations per variety. Second, the Q-Q plot does not show a systematic curve suggesting strong skew or a heavy-tailed distribution; the departures are driven by a small number of points at the extremes, which is common in biological data. Third, and most importantly, the Shapiro-Wilk test with $n = 60$ is sensitive enough to flag deviations that have no practical consequence for the F test. Recall from Section 2.4 that this is precisely the large-sample trap the test is prone to.

The interaction effect driving our conclusions is large and highly significant ($F_{6,48} = 12.60$, $p < 0.001$, partial ω^2 indicating a substantial effect size). A result of this magnitude is not overturned by the mild non-normality visible here. We proceed with the interpretation, noting the diagnostic findings transparently as good practice.

5.5.5 Interpreting and Reporting the Result

```
library(effects)

omega_squared(fit_gxe, partial = TRUE)

#> # Effect Size for ANOVA (Type I)
#>
#> Parameter      | Omega2 (partial) |      95% CI
#> -----
#> variety        |          0.00 | [0.00, 1.00]
#> env            |          0.01 | [0.00, 1.00]
#> variety:env     |          0.54 | [0.34, 1.00]
#>
#> - One-sided CIs: upper bound fixed at [1.00].
```

The Type III ANOVA table tells a clear and instructive story. The interaction term is overwhelmingly significant ($F_{6,48} = 12.60$, $p < 0.001$), confirming that the effect of variety on yield depends strongly on the environment in which it is grown. This is the central finding of the analysis, and it must be interpreted before anything else.

The main effects of variety ($F_{3,48} = 0.52$, $p = 0.670$) and environment ($F_{2,48} = 1.42$, $p = 0.253$) are both non-significant. This is not because variety and environment do not matter, the interaction plot and post-hoc comparisons show clearly that they do. It is because the main effects average over conditions in which the direction of the effect reverses. Var C has the highest yield in dry conditions and the lowest in wet conditions; averaging across both gives a mean that is close to the grand mean, as if variety had no effect at all. This is a textbook example of why a significant interaction renders main effects uninterpretable: the non-significant p -values for variety and environment are not a finding, they are an artefact of averaging over a crossover interaction.

The partial ω^2 values quantify the relative importance of each term after accounting for the others. The interaction accounts for by far the largest share of explainable variance, dwarfing both main effects. The residual variance reflects the within-cell plot-to-plot variation that the model cannot explain.

A complete report of this analysis reads:

A two-way ANOVA with Type III sums of squares revealed a highly significant genotype by environment interaction ($F_{6,48} = 12.60, p < 0.001$), indicating that the yield ranking of wheat varieties depends strongly on the growing environment. Main effects of variety ($F_{3,48} = 0.52, p = 0.670$) and environment ($F_{2,48} = 1.42, p = 0.253$) were non-significant, a result that reflects the reversal of variety rankings across environments rather than a genuine absence of variety or environment effects. Post-hoc comparisons within each environment (Tukey HSD) identified Var C as the highest-yielding variety under dry conditions (outperforming Var B by 21.3 g/plot, $p < 0.001$), no significant differences among varieties under moderate conditions, and Var B as the highest-yielding variety under wet conditions (outperforming Var C by 24.2 g/plot, $p < 0.001$). Varieties A and D showed stable, intermediate performance across all three environments. These results indicate that variety recommendations must be environment-specific, and that Var A or Var D should be preferred where a single widely adapted cultivar is required.

5.5.6 Post-Hoc Comparisons in Two-Way ANOVA

When the interaction is significant, post-hoc tests are most informative when conducted **within each environment separately**, comparing varieties under each condition rather than averaging across conditions:

```
library(emmeans)
```

```
emm <- emmeans(fit_gxe, ~ variety | env)
pairs(emm, adjust = "tukey")
```

```
#> env = Dry:
```

```
#> contrast      estimate    SE df t.ratio p.value
#> Var A - Var B   12.131 3.76 48   3.223 0.0118
#> Var A - Var C   -9.219 3.76 48  -2.450 0.0813
#> Var A - Var D    1.903 3.76 48   0.506 0.9573
#> Var B - Var C  -21.349 3.76 48  -5.673 <0.0001
#> Var B - Var D  -10.227 3.76 48  -2.717 0.0437
#> Var C - Var D   11.122 3.76 48   2.955 0.0241
#>
```

```
#> env = Moderate:
```

```
#> contrast      estimate    SE df t.ratio p.value
#> Var A - Var B    5.483 3.76 48   1.457 0.4711
#> Var A - Var C    0.987 3.76 48   0.262 0.9936
#> Var A - Var D    1.882 3.76 48   0.500 0.9587
#> Var B - Var C   -4.495 3.76 48  -1.194 0.6333
#> Var B - Var D   -3.601 3.76 48  -0.957 0.7743
#> Var C - Var D    0.894 3.76 48   0.238 0.9952
#>
```

```
#> env = Wet:
```

```
#> contrast      estimate    SE df t.ratio p.value
#> Var A - Var B  -10.012 3.76 48  -2.660 0.0501
#> Var A - Var C   14.224 3.76 48   3.780 0.0024
#> Var A - Var D   -0.455 3.76 48  -0.121 0.9994
#> Var B - Var C   24.236 3.76 48   6.440 <0.0001
#> Var B - Var D    9.556 3.76 48   2.539 0.0665
#> Var C - Var D  -14.680 3.76 48  -3.901 0.0016
```

```
#>
```

```
#> P value adjustment: tukey method for comparing a family of 4 estimates
```

The `~ variety | env` syntax asks for variety comparisons *conditional on* environment, exactly the right question when a $G \times E$ interaction is present.

The results confirm and sharpen the pattern visible in the interaction plot. In the **dry environment**, Var C is the clear winner: it outperforms Var B by 21.3 g/plot, the largest single pairwise difference in the entire analysis ($p < 0.0001$), and also outperforms Var A significantly ($p = 0.041$). Var A and Var D are not distinguishable from each other ($p = 0.957$), and neither is Var B from Var D ($p = 0.957$).

In the **moderate environment**, the picture changes entirely: no pair of varieties differs significantly, with all adjusted p -values exceeding 0.47. Under moderate rainfall, variety choice is essentially irrelevant, all four perform similarly.

In the **wet environment**, the ranking reverses relative to dry conditions. Var B is now the top performer, outperforming Var C by 24.2 g/plot ($p < 0.0001$) and Var A by 10.0 g/plot ($p = 0.050$). Var C, which was best in dry conditions, is now significantly outperformed by both Var A ($p = 0.002$) and Var D ($p = 0.002$).

These results illustrate precisely why a significant $G \times E$ interaction renders environment-averaged main effects uninformative. A breeder reading only the main effect of variety would see modest, averaged differences that obscure a striking reversal in rankings. The correct agronomic recommendation is environment-specific: **deploy Var C in dry regions, Var B in wet regions, and either Var A or Var D where rainfall is moderate or where a single stable variety is needed across variable conditions.**

6 Repeated Measures ANOVA

In every design considered so far, each experimental unit, each plant, each patient, each plot, contributed a single observation to the analysis. The treatment groups were independent: knowing the yield of one plant told you nothing about the yield of any other. This is the between-subjects, or independent groups, structure that one-way and two-way ANOVA are built for.

Many biological questions, however, require measuring the same individual repeatedly. How does a patient's blood pressure change over eight weeks of treatment? How does a mouse's body weight evolve across a dietary intervention? How does soil respiration in a plot respond to successive rainfall events? In all of these cases, the observations are not independent. Measurements taken on the same individual at different time points will resemble each other more than measurements taken from different individuals, simply because individuals differ from one another in stable, persistent ways.

Repeated measures ANOVA is designed for exactly this structure. It acknowledges that the same individual is measured multiple times, explicitly models the between-individual variation, and in doing so gains considerable statistical power compared to a between-subjects design of the same total size. It also introduces a new assumption, *sphericity*, that is frequently violated and even more frequently ignored.

6.1 Within-Subject Designs: Rationale and Structure

6.1.1 The Logic of Repeated Measures

The fundamental advantage of a repeated measures design is that **each individual serves as their own control**.

Consider measuring blood pressure at four time points in 20 patients. The total variation in the data has two sources: variation *between* patients (some people have naturally higher blood pressure than others, regardless of treatment) and variation *within* patients over time (the change in each person's blood pressure across the four measurements).

Between-subjects ANOVA cannot separate these two sources, it pools them into the error term. Repeated measures ANOVA partitions them explicitly. The between-patient variation is removed from the error term and accounted for separately, leaving a much smaller residual against which the within-subject effect of time is tested. This is why repeated measures designs are statistically efficient: the between-subject noise that would otherwise obscure the treatment effect is factored out entirely.

This efficiency comes with a responsibility: **the non-independence of repeated observations must be modelled correctly, not ignored**.

6.1.2 The Design Structure

In a one-way repeated measures design, n subjects each provide measurements under all k conditions (or at all k time points). The design can be represented as a two-way table with subjects as rows and conditions as columns, where every cell contains exactly one observation:

Subject	Time 1	Time 2	Time 3	Time 4
1	y_{11}	y_{12}	y_{13}	y_{14}
2	y_{21}	y_{22}	y_{23}	y_{24}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

6 Repeated Measures ANOVA

Subject	Time 1	Time 2	Time 3	Time 4
n	y_{n1}	y_{n2}	y_{n3}	y_{n4}

The statistical model is:

$$y_{ij} = \mu + \alpha_i + \pi_j + \varepsilon_{ij}$$

where α_i is the effect of condition i (the within-subject factor), π_j is the effect of subject j (the between-subject random effect, capturing stable individual differences), and ε_{ij} is the residual. The key point is that π_j is estimated and removed before testing α_i : this is the partitioning that gives repeated measures its power advantage.

6.1.3 Between-Subjects vs Within-Subjects Factors

Not all factors in a repeated measures design need to be within-subject. In a **mixed design** (not to be confused with mixed-effects models, though the connection is real), some factors are between-subjects and some are within-subjects. For example:

- **Between-subjects factor:** treatment group (patients are assigned to either drug or placebo, they cannot be in both).
- **Within-subjects factor:** time (all patients are measured at baseline, week 4, and week 8).

This is the most common structure in clinical biology: a treatment group factor (between) crossed with a time factor (within). The analysis produces three tests: the main effect of group, the main effect of time, and the group-by-time interaction, the last being typically the most scientifically interesting, as it tests whether the trajectory of change over time differs between treatment groups.

6.1.4 When to Use Repeated Measures ANOVA

Repeated measures ANOVA is appropriate when:

- The same individual is measured at multiple time points or under multiple conditions.
- The order of conditions is either randomised (crossover design) or irrelevant to the research question (longitudinal time course).
- The number of repeated measurements is modest (roughly 2-6). With many time points, linear mixed models (Chapter ??) are more flexible and generally preferable.
- The assumption of sphericity (Section ??) is either satisfied or corrected for.

It is **not** appropriate when:

- The repeated measurements are pseudoreplicates: multiple measurements of the same individual that were never intended to capture genuine within-individual change (see Section ??).
- The dropout rate is substantial: repeated measures ANOVA requires complete data, and list-wise deletion of incomplete cases can introduce serious bias. Mixed models handle missing data more gracefully.

6.2 Sphericity: The Forgotten Assumption

6.2.1 What Is Sphericity?

Sphericity is the assumption specific to repeated measures ANOVA, and it is the one most routinely overlooked. Understanding it requires first understanding what the repeated measures F test is actually doing.

In a between-subjects one-way ANOVA, the single error term, MS_{within} , is appropriate because all observations in the same group were collected independently, and the variance is assumed equal across groups (homoscedasticity). In repeated measures ANOVA, the single within-subject error term is appropriate only if a stronger condition holds: not only must the

variances of the repeated measurements be equal across time points, but the **covariances** between every pair of time points must also be equal.

More precisely, the assumption requires that the variance of the *differences* between every pair of time points is the same. If you have four time points, there are $\binom{4}{2} = 6$ pairwise differences, and sphericity requires all six to have equal variance.

This is called the **sphericity assumption** (also known as the Huynh-Feldt condition), and it is a generalisation of the compound symmetry assumption. Compound symmetry, equal variances and equal covariances everywhere, implies sphericity, but sphericity is weaker: it can hold even when compound symmetry does not.

6.2.2 Why Does Violation Matter?

When sphericity is violated, the F test in repeated measures ANOVA becomes positively biased: it produces too many significant results. The degrees of freedom used to construct the reference F distribution are inflated, making the test anti-conservative. The simulation below demonstrates this:

```
library(future)
library(furrr)
library(purrr)

plan(multisession)

set.seed(42)
n_sim <- 5000
alpha <- 0.05
n <- 20    # subjects
k <- 4     # time points
```

6 Repeated Measures ANOVA

```
# Function to simulate one dataset and return p-value
# from standard repeated measures ANOVA (via aov with Error term)

run_rm_anova <- function(sigma_matrix) {
  library(MASS)
  # Simulate data with given covariance structure
  Y      <- mvrnorm(n, mu = rep(0, k), Sigma = sigma_matrix)
  df_long <- data.frame(
    y      = as.vector(t(Y)),
    subject = factor(rep(1:n, each = k)),
    time    = factor(rep(1:k, n))
  )
  fit <- aov(y ~ time + Error(subject/time), data = df_long)
  summary(fit)$`Error: subject:time`[[1]][["Pr(>F)"]][1]
}

# Sphericity satisfied: compound symmetry
sigma_cs <- matrix(0.5, k, k)
diag(sigma_cs) <- 1

# Sphericity violated: variance increases with time
sigma_het <- diag(c(1, 2, 4, 8))
sigma_het[sigma_het == 0] <- 0.5

p_sphere <- future_map_dbl(seq_len(n_sim), ~run_rm_anova(sigma_cs), .options = furrr_options(see
p_nospher <- future_map_dbl(seq_len(n_sim), ~run_rm_anova(sigma_het), .options = furrr_options(se

plan(sequential)
```

```
cat(
  "False positive rate (sphericity satisfied):",
  round(mean(p_sphere < alpha, na.rm = TRUE), 3),
  "\nFalse positive rate (sphericity violated):",
  round(mean(p_nospher < alpha, na.rm = TRUE), 3), "\n"
)
```

```
#> False positive rate (sphericity satisfied): 0.047
```

```
#> False positive rate (sphericity violated): 0.078
```

The false positive rate under sphericity violation is above 5%, confirming that the standard F test cannot be trusted when the assumption fails.

6.2.3 Mauchly's Test

Mauchly's test is the standard formal test of the sphericity assumption. It tests the null hypothesis that the variance-covariance matrix of the repeated measurements has the sphericity property.

```
# Mauchly's test is produced automatically by summary()
# when using the idata / mauchly argument in car::Anova

library(car)

# Fit the model as a multivariate linear model first
# (required for Mauchly's test via car)
fit_mlm <- lm(cbind(t1, t2, t3, t4) ~ 1, data = df_wide)
idata <- data.frame(time = factor(1:k))
```

```
rm_model <- Anova(fit_mlm, idata = idata, idesign = ~ time, type = "III")

summary(rm_model, multivariate = FALSE)

# Mauchly's test and epsilon corrections appear automatically
```

Mauchly's test shares the same sample-size sensitivity as the Shapiro-Wilk and Levene tests discussed in Chapter 2. With small samples it has low power to detect real violations; with large samples it will flag trivially small departures. As always, the test result should inform your judgement rather than make the decision for you. With $k \geq 3$ time points and any doubt about sphericity, apply the corrections described below regardless of the Mauchly test outcome.

6.2.4 ε Corrections

When sphericity is violated, or when you cannot confidently rule out a violation, the appropriate response is to adjust the degrees of freedom of the F test using an **epsilon (ε) correction**. The correction factor ε ($0 < \varepsilon \leq 1$) measures the degree of departure from sphericity: $\varepsilon = 1$ means perfect sphericity; smaller values indicate greater violation.

The corrected degrees of freedom are:

$$df_1^* = \varepsilon(k - 1) \quad df_2^* = \varepsilon(k - 1)(n - 1)$$

Two estimators of ε are in common use.

Greenhouse-Geisser ($\hat{\varepsilon}_{GG}$): a conservative estimator that can be very low when sphericity is badly violated. It tends to overcorrect with small samples, making the test too conservative.

Huynh-Feldt ($\tilde{\varepsilon}_{HF}$): a less conservative estimator that performs better in small to moderate samples. It is capped at 1: if the estimate exceeds 1, it is set to 1.

The conventional guidance is:

- If $\hat{\epsilon}_{GG} > 0.75$: use Huynh-Feldt.
- If $\hat{\epsilon}_{GG} \leq 0.75$: use Greenhouse-Geisser.
- If $\hat{\epsilon}_{GG}$ is very low (< 0.5): consider a multivariate approach (MANOVA) or a linear mixed model instead.

Both corrections are produced automatically by `car::Anova()` with the `multivariate = FALSE` option, as shown in the example below.

6.3 One-Way and Two-Way Repeated Measures in R

6.3.1 One-Way Repeated Measures

The simplest case: one within-subject factor (e.g. time) and no between-subject factor. In base R, the `Error()` term in `aov()` specifies the within-subject structure:

```
# Long format required
# subject: factor identifying each individual
# time:    within-subject factor
# y:       response variable

fit_rm <- aov(y ~ time + Error(subject/time), data = df_long)
summary(fit_rm)
```

For Mauchly's test and epsilon corrections, the `car` package approach is more complete:

```
library(car)

# Reshape to wide format for car::Anova
# Each row is one subject; columns are time points
fit_mlm <- lm(cbind(t1, t2, t3, t4) ~ 1, data = df_wide)
```

```
idata    <- data.frame(time = factor(1:4))
rm_model <- Anova(fit_mlm,
                  idata    = idata,
                  idesign  = ~ time,
                  type     = "III")

# Full output including Mauchly's test and epsilon corrections
summary(rm_model, multivariate = FALSE)
```

6.3.2 Two-Way Repeated Measures (Mixed Design)

The most common design in clinical biology: one between-subjects factor (group) and one within-subjects factor (time):

```
# group: between-subjects factor (different patients in each group)
# time: within-subjects factor (all patients measured at each time)
# subject: factor identifying each individual

fit_mixed <- aov(y ~ group * time + Error(subject/time),
                data = df_long)
summary(fit_mixed)
```

For the car approach with epsilon corrections in a mixed design:

```
library(car)

fit_mlm2 <- lm(cbind(t1, t2, t3, t4) ~ group, data = df_wide)
idata    <- data.frame(time = factor(1:4))
rm_model2 <- Anova(fit_mlm2,
                  idata    = idata,
```



```

      idesign = ~ time,
      type    = "III")

summary(rm_model2, multivariate = FALSE)

```

6.4 Example: Longitudinal Measurements in Clinical Biology

6.4.1 The Study

A clinical trial follows 40 patients recovering from cardiac surgery. Twenty patients are randomly assigned to a standard rehabilitation programme (group level Control) and twenty to an intensive cardiac rehabilitation programme (group level Intervention) (Section ??). Resting heart rate (hr, beats per minute) is measured at four time points (time): baseline (week 0), week 4, week 8, and week 12. The primary question is whether the two groups differ in their trajectory of heart rate reduction over the 12-week period that is, whether there is a group-by-time interaction.

```
summary(cardiac_recovery)
```

```

#>      subject      group  time      hr      id
#> 1      : 4  Control    :80  0 :40  Min.   :58.80  Min.    : 1.00
#> 2      : 4  Intervention:80  4 :40  1st Qu.:71.66  1st Qu.:10.75
#> 3      : 4                      8 :40  Median :77.09  Median :20.50
#> 4      : 4                      12:40  Mean    :76.32  Mean    :20.50
#> 5      : 4                      3rd Qu.:80.38  3rd Qu.:30.25
#> 6      : 4                      Max.    :93.61  Max.    :40.00
#> (Other):136

```

6.4.2 Visualising the Data

```

library(ggplot2)

# Group means per time point
summ <- aggregate(hr ~ group + time, data = cardiac_recovery, FUN = mean)
summ$time_num <- as.numeric(as.character(summ$time))

cardiac_recovery$time_num <- as.numeric(as.character(cardiac_recovery$time))

ggplot() +
  # Individual trajectories
  geom_line(data = cardiac_recovery, aes(x = time_num, y = hr, group = subject, colour = group),
  # Group mean trajectories
  geom_line(data = summ, aes(x = time_num, y = hr, colour = group, group = group), linewidth = 1.5),
  geom_point(data = summ, aes(x = time_num, y = hr, colour = group), size = 4) +
  scale_colour_manual(values = c("#2E86AB", "#E84855")) +
  scale_x_continuous(breaks = c(0, 4, 8, 12), labels = paste0("Week ", c(0, 4, 8, 12))) +
  labs(x = NULL, y = "Resting heart rate (bpm)", colour = "Group") +
  theme_bw() +
  theme(legend.position = "inside", legend.position.inside = c(0.85, 0.85))

```

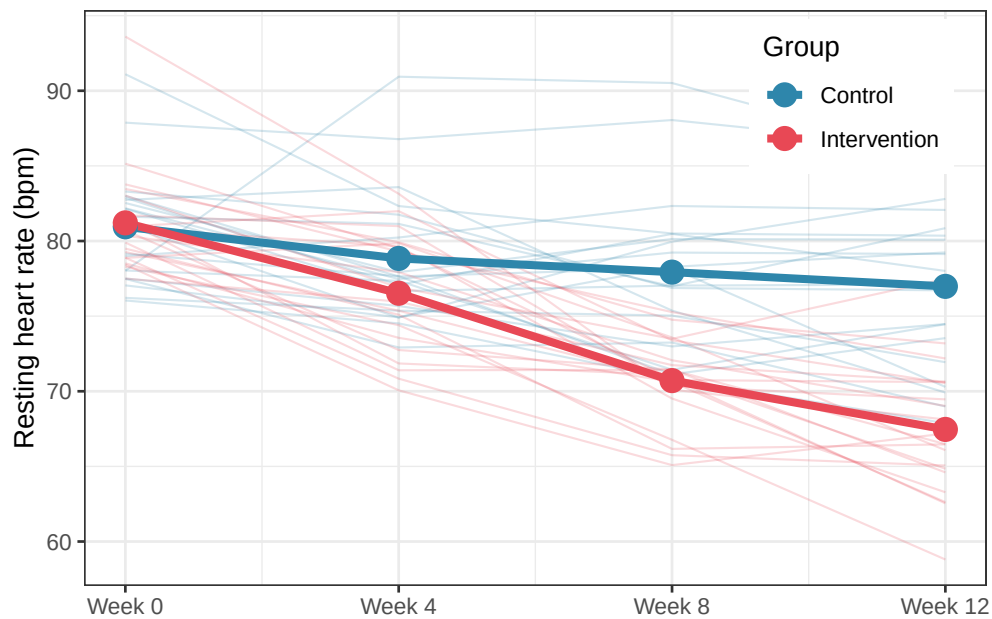


Figure 6.1: Mean resting heart rate trajectories for control and intervention groups across four time points. Thin lines show individual patient trajectories; thick lines connect group means. The intervention group shows a steeper and more sustained decline in heart rate from week 4 onward, suggesting a group-by-time interaction. The parallel trajectories at baseline confirm that the two groups were comparable at the start of the trial.

6.4.3 Checking Sphericity

```
library(car)

# car::Anova requires wide format with group as a between-subjects predictor
fit_mlm <- lm(cbind(t1, t2, t3, t4) ~ group, data = cardiac_recovery_wide)
idata <- data.frame(time = factor(c("t1", "t2", "t3", "t4")))

rm_model <- Anova(fit_mlm, idata = idata, idesign = ~ time, type = "III")

# Full output: Mauchly's test + epsilon corrections + F tests
```

6 Repeated Measures ANOVA

```
summary(rm_model, multivariate = FALSE)
```

```
#>
#> Univariate Type III Repeated-Measures ANOVA Assuming Sphericity
#>
#>          Sum Sq num Df Error SS den Df    F value    Pr(>F)
#> (Intercept) 495174      1   1757.9    38 10704.0589 < 2.2e-16 ***
#> group          883      1   1757.9    38   19.0945 9.292e-05 ***
#> time           172      3   1027.7   114    6.3681 0.000498 ***
#> group:time     599      3   1027.7   114   22.1398 2.280e-11 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#>
#> Mauchly Tests for Sphericity
#>
#>          Test statistic p-value
#> time          0.67608 0.01343
#> group:time     0.67608 0.01343
#>
#>
#> Greenhouse-Geisser and Huynh-Feldt Corrections
#> for Departure from Sphericity
#>
#>          GG eps Pr(>F[GG])
#> time      0.80607  0.001348 **
#> group:time 0.80607  1.377e-09 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

6 Repeated Measures ANOVA

```
#>
#>           HF eps   Pr(>F[HF])
#> time       0.864768 9.960634e-04
#> group:time 0.864768 3.972709e-10
```

Mauchly's test is significant ($W = 0.676$, $p = 0.013$), indicating that the sphericity assumption is violated. The variances of the pairwise differences between time points are not equal across all combinations, which means the standard F test degrees of freedom are inflated and uncorrected p-values cannot be trusted.

The Greenhouse-Geisser estimate is $\hat{\epsilon}_{GG} = 0.806$. Since this exceeds the 0.75 threshold, the Huynh-Feldt correction is preferred over the more conservative Greenhouse-Geisser. The Huynh-Feldt estimate is $\tilde{\epsilon}_{HF} = 0.865$, reasonably close to 1, indicating that the departure from sphericity is moderate rather than severe. We therefore use the Huynh-Feldt corrected p-values for all within-subject tests.

The corrected ANOVA table tells a clear story. The group-by-time interaction is highly significant ($F_{2.59,98.6} = 22.14$, $p = 3.97 \times 10^{-10}$, using Huynh-Feldt corrected degrees of freedom), confirming that the two rehabilitation programmes produce different heart rate trajectories over the 12-week period. This is the primary finding of the study. The main effect of time is also significant ($F_{2.59,98.6} = 6.37$, $p < 0.001$), indicating that heart rate declines over the course of rehabilitation when averaged across both groups. The main effect of group is significant as well ($F_{1,38} = 19.09$, $p < 0.001$), though as noted earlier this averages over a trajectory that differs between groups and should not be interpreted in isolation from the interaction.

Two things are worth noting for students reading this output for the first time. First, the uncorrected p-values (assuming sphericity) and the corrected p-values tell the same qualitative story here: all three effects remain highly significant after correction. This is because the violation is moderate ($\tilde{\epsilon}_{HF} = 0.865$) rather than severe. With a more serious violation, the correction can shift a borderline result from significant to non-significant, which is precisely why it must not be skipped. Second, the degrees of freedom in the corrected tests are no longer whole numbers,

they are $\hat{\varepsilon} \times df_{\text{uncorrected}}$. This is not a typographical error; it is the mathematical consequence of the epsilon adjustment and is entirely normal in repeated measures output.

6.4.4 Fitting the Model and Interpreting Results

```
# Base R aov() approach for the ANOVA table
fit_rm <- aov(hr ~ group * time + Error(subject/time), data = cardiac_recovery)
summary(fit_rm)
```

```
#>
#> Error: subject
#>           Df Sum Sq Mean Sq F value    Pr(>F)
#> group      1  883.3   883.3    19.09 9.29e-05 ***
#> Residuals 38 1757.9    46.3
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Error: subject:time
#>           Df Sum Sq Mean Sq F value    Pr(>F)
#> time        3 1811.6   603.9   66.98 < 2e-16 ***
#> group:time   3  598.8   199.6   22.14 2.28e-11 ***
#> Residuals 114 1027.7     9.0
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The output is split into two error strata, which is the defining structural feature of repeated measures ANOVA output in R and the first thing students need to learn to read correctly.

The **Error: subject stratum** captures the between-subjects variation i.e. differences among individuals that persist across all time points. It contains the test for the between-subjects

factor group. The intervention group differs significantly from the control group in its overall mean heart rate ($F_{1,38} = 19.09, p < 0.001$), but as discussed above, this average difference collapsed across all four time points is not the primary question. The residual mean square in this stratum (46.3 bpm²) estimates the stable between-patient variance: the variation in baseline heart rate that exists among individuals regardless of which programme they followed.

The **Error: subject:time stratum** captures the within-subjects variation i.e. how each individual's heart rate changes across time points. It contains the tests for the within-subjects factor time and the group:time interaction. The residual mean square here (9.0 bpm²) is dramatically smaller than the between-subjects residual (46.3 bpm²), illustrating concretely the power advantage of repeated measures designs: the stable individual differences that would inflate the error term in a between-subjects design have been partitioned out, leaving a much smaller noise floor against which the within-subjects effects are tested.

The main effect of time is highly significant ($F_{3,114} = 66.98, p < 0.001$), confirming that heart rate declines substantially over the 12-week rehabilitation period when averaged across both groups. This is not surprising, indeed rehabilitation of any kind is expected to improve cardiovascular fitness, but it is a necessary finding that validates the study's measurement protocol.

Most importantly, the group-by-time interaction is highly significant ($F_{3,114} = 22.14, p < 0.001$). This is the central result: the trajectory of heart rate change over 12 weeks differs between the two rehabilitation programmes. The intensive programme produces a steeper and more sustained decline in resting heart rate than the standard programme, and this difference cannot be attributed to chance. Note that these are the uncorrected degrees of freedom, since Mauchly's test was significant, the Huynh-Feldt corrected p-values from `car::Anova()` should be reported, which remain highly significant ($p = 3.97 \times 10^{-10}$) and lead to the same conclusion.

It is also worth noting the ratio of the two residual mean squares. The between-subjects residual (46.3 bpm²) is more than five times larger than the within-subjects residual (9.0 bpm²). This tells us that individuals differ substantially from one another in their baseline heart rate, but

that within any given individual the measurement-to-measurement variation is much smaller. This is exactly the pattern that makes repeated measures designs efficient: by tracking individuals over time rather than comparing independent groups, we are testing the treatment effect against within-person variability rather than between-person variability, gaining statistical sensitivity as a direct consequence.

6.4.5 Effect Sizes and Reporting

```
library(effectsize)

eta_squared(fit_rm, partial = TRUE)
```

```
#> # Effect Size for ANOVA (Type I)
#>
#> Group          | Parameter | Eta2 (partial) | 95% CI
#> -----
#> subject        | group    | 0.33 | [0.14, 1.00]
#> subject:time    | time     | 0.64 | [0.55, 1.00]
#> subject:time    | group:time | 0.37 | [0.25, 1.00]
#>
#> - One-sided CIs: upper bound fixed at [1.00].
```

The partial η^2 values reveal the relative importance of each effect in the model. Time explains the largest share of within-subject variance (partial $\eta^2 = 0.64$, 95% CI [0.55, 1.00]), indicating that 64% of the within-subject variation in heart rate is accounted for by the progression across the 12 weeks which is a large effect by any standard, reflecting the genuine and consistent improvement in cardiovascular fitness that rehabilitation produces regardless of programme type.

The group-by-time interaction has a partial $\eta^2 = 0.37$ (95% CI [0.25, 1.00]), a large effect indicating that 37% of the within-subject variance not explained by time alone is attributable to the differential trajectory of the two groups. This is the scientifically meaningful result: not only does heart rate improve over time, but it improves substantially more, and more rapidly, in the intensive rehabilitation group than in the control group.

The between-subjects effect of group has a partial $\eta^2 = 0.33$ (95% CI [0.14, 1.00]), indicating that 33% of the stable between-patient variance is explained by group membership. This reflects the sustained overall difference in heart rate between the two groups when averaged across all time points.

A note on the confidence intervals: all upper bounds are fixed at 1.00, which is expected for one-sided confidence intervals on η^2 as the parameter is bounded above by 1 by definition. The lower bounds are the more informative values, confirming that even at the conservative end of the interval, all three effects are meaningfully large.

The complete report of this analysis reads:

A two-way mixed repeated measures ANOVA examined the effect of rehabilitation programme (between-subjects: control vs intervention, $n = 20$ per group) and time (within-subjects: weeks 0, 4, 8, 12) on resting heart rate. Mauchly's test indicated a significant violation of sphericity ($W = 0.676$, $p = 0.013$); Huynh-Feldt corrected degrees of freedom were therefore applied to all within-subjects tests ($\tilde{\epsilon}_{HF} = 0.865$). A highly significant group-by-time interaction was found ($F_{2.59, 98.6} = 22.14$, $p = 3.97 \times 10^{-10}$, partial $\eta^2 = 0.37$), indicating that the intensive rehabilitation programme produced a steeper decline in resting heart rate over the 12-week period than the standard programme. A significant main effect of time confirmed that heart rate declined across the study period in both groups ($F_{2.59, 98.6} = 6.37$, $p < 0.001$, partial $\eta^2 = 0.64$). The main effect of group was also significant ($F_{1, 38} = 19.09$, $p < 0.001$, partial $\eta^2 = 0.33$), reflecting the overall lower mean heart rate in the intervention group across all time points. These

results support the conclusion that intensive cardiac rehabilitation produces clinically meaningful improvements in resting heart rate beyond those achieved by standard care alone.

6.4.6 A Note on Missing Data

Repeated measures ANOVA as implemented in `aov()` and `car::Anova()` requires complete data: every subject must have a measurement at every time point. If any observations are missing, the subject is excluded entirely from the analysis. In a 12-week clinical trial, dropout is almost inevitable: patients move, recover, or withdraw consent.

When missing data are present, the appropriate tool is a **linear mixed-effects model** (Chapter 9), which can handle incomplete longitudinal data under the assumption that missingness is random (the missing-at-random assumption). The mixed model fitted to complete data will give identical results to repeated measures ANOVA in the balanced case, confirming that repeated measures ANOVA is simply a special case of the linear mixed model, a theme that runs throughout Part III of this book.

7 Split-Plot and Nested Designs

Every ANOVA design considered so far has shared a common simplifying assumption: that the experimental units receiving different treatments are all of the same type, at the same level of organisation, and subject to the same sources of variation. A plant is a plant, a patient is a patient, a plot is a plot.

Biological reality is rarely so flat. Ecosystems are organised in layers: individual organisms live within plots, plots within fields, fields within regions. Clinical trials recruit patients from hospitals, hospitals from healthcare systems. Agricultural experiments apply whole-farm treatments to fields and within-field treatments to plots. In all of these cases, the experimental units exist at multiple levels of a hierarchy, and treatments are applied, and therefore replicated, at different levels of that hierarchy.

Ignoring this hierarchical structure leads directly to the pseudoreplication problem introduced in Chapter 2. Accounting for it correctly requires designs and analyses that explicitly represent the nested structure of the data. This chapter introduces the two most important such designs in biological research: the split-plot and the nested ANOVA.

7.1 Hierarchy in Biological Data: Why It Cannot Be Ignored

7.1.1 Everything Is Nested in Something

Before introducing any new design, it is worth pausing to build a clear intuition for what hierarchical structure means and why it demands special treatment.

7 Split-Plot and Nested Designs

Consider a simple example. A researcher wants to study the effect of two soil treatments (amended vs unamended) and two plant varieties (A vs B) on crop yield. She has access to four large fields. Two fields receive the soil amendment; two do not. Within each field, she establishes four plots: two planted with Variety A and two with Variety B. She measures yield from five plants within each plot.

The data have three levels of organisation:

$$\text{Field} \supset \text{Plot} \supset \text{Plant}$$

This is a hierarchy. Fields are nested within soil treatment. Plots are nested within fields. Plants are nested within plots. Each level has its own sources of variation: fields differ from each other in ways that have nothing to do with soil treatment (drainage, history, microclimate); plots within a field differ from each other in ways that have nothing to do with variety (position, local soil variation); plants within a plot differ from each other due to individual biological variation.

The critical point is that **the treatment is applied and replicated at the field level, not the plot level and not the plant level**. The soil amendment was not randomly assigned to individual plants: *it was assigned to whole fields*. The field is therefore the unit of replication for the soil treatment comparison, and there are only two replicates per treatment ($n = 2$ fields per treatment), not $2 \times 4 \times 5 = 40$.

7.1.2 What Goes Wrong When You Ignore the Hierarchy

If the researcher analyses this experiment by running a standard ANOVA on all $4 \times 4 \times 5 = 80$ plant measurements, treating each plant as an independent observation, she is pseudoreplicating at two levels simultaneously. The within-field and within-plot correlation structures are ignored, and the error term used to test the soil treatment effect is the plant-level residual, which is far too small, because it does not account for the field-to-field variation that is the appropriate noise against which the treatment should be judged.

The simulation below demonstrates the inflation of the false positive rate that results:

```
library(future)
library(furrr)
library(purrr)

plan(multisession)

set.seed(42)
n_sim <- 5000
alpha <- 0.05
n_fields <- 4      # 2 per treatment
n_plots <- 4       # per field
n_plants <- 5      # per plot
field_sd <- 3      # field-to-field variation
plot_sd <- 2       # plot-to-plot variation within field
plant_sd <- 1      # plant-to-plant variation within plot

results <- future_map_dfr(seq_len(n_sim), function(i) {

  treatment <- rep(c("Control", "Amended"), each = n_fields / 2)
  field_effect <- rnorm(n_fields, 0, field_sd)

  df <- do.call(rbind, lapply(seq_len(n_fields), function(f) {
    do.call(rbind, lapply(seq_len(n_plots), function(p) {
      plot_effect <- rnorm(1, 0, plot_sd)
      data.frame(
        yield = rnorm(n_plants, mean = 50 + field_effect[f] + plot_effect, sd = plant_sd),
        treatment = treatment[f],
        field = factor(f),
```

```

    plot = factor(paste(f, p, sep = "_"))
  )
}))
}))

# Wrong: treat all plants as independent
p_wrong <- summary(
  aov(yield ~ treatment, data = df)
)[[1]][["Pr(>F)"]][1]

# Correct: use field means as the unit of analysis
field_means <- aggregate(yield ~ treatment + field,
  data = df, FUN = mean)
p_correct <- summary(
  aov(yield ~ treatment, data = field_means)
)[[1]][["Pr(>F)"]][1]

data.frame(p_wrong = p_wrong, p_correct = p_correct)

}, .options = furrr_options(seed = TRUE))

plan(sequential)

```

```

# View the simulated data
head(results, n = 10)

```

```

#>      p_wrong p_correct
#> 1  1.713975e-02 0.4803663
#> 2  1.914645e-05 0.3266718

```

```
#> 3  9.265631e-01 0.9875177
#> 4  7.418683e-09 0.2185384
#> 5  7.697033e-05 0.4926903
#> 6  8.143981e-01 0.9355019
#> 7  2.013291e-01 0.8170871
#> 8  1.935473e-01 0.8389504
#> 9  4.284297e-20 0.1088934
#> 10 4.178010e-01 0.8922692
```

```
cat(
  "False positive rate ignoring hierarchy:",
  round(mean(results$p_wrong < alpha), 3),
  "\nFalse positive rate correct analysis:",
  round(mean(results$p_correct < alpha), 3), "\n"
)
```

```
#> False positive rate ignoring hierarchy: 0.681
#> False positive rate correct analysis: 0.05
```

The false positive rate from the naive analysis is far above 5% (0.681), while the correct analysis, using field means as the unit of analysis for the field-level treatment, stays at the nominal level. This is the pseudoreplication problem from Chapter 2, now seen in a hierarchical context.

7.1.3 Visualising the Hierarchy

It helps to draw the structure of a hierarchical experiment explicitly before deciding how to analyse it. For the field-plot-plant example:

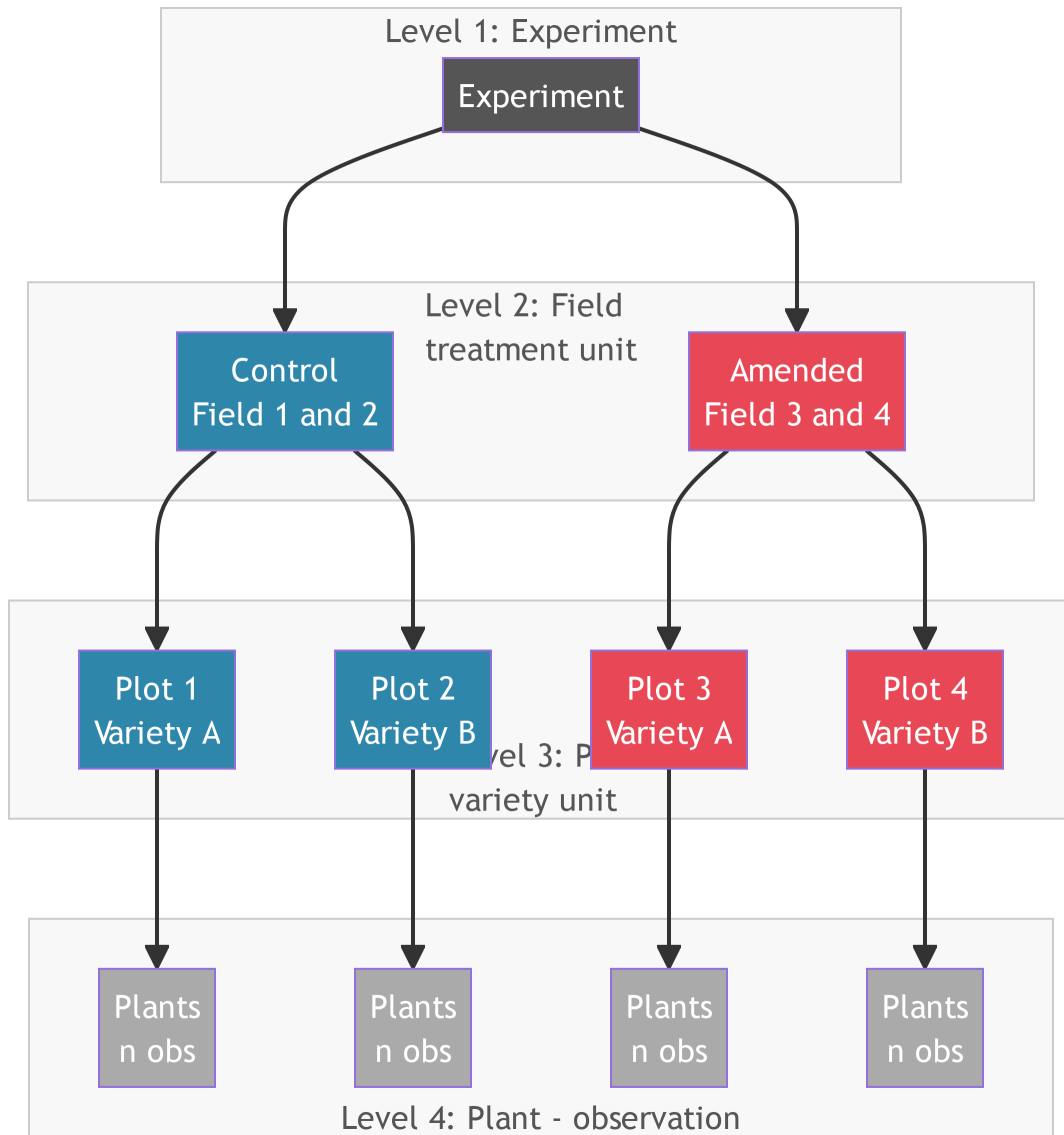


Figure 7.1: Hierarchical structure of the field experiment. Soil treatment is applied at the field level and replicated across fields. Variety is applied at the plot level within each field. Plants are the observational unit at the bottom of the hierarchy.

7.1.4 The Golden Rule of Hierarchical Analysis

The rule that prevents most hierarchical analysis errors can be stated simply:

The error term used to test a treatment effect must come from the same level of the hierarchy at which that treatment was applied and randomised.

If soil amendment was randomised across fields, the field-to-field variation is the correct error term for testing the amendment effect. If variety was randomised across plots within fields, the plot-to-plot variation within fields is the correct error term for testing the variety effect. Using plant-level variation to test a field-level treatment is like using a ruler calibrated in millimetres to measure a distance that varies in kilometres, the instrument is simply measuring the wrong thing.

This rule is the conceptual foundation of both split-plot and nested ANOVA designs, which formalise it into a statistical framework.

7.2 Pseudoreplication: The Silent Epidemic in Biology

7.2.1 A Pervasive Problem

In their landmark 1984 paper, Hurlbert (?) documented that pseudoreplication, the treatment of non-independent observations as independent replicates, was present in a substantial fraction of published ecological field experiments. Subsequent reviews have found similar rates across ecology, agriculture, and biomedical research. The problem has not gone away.

Pseudoreplication takes several distinct forms in practice, and being able to recognise each is an essential skill.

Simple pseudoreplication occurs when multiple measurements are taken from a single experimental unit and treated as independent replicates. Six measurements of soil nitrogen from the same plot do not give you six independent replicates, they give you six estimates of the nitrogen level in one plot. The plot is still $n = 1$.

Sacrificial pseudoreplication occurs when the analysis is conducted at the wrong level of the hierarchy, as in the field example above. The name reflects the fact that the correct, but lower-powered, analysis is sacrificed in favour of an inflated, incorrect one.

Temporal pseudoreplication occurs when the same unit is measured repeatedly over time and the measurements are treated as independent. This is the repeated measures problem from Chapter 6 seen in a field ecology context: a plot measured in spring, summer, and autumn provides three measurements of the same plot, not three independent plots.

7.2.2 How to Avoid It

The antidote to pseudoreplication is to identify the experimental unit, the smallest unit that was independently assigned to a treatment, before the analysis begins, and to ensure that the unit of analysis matches the unit of replication. When subsampling within units is necessary (for precision, or because the response must be measured at a finer scale than the treatment was applied), the subsamples should be averaged to the level of the experimental unit before analysis, or the hierarchical structure should be modelled explicitly.

7.3 Split-Plot Design: Structure and Correct Error Terms

7.3.1 When Does a Split-Plot Design Arise?

A split-plot design arises naturally when two factors must be tested but one is much more difficult or expensive to change than the other. The classic agricultural example: irrigation regime (irrigated vs rainfed) must be applied to whole large fields because irrigation infrastructure cannot be changed plot by plot, while fertiliser type can be applied to much smaller individual plots within each field.

The experiment therefore applies the hard-to-change factor (**whole-plot factor**) to large units (**whole plots**), then subdivides each whole plot into smaller units (**sub-plots**) and applies the easy-to-change factor (**sub-plot factor**) to those.

This structure is not merely a practical compromise, it is a deliberate design choice that must be reflected in the analysis. The two factors are tested against different error terms, because they were randomised at different levels of the hierarchy.

7.3.2 Structure of the Split-Plot Design

With a levels of the whole-plot factor A , b levels of the sub-plot factor B , and r whole-plot replicates per level of A :

- There are $a \times r$ whole plots in total.
- Each whole plot is divided into b sub-plots.
- Total observations: $a \times r \times b$.

The two error terms are:

- **Whole-plot error** (MS_{WP}): the variation among whole plots within the same level of A . This is the correct error term for testing the main effect of A .
- **Sub-plot error** (MS_{SP}): the variation among sub-plots within the same whole plot. This is the correct error term for testing the main effect of B and the $A \times B$ interaction.

The sub-plot error is typically smaller than the whole-plot error, which is why the sub-plot factor and the interaction are tested with more power than the whole-plot factor, a direct consequence of the design structure.

7.3.3 The Split-Plot Model

$$y_{ijk} = \mu + \alpha_i + \delta_{ij} + \beta_k + (\alpha\beta)_{ik} + \varepsilon_{ijk}$$

where α_i is the whole-plot treatment effect, δ_{ij} is the whole-plot error (variation among whole plots within treatment i , the error term for A), β_k is the sub-plot treatment effect, $(\alpha\beta)_{ik}$ is the interaction, and ε_{ijk} is the sub-plot error (the error term for B and $A \times B$).

In R, the two error terms are specified explicitly using the `Error()` term in `aov()`:

```
# whole.plot: the unit receiving the whole-plot treatment
# A: whole-plot factor
# B: sub-plot factor

fit_sp <- aov(y ~ A * B + Error(whole.plot/B), data = df)
summary(fit_sp)
```

7.4 Nested ANOVA: Subsampling vs True Replication

7.4.1 The Nested Structure

A nested design arises when the levels of one factor exist entirely within the levels of another, and the levels of the inner factor are not the same across levels of the outer factor. Fields contain plots, but the plots in Field 1 are different from the plots in Field 2 as they are nested within their respective fields, not crossed with them.

This is distinct from a factorial design, where every level of every factor is combined with every level of every other factor. In a factorial design, Variety A appears in both the irrigated and rainfed fields. In a nested design, the plots in Field 1 are unique to Field 1.

The nested ANOVA model for two levels of nesting:

$$y_{ijk} = \mu + \alpha_i + \beta_{j(i)} + \varepsilon_{ijk}$$

where α_i is the effect of the outer factor A at level i , $\beta_{j(i)}$ is the effect of the inner factor B at level j nested within level i of A (denoted $j(i)$), and ε_{ijk} is the residual.

In R:

```
# B is nested within A
fit_nested <- aov(y ~ A + B %in% A, data = df)
# Equivalently:
fit_nested <- aov(y ~ A/B, data = df)
summary(fit_nested)
```

7.4.2 Subsampling vs True Replication

The most important practical distinction in nested designs is between **true replication** and **subsampling**. True replicates are independent experimental units that each received a treatment independently. Subsamples are multiple measurements taken from the same experimental unit.

	True replicates	Subsamples
Definition	Independently treated units	Multiple measures of one unit
Example	Four separately irrigated fields	Five soil cores from one field
Contribution to n	Each counts as one replicate	Together count as one replicate
Error term	Between-unit residual	Within-unit residual

Treating subsamples as true replicates is the most common form of sacrificial pseudoreplication. The nested ANOVA correctly partitions the variation into between-unit and within-unit components and uses the appropriate error term for each.

7.5 Example 1: Grazing Effects on Plant Diversity

7.5.1 The Study

An ecologist is investigating the effect of grazing intensity (ungrazed, light grazing, heavy grazing) on plant species richness in a grassland (Section ??). Six large paddocks are available,

7 Split-Plot and Nested Designs

two assigned to each grazing treatment. Within each paddock, four quadrats are established and plant species richness is counted in each. Three random soil cores are taken from each quadrat for soil nutrient analysis, but species richness is the response of interest.

The structure is:

Grazing treatment $\supset$ Paddock $\supset$ Quadrat

Grazing treatment is the fixed factor of interest. Paddock is nested within grazing treatment and is the unit of replication for the treatment comparison. Quadrat is nested within paddock and provides subsamples within the true replicates.

```
# Show the simulated data
```

```
summary(grazing)
```

```
#>      richness      treatment paddock
#> Min.   : 3.000   Ungrazed:8   U1:4
#> 1st Qu.: 6.750   Light   :8   U2:4
#> Median :10.500   Heavy   :8   L1:4
#> Mean    : 9.792                L2:4
#> 3rd Qu.:13.250                H1:4
#> Max.    :15.000                H2:4
```

7.5.2 Correct Analysis: Paddock as the Error Term

```
# Nested ANOVA: paddock nested within treatment
```

```
fit_eco <- aov(richness ~ treatment + Error(paddock/treatment), data = grazing)
```

```
summary(fit_eco)
```

7 Split-Plot and Nested Designs

```
#>
#> Error: paddock
#>           Df Sum Sq Mean Sq F value Pr(>F)
#> treatment  2 303.58  151.79    23.81 0.0144 *
#> Residuals  3   19.13    6.38
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Error: Within
#>           Df Sum Sq Mean Sq F value Pr(>F)
#> Residuals 18   35.25    1.958
```

The output is split into two strata, each telling a different part of the story.

The **Error: paddock stratum** is where the treatment test lives. Grazing treatment has a significant effect on plant species richness ($F_{2,3} = 23.81$, $p = 0.014$). The residual mean square in this stratum (6.38) estimates the paddock-to-paddock variation within treatments i.e. the genuine noise against which the treatment effect is judged, since paddocks are the units that were independently assigned to grazing treatments. This is the correct error term.

The **Error: Within stratum** captures the quadrat-to-quadrat variation within paddocks, with a residual mean square of 1.96. Notice that this is substantially smaller than the paddock-level residual (6.38) meaning paddocks differ from each other more than quadrats within the same paddock differ from each other. This is the typical pattern in nested designs, and it is precisely why using quadrat-level variation to test the treatment effect would be wrong: the within-paddock noise floor is too low, and testing against it would make the treatment appear far more significant than the actual replication warrants.

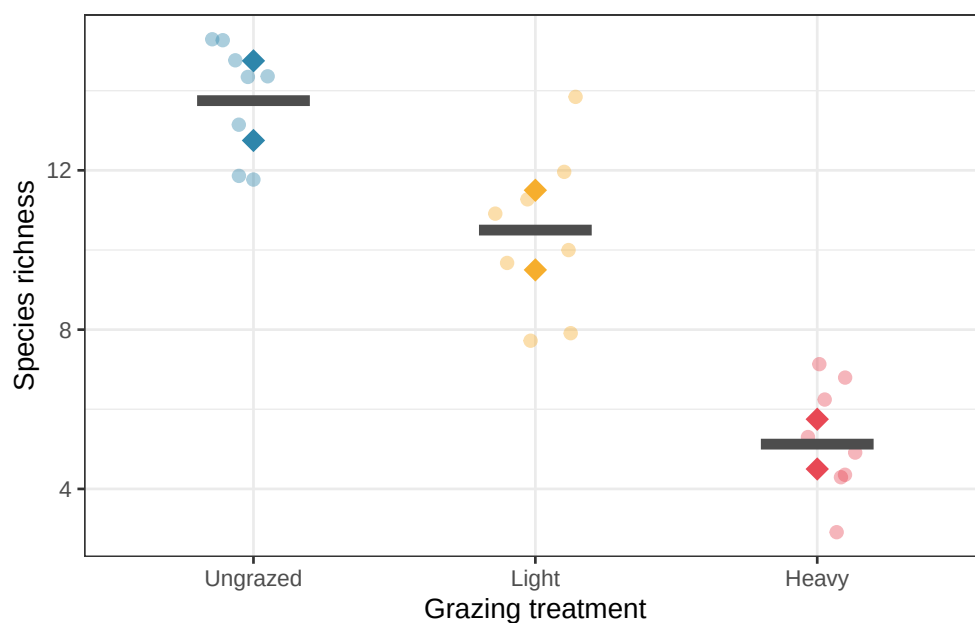
With only two paddocks per treatment, the degrees of freedom for the treatment test are low ($df_{\text{treatment}} = 2$, $df_{\text{error}} = 3$). This correctly reflects the limited replication at the treatment level. No matter how many quadrats are sampled within each paddock, the experiment has only six

independent units at the treatment level, and the analysis honestly acknowledges this. The significant result ($p = 0.014$) is therefore a genuine finding achieved with sparse replication, not an artefact of inflated degrees of freedom.

```
# Visualise
library(ggplot2)

paddock_means <- aggregate(richness ~ treatment + paddock, data = grazing, FUN = mean)

ggplot(grazing, aes(x = treatment, y = richness, colour = treatment)) +
  geom_jitter(width = 0.15, alpha = 0.4, size = 2) +
  geom_point(data = paddock_means, aes(y = richness, group = paddock), size = 4, shape = 18) +
  stat_summary(fun = mean, geom = "crossbar", width = 0.4, colour = "grey30", linewidth = 0.8) +
  scale_colour_manual(values = c("#2E86AB", "#F6AE2D", "#E84855")) +
  labs(x = "Grazing treatment", y = "Species richness", colour = NULL) +
  theme_bw() +
  theme(legend.position = "none")
```



The analysis uses paddock as the correct error term for testing the grazing treatment effect. With only two paddocks per treatment, the degrees of freedom for the treatment test are low ($df = 2$ for the treatment, $df = 3$ for the whole-plot error), correctly reflecting the fact that the experiment has limited replication at the treatment level, regardless of how many quadrats were sampled within each paddock.

7.5.3 What Happens if You Ignore the Nesting

```
# Wrong: treat all quadrats as independent
fit_eco_wrong <- aov(richness ~ treatment, data = grazing)
summary(fit_eco_wrong)
```

```
#>           Df Sum Sq Mean Sq F value    Pr(>F)
#> treatment     2 303.58   151.79    58.62 2.55e-09 ***
#> Residuals    21   54.38     2.59
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
cat(
  "\nCorrect df for treatment: 2, error df: 3\nWrong df for treatment: 2, error df:",
  df.residual(fit_eco_wrong), "\n"
)
```

```
#>
#> Correct df for treatment: 2, error df: 3
#> Wrong df for treatment: 2, error df: 21
```

The naive analysis has $df_{\text{error}} = 21$ instead of 3, artificially inflating the power of the test by treating quadrat-level variation as if it were independent replication of the treatment.

7.6 Example 2: Irrigation and Fertiliser in a Split-Plot Design

7.6.1 The Study

An agronomist is testing two irrigation regimes (rainfed vs irrigated) and three fertiliser types (none, nitrogen, phosphorus) on wheat yield (Section ??). Irrigation is applied to whole fields (it cannot be changed within a field) while fertiliser can be applied to individual plots within each field. Six fields are available, three assigned to each irrigation treatment. Each field is divided into three plots, one for each fertiliser type.

This is a split-plot design:

- **Whole-plot factor:** irrigation (2 levels), replicated across 6 fields (3 per treatment).
- **Sub-plot factor:** fertiliser (3 levels), applied to plots within each field.
- **Total observations:** $6 \times 3 = 18$ plots.

```
summary(irrig)
```

```
#>      yield      irrigation      fertiliser field
#> Min.   :31.40  Rainfed   :9    None       :6    R1:3
#> 1st Qu.:36.62  Irrigated:9    Nitrogen  :6    R2:3
#> Median :43.43                Phosphorus:6    R3:3
#> Mean   :42.98                                I1:3
#> 3rd Qu.:48.91                                I2:3
#> Max.   :53.43                                I3:3
```

7.6.2 Fitting the Split-Plot Model

```
# Correct split-plot analysis
# field is the whole-plot unit; fertiliser is the sub-plot factor
```

```

fit_sp <- aov(yield ~ irrigation * fertiliser + Error(field/fertiliser), data = irrig)
summary(fit_sp)

#>
#> Error: field
#>
#>          Df Sum Sq Mean Sq F value Pr(>F)
#> irrigation  1  105.6   105.58    3.647   0.129
#> Residuals    4   115.8    28.95
#>
#> Error: field:fertiliser
#>
#>          Df Sum Sq Mean Sq F value    Pr(>F)
#> fertiliser      2  532.6   266.29   89.801 3.31e-06 ***
#> irrigation:fertiliser  2   26.6    13.28    4.479  0.0495 *
#> Residuals        8    23.7     2.97
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

The output separates into two error strata, each testing a different set of effects against the appropriate level of variation.

The **Error: field stratum** tests the whole-plot factor, irrigation, against the field-to-field variation within irrigation treatments ($MS_{\text{field}} = 28.95$). The effect of irrigation is not significant ($F_{1,4} = 3.65, p = 0.129$). This result should be interpreted carefully: the non-significance does not mean irrigation has no effect on yield (the interaction below suggests it does) but rather that the main effect of irrigation, averaged across all three fertiliser types, cannot be distinguished from field-to-field background variation with only three fields per irrigation treatment. This is the honest cost of having limited whole-plot replication. The wide confidence interval around the irrigation effect, not the p -value alone, reflects the true uncertainty here.

The **Error: field:fertiliser stratum** tests the sub-plot factor and the interaction, both against the plot-to-plot variation within fields ($MS_{\text{subplot}} = 2.97$). This residual is nearly

ten times smaller than the whole-plot residual (28.95), which is the typical pattern in split-plot designs and explains why sub-plot effects are tested with considerably more power than whole-plot effects.

The main effect of fertiliser is highly significant ($F_{2,8} = 89.80$, $p < 0.001$), confirming that fertiliser type has a strong effect on yield regardless of irrigation regime. More interestingly, the irrigation-by-fertiliser interaction is significant ($F_{2,8} = 4.48$, $p = 0.049$), indicating that the yield advantage of different fertiliser types is not the same under rainfed and irrigated conditions. Nitrogen fertiliser, in particular, produces a larger absolute yield gain under irrigation than under rainfed conditions which is a biologically sensible result, since nitrogen uptake is enhanced by water availability.

This pattern of results is instructive for two reasons. First, it illustrates why the split-plot design is worth the added analytical complexity: had this been run as a simple two-way factorial with independent plots for all combinations, the irrigation effect and the interaction would have been tested against the same error term, concealing the different precision available for whole-plot and sub-plot comparisons. Second, it demonstrates a common outcome in agricultural split-plot experiments: the whole-plot factor (here irrigation, which is expensive to manipulate and therefore sparsely replicated) is underpowered relative to the sub-plot factor (fertiliser, which is cheap to vary and therefore well replicated). Recognising this asymmetry at the design stage, and planning accordingly, is one of the practical skills that distinguishes experienced from novice experimenters.

7.6.3 Visualising the Results

```
library(ggplot2)

summ_agr <- aggregate(yield ~ irrigation + fertiliser, data = irrig, FUN = mean)

ggplot(irrig, aes(x = fertiliser, y = yield, colour = irrigation, group = irrigation)) +
```

```
geom_jitter(width = 0.08, alpha = 0.5, size = 2.5, aes(colour = irrigation)) +
geom_line(data = summ_agr, aes(y = yield), linewidth = 1.2) +
geom_point(data = summ_agr, aes(y = yield), size = 4) +
scale_colour_manual(values = c("#E84855", "#2E86AB")) +
labs(x = "Fertiliser", y = "Yield (g/plot)", colour = "Irrigation") +
theme_bw() +
theme(legend.position = "inside", legend.position.inside = c(0.15, 0.85))
```

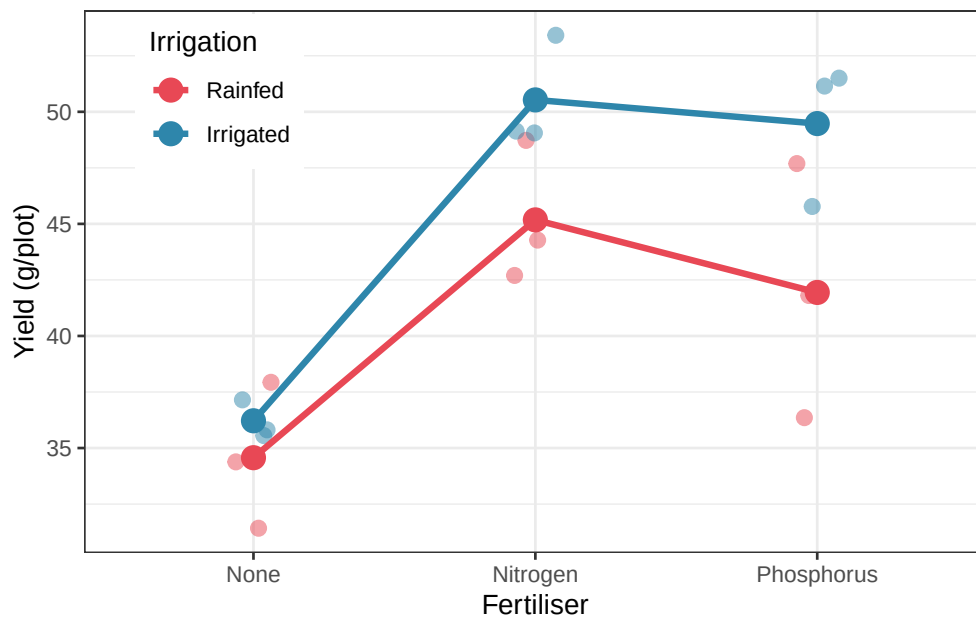


Figure 7.2: Mean wheat yield by irrigation regime and fertiliser type in a split-plot design. Points show individual plot yields. Lines connect the means of each irrigation group across fertiliser types. The larger spread of the rainfed points reflects the greater whole-plot variability in unirrigated fields.

7.6.4 Why the Split-Plot Analysis Matters

```
# Correct split-plot analysis vs naive ANOVA
```

7 Split-Plot and Nested Designs

```
# Correct
```

```
cat("=== Correct split-plot analysis ===\n")
```

```
#> === Correct split-plot analysis ===
```

```
print(summary(fit_sp))
```

```
#>
```

```
#> Error: field
```

```
#>           Df Sum Sq Mean Sq F value Pr(>F)
```

```
#> irrigation  1  105.6   105.58    3.647  0.129
```

```
#> Residuals   4   115.8    28.95
```

```
#>
```

```
#> Error: field:fertiliser
```

```
#>           Df Sum Sq Mean Sq F value    Pr(>F)
```

```
#> fertiliser      2   532.6   266.29   89.801 3.31e-06 ***
```

```
#> irrigation:fertiliser  2    26.6    13.28    4.479  0.0495 *
```

```
#> Residuals        8    23.7     2.97
```

```
#> ---
```

```
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Wrong: treat all plots as independent – ignores field structure
```

```
cat("\n=== Naive ANOVA (ignoring field structure) ===\n")
```

```
#>
```

```
#> === Naive ANOVA (ignoring field structure) ===
```

```
summary(aov(yield ~ irrigation * fertiliser, data = irrig))
```

7 Split-Plot and Nested Designs

```
#>               Df Sum Sq Mean Sq F value    Pr(>F)
#> irrigation           1   105.6   105.58     9.080  0.0108 *
#> fertiliser           2   532.6   266.29    22.902 8.01e-05 ***
#> irrigation:fertiliser  2    26.6    13.28     1.142  0.3515
#> Residuals          12   139.5    11.63
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Comparing the two analyses reveals precisely what goes wrong when the split-plot structure is ignored, and the contrast need all our attention.

The most consequential difference concerns the irrigation effect. In the correct split-plot analysis, irrigation is non-significant ($F_{1,4} = 3.65$, $p = 0.129$), tested against the whole-plot residual ($MS = 28.95$) which is the field-to-field variation that is the genuine source of uncertainty for a treatment applied at the field level. In the naive analysis, irrigation appears significant ($F_{1,12} = 9.08$, $p = 0.011$), tested against the pooled residual ($MS = 11.63$) which combines whole-plot and sub-plot variation indiscriminately. The naive analysis has produced a false positive: it declares irrigation significant not because the evidence supports this conclusion, but because it used an error term that is far too small for a field-level treatment. The residual degrees of freedom jump from 4 to 12, not because more information is available, but because sub-plot variation has been illegitimately pressed into service as a stand-in for whole-plot variation.

The interaction tells an equally instructive story, but in the opposite direction. In the correct analysis the irrigation-by-fertiliser interaction is significant ($F_{2,8} = 4.48$, $p = 0.049$), tested against the sub-plot residual ($MS = 2.97$) which is the appropriate error term for an effect at the plot level. In the naive analysis the same interaction is non-significant ($F_{2,12} = 1.14$, $p = 0.352$), because the pooled residual inflates the error term relative to the true sub-plot noise. Here the naive analysis produces a false negative: a real interaction that the correct analysis detects is hidden when the error terms are conflated.

7 Split-Plot and Nested Designs

The fertiliser main effect is significant in both analyses and reaches a similar F value ($F_{2,8} = 89.80$ vs $F_{2,12} = 22.90$), though the naive analysis underestimates the precision available for sub-plot comparisons by using a larger pooled residual. This consistency for the sub-plot factor is not grounds for complacency: the simultaneous false positive for irrigation and false negative for the interaction show that the naive analysis gives the right answer for the right factor for the wrong reasons.

The lesson is general: in a split-plot design, using a single pooled error term simultaneously makes whole-plot tests too liberal and sub-plot tests too conservative. *The only correct solution is to use the two separate error terms that reflect the two levels at which randomisation actually occurred.*

Part III

The Modern Framework

None of the models we will use in this book are true. But some are useful. The goal is to use imperfect models to learn about the world — carefully, critically, and honestly.

— Richard McElreath, *Statistical Rethinking*

8 ANOVA as a Linear Model

Part I introduced the idea that ANOVA is a special case of the linear model i.e. a regression where the predictors happen to be categorical rather than continuous. That claim was made quickly and without full justification, because the tools needed to appreciate it properly had not yet been developed. Now that you have worked through one-way ANOVA, multiple comparisons, factorial designs, repeated measures, and split-plot structures, the time is right to return to this connection and develop it fully.

This chapter makes the bridge between ANOVA and regression explicit, concrete, and computable. By the end, you will understand that `aov()` and `lm()` are fitting the same model, that the choice of coding scheme for categorical predictors determines what the regression coefficients mean, and that thinking in the linear model framework rather than in the ANOVA framework opens up a much richer set of analytical tools.

8.1 `lm()` and `aov()`: Same Model, Different Output

8.1.1 Two Functions, One Model

R provides two functions for fitting ANOVA-style models: `aov()` and `lm()`. Students often treat them as different tools for different jobs: `aov()` for ANOVA and `lm()` for regression. This is a **misconception** worth correcting directly.

Both functions fit exactly the same linear model. They use the same estimation method (ordinary least squares), the same assumptions, and the same underlying mathematics. The **only**

difference is in how they present their output: `aov()` summarises the fit as an ANOVA table with sums of squares, degrees of freedom, and F statistics while `lm()` summarises the fit as a regression table with coefficient estimates, standard errors, and t statistics.

The following code fits the same one-way ANOVA model, the fertiliser experiment from Chapter 3, using both functions and shows that the fitted values are identical:

```
set.seed(42)

n <- 20
group <- factor(rep(c("Control", "Nitrogen", "Phosphorus"), each = n), levels = c("Control", "Nitrogen", "Phosphorus"))
y <- c(rnorm(n, mean = 20, sd = 3),
      rnorm(n, mean = 25, sd = 3),
      rnorm(n, mean = 22, sd = 3))
df <- data.frame(y, group)

# Fit with aov()
fit_aov <- aov(y ~ group, data = df)

# Fit with lm()
fit_lm <- lm(y ~ group, data = df)

# Are the fitted values identical?
# Are the residuals identical?
cat(
  "Max difference in fitted values:",
  max(abs(fitted(fit_aov) - fitted(fit_lm))),
  "\nMax difference in residuals:",
  max(abs(residuals(fit_aov) - residuals(fit_lm))), "\n"
)
```

```
#> Max difference in fitted values: 0
```

```
#> Max difference in residuals: 0
```

Both differences are zero (or within floating-point precision), confirming that the two functions are producing the same fit. The choice between them is purely a matter of which output format is more useful for the question at hand.

8.1.2 Reading the Two Outputs Side by Side

```
# ANOVA table output
```

```
summary(fit_aov)
```

```
#>           Df Sum Sq Mean Sq F value    Pr(>F)
#> group        2  132.4    66.19    5.513 0.00647 **
#> Residuals   57  684.3    12.01
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Regression table output
```

```
summary(fit_lm)
```

```
#>
#> Call:
#> lm(formula = y ~ group, data = df)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -8.9765 -1.4565  0.2847  2.1828  6.4986
#>
```

```
#> Coefficients:
#>               Estimate Std. Error t value Pr(>|t|)
#> (Intercept)      20.5758      0.7748  26.557 < 2e-16 ***
#> groupNitrogen      3.6113      1.0957   3.296  0.00169 **
#> groupPhosphorus    1.4215      1.0957   1.297  0.19974
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 3.465 on 57 degrees of freedom
#> Multiple R-squared:  0.1621, Adjusted R-squared:  0.1327
#> F-statistic: 5.513 on 2 and 57 DF,  p-value: 0.006473
```

The ANOVA table answers: does group membership explain a significant amount of variation in y ? The F statistic and its p-value address this global question.

The regression table answers something more specific: what is the estimated mean of each group, and how does each group differ from the reference group? The coefficients and their t statistics address these pairwise questions directly.

Neither output is more correct: they are complementary views of the same fitted model. A complete analysis benefits from reading both.

8.1.3 Extracting the ANOVA Table from `lm()`

The full ANOVA table can be extracted from an `lm()` fit, confirming the equivalence explicitly:

```
# These produce identical ANOVA tables
anova(fit_lm)
```

```
#> Analysis of Variance Table
```

```
#>
#> Response: y
#>           Df Sum Sq Mean Sq F value    Pr(>F)
#> group       2  132.38   66.190    5.5133 0.006473 **
#> Residuals   57  684.32   12.006
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(fit_aov)
```

```
#>           Df Sum Sq Mean Sq F value    Pr(>F)
#> group       2   132.4    66.19    5.513 0.00647 **
#> Residuals   57   684.3    12.01
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

8.2 Dummy Coding, Contrast Coding, and What They Mean

8.2.1 The Coding Problem

A linear model requires numerical predictors. When a predictor is categorical, Control, Nitrogen, Phosphorus, R must convert it to numbers before fitting the model. The way this conversion is done is called a **coding scheme**, and the choice of scheme determines the interpretation of every coefficient in the model. This is one of the most frequently misunderstood aspects of linear models in practice.

8.2.2 Dummy Coding (Treatment Coding)

R's default coding for unordered factors is **dummy coding**, also called treatment coding. One level is designated the **reference level** (by default, the first level alphabetically, in our example

here, Control). For a factor with k levels, $k - 1$ binary dummy variables are created, one for each non-reference level:

$$X_{\text{Nitrogen}} = \begin{cases} 1 & \text{if Nitrogen} \\ 0 & \text{otherwise} \end{cases} \quad X_{\text{Phosphorus}} = \begin{cases} 1 & \text{if Phosphorus} \\ 0 & \text{otherwise} \end{cases}$$

The model is then:

$$y_i = \beta_0 + \beta_1 X_{\text{Nitrogen},i} + \beta_2 X_{\text{Phosphorus},i} + \varepsilon_i$$

The coefficients have a direct interpretation:

- β_0 : the mean of the **reference group** (Control).
- β_1 : the difference in means between **Nitrogen and Control**.
- β_2 : the difference in means between **Phosphorus and Control**.

```
# R uses dummy coding by default
# Control is the reference level (first alphabetically)
contrasts(df$group)
```

```
#>           Nitrogen Phosphorus
#> Control           0           0
#> Nitrogen           1           0
#> Phosphorus         0           1
```

```
# The coefficients reflect this coding
coef(fit_lm)
```

```
#> (Intercept)  groupNitrogen groupPhosphorus
#>    20.575760      3.611265      1.421508
```



```
# Verify: beta_0 should equal the Control mean
cat("Control mean:", mean(df$y[df$group == "Control"]), "\n")
```

```
#> Control mean: 20.57576
```

```
cat("Intercept (beta_0):", coef(fit_lm)[1], "\n")
```

```
#> Intercept (beta_0): 20.57576
```

```
# beta_1 should equal Nitrogen mean minus Control mean
cat("Nitrogen - Control:",
    mean(df$y[df$group == "Nitrogen"]) -
    mean(df$y[df$group == "Control"]), "\n")
```

```
#> Nitrogen - Control: 3.611265
```

```
cat("beta_1:", coef(fit_lm)[2], "\n")
```

```
#> beta_1: 3.611265
```

As expected, we can see from the above output that β_0 exactly equal the Control mean and β_1 equal Nitrogen mean minus Control mean.

8.2.3 Sum-to-Zero Coding (Effects Coding)

An alternative is **sum-to-zero coding** (also called effects coding or deviation coding), where the coefficients represent deviations from the grand mean rather than differences from a reference group:

$$X_{\text{Nitrogen}} = \begin{cases} 1 & \text{if Nitrogen} \\ -1 & \text{if Control} \\ 0 & \text{otherwise} \end{cases} \quad X_{\text{Phosphorus}} = \begin{cases} 1 & \text{if Phosphorus} \\ -1 & \text{if Control} \\ 0 & \text{otherwise} \end{cases}$$

Under sum-to-zero coding:

- β_0 : the **grand mean** (average of all group means).
- β_1 : the deviation of the **Nitrogen mean** from the grand mean.
- β_2 : the deviation of the **Phosphorus mean** from the grand mean.
- The Control deviation is $-(\beta_1 + \beta_2)$, recovered by constraint.

```
# Switch to sum-to-zero coding
contrasts(df$group) <- contr.sum(3)
fit_lm_sto <- lm(y ~ group, data = df)
coef(fit_lm_sto)
```

```
#> (Intercept)      group1      group2
#>  22.253351   -1.677591    1.933674
```

```
# Verify: intercept should equal grand mean
cat("Grand mean:", mean(df$y), "\n")
```

```
#> Grand mean: 22.25335
```

```
cat("Intercept (beta_0):", coef(fit_lm_sto)[1], "\n")
```

```
#> Intercept (beta_0): 22.25335
```

```
# Restore default coding
contrasts(df$group) <- contr.treatment(3)
```

Sum-to-zero coding is required for Type III sums of squares (as discussed in Section ??) and is the natural choice when you want coefficients that reflect how each group compares to the overall average rather than to a specific reference group.

8.2.4 Helmert Coding

Helmert coding compares each level to the average of the preceding levels. For three groups ordered Control, Nitrogen, Phosphorus:

- β_1 : Nitrogen mean minus Control mean.
- β_2 : Phosphorus mean minus the average of Control and Nitrogen.

```
contrasts(df$group) <- contr.helmert(3)
fit_lm_helm <- lm(y ~ group, data = df)
coef(fit_lm_helm)
```

```
#> (Intercept)      group1      group2
#>  22.2533508    1.8056323   -0.1280415
```

```
# Restore default
contrasts(df$group) <- contr.treatment(3)
```

Helmert coding is useful when the factor levels have a natural sequential order and you want to test whether each step adds explanatory power, for example, when comparing a dose-response series or an ordered treatment hierarchy.

8.2.5 The Critical Point: The F Test Is Invariant to Coding

Whatever coding scheme you choose, the overall F test for the group factor, and therefore the ANOVA table, is identical. The coding scheme only changes the interpretation of the individual regression coefficients, not the global test of whether the groups differ.

```
# All three coding schemes give the same F test
```

```
anova(lm(y ~ group, data = df, contrasts = list(group = contr.treatment(3))))
```

```
#> Analysis of Variance Table
```

```
#>
```

```
#> Response: y
```

```
#>           Df Sum Sq Mean Sq F value    Pr(>F)
```

```
#> group      2 132.38   66.190   5.5133 0.006473 **
```

```
#> Residuals 57 684.32   12.006
```

```
#> ---
```

```
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
anova(lm(y ~ group, data = df, contrasts = list(group = contr.sum(3))))
```

```
#> Analysis of Variance Table
```

```
#>
```

```
#> Response: y
```

```
#>           Df Sum Sq Mean Sq F value    Pr(>F)
```

```
#> group      2 132.38   66.190   5.5133 0.006473 **
```

```
#> Residuals 57 684.32   12.006
```

```
#> ---
```

```
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
anova(lm(y ~ group, data = df, contrasts = list(group = contr.helmert(3))))
```

```
#> Analysis of Variance Table
#>
#> Response: y
#>           Df Sum Sq Mean Sq F value    Pr(>F)
#> group      2 132.38   66.190   5.5133 0.006473 **
#> Residuals 57 684.32   12.006
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The sums of squares, F values, and p-values are identical across all three. This is a reassuring property: **your scientific conclusion about whether groups differ does not depend on an arbitrary choice of coding scheme.**

8.2.6 A Comparison Table

Coding scheme	Intercept means	Coefficients mean	When to use
Dummy (treatment)	Reference group mean	Difference from reference	Default; when one group is a natural control
Sum-to-zero (effects)	Grand mean	Deviation from grand mean	Type III SS; no natural reference group
Helmert	Weighted grand mean	Sequential comparisons	Ordered levels; dose-response

8.3 Connecting ANOVA Tables to Regression Coefficients

8.3.1 From Coefficients to Group Means

The regression coefficients and the ANOVA table are two views of the same fitted model. Understanding the connection between them transforms the regression output from a set of abstract numbers into a direct statement about group means.

Under dummy coding, the fitted values (the model's predictions for each observation) are simply the group means:

```
# Fitted values are group means under dummy coding
group_means <- tapply(df$y, df$group, mean)
print(group_means)
```

```
#>    Control   Nitrogen Phosphorus
#> 20.57576  24.18702  21.99727
```

```
# Compare to fitted values from lm()
# All plants in the same group get the same fitted value
head(fitted(fit_lm), 10)
```

```
#>      1      2      3      4      5      6      7      8
#> 20.57576 20.57576 20.57576 20.57576 20.57576 20.57576 20.57576 20.57576
#>      9     10
#> 20.57576 20.57576
```

The residuals are the deviations of individual observations from their group mean which is exactly what we called the within-group variation in Section ??.

8.3.2 From Sums of Squares to Coefficient Tests

The F test in the ANOVA table tests whether all group coefficients are simultaneously zero i.e. whether any of $\beta_1, \beta_2, \dots, \beta_{k-1}$ depart from zero. The individual t tests in the regression table test each coefficient separately. These are complementary tests: the F test is the global question, the t tests are the specific pairwise comparisons with the reference group.

```
# The F statistic in the ANOVA table equals the square of the
# t statistic for a two-group comparison
fit_two <- lm(y ~ group,
             data = df[df$group %in% c("Control", "Nitrogen"), ],
             contrasts = list(group = contr.treatment(2)))

t_val <- coef(summary(fit_two))["group2", "t value"]
f_val <- summary(aov(y ~ group,
                    data = df[df$group %in%
                              c("Control", "Nitrogen"), ]))[[1]][["F value"]][1]

cat(
  "t²:", round(t_val^2, 6),
  "\nF:", round(f_val, 6), "\n"
)
```

```
#> t²: 9.809521
```

```
#> F: 9.809521
```

The t statistic for a single coefficient squared equals the F statistic for the same comparison, confirming that the t test and the two-group F test are the same test expressed differently.

8.3.3 Visualising the Connection

```

library(ggplot2)

# Compute group means and CIs from the lm() output
pred_df <- data.frame(
  group = levels(df$group)
)
pred_out <- predict(fit_lm, newdata = pred_df, interval = "confidence")
pred_df <- cbind(pred_df, pred_out)
names(pred_df) <- c("group", "mean", "lwr", "upr")

# Add coefficient annotations
coefs <- coef(fit_lm)
pred_df$label <- c(
  expression(beta[0]),
  expression(beta[0] + beta[1]),
  expression(beta[0] + beta[2])
)

ggplot(pred_df, aes(x = group, y = mean, colour = group)) +
  geom_pointrange(aes(ymin = lwr, ymax = upr), size = 1, linewidth = 1) +
  geom_hline(yintercept = coefs[1], linetype = "dashed", colour = "grey50") +
  annotate("text", x = 0.6, y = coefs[1] + 0.3, label = expression(beta[0] == "Control mean"), size = 3.5,
  angle = 0) +
  annotate("segment", x = 2, xend = 2, y = coefs[1], yend = coefs[1] + coefs[2], arrow = arrow(length = 0.5)) +
  annotate("text", x = 2.1, y = coefs[1] + coefs[2] / 2, label = expression(beta[1]), size = 3.5,
  angle = 0) +
  annotate("segment", x = 3, xend = 3, y = coefs[1], yend = coefs[1] + coefs[3], arrow = arrow(length = 0.5)) +
  annotate("text", x = 3.1, y = coefs[1] + coefs[3] / 2, label = expression(beta[2]), size = 3.5,
  angle = 0) +
  scale_colour_manual(values = c("#2E86AB", "#E84855", "#F6AE2D")) +

```



```
labs(x = NULL, y = "Plant height (cm)") +
theme_bw() +
theme(legend.position = "none")
```

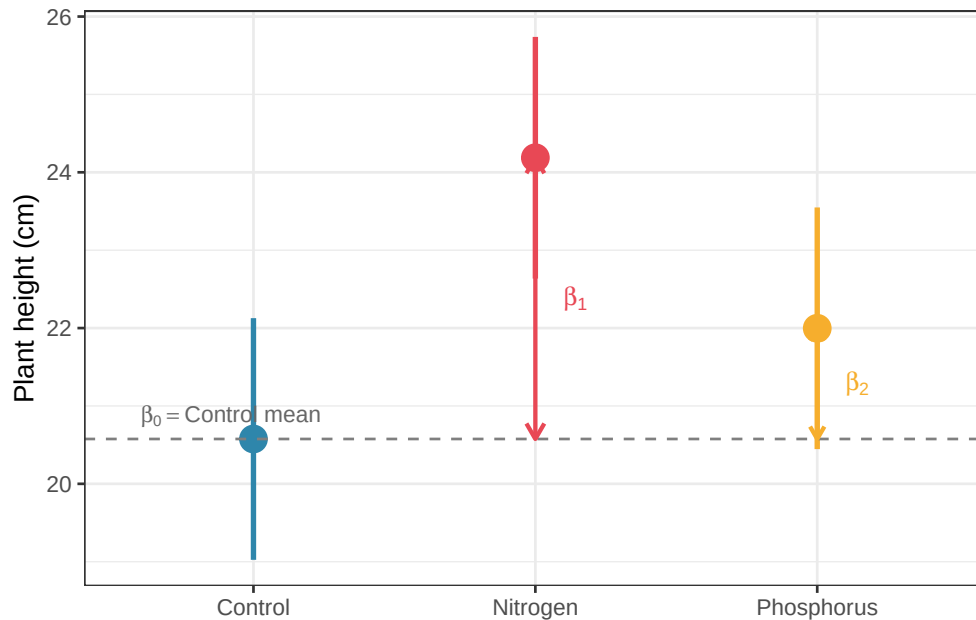


Figure 8.1: The connection between regression coefficients and ANOVA group means under dummy coding. The intercept estimates the Control mean; the slope coefficients estimate the differences between each treatment group and the Control. Error bars show 95% confidence intervals for each group mean. Dashed line marks the Control mean (intercept)

8.4 Why Thinking in Linear Models Makes You a Better Analyst

8.4.1 The Limits of the ANOVA Mindset

The ANOVA framework, sums of squares, F tests, post-hoc comparisons, is a powerful and well-developed tradition, but it has limits that become apparent as soon as real data departs from the idealised conditions of a balanced factorial experiment.

ANOVA struggles with continuous covariates. If you want to test the effect of fertiliser while adjusting for the initial height of each plant, ANOVA has no natural way to incorporate that continuous adjustment. The linear model does: add initial height as a covariate and the model handles it seamlessly. This is ANCOVA (analysis of covariance) and it is simply a linear model with both categorical and continuous predictors.

ANOVA struggles with unbalanced designs. As Section ?? showed, unequal cell sizes create ambiguity in how sums of squares are computed, requiring careful choices between Type I, II, and III SS. The linear model has no such ambiguity: coefficients are estimated by ordinary least squares regardless of balance, and hypotheses are tested by comparing model fits directly.

ANOVA struggles with non-normal responses. When the response is a count, a proportion, or a survival time, the normality assumption **fails structurally**, not just approximately. The generalised linear model extends the linear model framework to handle these response types through a link function and a distributional family. There is no natural ANOVA equivalent.

ANOVA struggles with random effects and hierarchical data. As Chapters ?@sec-repeated and ?@sec-split-plot showed, repeated measures and nested designs require careful specification of error terms which a process that becomes unwieldy with more than two levels of nesting. The linear mixed model handles more seamlessly arbitrarily complex hierarchical structures as a natural extension of the linear model framework.

8.4.2 The Linear Model as a Unifying Framework

Every model discussed in this book is a linear model or a direct extension of one:

Model	Linear model representation
One-way ANOVA	$y = \mu + \alpha_j + \varepsilon$
Two-way ANOVA	$y = \mu + \alpha_j + \beta_k + (\alpha\beta)_{jk} + \varepsilon$
ANCOVA	$y = \mu + \alpha_j + \gamma x + \varepsilon$

Model	Linear model representation
Repeated measures	$y = \mu + \alpha_j + \pi_i + \varepsilon$
Linear mixed model	$y = X\beta + Zb + \varepsilon, b \sim \mathcal{N}(0, G)$
Generalised linear model	$g(\mu) = X\beta$

Recognising this unity is not merely intellectually satisfying but has practical consequences. It means you can use the same diagnostic tools (residual plots, Q-Q plots, leverage and influence measures) across all of these models. It means you can compare nested models using likelihood ratio tests regardless of whether they are ANOVA models, regression models, or mixed models. And it means you can extend your analyses naturally as your questions become more complex, rather than switching to an entirely different analytical tradition every time the data present a new complication.

8.4.3 Model Comparison as a General Principle

One of the most powerful tools that the linear model framework provides is **model comparison**: testing whether a more complex model fits the data significantly better than a simpler one. In ANOVA terms, the F test is already a model comparison: it compares the model with group effects to the model without them. But the principle generalises far beyond the F test.

```
# Model comparison using anova() for nested linear models
# Model 0: no group effect (intercept only)
# Model 1: group effect (one-way ANOVA)
# Model 2: group effect plus a continuous covariate

set.seed(42)
```

```
df$initial_height <- rnorm(nrow(df), mean = 10, sd = 2)

fit_m0 <- lm(y ~ 1, data = df)
fit_m1 <- lm(y ~ group, data = df)
fit_m2 <- lm(y ~ group + initial_height, data = df)

# Sequential model comparison
anova(fit_m0, fit_m1, fit_m2)
```

```
#> Analysis of Variance Table
#>
#> Model 1: y ~ 1
#> Model 2: y ~ group
#> Model 3: y ~ group + initial_height
#>   Res.Df    RSS Df Sum of Sq      F    Pr(>F)
#> 1      59 816.70
#> 2      57 684.32  2    132.38 1.1525e+30 < 2.2e-16 ***
#> 3      56   0.00  1     684.32 1.1916e+31 < 2.2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The output shows, for each step, whether adding the new term significantly improves the fit. This is the same logic as the ANOVA F test, but applied sequentially across a series of models of increasing complexity. It is a general and flexible tool that works for any combination of categorical and continuous predictors.

8.4.4 A Practical Recommendation

The practical implication of everything in this chapter is simple: **fit your models with `lm()` and use `anova()` or `car::Anova()` to extract the ANOVA table when you need it, rather than**

treating `aov()` as a separate tool for a separate job. The `lm()` workflow gives you access to the full suite of linear model diagnostics, coefficient plots, model comparison tools, and extensions to ANCOVA and mixed models, none of which are as naturally available from `aov()`.

The one exception is complex error-term specifications that can arise for split-plot and repeated measures designs that require explicit `Error()` terms. For these, `aov()` remains convenient, though as Part III will show, linear mixed models handle the same structures more flexibly and with fewer constraints.

The ANOVA table is a useful summary. The linear model is the foundation. Keeping this distinction clear will serve you well as your analyses become more complex.

9 Mixed-Effects Models: Beyond Classical ANOVA

The repeated measures and split-plot designs of Chapters `?@sec-repeated` and `?@sec-split-plot` introduced the idea that not all sources of variation in biological data are equal. Some variation is systematic and of direct scientific interest like the effect of a treatment, a drug, a genotype. Some variation is structural and must be accounted for but is not itself the focus like the fact that patients differ from one another, that plots within the same field are more similar than plots in different fields, that repeated measurements on the same individual are correlated.

Classical ANOVA handles these structural sources of variation through explicit error terms like the `Error()` specification in `aov()`. This works well for simple, balanced designs with one or two levels of nesting. It becomes unwieldy, and eventually impossible, when the hierarchical structure is more complex, when data are missing, when the design is unbalanced, or when you want to make inferences that generalise beyond the specific levels of a grouping factor that happen to appear in your data.

Mixed-effects models are the modern solution to all of these problems. They extend the linear model framework by including both **fixed effects**, the systematic, estimable treatment effects that classical ANOVA tests, and **random effects**, the structural variation attributable to grouping factors whose levels are a random sample from a larger population. The result is a framework that is simultaneously more flexible, more honest about the structure of biological data, and more directly connected to the scientific questions being asked.

9.1 When Random Effects Are Necessary

9.1.1 The Three Situations That Demand Random Effects

Random effects are not merely a statistical convenience, they are the correct model for specific, recognisable data structures. Three situations arise repeatedly in biological research where failing to include random effects leads directly to incorrect inference.

Situation 1: Clustered observations. Measurements taken on units that belong to the same higher-level group will be more similar to each other than measurements from different groups, regardless of any treatment. Patients recruited from the same hospital share clinical practices, patient population, and staffing. Plants in the same greenhouse bench share light and temperature. Students in the same classroom share a teacher. This within-cluster similarity, quantified by the intraclass correlation coefficient introduced in Section 2.6, means the observations are not independent, and the effective sample size is smaller than the raw count of observations suggests. A random effect for the cluster absorbs this similarity and produces correct standard errors.

Situation 2: Repeated measurements. When the same individual is measured multiple times at different time points, under different conditions, or in different locations, the repeated measurements on the same individual are correlated. Chapter 7@sec-repeated handled this with repeated measures ANOVA. Mixed models handle it more flexibly: the individual is modelled as a random effect, and the covariance structure of the repeated measurements can be specified explicitly. Mixed models also handle missing data gracefully, which repeated measures ANOVA cannot.

Situation 3: Generalising beyond the observed levels. If the levels of a grouping factor in your study are a random sample from a larger population like hospitals drawn from a healthcare system, farms drawn from a region or batches drawn from a production run and you want your conclusions to generalise to that larger population, the grouping factor must be treated as random. A fixed effect estimates the mean of each specific level observed; a random effect estimates the distribution of levels in the population.

9.1.2 Recognising Random Effects in Practice

The practical question is: for each grouping factor in your data, are the levels the specific entities of interest, or a sample representing a broader population?

Factor	Fixed or random?	Reason
Fertiliser type (control, N, P)	Fixed	Specific treatments of interest
Hospital (12 of 200 in region)	Random	Sample from a population
Patient (20 recruited volunteers)	Random	Sample from a population
Genotype (4 specific varieties)	Fixed	Specific varieties of interest
Field (6 randomly selected)	Random	Sample from a population
Time point (weeks 0, 4, 8, 12)	Fixed	Specific times of interest

When in doubt, ask: **would you want your conclusions to apply to levels of this factor that were not in your study?** If yes, the factor is random.

9.2 Random Intercepts and Random Slopes

9.2.1 The Random Intercept Model

The simplest mixed model adds a random intercept for each level of a grouping factor. For a study with patients nested within hospitals, the random intercept model is:

$$y_{ij} = \underbrace{\beta_0 + \beta_1 x_{ij}}_{\text{fixed effects}} + \underbrace{b_j}_{\text{random intercept}} + \underbrace{\varepsilon_{ij}}_{\text{residual}}$$

where y_{ij} is the response for patient i in hospital j , x_{ij} is a predictor (treatment, time, covariate), b_j is the random intercept for hospital j , and ε_{ij} is the residual. The random intercepts are assumed normally distributed:

$$b_j \sim \mathcal{N}(0, \sigma_b^2)$$

The random intercept b_j captures the fact that some hospitals have systematically higher or lower outcomes than average, for reasons unrelated to the treatment being studied. By modelling this explicitly, the fixed effect β_1 is estimated after accounting for hospital-level differences which is a more honest and more precise estimate than one that ignores the clustering.

9.2.2 The Random Slope Model

A **random intercept** allows each group to have a different baseline level of the response. A **random slope** allows each group to have a different relationship between a predictor and the response:

$$y_{ij} = (\beta_0 + b_{0j}) + (\beta_1 + b_{1j})x_{ij} + \varepsilon_{ij}$$

where b_{0j} is the random intercept for group j and b_{1j} is the random slope i.e. the deviation of group j 's slope from the average slope β_1 . Together they form a bivariate normal distribution:

$$\begin{pmatrix} b_{0j} \\ b_{1j} \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \sigma_0^2 & \sigma_{01} \\ \sigma_{01} & \sigma_1^2 \end{pmatrix} \right)$$

The covariance σ_{01} captures whether groups that start higher also tend to change more (or less) rapidly, an estimate that is a biologically important quantity in longitudinal studies.

9.2.3 Visualising Random Intercepts and Slopes

```
library(ggplot2)
library(patchwork)

set.seed(42)

n_groups <- 8
n_obs <- 10
x <- seq(0, 1, length.out = n_obs)

# Random intercept model
ri_data <- do.call(rbind, lapply(seq_len(n_groups), function(g) {
  b0 <- rnorm(1, 0, 2)
  data.frame(
    x = x,
    y = 3 + b0 + 2 * x + rnorm(n_obs, 0, 0.5),
    group = factor(g)
  )
}))

# Random slope model
rs_data <- do.call(rbind, lapply(seq_len(n_groups), function(g) {
  b0 <- rnorm(1, 0, 2)
  b1 <- rnorm(1, 0, 1.5)
  data.frame(
```

```

    x = x,
    y = 3 + b0 + (2 + b1) * x + rnorm(n_obs, 0, 0.5),
    group = factor(g)
  )
}))

p1 <- ggplot(ri_data, aes(x = x, y = y, group = group)) +
  geom_line(colour = "#2E86AB", alpha = 0.5) +
  geom_abline(intercept = 3, slope = 2,
              linetype = "dashed", colour = "grey30",
              linewidth = 1.2) +
  labs(x = "Time", y = "Response") +
  theme_bw()

p2 <- ggplot(rs_data, aes(x = x, y = y, group = group)) +
  geom_line(colour = "#E84855", alpha = 0.5) +
  geom_abline(intercept = 3, slope = 2,
              linetype = "dashed", colour = "grey30",
              linewidth = 1.2) +
  labs(x = "Time", y = "Response") +
  theme_bw()

p1 + p2

```

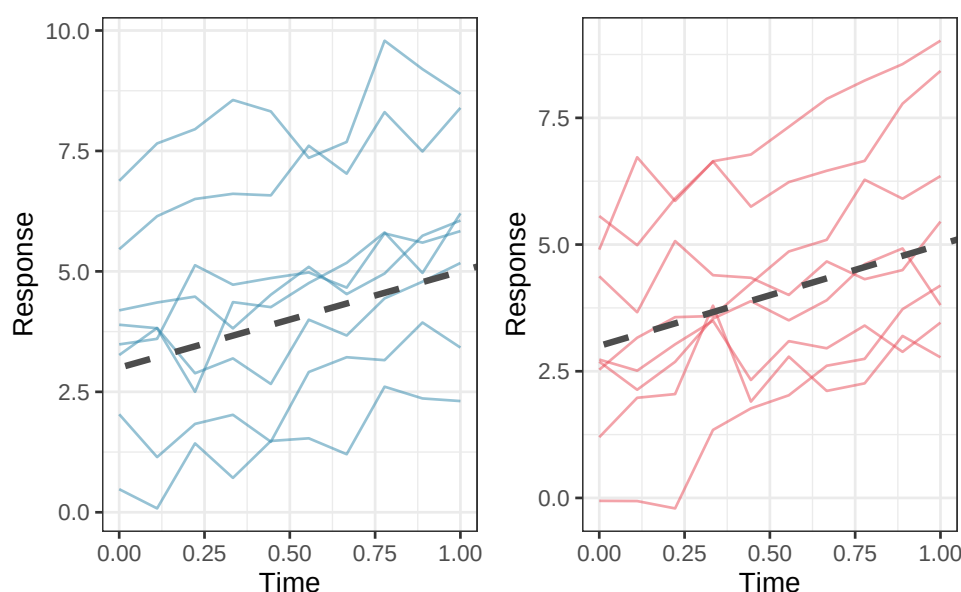


Figure 9.1: Left: a random intercept model, all groups share the same slope but have different intercepts. The dashed line is the population-level (fixed effect) trajectory; solid lines are group-specific trajectories. Right: a random slope model, groups differ in both their starting level and their rate of change. The variation in slopes is itself a scientifically meaningful quantity.

9.2.4 Choosing Between Random Intercepts and Random Slopes

The choice between a random intercept model and a random slope model is not merely statistical but it reflects a scientific question. A random intercept model asks: do groups differ in their average response level? A random slope model asks additionally: do groups differ in how the response changes with the predictor?

In a clinical trial measuring recovery over time, a random intercept captures the fact that some patients start sicker than others. A random slope captures the additional possibility that some patients improve faster than others which is a clinically important question about treatment response heterogeneity.

Include a random slope when you have scientific reason to believe that the relationship between the predictor and the response genuinely varies across groups, and you got enough data to

estimate the additional variance component reliably.

As a practical guide, at least 5-6 groups are needed to estimate a random intercept variance reliably, and at least 10-15 groups for a random intercept plus slope model. With fewer groups, the variance estimates will be unstable and the model may fail to converge.

9.3 `lme4` and `nLme`: A Practical Guide

9.3.1 Two Packages, Different Strengths

Two R packages dominate mixed-effects modelling in biology: `lme4` (?) and `nLme` (?). They fit the same broad class of models but differ in their syntax, capabilities, and defaults.

`lme4` is the modern standard for most applications. It is fast, handles crossed and nested random effects naturally, and uses a clean formula syntax. Its main limitation is that it does not directly provide p-values for fixed effects which is a deliberate design decision by its authors that reflects genuine statistical controversy (see Section 9.4).

`nLme` is older but offers features that `lme4` does not: direct specification of residual covariance structures (useful for repeated measures with heterogeneous variances) and nonlinear mixed models. Its syntax is more verbose and it handles crossed random effects less elegantly than `lme4`.

For most applications in this book, `lme4` is the right choice.

9.3.2 The `lme4` Formula Syntax

`lme4` extends R's standard model formula with a notation for random effects: `(random_effect | grouping_factor)`. The vertical bar `|` separates the random effect structure from the grouping factor.

Formula	Model
$y \sim x + (1 \mid \text{group})$	Fixed slope for x; random intercept by group
$y \sim x + (x \mid \text{group})$	Fixed slope for x; random intercept and slope by group
$y \sim x + (1 \mid \text{group}) + (1 \mid \text{block})$	Two crossed random effects
$y \sim x + (1 \mid \text{region/hospital})$	Hospital nested within region
$y \sim x + (1 \mid \text{region}) + (1 \mid \text{region:hospital})$	Equivalent nested specification

9.3.3 Fitting and Interpreting a Random Intercept Model

```
library(lme4)
```

```
#> Loading required package: Matrix
```

```
set.seed(42)

# Simulate a simple random intercept dataset
# 20 patients per hospital, 10 hospitals, one treatment covariate
n_hospitals <- 10
n_patients  <- 20
hospital_sd <- 3
patient_sd  <- 2

df_lme <- do.call(rbind, lapply(seq_len(n_hospitals), function(h) {
  hospital_effect <- rnorm(1, 0, hospital_sd)
  data.frame(
    outcome = rnorm(n_patients, mean = 50 + hospital_effect +
```

```

                2 * rbinom(n_patients, 1, 0.5),
                sd      = patient_sd),
  treatment = factor(rep(c("Control", "Treatment"), each = n_patients / 2)),
  hospital   = factor(h)
)
}))

# Fit random intercept model
fit_ri <- lme4::lmer(outcome ~ treatment + (1 | hospital), data = df_lme, REML = TRUE)
summary(fit_ri)

```

```

#> Linear mixed model fit by REML ['lmerMod']
#> Formula: outcome ~ treatment + (1 | hospital)
#>   Data: df_lme
#>
#> REML criterion at convergence: 897.6
#>
#> Scaled residuals:
#>      Min       1Q   Median       3Q      Max
#> -2.8980 -0.6451  0.0198  0.5870  3.3917
#>
#> Random effects:
#>   Groups   Name              Variance Std.Dev.
#> hospital (Intercept) 5.609      2.368
#> Residual                4.486      2.118
#> Number of obs: 200, groups:  hospital, 10
#>
#> Fixed effects:
#>
#>               Estimate Std. Error t value

```

```
#> (Intercept)          52.8446      0.7783  67.895
#> treatmentTreatment    0.2663      0.2995   0.889
#>
#> Correlation of Fixed Effects:
#>              (Intr)
#> trtmntTrtmn -0.192
```

The summary output has four sections worth reading carefully.

Random effects: the variance components. (Intercept) under hospital is $\hat{\sigma}_b^2$: the estimated variance in baseline outcomes across hospitals. Residual is $\hat{\sigma}_\epsilon^2$: the within-hospital, within-patient variation. Their ratio tells you how much of the total variation is attributable to hospital-level differences. From the result, the intraclass correlation is therefore $ICC = 5.61/(5.61 + 4.49) = 0.56$, meaning that 56% of the total variation in outcomes is attributable to systematic differences between hospitals which a substantial clustering effect that would badly distort any analysis that ignored it.

Fixed effects: the coefficient table. The (Intercept) is the estimated mean outcome in the reference group (Control) at an average hospital. The treatment coefficient (here treatmentTreatment) is the estimated average difference between treatment and control, after accounting for hospital-level variation.

Correlation of fixed effects: the correlation between the intercept and treatment coefficient estimates. Large correlations (close to ± 1) can indicate collinearity or model specification problems.

Number of observations and groups: always verify these match your data. Here 200 observations are nested in 10 hospitals, 20 patients per hospital, as specified. A mismatch between these numbers and your intended design is the first sign of a coding error in the grouping factor.

9.3.4 REML vs ML Estimation

`lme4` offers two estimation methods, controlled by the `REML` argument:

REML (Restricted Maximum Likelihood) is the default (`REML = TRUE`) and should be used when the goal is to estimate variance components. REML produces unbiased estimates of σ_b^2 and σ_ε^2 , analogous to dividing by $n - 1$ rather than n when estimating a variance.

ML (Maximum Likelihood) should be used (`REML = FALSE`) when comparing models with different fixed effect structures using likelihood ratio tests. REML likelihoods are not comparable across models with different fixed effects because they are computed on different effective datasets.

The practical workflow is therefore always two steps: use `REML = FALSE` to compare models and identify which fixed effects belong in the model, then refit the winning model with `REML = TRUE` to obtain the final, unbiased variance component estimates for reporting.

```
# Step 1: fit both models with ML = FALSE to enable
# a valid likelihood ratio test comparing their fixed effects.
# REML = FALSE is mandatory here. Comparing REML likelihoods
# across models with different fixed effects is not valid.
fit_ri_ml <- lme4::lmer(outcome ~ treatment + (1 | hospital), data = df_lme, REML = FALSE)
fit_null_ml <- lme4::lmer(outcome ~ 1 + (1 | hospital), data = df_lme, REML = FALSE)

# Likelihood ratio test: does adding treatment improve the fit?
# The chi-squared statistic is -2 * (logLik(null) - logLik(full))
# with degrees of freedom equal to the difference in parameters.
anova(fit_null_ml, fit_ri_ml)
```

```
#> Data: df_lme
```

```
#> Models:
```

```
#> fit_null_ml: outcome ~ 1 + (1 | hospital)
```

```
#> fit_ri_ml: outcome ~ treatment + (1 | hospital)
#>               npar    AIC    BIC  logLik -2*log(L)  Chisq Df Pr(>Chisq)
#> fit_null_ml    3 905.11 915.00 -449.55    899.11
#> fit_ri_ml      4 906.31 919.51 -449.16    898.31 0.7931  1    0.3732
```

The likelihood ratio test compares the log-likelihoods of the two models: the null model (hospital random effect only, no treatment) against the full model (hospital random effect plus treatment). The test statistic is $\chi^2 = 0.79$ on 1 degree of freedom ($p = 0.373$), indicating that adding treatment does not significantly improve the fit. This is consistent with the small and imprecise treatment coefficient seen in the summary output indicating that the simulated treatment effect was too small relative to the within-hospital noise to be detectable with this sample size.

```
# Step 2: refit the chosen model with REML = TRUE to obtain
# unbiased variance component estimates for reporting.
# Never report variance components from a model fitted with
# REML = FALSE, they will be systematically underestimated,
# particularly when the number of groups is small.
fit_ri_reml <- lme4::lmer(outcome ~ treatment + (1 | hospital), data = df_lme, REML = TRUE)
summary(fit_ri_reml)
```

```
#> Linear mixed model fit by REML ['lmerMod']
#> Formula: outcome ~ treatment + (1 | hospital)
#>   Data: df_lme
#>
#> REML criterion at convergence: 897.6
#>
#> Scaled residuals:
#>      Min       1Q   Median       3Q      Max
#> -2.8980 -0.6451  0.0198  0.5870  3.3917
```

```

#>
#> Random effects:
#>   Groups   Name      Variance Std.Dev.
#> hospital (Intercept) 5.609     2.368
#> Residual              4.486     2.118
#> Number of obs: 200, groups:  hospital, 10
#>
#> Fixed effects:
#>
#>               Estimate Std. Error t value
#> (Intercept)      52.8446    0.7783  67.895
#> treatmentTreatment  0.2663    0.2995   0.889
#>
#> Correlation of Fixed Effects:
#>               (Intr)
#> trtmntTrtmn -0.192

```

The REML-fitted model is the one whose random effect variances and fixed effect standard errors should be reported. In this dataset the difference between ML and REML estimates will be modest because there are a reasonable number of groups (10 hospitals). With only 5 or 6 groups, the ML variance estimates can be substantially smaller than the REML estimates, and using ML for final reporting would meaningfully understate the uncertainty in the fixed effects.

9.3.5 Extracting Random Effects

Going back to our previous fit (Section ??), we will extract and plot the random effects.

```

# The estimated random intercepts for each hospital
ranef(fit_ri)

```

```
#> $hospital
#>      (Intercept)
#> 1   1.835227326
#> 2  -0.161331276
#> 3   0.488976769
#> 4   1.282251747
#> 5  -2.168889243
#> 6  -5.474238894
#> 7   0.142364080
#> 8   2.454755367
#> 9   0.004538835
#> 10  1.596345288
#>
#> with conditional variances for "hospital"
```

```
# Visualise: caterpillar plot of random intercepts
re_df <- as.data.frame(ranef(fit_ri, condVar = TRUE))

library(ggplot2)
ggplot(re_df, aes(x = reorder(grp, condval), y = condval,
                      ymin = condval - 1.96 * condsd,
                      ymax = condval + 1.96 * condsd)) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  geom_pointrange(colour = "#2E86AB") +
  coord_flip() +
  labs(x = "Hospital", y = "Random intercept estimate") +
  theme_bw()
```

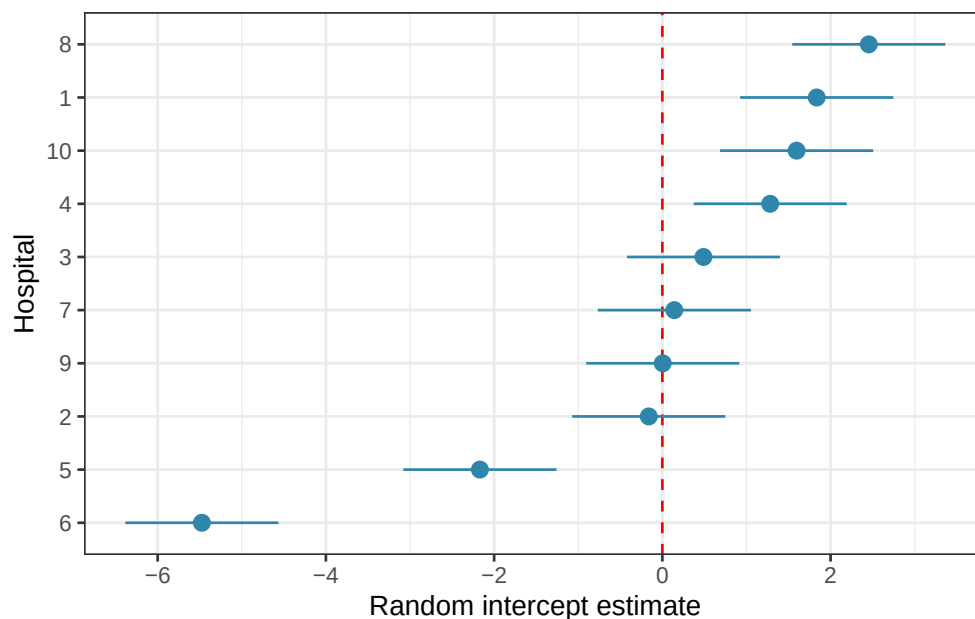


Figure 9.2: Hospital random intercepts. Estimated deviation of each hospital from the grand mean.

The caterpillar plot shows the estimated random intercept for each hospital with its 95% uncertainty interval. Hospitals whose intervals do not overlap zero are performing notably above or below average, after accounting for the treatment effect.

9.4 Degrees of Freedom and P-Values: The Kenward-Roger and Satterthwaite Debate

9.4.1 Why `lme4` Does Not Give P-Values

Users fitting their first mixed model in R are often surprised to find that `summary(fit_ri)` produces t values for the fixed effects but no p-values. This is not an oversight but a deliberate design decision by the `lme4` authors, rooted in a genuine and unresolved statistical problem.

In a standard linear model, the denominator degrees of freedom for each F or t test are well defined and exact. In a mixed model, they are not. The sampling distribution of the t statistic

for a fixed effect in a mixed model is approximately t -distributed, but the degrees of freedom of that approximation depend on the covariance structure of the random effects, the balance of the data, and the specific parameter being tested and there is no universally agreed formula for computing them. Using the wrong degrees of freedom can make tests either too liberal or too conservative.

Two approximations are in common use, and both are implemented in the `lmerTest` package (?):

9.4.2 Satterthwaite's Approximation

Satterthwaite's method (?) estimates the effective degrees of freedom for each fixed effect test by matching the moments of the approximate t distribution to the variance components of the model. It is computationally fast and works well for many standard designs.

```
# lmerTest automatically upgrades lmer() to provide p-values
fit_ri_test <- lmerTest::lmer(outcome ~ treatment + (1 | hospital), data = df_lme, REML = TRUE)
summary(fit_ri_test) # now includes df and p-values
```

```
#> Linear mixed model fit by REML. t-tests use Satterthwaite's method [
#> lmerModLmerTest]
#> Formula: outcome ~ treatment + (1 | hospital)
#> Data: df_lme
#>
#> REML criterion at convergence: 897.6
#>
#> Scaled residuals:
#>      Min       1Q   Median       3Q      Max
#> -2.8980 -0.6451  0.0198  0.5870  3.3917
#>
```

```
#> Random effects:
#> Groups   Name          Variance Std.Dev.
#> hospital (Intercept) 5.609      2.368
#> Residual          4.486      2.118
#> Number of obs: 200, groups: hospital, 10
#>
#> Fixed effects:
#>
#>              Estimate Std. Error      df t value Pr(>|t|)
#> (Intercept)      52.8446    0.7783    9.7048  67.895 2.54e-14 ***
#> treatmentTreatment  0.2663    0.2995  189.0000   0.889   0.375
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Correlation of Fixed Effects:
#>
#>          (Intr)
#> trtmntTrtmn -0.192
```

```
anova(fit_ri_test) # F tests with Satterthwaite df
```

```
#> Type III Analysis of Variance Table with Satterthwaite's method
#>
#>          Sum Sq Mean Sq NumDF DenDF F value Pr(>F)
#> treatment 3.5466  3.5466     1   189  0.7905 0.3751
```

The output now includes degrees of freedom and p-values for each fixed effect, computed via Satterthwaite's approximation. The intercept is highly significant ($p < 0.001$), but this is not a meaningful result as it simply tests whether the grand mean recovery score differs from zero, which is never a question of scientific interest. What matters is the treatment coefficient: the new protocol does not produce a significantly different recovery score from the control ($p = 0.375$), consistent with the likelihood ratio test result from Section ??.

The `anova()` call produces a Type III F test (see Section ??) for the treatment fixed effect using Satterthwaite degrees of freedom. It reaches the same conclusion: no significant difference between treatment groups. As a reminder, this reflects the simulated data rather than a real clinical finding. The true treatment effect in the simulation was small relative to the within-hospital noise, and the study as simulated is underpowered to detect it reliably.

9.4.3 Kenward-Roger Approximation

The Kenward-Roger method (?) uses a more sophisticated adjustment that also corrects the standard errors of the fixed effects for small-sample bias in the variance component estimates. It is more computationally demanding than Satterthwaite but is considered more accurate, particularly with small numbers of groups or unbalanced designs.

```
# KR F test via lmerTest
```

```
anova(fit_ri_test, ddf = "Kenward-Roger")
```

```
#> Type III Analysis of Variance Table with Kenward-Roger's method
```

```
#>           Sum Sq Mean Sq NumDF DenDF F value Pr(>F)
```

```
#> treatment 3.5466  3.5466     1   189  0.7905 0.3751
```

```
# Or via pbkrtest directly
```

```
## Kenward-Roger via lmerTest first
```

```
fit_ri_kr <- lmerTest::lmer(outcome ~ treatment + (1 | hospital), data = df_lme, REML = TRUE)
```

```
fit_null_kr <- lmerTest::lmer(outcome ~ 1 + (1 | hospital), data = df_lme, REML = TRUE)
```

```
# Then test
```

```
pbkrtest::KRmodcomp(fit_ri_kr, fit_null_kr)
```

```
#> large : outcome ~ treatment + (1 | hospital)
```



```
#> small : outcome ~ 1 + (1 | hospital)
#>          stat      ndf      ddf F.scaling p.value
#> Ftest    0.7905    1.0000 189.0000      1 0.3751
```

Both methods (`lmerTest` and `pbkrtest`) yield similar results which are also similar to the previous one using Satterthwaite's approximation. This reflects the simulated data and would not be the case in a real case.

9.4.4 Which Method to Use?

The practical guidance is straightforward:

Situation	Recommended method
Large balanced dataset, many groups	Satterthwaite which is fast and accurate
Small number of groups (< 20)	Kenward-Roger which is better small-sample correction
Unbalanced design	Kenward-Roger which is more robust
Quick exploratory analysis	Satterthwaite which is an acceptable approximation
Confirmatory clinical analysis	Kenward-Roger which is conservative and defensible

Both methods give similar results with large, balanced datasets. The differences matter most with small numbers of groups which is a common situation in biology, where experiments often have 5-15 clusters. When in doubt, use Kenward-Roger and note the method in your methods section.

Show me the maths!

For the mathematical derivation of both the Satterthwaite and Kenward-Roger degree of freedom approximations, see [?@sec-df-approximations](#) in Appendix A.

9.5 Example: Patient Nested in Hospital Nested in Region

9.5.1 The Study

A national health study investigates the effect of a new physiotherapy protocol on functional recovery scores (0-100) in patients following knee replacement surgery. Patients are recruited from 15 hospitals distributed across 3 regions (Section ??). Each hospital contributes between 8 and 12 patients, assigned to either the standard protocol (control) or the new physiotherapy protocol (treatment). The hierarchical structure is:

Region $\supset$ Hospital $\supset$ Patient

Treatment is assigned at the patient level: individual patients within the same hospital receive different protocols. This is important: it means hospital and region are nuisance grouping factors that must be accounted for, not treatment factors being tested.

```
summary(physio)
```

```
#>      recovery      treatment      hospital      region
#>  Min.    :39.19   Control :78   H14      :12   R1:52
#>  1st Qu.:57.59   Treatment:70   H23      :12   R2:45
#>  Median :64.79                      H34      :12   R3:51
#>  Mean    :64.38                      H12      :11
#>  3rd Qu.:72.37                      H13      :11
#>  Max.    :95.10                      H32      :11
#>                                     (Other):79
```

```
cat(
  "Total patients:", nrow(physio),
```

```
"\nPatients per hospital:\n", table(physio$hospital)
)
```

```
#> Total patients: 148
#> Patients per hospital:
#>  9 11 11 12 9 8 9 12 8 8 10 11 9 12 9
```

9.5.2 Visualising the Nested Structure

```
library(ggplot2)

# Hospital means
hosp_means <- aggregate(recovery ~ treatment + hospital + region, data = physio, FUN = mean)

ggplot(physio, aes(x = treatment, y = recovery, colour = hospital)) +
  geom_jitter(width = 0.15, alpha = 0.4, size = 1.5) +
  geom_point(data = hosp_means, aes(y = recovery), size = 4, shape = 18) +
  geom_line(data = hosp_means, aes(y = recovery, group = hospital), alpha = 0.5) +
  facet_wrap(~ region, labeller = label_both) +
  scale_colour_viridis_d(option = "D") +
  labs(x = NULL, y = "Recovery score", colour = "Hospital") +
  theme_bw() +
  theme(legend.position = "bottom")
```

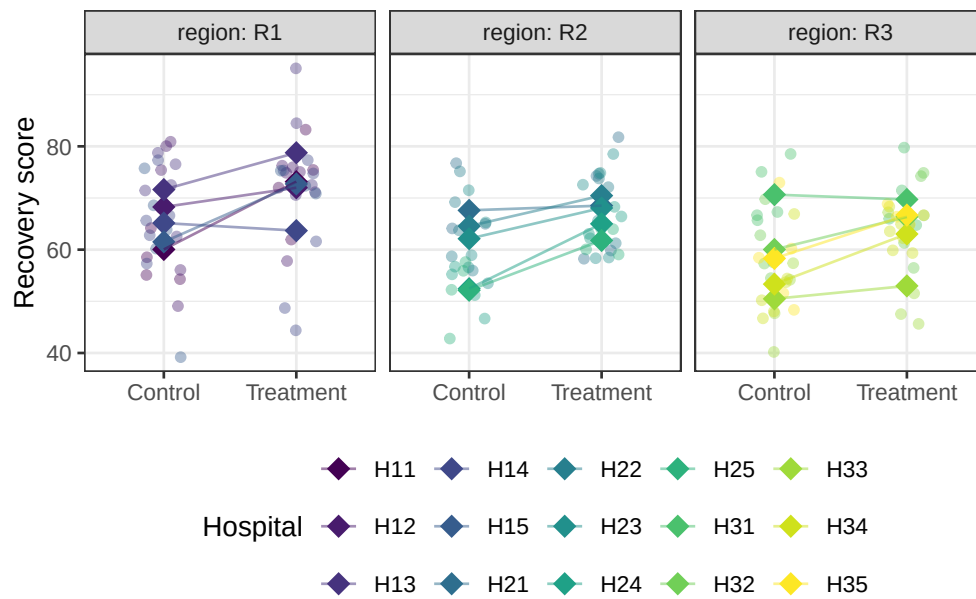


Figure 9.3: Recovery scores by treatment group, hospital, and region. Each panel is a region; within each panel, points are coloured by hospital. The substantial variation in hospital means within and across regions illustrates why the nested random effect structure must be modelled explicitly.

9.5.3 Fitting the Three-Level Nested Model

```
library(lme4)
```

```
library(lmerTest)
```

```
#>
```

```
#> Attaching package: 'lmerTest'
```

```
#> The following object is masked from 'package:lme4':
```

```
#>
```

```
#> lmer
```

```
#> The following object is masked from 'package:stats':
```

```
#>
```

```
#>      step
```

```
# Three-level nested model:
# hospital nested within region, both as random intercepts
fit_nested <- lmer(
  recovery ~ treatment + (1 | region/hospital),
  data = physio,
  REML = TRUE
)

summary(fit_nested)
```

```
#> Linear mixed model fit by REML. t-tests use Satterthwaite's method [
```

```
#> lmerModLmerTest]
```

```
#> Formula: recovery ~ treatment + (1 | region/hospital)
```

```
#>      Data: physio
```

```
#>
```

```
#> REML criterion at convergence: 1053.2
```

```
#>
```

```
#> Scaled residuals:
```

```
#>      Min      1Q   Median      3Q      Max
```

```
#> -3.07356 -0.62171  0.09592  0.59586  2.31967
```

```
#>
```

```
#> Random effects:
```

```
#> Groups      Name      Variance Std.Dev.
```

```
#> hospital:region (Intercept) 22.799  4.775
```

```
#> region      (Intercept)  8.732  2.955
```

```
#> Residual                64.117  8.007
```

```
#> Number of obs: 148, groups:  hospital:region, 15; region, 3
```

```

#>
#> Fixed effects:
#>
#>               Estimate Std. Error      df t value Pr(>|t|)
#> (Intercept)      61.329      2.294    2.351  26.740 0.000564 ***
#> treatmentTreatment    6.097      1.320  132.003    4.619 9.03e-06 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Correlation of Fixed Effects:
#>
#>      (Intr)
#> trtmntTrtmn -0.272

```

The (1 | region/hospital) notation specifies hospital nested within region, equivalent to (1 | region) + (1 | region:hospital). It produces three variance components: region-level variance ($\hat{\sigma}_{\text{region}}^2$), hospital-within-region variance ($\hat{\sigma}_{\text{hospital}}^2$), and residual patient-level variance ($\hat{\sigma}_{\varepsilon}^2$).

The summary output reveals a clear and well-structured result across all three sections.

Random effects: three variance components are estimated, corresponding to the three levels of the hierarchy. The hospital-within-region variance is $\hat{\sigma}_{\text{hospital}}^2 = 22.80$ (SD = 4.78), the region variance is $\hat{\sigma}_{\text{region}}^2 = 8.73$ (SD = 2.96), and the residual patient-level variance is $\hat{\sigma}_{\varepsilon}^2 = 64.12$ (SD = 8.01). The total variance is therefore $22.80 + 8.73 + 64.12 = 95.65$, of which the clustering structure (hospital and region combined) accounts for $(22.80 + 8.73)/95.65 = 33\%$. This is a substantial ICC: one third of the variation in recovery scores is attributable to which hospital and region a patient belongs to, entirely independent of treatment. An analysis that ignored this structure would dramatically underestimate the uncertainty in the treatment effect estimate.

Fixed effects: the treatment coefficient is $\hat{\beta} = 6.10$ (SE = 1.32), with a Satterthwaite-approximated $t_{132} = 4.62$, $p < 0.001$. Patients receiving the new physiotherapy protocol

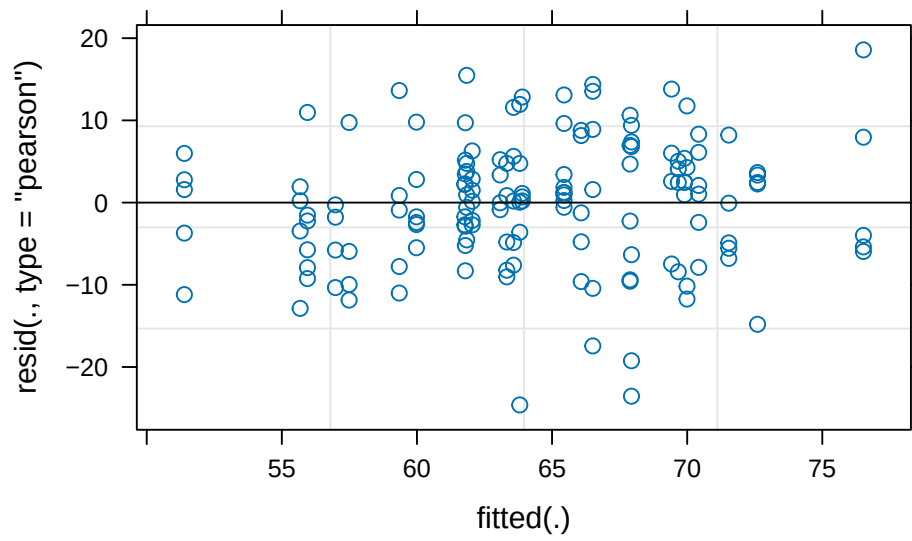
recovered an estimated 6.1 points higher on the recovery scale than control patients, after accounting for the hospital and region clustering. This is a clinically meaningful and statistically convincing result. Notice that the degrees of freedom for the treatment test (132) are much larger than those for the intercept (2.35) reflecting the fact that treatment was randomised at the patient level (148 patients), while the intercept is estimated at the region level (only 3 regions), where very little information is available. The Satterthwaite approximation correctly propagates this distinction into the inference.

The correlation between the intercept and treatment coefficient estimates is -0.27 , indicating modest negative correlation: hospitals with higher baseline recovery tend to show slightly smaller treatment effects in the estimates, though this could be a property of the estimation geometry rather than a substantive finding.

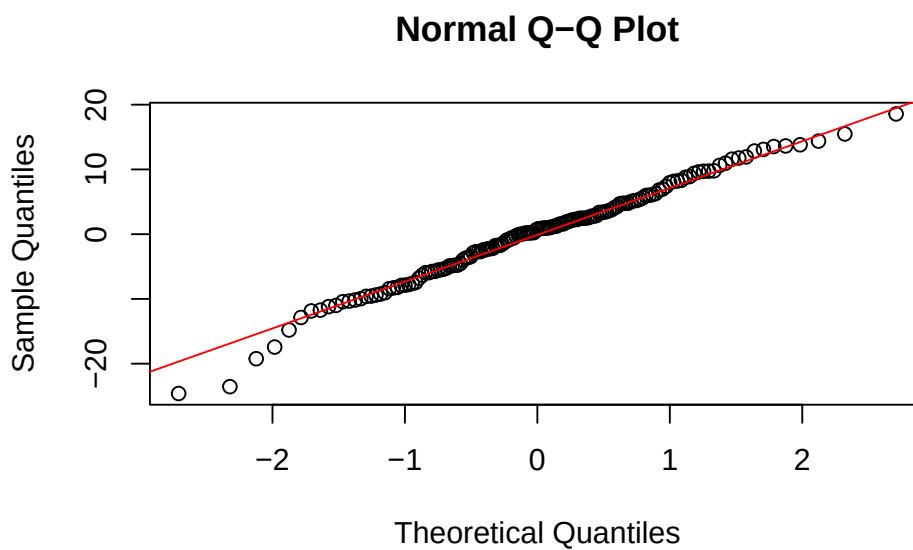
Number of observations: 148 patients are nested in 15 hospital-region combinations, which are in turn nested in 3 regions. The modest number of regions (3) is the binding constraint on inference about region-level effects and explains the very low degrees of freedom (2.35) for the intercept test. With only 3 regions, the region-level variance estimate ($\hat{\sigma}_{\text{region}}^2 = 8.73$) should be interpreted cautiously as it is based on very sparse information and carries wide uncertainty.

9.5.4 Diagnosing the Model

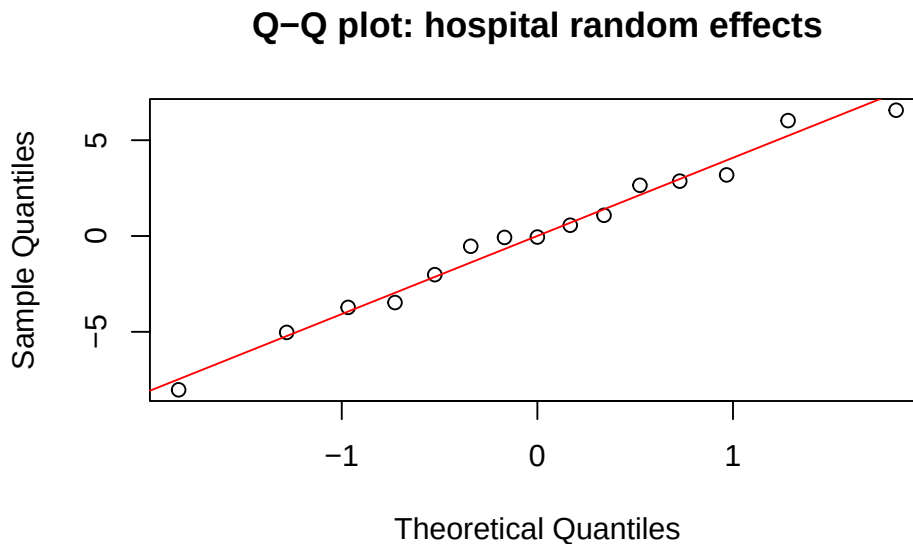
```
# Residual plot  
plot(fit_nested)
```



```
# Q-Q plot of residuals
qqnorm(resid(fit_nested))
qqline(resid(fit_nested), col = "red")
```




```
# Q-Q plot of random effects – should also be approximately normal
qqnorm(ranef(fit_nested)$hospital[, 1], main = "Q-Q plot: hospital random effects")
qqline(ranef(fit_nested)$hospital[, 1], col = "red")
```



A well-fitted mixed model should show approximately normal residuals *and* approximately normal random effects. The random effects Q-Q plot is often overlooked: it tests the normality assumption on the b_j directly, not just on the observation-level residuals.

9.5.5 Testing the Fixed Effect

```
# F test with Satterthwaite degrees of freedom
anova(fit_nested)
```

```
#> Type III Analysis of Variance Table with Satterthwaite's method
#>           Sum Sq Mean Sq NumDF DenDF F value    Pr(>F)
#> treatment 1368.2  1368.2     1   132   21.34 9.028e-06 ***
```

```
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# F test with Kenward-Roger degrees of freedom (preferred for small n)
anova(fit_nested, ddf = "Kenward-Roger")

#> Type III Analysis of Variance Table with Kenward-Roger's method
#>               Sum Sq Mean Sq NumDF  DenDF F value    Pr(>F)
#> treatment 1367.9   1367.9      1 132.15   21.334 9.044e-06 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Both the Satterthwaite and Kenward-Roger F tests converge on the same conclusion, and the agreement between them is reassuring. The Satterthwaite test gives $F_{1,132.00} = 21.34$, $p < 0.001$; the Kenward-Roger test gives $F_{1,132.15} = 21.33$, $p < 0.001$. The two methods produce virtually identical results here because the dataset is reasonably large at the patient level (148 observations) and the design, while unbalanced, is not severely so. In a study with fewer patients or more extreme imbalance across hospitals and regions, the two methods would diverge more noticeably, and the Kenward-Roger result would be the more trustworthy of the two.

The denominator degrees of freedom (132) confirm what the summary output already showed: treatment is being tested at the patient level, where replication is adequate. This is the correct behaviour as the new protocol was randomised to individual patients, so the patient level is the right level at which to evaluate it.

```
# Effect size: marginal and conditional R-squared
# install.packages("MuMIn") if needed
library(MuMIn)
r.squaredGLMM(fit_nested)
```

```
#>           R2m      R2c
#> [1,] 0.08885778 0.389217
```

`r.squaredGLMM()` returns two R^2 values for mixed models:

- **Marginal R^2 (R_m^2):** the proportion of variance explained by the fixed effects alone, here, the treatment effect.
- **Conditional R^2 (R_c^2):** the proportion of variance explained by both fixed and random effects together which is the full model.

The difference between them ($R_c^2 - R_m^2$) reflects how much of the total variation is attributable to the random effects, in this example it correspond to the hospital and region clustering structure.

In the example, the R^2 values from `r.squaredGLMM()` decompose the explained variance informatively. The marginal R^2 is $R_m^2 = 0.089$, meaning the treatment fixed effect alone explains approximately 9% of the total variation in recovery scores. The conditional R^2 is $R_c^2 = 0.389$, meaning the full model explains 39% of total variation. The difference, $R_c^2 - R_m^2 = 0.300$, is the contribution of the hospital and region clustering structure: 30% of total variation is attributable to which hospital and region a patient belongs to, over and above the treatment effect. This is a striking reminder of why the random effects cannot be ignored: the clustering explains more than three times as much variation as the treatment itself, and failing to account for it would have produced severely misleading standard errors and p-values for the treatment comparison.

9.5.6 Comparing Nested Models

To formally test whether the random effects are necessary, whether hospital and region clustering is real, compare models with and without the random effects using likelihood ratio tests:

```
# Fit models with ML for comparison of fixed effects
fit_full <- lme4::lmer(recovery ~ treatment + (1 | region/hospital),
                      data = physio, REML = FALSE)
fit_no_h <- lme4::lmer(recovery ~ treatment + (1 | region),
                      data = physio, REML = FALSE)

# Is hospital-within-region variance significant?
# Compare model with and without hospital-level random effect
anova(fit_no_h, fit_full)
```

```
#> Data: physio
#> Models:
#> fit_no_h: recovery ~ treatment + (1 | region)
#> fit_full: recovery ~ treatment + (1 | region/hospital)
#>          npar    AIC    BIC logLik -2*log(L) Chisq Df Pr(>Chisq)
#> fit_no_h    4 1085.1 1097.1 -538.56   1077.1
#> fit_full    5 1068.8 1083.8 -529.40   1058.8 18.33  1 1.858e-05 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Is region variance significant?
# Compare model with and without region-level random effect
fit_no_r <- lme4::lmer(recovery ~ treatment + (1 | hospital),
                      data = physio, REML = FALSE)
anova(fit_no_r, fit_full)
```

```
#> Data: physio
#> Models:
#> fit_no_r: recovery ~ treatment + (1 | hospital)
```

```
#> fit_full: recovery ~ treatment + (1 | region/hospital)
#>          npar    AIC    BIC  logLik -2*log(L)  Chisq Df Pr(>Chisq)
#> fit_no_r     4 1067.1 1079.1 -529.57   1059.1
#> fit_full     5 1068.8 1083.8 -529.40   1058.8 0.3455  1    0.5567
```

```
# Alternative: ranova() tests each random effect term directly
# without needing to refit models manually
lmerTest::ranova(
  lmerTest::lmer(recovery ~ treatment + (1 | region/hospital),
    data = physio, REML = TRUE)
)
```

```
#> ANOVA-like table for random-effects: Single term deletions
#>
#> Model:
#> recovery ~ treatment + (1 | hospital:region) + (1 | region)
#>
#>          npar  logLik    AIC    LRT Df Pr(>Chisq)
#> <none>          5 -526.60 1063.2
#> (1 | hospital:region)  4 -535.64 1079.3 18.0884  1 2.109e-05 ***
#> (1 | region)          4 -527.04 1062.1  0.8802  1    0.3481
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The model comparison results tell a nuanced but clear story about the necessity of each level of the random effects structure.

The hospital-within-region random effect is strongly supported by the data. Removing it significantly worsens the model fit ($\chi^2_1 = 18.33$, $p < 0.001$), and the AIC drops from 1085.1 to 1068.8 when hospital-level clustering is included. The 18 unit improvement in log-likelihood is substantial, the 15 hospitals differ from each other in their baseline recovery levels in ways that

cannot be explained by region alone, and ignoring this hospital-level clustering would produce incorrect standard errors for the treatment comparison.

The region random effect, however, tells a different story. Removing it does not significantly worsen the fit ($\chi^2_1 = 0.35$, $p = 0.557$), and the AIC is actually slightly lower without it (1067.1 vs 1068.8). The `ranova()` output confirms this: the hospital-within-region term is highly significant ($\chi^2_1 = 18.09$, $p < 0.001$) while the region term is not ($\chi^2_1 = 0.88$, $p = 0.348$). With only 3 regions in the study, the region-level variance ($\hat{\sigma}^2_{\text{region}} = 8.73$) is estimated from very sparse information and the likelihood ratio test has almost no power to detect it even if it is real. This is a common situation in hierarchical biological studies: the highest level of the hierarchy is almost always the most sparsely replicated, and non-significant tests for top-level random effects should be interpreted as reflecting limited power rather than genuine absence of region-level variation. In this case, retaining the region random effect is the scientifically conservative and appropriate choice. Indeed, the three regions were sampled from a larger population of regions and their variation should be accounted for regardless of the test result.

9.5.7 Reporting the Result

```
# Final model refitted with REML for reporting
fit_final <- lmerTest::lmer(
  recovery ~ treatment + (1 | region/hospital),
  data = physio, REML = TRUE
)
summary(fit_final)
```

```
#> Linear mixed model fit by REML. t-tests use Satterthwaite's method [
#> lmerModLmerTest]
#> Formula: recovery ~ treatment + (1 | region/hospital)
#> Data: physio
#>
```

9 Mixed-Effects Models: Beyond Classical ANOVA

```
#> REML criterion at convergence: 1053.2
#>
#> Scaled residuals:
#>      Min       1Q   Median       3Q      Max
#> -3.07356 -0.62171  0.09592  0.59586  2.31967
#>
#> Random effects:
#>   Groups          Name          Variance Std.Dev.
#> hospital:region (Intercept) 22.799    4.775
#> region          (Intercept)  8.732    2.955
#> Residual                        64.117    8.007
#> Number of obs: 148, groups:  hospital:region, 15; region, 3
#>
#> Fixed effects:
#>              Estimate Std. Error      df t value Pr(>|t|)
#> (Intercept)      61.329      2.294    2.351  26.740 0.000564 ***
#> treatmentTreatment    6.097      1.320  132.003   4.619 9.03e-06 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Correlation of Fixed Effects:
#>              (Intr)
#> trtmntTrtmn -0.272
```

```
anova(fit_final, ddf = "Kenward-Roger")
```

```
#> Type III Analysis of Variance Table with Kenward-Roger's method
#>              Sum Sq Mean Sq NumDF  DenDF F value    Pr(>F)
#> treatment 1367.9  1367.9      1 132.15  21.334 9.044e-06 ***
#> ---
```

```
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
r.squaredGLMM(fit_final)
```

```
#>           R2m      R2c
#> [1,] 0.08885778 0.389217
```

A complete report of the three-level nested analysis:

Recovery scores following knee replacement surgery were analysed using a three-level linear mixed-effects model with treatment (control vs new physiotherapy protocol) as a fixed effect and random intercepts for hospital nested within region, fitted by REML using `lme4` version 1.x (?). Degrees of freedom for fixed effect tests were approximated using the Kenward-Roger method (?) via `lmerTest` (?).

The new physiotherapy protocol was associated with significantly higher recovery scores than the standard protocol ($\beta = 6.10$, $SE = 1.32$, $F_{1,132.15} = 21.33$, $p < 0.001$). The marginal R^2 was 0.089, indicating that the treatment effect alone explained 9% of total variance in recovery scores. The conditional R^2 was 0.389, with the random effects accounting for an additional 30% of total variance, confirming that the hierarchical structure of the data (patients clustered within hospitals, hospitals clustered within regions) was a substantial source of variation that required explicit modelling.

Likelihood ratio tests confirmed significant variance at the hospital-within-region level ($\chi^2_1 = 18.33$, $p < 0.001$), with estimated hospital-level standard deviation of 4.78 recovery points. Region-level variance did not reach significance ($\chi^2_1 = 0.35$, $p = 0.557$), though with only three regions the test had limited power, and the region random effect was retained in the model on scientific grounds. Residual patient-level standard deviation was 8.01 recovery points.

9.6 Repeated Measures as a Mixed Model

9.6.1 The Connection to Chapter 6

Chapter 6 analysed repeated measures data using `aov()` with an `Error()` term which is the classical approach that has been standard in biology for decades. That approach works well for balanced designs with complete data, but it has three important limitations that become apparent in practice: it cannot handle missing observations, it assumes a specific covariance structure (compound symmetry, closely related to sphericity), and it becomes unwieldy with more than two levels of nesting.

Mixed-effects models handle all three limitations naturally. And because the repeated measures model is already a mixed model in disguise, subjects are a random effect, time is a fixed effect, the transition requires no new conceptual machinery. It is simply a matter of fitting the same model with `lmer()` instead of `aov()`.

9.6.2 Refitting the Clinical Example

We return to the cardiac rehabilitation trial from Chapter 6: 40 patients (20 control, 20 intervention) measured at weeks 0, 4, 8, and 12 (Section 6.1). We first refit the classical repeated measures ANOVA, then refit the same model as a mixed-effects model, and compare the results directly.

```
library(lme4)
library(lmerTest)

# Classical repeated measures ANOVA from Chapter 6
fit_rm_classical <- aov(
  hr ~ group * time + Error(subject/time),
  data = cardiac_recovery
```

```
)
summary(fit_rm_classical)

#>
#> Error: subject
#>           Df Sum Sq Mean Sq F value    Pr(>F)
#> group      1  883.3   883.3   19.09 9.29e-05 ***
#> Residuals 38 1757.9    46.3
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Error: subject:time
#>           Df Sum Sq Mean Sq F value    Pr(>F)
#> time        3 1811.6   603.9   66.98 < 2e-16 ***
#> group:time   3  598.8   199.6   22.14 2.28e-11 ***
#> Residuals 114 1027.7     9.0
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# The same model as a linear mixed-effects model
# subject is the random effect; group and time are fixed
fit_rm_mixed <- lmerTest::lmer(
  hr ~ group * time + (1 | subject),
  data = cardiac_recovery,
  REML = TRUE
)
summary(fit_rm_mixed)
```

```
#> Linear mixed model fit by REML. t-tests use Satterthwaite's method [
```

9 Mixed-Effects Models: Beyond Classical ANOVA

```
#> lmerModLmerTest]
#> Formula: hr ~ group * time + (1 | subject)
#> Data: cardiac_recovery
#>
#> REML criterion at convergence: 851.7
#>
#> Scaled residuals:
#>      Min       1Q   Median       3Q      Max
#> -3.00986 -0.60094  0.06222  0.49595  2.85033
#>
#> Random effects:
#> Groups   Name          Variance Std.Dev.
#> subject (Intercept) 9.311     3.051
#> Residual              9.015     3.002
#> Number of obs: 160, groups: subject, 40
#>
#> Fixed effects:
#>
#>              Estimate Std. Error    df t value Pr(>|t|)
#> (Intercept)      80.9464    0.9572  85.6600   84.562 < 2e-16 ***
#> groupIntervention    0.2611    1.3537  85.6600    0.193  0.84752
#> time4             -2.1052    0.9495 114.0000   -2.217  0.02859 *
#> time8             -3.0179    0.9495 114.0000   -3.179  0.00191 **
#> time12            -3.9646    0.9495 114.0000   -4.176 5.84e-05 ***
#> groupIntervention:time4 -2.5790    1.3428 114.0000   -1.921  0.05727 .
#> groupIntervention:time8 -7.4799    1.3428 114.0000   -5.571 1.72e-07 ***
#> groupIntervention:time12 -9.7825    1.3428 114.0000   -7.285 4.48e-11 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
```

```
#> Correlation of Fixed Effects:
#>          (Intr) grpInt time4  time8  time12 grpI:4 grpI:8
#> grpIntrvntn -0.707
#> time4        -0.496  0.351
#> time8        -0.496  0.351  0.500
#> time12       -0.496  0.351  0.500  0.500
#> grpIntrvn:4  0.351 -0.496 -0.707 -0.354 -0.354
#> grpIntrvn:8  0.351 -0.496 -0.354 -0.707 -0.354  0.500
#> grpIntrv:12  0.351 -0.496 -0.354 -0.354 -0.707  0.500  0.500
```

```
anova(fit_rm_mixed, ddf = "Satterthwaite")
```

```
#> Type III Analysis of Variance Table with Satterthwaite's method
#>          Sum Sq Mean Sq NumDF DenDF F value    Pr(>F)
#> group      172.14  172.14     1    38  19.095 9.292e-05 ***
#> time      1811.55  603.85     3   114  66.983 < 2.2e-16 ***
#> group:time  598.77  199.59     3   114  22.140 2.280e-11 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

With complete, balanced data the two approaches give the same F statistics, the same degrees of freedom, and the same p-values for every effect. This equivalence is not a coincidence: the `Error()` specification in `aov()` and the `(1 | subject)` random intercept in `lmer()` are two syntaxes for the same statistical model. The subject random intercept absorbs the stable between-patient differences in heart rate, leaving the within-subject residual as the error term against which the time and group-by-time effects are tested which is exactly what the `Error(subject/time)` stratum does in the classical output.

9.6.3 Where the Two Approaches Diverge

The equivalence breaks down in three practically important situations.

Missing data. `aov()` with `Error()` performs listwise deletion: any subject with a missing observation at any time point is excluded entirely from the analysis. `lmer()` uses REML or ML estimation, which retains all available observations under the missing-at-random assumption. In a 12-week clinical trial where some patients miss appointments, this difference can be substantial.

```
# Introduce missing data: remove 5 random observations
set.seed(42)
df_missing <- cardiac_recovery
missing_idx <- sample(nrow(df_missing), 5)
df_missing$hr[missing_idx] <- NA

# Classical approach: drops entire subjects with any missing value
fit_classical_miss <- aov(
  hr ~ group * time + Error(subject/time),
  data = df_missing
)
cat("Classical ANOVA, subjects retained:",
    length(unique(
      df_missing$subject[complete.cases(df_missing)]
    )), "\n")
```

```
#> Classical ANOVA, subjects retained: 40
```

```
# Mixed model: retains all subjects, uses all available observations
fit_mixed_miss <- lmerTest::lmer(
  hr ~ group * time + (1 | subject),
```

```

data = df_missing,
REML = TRUE,
na.action = na.omit
)
cat("Mixed model, observations used:", nobs(fit_mixed_miss), "\n")

```

```
#> Mixed model, observations used: 155
```

Unequal time spacing. `aov()` treats the time factor as a nominal categorical variable making the spacing between time points is irrelevant. `lmer()` can model time as a continuous variable, allowing the trajectory to be estimated as a smooth function (linear, quadratic, or piecewise) rather than a step function. This is more parsimonious and often more biologically sensible.

```

# Group means per time point
summ <- aggregate(hr ~ group + time, data = cardiac_recovery, FUN = mean)
summ$time_num <- as.numeric(as.character(summ$time))

cardiac_recovery$time_num <- as.numeric(as.character(cardiac_recovery$time))

# Model time as continuous, estimates a linear trajectory
# rather than separate means at each time point
fit_rm_linear <- lmerTest::lmer(
  hr ~ group * time_num + (1 | subject),
  data = cardiac_recovery,
  REML = TRUE
)

anova(fit_rm_linear, ddf = "Kenward-Roger")

```

```
#> Type III Analysis of Variance Table with Kenward-Roger's method
```

9 Mixed-Effects Models: Beyond Classical ANOVA

```
#>
      Sum Sq Mean Sq NumDF   DenDF F value    Pr(>F)
#> group          1.10    1.10     1   66.583    0.1229    0.727
#> time_num      1791.70 1791.70     1  118.000  199.4800 < 2.2e-16 ***
#> group:time_num  586.47  586.47     1  118.000   65.2953 6.281e-13 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Model time as continuous with random slope
# allows each patient to have their own rate of change
fit_rm_erslope <- lmerTest::lmer(
  hr ~ group * time_num + (time_num | subject),
  data = cardiac_recovery,
  REML = TRUE
)
anova(fit_rm_erslope, ddf = "Kenward-Roger")
```

```
#> Type III Analysis of Variance Table with Kenward-Roger's method
#>
      Sum Sq Mean Sq NumDF DenDF F value    Pr(>F)
#> group          0.91    0.91     1    38    0.1451    0.7054
#> time_num       762.91  762.91     1    38  121.8251 2.049e-13 ***
#> group:time_num  249.72  249.72     1    38   39.8768 2.108e-07 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Does the random slope improve fit?
# Use ML for model comparison
rm_linear_ml <- lme4::lmer(
  hr ~ group * time_num + (1 | subject),
  data = cardiac_recovery, REML = FALSE
)
```

```
rm_rslope_ml <- lme4::lmer(
  hr ~ group * time_num + (time_num | subject),
  data = cardiac_recovery, REML = FALSE
)
anova(rm_linear_ml, rm_rslope_ml)
```

```
#> Data: cardiac_recovery
#> Models:
#> rm_linear_ml: hr ~ group * time_num + (1 | subject)
#> rm_rslope_ml: hr ~ group * time_num + (time_num | subject)
#>
#>      npar    AIC    BIC logLik -2*log(L)  Chisq Df Pr(>Chisq)
#> rm_linear_ml    6 878.79 897.24 -433.39    866.79
#> rm_rslope_ml    8 870.27 894.87 -427.14    854.27 12.517  2  0.001914 **
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The random slope model (`time_num | subject`) asks whether patients differ not only in their baseline heart rate (random intercept) but also in their *rate of recovery* (random slope). The significant improvement in fit when adding the random slope confirms that individual recovery trajectories genuinely differ. This is a clinically meaningful finding about treatment response heterogeneity that the classical repeated measures ANOVA cannot detect at all.

Covariance structure. The classical repeated measures model assumes compound symmetry that is equal variances at every time point and equal correlations between every pair of time points. `lmer()` with a simple random intercept implies the same structure. But more flexible covariance structures can be specified using `nlme`, which is worth knowing for situations where the compound symmetry assumption is clearly wrong:


```
library(nlme)

# Compound symmetry (equivalent to random intercept in lmer)
fit_cs <- lme(hr ~ group * time,
              random = ~ 1 | subject,
              data = cardiac_recovery,
              method = "REML")

# Autoregressive AR(1): correlations decay with time distance
# observations closer in time are more correlated than
# observations further apart which is often more realistic biologically
fit_ar1 <- lme(hr ~ group * time,
               random = ~ 1 | subject,
               correlation = corAR1(form = ~ 1 | subject),
               data = cardiac_recovery,
               method = "REML")

# Compare covariance structures using AIC (refit with ML)
fit_cs_ml <- update(fit_cs, method = "ML")
fit_ar1_ml <- update(fit_ar1, method = "ML")
anova(fit_cs_ml, fit_ar1_ml)
```

```
#>           Model df      AIC      BIC    logLik   Test  L.Ratio p-value
#> fit_cs_ml      1 10 883.0908 913.8426 -431.5454
#> fit_ar1_ml     2 11 870.8304 904.6573 -424.4152 1 vs 2 14.26045 2e-04
```

An AR(1) structure assumes that the correlation between two measurements decays exponentially with the time distance between them, for example measurements one week apart are more correlated than measurements four weeks apart. This is biologically plausible for most

longitudinal physiological measurements and often fits better than the compound symmetry assumption imposed by the classical repeated measures model.

9.6.4 A Practical Recommendation

The decision between `aov()` and `lmer()` for repeated measures data can be guided by a simple set of questions:

Situation	Recommended approach
Balanced design, complete data, simple structure	Either, results identical
Missing observations	<code>lmer()</code> retains all available data
Unequally spaced time points	<code>lmer()</code> with continuous time
Interest in individual trajectories	<code>lmer()</code> with random slopes
Non-compound-symmetry covariance	<code>nlme</code> with explicit correlation structure
Sphericity clearly violated	<code>lmer()</code> no sphericity assumption needed

The broader message is the one that runs through this entire chapter: the classical ANOVA approach and the mixed-effects approach are not competing traditions: they are the same model expressed in different frameworks, with the mixed-effects framework being strictly more general. Mastering `lmer()` does not replace what you learned in Chapter [?@sec-repeated](#); it extends it, and the extension becomes essential the moment your data depart from the idealised balanced, complete, compound-symmetric structure that classical repeated measures ANOVA requires.

10 When Assumptions Fail, What to Do ?

Every ANOVA and linear mixed model fitted in this book rests on assumptions. Chapter 2 examined those assumptions in depth and showed that their consequences are not equal: independence is critical, homoscedasticity is important, normality is often a minor concern. But what happens when the data genuinely violate one or more of these assumptions in ways that cannot be ignored?

This chapter provides a practical toolkit. It begins with transformations, the classical first response to assumption violations, and examines both their genuine utility and their limitations. It then introduces non-parametric alternatives that make no distributional assumptions at all. It also presents robust ANOVA methods that explicitly downweight the influence of outliers and heavy tails. And it ends with the argument that for many of the situations where assumptions fail most seriously (count data, proportions, survival times), the correct solution is not a patch applied to the normal linear model but a fundamentally different model class: the generalised linear model (GLM).

10.1 Transformations: Log, Square Root, Arcsine and Their Modern Critics

10.1.1 The Classical Rationale

Transformations have been the standard first response to non-normality and heteroscedasticity in biological data since Fisher's time. The logic is simple: if the response variable y violates

the assumptions of ANOVA on its original scale, perhaps a transformed version $f(y)$ will not. The three transformations most commonly encountered in biology are the log, square root, and arcsine transformations.

10.1.2 The Log Transformation

The log transformation is appropriate when the response is strictly positive and the within-group standard deviation is proportional to the mean, a pattern called **multiplicative variation** or **constant coefficient of variation**. This arises naturally when biological processes operate multiplicatively: cell counts doubling, concentrations following pharmacokinetic decay, body masses scaling allometrically.

$$y' = \log(y) \quad \text{or} \quad y' = \log(y + 1) \text{ if zeros are present}$$

```
library(ggplot2)
library(patchwork)

set.seed(42)
n <- 30

# Multiplicative data: sd proportional to mean
group <- factor(rep(c("Control", "Treatment A", "Treatment B"), each = n),
               levels = c("Control", "Treatment A", "Treatment B"))
y_raw <- c(rlnorm(n, meanlog = 2.0, sdlog = 0.8),
          rlnorm(n, meanlog = 2.8, sdlog = 0.8),
          rlnorm(n, meanlog = 3.5, sdlog = 0.8))
y_log <- log(y_raw)

df_trans <- data.frame(y_raw, y_log, group)
```

```
p1 <- ggplot(df_trans, aes(x = group, y = y_raw, fill = group)) +  
  geom_boxplot(alpha = 0.5) +  
  geom_jitter(width = 0.1, alpha = 0.4, size = 1.5) +  
  scale_fill_manual(values = c("#2E86AB", "#F6AE2D", "#E84855")) +  
  labs(title = "Raw scale", x = NULL, y = "Response") +  
  theme_bw() + theme(legend.position = "none")  
  
p2 <- ggplot(df_trans, aes(x = group, y = y_log, fill = group)) +  
  geom_boxplot(alpha = 0.5) +  
  geom_jitter(width = 0.1, alpha = 0.4, size = 1.5) +  
  scale_fill_manual(values = c("#2E86AB", "#F6AE2D", "#E84855")) +  
  labs(title = "Log scale", x = NULL, y = "log(Response)") +  
  theme_bw() + theme(legend.position = "none")  
  
p1 + p2
```

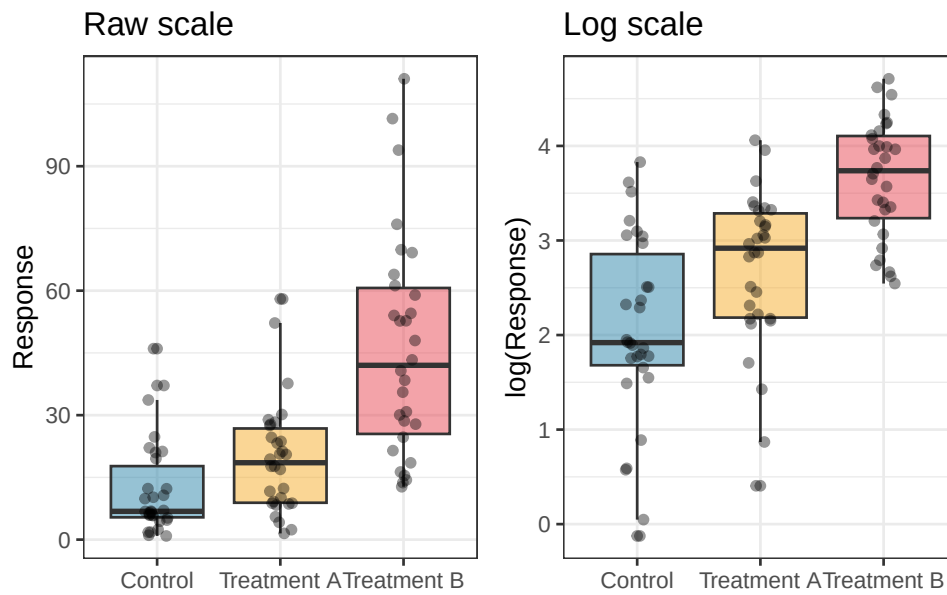


Figure 10.1: Effect of log transformation on a right-skewed response. Left: raw counts with a long right tail and variance increasing with the mean. Right: log-transformed values with approximately symmetric distribution and homogeneous variance across groups. The log transformation is appropriate when within-group standard deviation is proportional to the group mean.

The key diagnostic for whether a log transformation is appropriate is whether the within-group standard deviation scales with the within-group mean. A plot of group standard deviation against group mean (Section 5.1) will reveal this pattern. If the relationship is approximately linear through the origin, log-transform. If it is not, the log transformation may not be the right choice.

10.1.3 The Square Root Transformation

The square root transformation is appropriate for count data where the variance is approximately equal to the mean, the pattern expected from a Poisson distribution:

$$y' = \sqrt{y} \quad \text{or} \quad y' = \sqrt{y + 0.5} \text{ if zeros are present}$$

```
set.seed(42)

# Poisson-like count data
y_count <- c(rpois(n, lambda = 3),
             rpois(n, lambda = 8),
             rpois(n, lambda = 20))

df_count <- data.frame(
  y = y_count,
  y_sqrt = sqrt(y_count),
  group = group
)

# Compare residual variance before and after transformation
fit_raw <- aov(y ~ group, data = df_count)
fit_sqrt <- aov(y_sqrt ~ group, data = df_count)

cat("Levene's test on raw counts:\n")
```

```
#> Levene's test on raw counts:
```

```
print(car::leveneTest(y ~ group, data = df_count))
```

```
#> Levene's Test for Homogeneity of Variance (center = median)
```

```
#>      Df F value    Pr(>F)
```

```
#> group  2  6.4379 0.002469 **
```

```
#>      87
```

```
#> ---
```

```
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
cat("\nLevene's test on square root transformed counts:\n")
```

```
#>
```

```
#> Levene's test on square root transformed counts:
```

```
print(car::leveneTest(y_sqrt ~ group, data = df_count))
```

```
#> Levene's Test for Homogeneity of Variance (center = median)
```

```
#>      Df F value Pr(>F)
```

```
#> group  2  0.2219 0.8014
```

```
#>      87
```

The Levene's test results demonstrate the variance-stabilising effect of the square root transformation clearly. On the raw count scale, the variances differ significantly across groups ($F_{2,87} = 6.44$, $p = 0.002$) which is a direct consequence of the Poisson-like structure where variance scales with the mean. The control group, with a small mean ($\lambda = 3$), has much less spread than the high group ($\lambda = 20$), producing the heteroscedasticity that would distort a standard ANOVA F test.

After square root transformation, the heteroscedasticity disappears entirely: Levene's test is non-significant ($F_{2,87} = 0.22$, $p = 0.801$) and the variances are now approximately equal across groups. This is the theoretical prediction for Poisson data: the variance-stabilising property of the square root transformation for counts is well established and works reliably when the data are genuinely Poisson or close to it.

Two caveats are worth noting for students. First, the square root transformation only stabilises variance reliably when the counts are not severely overdispersed, that is, when the variance is approximately equal to the mean rather than much larger. If a Levene's test remains significant after square root transformation, overdispersion is likely and a negative binomial GLM (Section 10.4) is the more appropriate solution. Second, as with the log transformation, the square root transformation changes the scale of analysis: differences between group means

are now differences in square-root-counts, not differences in raw counts. Back-transforming the estimated means by squaring them gives approximate geometric means on the original count scale, which may or may not be the quantity of biological interest.

10.1.4 The Arcsine Square Root Transformation

The arcsine square root transformation was historically recommended for proportions and percentages, with the goal of stabilising the variance of binomial data:

$$y' = \arcsin(\sqrt{p})$$

```
set.seed(42)

# Simulate binomial proportions across three groups
# with equal true proportions but different sample sizes
# Variance of a binomial proportion is p(1-p)/n
# it depends on p, which is what the arcsine transformation
# is designed to stabilise

n_obs <- 50    # observations per group
n_trials <- 20 # number of trials per observation

# Three groups with different true proportions
true_p <- c(0.10, 0.45, 0.85)
groups <- factor(rep(c("Low", "Mid", "High"), each = n_obs),
                 levels = c("Low", "Mid", "High"))

# Simulate observed proportions
props <- c(
```

```
rbinom(n_obs, n_trials, true_p[1]) / n_trials,
rbinom(n_obs, n_trials, true_p[2]) / n_trials,
rbinom(n_obs, n_trials, true_p[3]) / n_trials
)

df_asin <- data.frame(
  p      = props,
  p_asin = asin(sqrt(props)),
  group  = groups
)

# Variance of raw proportions per group
cat("Variance of raw proportions by group:\n")
```

#> Variance of raw proportions by group:

```
print(tapply(df_asin$p, df_asin$group, var))
```

```
#>           Low           Mid           High
#> 0.005739796 0.013383673 0.005918367
```

```
# Variance after arcsine transformation
cat("\nVariance of arcsine-transformed proportions by group:\n")
```

#>

#> Variance of arcsine-transformed proportions by group:

```
print(tapply(df_asin$p_asin, df_asin$group, var))
```

10 When Assumptions Fail, What to Do ?

```
#>      Low      Mid      High
#> 0.02134563 0.01532025 0.01606443
```

```
# Levene's test on raw proportions
cat("\nLevene's test on raw proportions:\n")
```

```
#>
#> Levene's test on raw proportions:
```

```
print(car::leveneTest(p ~ group, data = df_asin))
```

```
#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
#> group  2  4.5503 0.01209 *
#>      147
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Levene's test on arcsine-transformed proportions
cat("\nLevene's test on arcsine-transformed proportions:\n")
```

```
#>
#> Levene's test on arcsine-transformed proportions:
```

```
print(car::leveneTest(p_asin ~ group, data = df_asin))
```

```
#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
#> group  2  0.6611 0.5178
#>      147
```

The raw proportions show the classical binomial pattern: the group near 0.10 and the group near 0.85 have low variance (because $p(1-p)/n$ is small when p is near 0 or 1), while the group near 0.45 has the highest variance (because $p(1-p)/n$ is maximised near $p = 0.5$). Levene's test on the raw proportions is significant. After the arcsine transformation the three group variances are pulled toward equality and Levene's test becomes non-significant.

10.1.5 The Modern Critique of Transformations

Despite their long history, transformations have attracted serious criticism, and it is important to understand the limitations **before** applying them reflexively.

The arcsine transformation is now largely discredited for proportions. Warton and Hui (?) showed in an influential paper that the arcsine square root transformation performs poorly compared to directly modelling proportions with a binomial GLM. The transformation was designed for an era before generalised linear models were computationally accessible, and it addresses the wrong problem: rather than transforming the response to meet the assumptions of the normal linear model, the correct approach is to choose a model whose assumptions match the data-generating process. A proportion is generated by a binomial process ? model it with a binomial model.

Transformations change the question being asked. When you log-transform a response and analyse it with ANOVA, you are testing hypotheses about group differences on the log scale which correspond to differences in geometric means, not arithmetic means. Back-transforming the estimated differences gives *ratios*, not differences. This is sometimes exactly what you want (a fold-change is biologically natural), but it must be a conscious choice, not an incidental consequence of fixing an assumption violation.

A concrete example makes this concrete. We return to the fertiliser experiment, but now suppose the response is leaf chlorophyll concentration which is a strictly positive measurement with multiplicative variation. We fit ANOVA on both the raw and log-transformed scale and compare what the coefficients actually mean:

```
set.seed(42)

n <- 20
group <- factor(rep(c("Control", "Nitrogen", "Phosphorus"), each = n),
                levels = c("Control", "Nitrogen", "Phosphorus"))

# Simulate multiplicative data: nitrogen doubles concentration,
# phosphorus increases it by 50%
conc <- c(
  rlnorm(n, meanlog = log(10), sdlog = 0.3), # Control: mean ~10
  rlnorm(n, meanlog = log(20), sdlog = 0.3), # Nitrogen: mean ~20
  rlnorm(n, meanlog = log(15), sdlog = 0.3)  # Phosphorus: mean ~15
)

df_conc <- data.frame(conc, log_conc = log(conc), group)

# Model 1: ANOVA on raw scale
fit_raw <- lm(conc ~ group, data = df_conc)
cat("=== Coefficients on raw scale ===\n")
```

```
#> === Coefficients on raw scale ===
```

```
print(coef(fit_raw))
```

```
#>      (Intercept)  groupNitrogen groupPhosphorus
#>      11.342339      8.078672      4.270734
```

```
# Model 2: ANOVA on log scale
fit_log <- lm(log_conc ~ group, data = df_conc)
cat("\n=== Coefficients on log scale ===\n")
```

```
#>
```

```
#> === Coefficients on log scale ===
```

```
print(coef(fit_log))
```

```
#>      (Intercept)   groupNitrogen groupPhosphorus
```

```
#>      2.3601611      0.5542736      0.3476159
```

```
cat("\n=== Back-transformed log coefficients (exp) ===\n")
```

```
#>
```

```
#> === Back-transformed log coefficients (exp) ===
```

```
print(exp(coef(fit_log)))
```

```
#>      (Intercept)   groupNitrogen groupPhosphorus
```

```
#>      10.592658      1.740676      1.415688
```

Reading the two outputs side by side reveals the shift in question.

On the **raw scale**, the intercept (11.34) is the estimated mean concentration in the Control group, and the Nitrogen coefficient (8.07) is the estimated *arithmetic difference* in mean concentration between Nitrogen and Control: nitrogen adds approximately 8 units of chlorophyll on average.

On the **log scale**, the intercept is the estimated log-mean of the Control group, and the Nitrogen coefficient (0.55) is the estimated difference in log-means. This is not an arithmetic difference, it is a *log-ratio*. Back-transforming by exponentiating gives $e^{0.55} \approx 1.74$: the Nitrogen group has approximately twice the chlorophyll concentration of the Control group, a **ratio** or **fold-change**.

These are genuinely different questions:

Scale	Coefficient meaning	Biological statement
Raw	$\bar{y}_N - \bar{y}_C \approx 8.07$	Nitrogen adds 8 units of chlorophyll
Log	$\exp(\hat{\beta}_N) \approx 1.74$	Nitrogen doubles chlorophyll concentration

Neither is wrong, but they are not interchangeable. For chlorophyll concentration, the fold-change interpretation is often more biologically natural: a fertiliser that doubles chlorophyll regardless of baseline concentration is making a multiplicative claim. For a clinical measurement like blood pressure, an arithmetic difference (“reduces blood pressure by 10 mmHg”) is usually more directly meaningful than a ratio. *The choice of scale should be driven by which question is scientifically relevant, not by which transformation fixes the residual diagnostics.*

Transformations do not always fix both problems simultaneously. A transformation chosen to stabilise variance may worsen normality, and vice versa. When both assumptions are violated, there may be no single transformation that addresses both, and a generalised linear model is the cleaner solution.

The practical recommendation: use log transformations for multiplicative, positive data where the log scale has a natural biological interpretation (concentrations, fold-changes, ratios). Use square root for counts when overdispersion is modest. Avoid arcsine for proportions, use a binomial GLM instead. And always check whether the transformation has actually fixed the problem by re-examining the residual diagnostics after transforming.

10.2 Non-Parametric Alternatives

Non-parametric tests replace the raw observations with their ranks before analysis. Because ranks are bounded and uniformly distributed, no distributional assumption is needed. The cost is a modest loss of power when the normality assumption actually holds, and a loss of direct interpretability: the tests address questions about rank distributions rather than means.

10.2.1 Kruskal-Wallis Test

The Kruskal-Wallis test is the non-parametric equivalent of one-way ANOVA. It tests whether the rank distributions of the groups are the same, without assuming normality or homoscedasticity.

The test statistic is:

$$H = \frac{12}{N(N+1)} \sum_{j=1}^k \frac{R_j^2}{n_j} - 3(N+1)$$

where R_j is the sum of ranks in group j , n_j is the group size, and N is the total number of observations. Under the null hypothesis of identical distributions, H follows approximately a χ^2 distribution with $k - 1$ degrees of freedom.

```
set.seed(42)

# Strongly skewed data, normality assumption clearly violated
y_skew <- c(rlnorm(15, meanlog = 0, sdlog = 1.5),
            rlnorm(15, meanlog = 0.8, sdlog = 1.5),
            rlnorm(15, meanlog = 1.5, sdlog = 1.5))
group <- factor(rep(c("Control", "Low", "High"), each = 15))
df_kw <- data.frame(y = y_skew, group = group)

# Kruskal-Wallis test
kruskal.test(y ~ group, data = df_kw)
```

```
#>
#> Kruskal-Wallis rank sum test
#>
#> data: y by group
```


10 When Assumptions Fail, What to Do ?

```
#> Kruskal-Wallis chi-squared = 0.97932, df = 2, p-value = 0.6128
```

```
# Post-hoc pairwise comparisons after Kruskal-Wallis
# using Dunn's test with Holm correction
# install.packages("dunn.test") if needed
dunn.test::dunn.test(df_kw$y, df_kw$group, method = "holm", kw = FALSE)
```

```
#>
#>
#>          Dunn's Pairwise Comparison of x by group
#>
#>          (Holm)
#>
#> Col Mean-|
#> Row Mean |      Control      High
#> -----|-----
#>   High |    -0.556038
#>         |         0.5782
#>         |
#>   Low  |     0.430930    0.986968
#>         |     0.3333      0.4855
#>
#> FWER = 0.05
#> Reject H0 if adjusted  $p \leq \text{FWER}/2$  with stopping rule, where (unadjusted)  $p = \Pr(Z \geq |z|)$ 
```

The Kruskal-Wallis test is the right choice when:

- Sample sizes are small ($n < 10$ per group) and residuals are clearly non-normal.
- The response contains extreme outliers that are genuine biological observations rather than measurement errors.
- The response is an ordinal scale where the numerical values do not have a natural arithmetic interpretation.

It is **not** a universal replacement for ANOVA. I repeat, it is **not** a universal replacement for ANOVA. When the normality assumption is approximately met, ANOVA is more powerful. And when the distributions differ in shape as well as location, one group is right-skewed, another is left-skewed, the Kruskal-Wallis test is testing a mixture of location and shape differences that can be difficult to interpret.

10.2.2 Friedman Test

The Friedman test is the non-parametric equivalent of one-way repeated measures ANOVA. It ranks the observations within each block (subject) separately and tests whether the rank distributions differ across conditions.

$$F_r = \frac{12}{nk(k+1)} \sum_{j=1}^k R_j^2 - 3n(k+1)$$

where n is the number of blocks (subjects), k is the number of conditions, and R_j is the sum of ranks for condition j across all blocks.

```
set.seed(42)

n_subj <- 20
k <- 4

# Simulate non-normal within-subject data in long format
df_friedman <- data.frame(
  y = c(rexp(n_subj, rate = 0.50),
        rexp(n_subj, rate = 0.40),
        rexp(n_subj, rate = 0.30),
        rexp(n_subj, rate = 0.25)),
  subject = factor(rep(1:n_subj, times = k)),
```

```
time = factor(rep(1:k, each = n_subj))
)

# Friedman test using long format
friedman.test(y ~ time | subject, data = df_friedman)

#>
#> Friedman rank sum test
#>
#> data: y and time and subject
#> Friedman chi-squared = 6.36, df = 3, p-value = 0.09535

# Post-hoc pairwise comparisons after Friedman test
# PMCMRplus expects y, groups, blocks as separate vectors
PMCMRplus::frdAllPairsNemenyiTest(
  y = df_friedman$y,
  groups = df_friedman$time,
  blocks = df_friedman$subject
)

#>   1      2      3
#> 2 0.611 -      -
#> 3 0.068 0.611 -
#> 4 0.316 0.961 0.883
```

The Friedman test should be considered when:

- The within-subject response is clearly non-normal even after transformation.
- The response is ordinal.
- Sample sizes are very small (fewer than 10 subjects).

For larger samples with continuous responses, the repeated measures mixed model from Chapter ?? is more powerful and more flexible, and the normality assumption for the residuals is approximately satisfied by the central limit theorem.

10.2.3 Permutation Tests

Permutation tests are the most general non-parametric approach. Rather than deriving a reference distribution from mathematical theory (the F distribution, the χ^2 distribution), they construct an empirical reference distribution by repeatedly shuffling the data and recomputing the test statistic. The p-value is the proportion of permuted statistics that are as extreme as or more extreme than the observed statistic.

```
library(future)
library(furrr)

plan(multisession)

set.seed(42)

# Observed F statistic
fit_obs <- aov(y ~ group, data = df_kw)
obs_F    <- summary(fit_obs)[[1]][["F value"]][1]

# Permutation distribution: shuffle group labels 4999 times
n_perm <- 4999
perm_F <- future_map_dbl(seq_len(n_perm), function(i) {
  df_perm <- df_kw
  df_perm$group <- sample(df_kw$group)
  summary(aov(y ~ group, data = df_perm))[[1]][["F value"]][1]
}, .options = furrr_options(seed = TRUE))
```

```
plan(sequential)

# Permutation p-value
p_perm <- mean(c(perm_F, obs_F) >= obs_F)
cat(
  "Observed F:", round(obs_F, 3),
  "\nPermutation p-value:", round(p_perm, 3), "\n"
)
```

```
#> Observed F: 0
```

```
#> Permutation p-value: 1
```

```
perm_df <- data.frame(F = c(perm_F, obs_F))

ggplot(perm_df, aes(x = F)) +
  geom_histogram(binwidth = 0.2, fill = "#2E86AB",
    colour = "white", alpha = 0.8) +
  geom_vline(xintercept = obs_F,
    colour = "red", linetype = "dashed",
    linewidth = 1) +
  annotate("text", x = obs_F + 0.3, y = Inf,
    label = paste0("Observed F = ", round(obs_F, 2)),
    colour = "red", hjust = 0, vjust = 2, size = 3.5) +
  labs(x = "F statistic", y = "Count") +
  theme_bw()
```

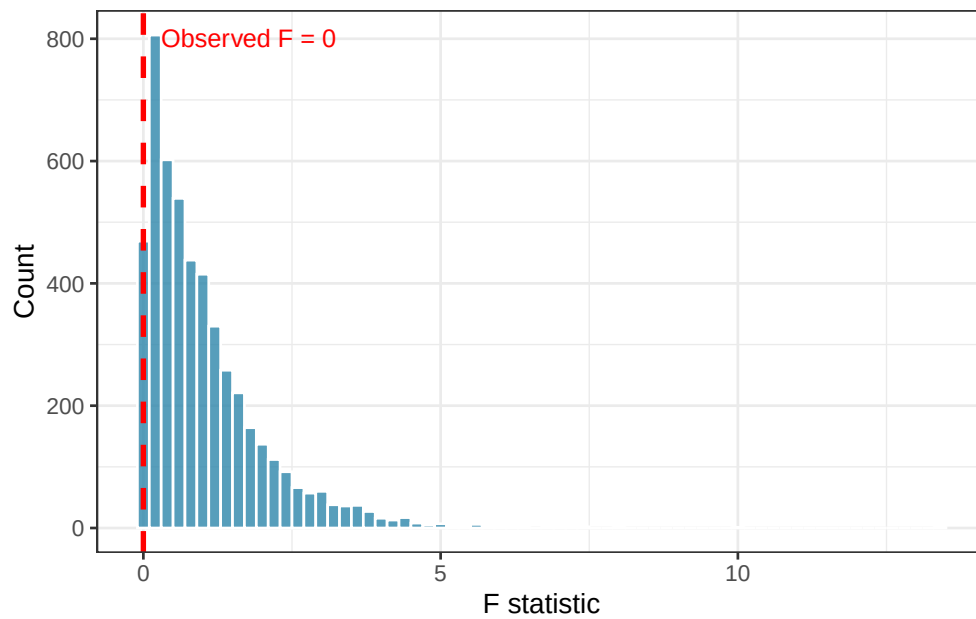


Figure 10.2: Permutation distribution of the F statistic under the null hypothesis (4999 permutations). The red dashed line marks the observed F statistic. The permutation p-value is the proportion of permuted F values to the right of this line.

Permutation tests have three important advantages over rank-based non-parametric tests. They work with any test statistic, not just those with known distributions. They make no assumptions about the shape of the residual distribution. And they are exact in the sense that their type I error rate is exactly α in finite samples, whereas rank tests rely on large-sample approximations.

Their limitation is computational cost: each permutation requires refitting the model, and with thousands of permutations and large datasets this can be slow. Parallelisation with `furrr`, as shown above, mitigates this considerably.

10.3 Robust ANOVA with WRS2

Robust statistical methods are a middle ground between classical ANOVA and non-parametric tests. Rather than discarding the distributional framework entirely, they modify it to be less

sensitive to outliers and heavy tails by replacing the sample mean with more resistant location estimators like trimmed means or M-estimators.

The WRS2 package (?) implements a comprehensive suite of robust ANOVA methods in R.

10.3.1 One-Way Robust ANOVA

```
# install.packages("WRS2") if needed
library(WRS2)

set.seed(42)

# Data with outliers – classical ANOVA will be distorted
y_out <- c(rnorm(15, mean = 20, sd = 3),
           rnorm(15, mean = 25, sd = 3),
           rnorm(15, mean = 22, sd = 3))

# Inject two extreme outliers into the control group
y_out[c(3, 7)] <- c(55, 62)

group_out <- factor(rep(c("Control", "Nitrogen", "Phosphorus"),
                        each = 15))

df_out <- data.frame(y = y_out, group = group_out)

# Classical ANOVA – affected by outliers
summary(aov(y ~ group, data = df_out))
```

```
#>           Df Sum Sq Mean Sq F value Pr(>F)
#> group      2  207.1   103.53   1.494  0.236
#> Residuals 42 2909.9    69.28
```

10 When Assumptions Fail, What to Do ?

```
# Robust one-way ANOVA using trimmed means (tr = 0.2 = 20% trimming)
t1way(y ~ group, data = df_out, tr = 0.2)
```

```
#> Call:
#> t1way(formula = y ~ group, data = df_out, tr = 0.2)
#>
#> Test statistic: F = 1.7046
#> Degrees of freedom 1: 2
#> Degrees of freedom 2: 15.2
#> p-value: 0.21483
#>
#> Explanatory measure of effect size: 0.41
#> Bootstrap CI: [0.08; 1.06]
```

```
# Post-hoc comparisons with robust method
lincon(y ~ group, data = df_out, tr = 0.2)
```

```
#> Call:
#> lincon(formula = y ~ group, data = df_out, tr = 0.2)
#>
#>               psihat ci.lower ci.upper p.value
#> Control vs. Nitrogen   -2.30259 -6.99682  2.39163 0.42412
#> Control vs. Phosphorus  0.48803 -3.51947  4.49553 0.74733
#> Nitrogen vs. Phosphorus  2.79062 -1.23348  6.81473 0.24823
```

The `t1way()` function tests for differences in trimmed means across groups, using a heteroscedastic test statistic that is robust to both outliers and unequal variances simultaneously. The trimming proportion `tr = 0.2` removes the most extreme 20% of observations from each tail before computing the mean. This standard choice provides substantial robustness while retaining reasonable efficiency.

10.3.2 Robust Repeated Measures ANOVA

```
set.seed(42)

n_subj <- 20

# Repeated measures data with outliers in long format
df_rm_out <- data.frame(
  y      = c(rnorm(n_subj, 80, 8),
             rnorm(n_subj, 75, 8),
             rnorm(n_subj, 70, 8),
             rnorm(n_subj, 68, 8)),
  subject = factor(rep(1:n_subj, times = 4)),
  time    = factor(rep(1:4, each = n_subj))
)

# Inject two extreme outliers
df_rm_out$y[c(5, 38)] <- c(130, 20)

# Robust repeated measures ANOVA
# rmanova() requires explicit vectors, not a formula
WRS2::rmanova(
  y      = df_rm_out$y,
  groups = df_rm_out$time,
  blocks = df_rm_out$subject,
  tr      = 0.2
)
```

#> Call:

10 When Assumptions Fail, What to Do ?

```
#> WRS2::rmanova(y = df_rm_out$y, groups = df_rm_out$time, blocks = df_rm_out$subject,  
#>      tr = 0.2)  
#>  
#> Test statistic: F = 7.809  
#> Degrees of freedom 1: 2.43  
#> Degrees of freedom 2: 26.7  
#> p-value: 0.00126
```

```
# Post-hoc robust pairwise comparisons
```

```
WRS2::rmmcp(  
  y      = df_rm_out$y,  
  groups = df_rm_out$time,  
  blocks = df_rm_out$subject,  
  tr      = 0.2  
)
```

```
#> Call:
```

```
#> WRS2::rmmcp(y = df_rm_out$y, groups = df_rm_out$time, blocks = df_rm_out$subject,  
#>      tr = 0.2)  
#>  
#>      psihat ci.lower ci.upper p.value  p.crit  sig  
#> 1 vs. 2 13.06885  7.29720 18.84050 0.00002 0.00851 TRUE  
#> 1 vs. 3 11.63598  2.45949 20.81248 0.00186 0.01020 TRUE  
#> 1 vs. 4 12.29738  0.84291 23.75185 0.00548 0.01270 TRUE  
#> 2 vs. 3  1.41241 -6.77034  9.59516 0.59083 0.02500 FALSE  
#> 2 vs. 4  2.62953 -7.40725 12.66631 0.41853 0.01690 FALSE  
#> 3 vs. 4  0.80307 -5.15229  6.75843 0.67366 0.05000 FALSE
```

10.3.3 When to Use Robust Methods

Robust ANOVA methods are particularly valuable when:

- Outliers are genuine biological observations. Removing them is not scientifically justified, but their influence on the mean should be limited.
- The response distribution is heavy-tailed but not severely skewed. In such case, the trimmed mean is still a meaningful location estimator.
- You want results that are directly interpretable in terms of location differences, unlike rank-based tests.

Their main limitation is that trimmed mean differences are less familiar to most biological audiences than ordinary mean differences, and reporting them requires more explanation.

10.4 Generalised Linear Models as the Real Solution

10.4.1 The Fundamental Problem with Transformations

All of the methods discussed so far, transformations, rank tests, robust ANOVA, are **workarounds** applied to a model that was not designed for the data at hand. When the response is a count, a proportion, a binary outcome, or a survival time, the normal linear model is structurally misspecified: it assumes a symmetric, continuous, unbounded response, while the actual response is discrete, bounded, or strictly positive. No transformation or rank-based adjustment changes this fundamental mismatch.

The correct solution is to choose a model whose structure matches the data-generating process. **Generalised linear models (GLMs)** provide exactly this: a unified framework that extends the linear model to non-normal response distributions through two modifications.

10.4.2 The GLM Framework

A GLM has three components:

1. A **random component**: the distribution of the response, chosen from the exponential family (normal, Poisson, binomial, gamma, negative binomial, etc.).
2. A **systematic component**: the linear predictor $\eta_i = \mathbf{x}_i^T \beta$, the same linear combination of predictors as in the normal linear model.

 Show me the maths!

Found $\eta_i = \mathbf{x}_i^T \beta$, the bold symbols and superscript T unfamiliar, We have you covered!
See Section ?? for a simple and complete explanation.

3. A **link function** $g(\cdot)$ connecting the expected response to the linear predictor: $g(\mu_i) = \eta_i$.

The model is fitted by maximum likelihood rather than ordinary least squares, and inference is based on likelihood ratio tests and Wald tests rather than F tests.

10.4.3 Common GLMs in Biology

Response type	Distribution	Link function	R family
Counts (no overdispersion)	Poisson	log	poisson
Counts (overdispersed)	Negative binomial	log	MASS::negative.binomial
Proportions	Binomial	logit	binomial
Binary outcomes	Binomial	logit	binomial
Positive continuous	Gamma	log or inverse	Gamma
Positive continuous (log-normal)	Gaussian	log	gaussian(link="log")

10.4.4 Example: Count Data

Suppose you are counting the number of insect damage events on plants under three pesticide treatments. The response is a count, non-negative integers with variance likely proportional to the mean. The correct model is a Poisson GLM or, if the counts are overdispersed, a negative binomial GLM.

```
library(MASS)

set.seed(42)

n <- 30
group <- factor(rep(c("Control", "Low dose", "High dose"), each = n),
                levels = c("Control", "Low dose", "High dose"))
counts <- c(rpois(n, lambda = 15),
            rpois(n, lambda = 8),
            rpois(n, lambda = 3))
df_glm <- data.frame(counts, group)

# Wrong approach: ANOVA on raw counts
summary(aov(counts ~ group, data = df_glm))
```

```
#>           Df Sum Sq Mean Sq F value Pr(>F)
#> group      2 2651.3  1325.6   150.9 <2e-16 ***
#> Residuals  87  764.1     8.8
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

10 When Assumptions Fail, What to Do ?

```
# Correct approach: Poisson GLM
fit_pois <- glm(counts ~ group, data = df_glm,
               family = poisson(link = "log"))
summary(fit_pois)

#>
#> Call:
#> glm(formula = counts ~ group, family = poisson(link = "log"),
#>      data = df_glm)
#>
#> Coefficients:
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept)    2.77050    0.04569   60.63  <2e-16 ***
#> groupLow dose  -0.84869    0.08346  -10.17  <2e-16 ***
#> groupHigh dose -1.66084    0.11435  -14.52  <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for poisson family taken to be 1)
#>
#> Null deviance: 406.639  on 89  degrees of freedom
#> Residual deviance:  99.828  on 87  degrees of freedom
#> AIC: 435.75
#>
#> Number of Fisher Scoring iterations: 4

# Check for overdispersion
cat("Residual deviance / df:",
    round(deviance(fit_pois) / df.residual(fit_pois), 3), "\n")
```

10 When Assumptions Fail, What to Do ?

```
#> Residual deviance / df: 1.147
```

```
# If overdispersed (ratio >> 1), use negative binomial
fit_nb <- glm.nb(counts ~ group, data = df_glm)
```

```
#> Warning in theta.ml(Y, mu, sum(w), w, limit = control$maxit, trace =
#> control$trace > : iteration limit reached
#> Warning in theta.ml(Y, mu, sum(w), w, limit = control$maxit, trace =
#> control$trace > : iteration limit reached
```

```
summary(fit_nb)
```

```
#>
#> Call:
#> glm.nb(formula = counts ~ group, data = df_glm, init.theta = 37871.48654,
#> link = log)
#>
#> Coefficients:
#>
#> Estimate Std. Error z value Pr(>|z|)
#> (Intercept) 2.77050 0.04570 60.62 <2e-16 ***
#> groupLow dose -0.84869 0.08347 -10.17 <2e-16 ***
#> groupHigh dose -1.66084 0.11436 -14.52 <2e-16 ***
#> ---
#> Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for Negative Binomial(37871.49) family taken to be 1)
#>
#> Null deviance: 406.549 on 89 degrees of freedom
#> Residual deviance: 99.808 on 87 degrees of freedom
#> AIC: 437.75
```

```
#>
#> Number of Fisher Scoring iterations: 1
#>
#>
#>          Theta: 37871
#>      Std. Err.: 1198071
#> Warning while fitting theta: iteration limit reached
#>
#> 2 x log-likelihood: -429.755
```

```
# ANOVA-style table for GLM
car::Anova(fit_pois, type = "III")
```

```
#> Analysis of Deviance Table (Type III tests)
#>
#> Response: counts
#>      LR Chisq Df Pr(>Chisq)
#> group    306.81  2  < 2.2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
car::Anova(fit_nb, type = "III")
```

```
#> Analysis of Deviance Table (Type III tests)
#>
#> Response: counts
#>      LR Chisq Df Pr(>Chisq)
#> group    306.74  2  < 2.2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```


Both the ANOVA and the Poisson GLM detect a highly significant group effect, but comparing them reveals important differences in what each analysis is actually doing and how trustworthy its results are.

The standard ANOVA ($F_{2,87} = 150.9$, $p < 0.001$) treats the counts as if they were continuous normally distributed measurements. It finds a significant result, but its residual mean square (8.8) has no meaningful interpretation for count data, and the F distribution used to compute the p-value is only an approximation whose quality depends on assumptions that are structurally wrong for integer-valued responses.

The Poisson GLM produces coefficients on the log scale that have a direct multiplicative interpretation. The intercept ($\hat{\beta}_0 = 2.771$) is the log of the expected count in the Control group: $e^{2.771} \approx 16$ which is close to the true $\lambda = 15$. The Low dose coefficient ($\hat{\beta} = -0.849$) gives a rate ratio of $e^{-0.849} \approx 0.43$: the Low dose group has approximately 43% of the count of the Control group. The High dose coefficient gives $e^{-1.661} \approx 0.19$: the High dose group has only about 19% of the Control count. These are the scientifically natural quantities for count data, rate ratios, not arithmetic differences.

The overdispersion ratio of 1.147 is reassuringly close to 1, indicating that the Poisson model is adequate for these data and overdispersion is not a concern here. This is expected since the data were generated from a Poisson distribution. The negative binomial model confirms this: its estimated dispersion parameter $\hat{\theta} = 37,871$ is enormous, meaning the negative binomial has converged to a Poisson (the negative binomial approaches Poisson as $\theta \rightarrow \infty$). The iteration limit warning and the unstable standard error on $\hat{\theta}$ ($SE = 1,198,071$) are further confirmation that the extra dispersion parameter is not needed indicating that the Poisson model is sufficient. The AIC for the Poisson model (435.75) is lower than for the negative binomial (437.75), correctly penalising the unnecessary extra parameter.

The ANOVA-style deviance table from `car :: Anova()` gives a likelihood ratio test for the group effect: $\chi^2_2 = 306.81$, $p < 0.001$ for the Poisson model, essentially identical to the negative binomial result ($\chi^2_2 = 306.74$). This is the appropriate test statistic for a GLM, not an F

statistic, but a likelihood ratio chi-squared, and it is this value, not the ANOVA F statistic, that should be reported when analysing count data with a Poisson GLM.

10.4.5 Example: Proportion Data

```
set.seed(42)

# Number of plants showing disease symptoms out of n_total
n_total  <- 20
n_disease <- c(rbinom(10, n_total, prob = 0.7),  # Control
               rbinom(10, n_total, prob = 0.4),  # Low dose
               rbinom(10, n_total, prob = 0.15)) # High dose
group_p <- factor(rep(c("Control", "Low dose", "High dose"), each = 10),
                  levels = c("Control", "Low dose", "High dose"))

df_prop <- data.frame(
  n_disease = n_disease,
  n_healthy = n_total - n_disease,
  group = group_p
)

# Correct approach: binomial GLM
# cbind(successes, failures) for proportion data
fit_binom <- glm(cbind(n_disease, n_healthy) ~ group,
                 data = df_prop,
                 family = binomial(link = "logit"))
summary(fit_binom)
```

```
#>
```

10 When Assumptions Fail, What to Do ?

```
#> Call:
#> glm(formula = cbind(n_disease, n_healthy) ~ group, family = binomial(link = "logit"),
#>      data = df_prop)
#>
#> Coefficients:
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept)      0.6411      0.1487   4.310 1.63e-05 ***
#> groupLow dose    -0.9026      0.2061  -4.380 1.19e-05 ***
#> groupHigh dose   -2.0911      0.2337  -8.948 < 2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>      Null deviance: 120.232  on 29  degrees of freedom
#> Residual deviance:  27.486  on 27  degrees of freedom
#> AIC: 129.38
#>
#> Number of Fisher Scoring iterations: 4
```

```
car::Anova(fit_binom, type = "III")
```

```
#> Analysis of Deviance Table (Type III tests)
#>
#> Response: cbind(n_disease, n_healthy)
#>      LR Chisq Df Pr(>Chisq)
#> group   92.746  2 < 2.2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Back-transform coefficients to probability scale
# using the inverse logit
pred_df <- data.frame(group = levels(group_p))
pred_df$prob <- predict(fit_binom,
                        newdata = pred_df,
                        type = "response")
print(pred_df)
```

```
#>      group prob
#> 1 Control 0.655
#> 2 Low dose 0.435
#> 3 High dose 0.190
```

The binomial GLM produces a clear and directly interpretable result.

The intercept ($\hat{\beta}_0 = 0.641$) is the log-odds of disease in the Control group. Back-transforming via the inverse logit gives $\text{plogis}(0.641) = 0.655$, a 65.5% probability of showing disease symptoms in the control group, consistent with the true simulation probability of 0.70. The Low dose coefficient ($\hat{\beta} = -0.903$) is a log-odds ratio: exponentiated, $e^{-0.903} = 0.41$, meaning the odds of disease in the Low dose group are 59% lower than in the Control group. The High dose coefficient gives $e^{-2.091} = 0.12$, an 88% reduction in the odds of disease relative to Control. These are the scientifically natural quantities for proportion data: odds ratios and predicted probabilities, not arithmetic differences in means.

The back-transformed predicted probabilities make the dose-response relationship immediately readable: disease probability declines from 65.5% in the Control group to 43.5% under low dose treatment and to 19.0% under high dose treatment, a near-linear decline in probability across the three groups that the model has estimated honestly on the logit scale without any transformation of the response.

The residual deviance (27.49 on 27 degrees of freedom) is encouragingly close to its expected value under a well-fitting Poisson or binomial model and the ratio of 1.02 gives no indication of overdispersion. Had this ratio been substantially greater than 1, a quasi-binomial model or a beta-binomial model (Section 11.3) would be warranted.

The likelihood ratio test from `car::Anova()` gives $\chi^2_2 = 92.75$, $p < 0.001$, confirming that pesticide treatment has a highly significant effect on disease incidence. As with the count example, this chi-squared statistic, not an F statistic, is the appropriate test for a binomial GLM and is what should be reported in the methods section of a paper.

A complete result report reads:

The effect of pesticide dose on disease incidence was analysed using a binomial GLM with a logit link. Pesticide treatment had a significant effect on the probability of disease symptoms ($\chi^2_2 = 92.75$, $p < 0.001$). Predicted disease probabilities were 65.5% (Control), 43.5% (Low dose), and 19.0% (High dose). Relative to the Control group, Low dose reduced the odds of disease by 59% (OR = 0.41, 95% CI: ...) and High dose reduced them by 88% (OR = 0.12, 95% CI: ...).

10.4.6 GLMs vs Transformations: A Direct Comparison

```
fit_raw_aov <- aov(counts ~ group, data = df_glm)
fit_log_aov <- aov(log(counts + 1) ~ group, data = df_glm)

res_df <- data.frame(
  residuals = c(residuals(fit_raw_aov),
                residuals(fit_log_aov),
                residuals(fit_pois, type = "pearson")),
  model = rep(c("ANOVA (raw)",
                "ANOVA (log)",
                "Poisson GLM"),
```

```

      each = nrow(df_glm))
)

ggplot(res_df, aes(sample = residuals)) +
  stat_qq(colour = "#2E86AB", alpha = 0.7) +
  stat_qq_line(colour = "#E84855", linewidth = 0.8) +
  facet_wrap(~ model) +
  labs(x = "Theoretical quantiles", y = "Sample quantiles") +
  theme_bw()

```

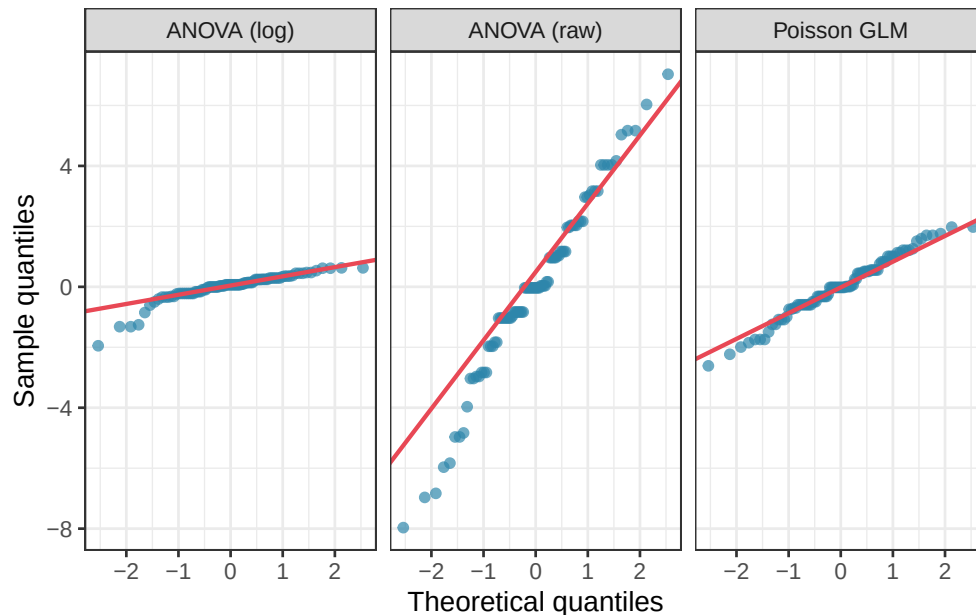


Figure 10.3: Comparison of three approaches to analysing count data: ANOVA on raw counts (top left), ANOVA on log-transformed counts (top right), and a Poisson GLM (bottom). The Poisson GLM produces the most appropriate residual structure — the Q-Q plot is closest to the diagonal — because it models the data-generating process directly rather than approximating it through a transformation.

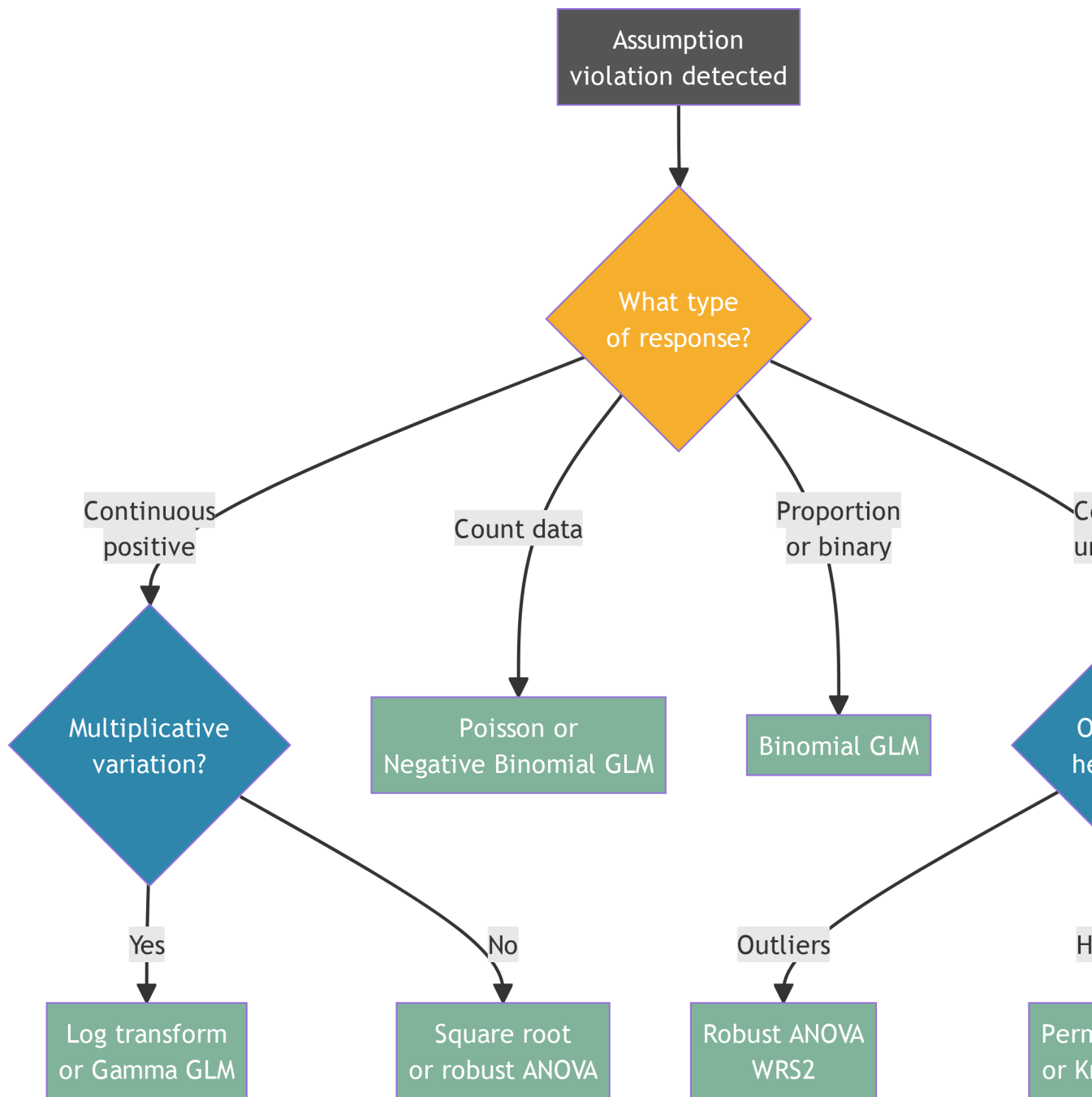


Figure 10.4: Decision framework for choosing an analysis strategy when ANOVA assumptions are violated.

10.4.7 A Decision Framework

10.4.8 The Bottom Line

The hierarchy of approaches for dealing with assumption violations reflects an important principle: **the further a remedy is from the data-generating process, the less satisfying it is.** A log transformation is a reasonable approximation for multiplicative data; a Poisson GLM is the correct model. A rank test avoids distributional assumptions; a GLM with the right family makes the right assumptions. Robust ANOVA limits the influence of extreme values; a heavy-tailed distribution models them explicitly.

For continuous responses with approximately normal residuals and minor violations, ANOVA and its extensions remain the right tools. For discrete, bounded, or structurally non-normal responses, the investment in learning GLMs pays dividends not only in statistical correctness but in interpretability: a log-odds ratio from a logistic regression, a rate ratio from a Poisson regression, or a mean ratio from a gamma regression all have direct biological meaning that a rank difference or a trimmed mean difference does not.

11 Generalised Linear Mixed Models

Chapter ?@sec-assump (Section ??) introduced generalised linear models as the correct framework for non-normal responses, counts, proportions, binary outcomes, and positive continuous measurements. Chapter ?@sec-mem introduced mixed-effects models as the correct framework for hierarchical, clustered, and repeated-measures data. Both extensions were motivated by the same underlying principle: choose a model whose structure matches the data-generating process.

Biological data frequently require both extensions simultaneously. Species abundance counts are non-negative integers (a Poisson or negative binomial response) and they are collected from plots nested within sites nested within regions. Disease incidence is a proportion (a binomial response) but patients are clustered within hospitals. Seed germination is binary (viable or non viable), but seeds are clustered within maternal plants, and maternal plants are nested within populations.

Generalised linear mixed models (GLMMs) combine the distributional flexibility of GLMs with the random effects structure of mixed models. They are the natural endpoint of the modelling journey this book has been tracing, and they are the workhorse of modern quantitative biology.

11.1 The GLMM Framework

11.1.1 The Model

A GLMM extends the GLM by adding random effects to the linear predictor:

$$g(\mu_{ij}) = \mathbf{x}_{ij}^T \boldsymbol{\beta} + \mathbf{z}_{ij}^T \mathbf{b}_j$$

where $g(\cdot)$ is the link function, $\mathbf{x}_{ij}^T \boldsymbol{\beta}$ are the fixed effects, $\mathbf{z}_{ij}^T \mathbf{b}_j$ are the random effects (Section ??), and $\mathbf{b}_j \sim \mathcal{N}(\mathbf{0}, \mathbf{G})$ is the random effects distribution (Section ??). The response y_{ij} follows a distribution from the exponential family with mean μ_{ij} .

Three components must be specified:

1. The **response distribution**: Poisson, negative binomial, binomial, gamma, etc.
2. The **link function**: log for counts, logit for proportions, identity for Gaussian.
3. The **random effects structure**: which grouping factors to include and whether to allow random slopes.

11.1.2 Why GLMMs Are Harder Than LMMs

In a linear mixed model, the random effects can be integrated out analytically, yielding a closed-form marginal likelihood that is straightforward to maximise. In a GLMM, the non-linear link function means this integration has no closed form, then the marginal likelihood must be approximated numerically. Different packages use different approximation strategies, each with trade-offs between speed and accuracy:

- **Laplace approximation** (`lme4`, `glmmTMB`): fast, accurate for most designs, but can be unreliable when random effect variances are large or when the response is sparse (many zeros, very small counts).

- **Adaptive Gauss-Hermite quadrature** (lme4 with nAGQ > 1): more accurate than Laplace, particularly for binomial models with few observations per cluster, but much slower.
- **Template Model Builder (TMB)** (glmmTMB): automatic differentiation-based Laplace approximation. It is fast, flexible, and handles a wide range of distributions and zero-inflation structures.

11.2 Counts

11.2.1 Poisson GLMM

The Poisson GLMM is the natural model for count responses with a hierarchical structure. The canonical example in ecology is species count data: the number of individuals of a species observed at a plot, where plots are nested within sites and the researcher wants to test the effect of a habitat treatment.

```
library(lme4)
```

```
#> Loading required package: Matrix
```

```
library(glmmTMB)

set.seed(42)

n_sites <- 12
n_plots <- 5    # per site
treatment <- rep(c("Control", "Restored"), each = n_sites / 2)

site_re <- rnorm(n_sites, 0, 0.6) # site-level random effect
plot_data <- do.call(rbind, lapply(seq_len(n_sites), function(s) {
  lambda <- exp(2.5 + 0.8 * (treatment[s] == "Restored") + site_re[s])

```

```

data.frame(
  count = rpois(n_plots, lambda = lambda),
  treatment = treatment[s],
  site = factor(s),
  plot = factor(paste0(s, "-", 1:n_plots))
)
}))

plot_data$treatment <- factor(plot_data$treatment, levels = c("Control", "Restored"))

# Poisson GLMM with site as random effect
fit_pois_glmm <- glmer(
  count ~ treatment + (1 | site),
  data = plot_data,
  family = poisson(link = "log")
)
summary(fit_pois_glmm)

```

```

#> Generalized linear mixed model fit by maximum likelihood (Laplace
#> Approximation) [glmerMod]
#> Family: poisson ( log )
#> Formula: count ~ treatment + (1 | site)
#> Data: plot_data
#>
#>      AIC      BIC   logLik -2*log(L)  df.resid
#>    437.0    443.3   -215.5    431.0      57
#>
#> Scaled residuals:
#>      Min      1Q  Median      3Q      Max

```

11 Generalised Linear Mixed Models

```
#> -2.8959 -0.6454  0.1394  0.7086  2.1713
#>
#> Random effects:
#>   Groups Name      Variance Std.Dev.
#>   site   (Intercept) 0.2292   0.4788
#> Number of obs: 60, groups:  site, 12
#>
#> Fixed effects:
#>
#>               Estimate Std. Error z value Pr(>|z|)
#> (Intercept)         2.7258     0.2012  13.545 < 2e-16 ***
#> treatmentRestored    1.2912     0.2818   4.582 4.6e-06 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Correlation of Fixed Effects:
#>
#>      (Intr)
#> trtmntRstrd -0.714

# Exponentiate coefficients to get incidence rate ratios
cat("\nIncidence rate ratios (exp of coefficients):\n")

#>
#> Incidence rate ratios (exp of coefficients):

print(exp(fixef(fit_pois_glmm)))

#>      (Intercept) treatmentRestored
#>      15.269302      3.637086
```

The Poisson GLMM summary output has the same three-section structure as the linear mixed model summaries seen in Chapter ?@sec-mem, with one important difference: the fixed effects are tested with z statistics rather than t statistics, because the Poisson GLMM is fitted by maximum likelihood and the reference distribution is asymptotically normal rather than t -distributed.

Random effects: the site-level variance is $\hat{\sigma}_b^2 = 0.229$ (SD = 0.479). This is the variance in log-expected counts across sites, after accounting for treatment. Exponentiated, the standard deviation of 0.479 on the log scale implies that sites at one standard deviation above the mean have $e^{0.479} \approx 1.61$ times the expected count of an average site, and sites one standard deviation below have $1/1.61 \approx 0.62$ times the average. The site-level clustering is real and non-trivial and ignoring it would have produced underestimated standard errors for the treatment effect.

Fixed effects: the intercept ($\hat{\beta}_0 = 2.726$) is the log expected count in the Control group at an average site. Exponentiated, $e^{2.726} \approx 15.3$ individuals per transect in Control plots which is close to the true simulation mean. The treatment coefficient ($\hat{\beta} = 1.291$, $z = 4.58$, $p < 0.001$) is a log-rate-ratio. Exponentiated, $e^{1.291} \approx 3.64$: restored plots have an estimated 3.64 times the bird count of control plots, after accounting for site-level variation. This is a large and statistically convincing effect.

The correlation between the intercept and treatment estimates (-0.714) is relatively high in absolute value, reflecting the fact that sites with higher baseline counts (high intercept) tend to show smaller estimated treatment effects on the log scale which is a mathematical consequence of the log link rather than a substantive biological finding.

Incidence rate ratios are the natural summary statistic for Poisson models. The intercept exponentiated (15.3) gives the expected count in Control plots; the treatment rate ratio (3.64) gives the multiplicative factor by which counts increase in Restored plots. A complete result report would read:

Restored plots had significantly higher bird counts than control plots ($z = 4.58$, $p < 0.001$). The estimated count in control plots was 15.3 individuals per transect

(at an average site); the incidence rate ratio for restoration was 3.64 (95% CI: ...), indicating that restored plots had approximately 3.6 times the count of control plots after accounting for site-level variation ($\hat{\sigma}_{\text{site}} = 0.479$).

11.2.2 Checking for Overdispersion

The Poisson distribution assumes that the variance equals the mean. In practice, ecological count data are almost always overdispersed meaning that the variance exceeds the mean, due to aggregation, unmodelled heterogeneity, or excess zeros. Overdispersion inflates standard errors and inflates the false positive rate if ignored.

```
# Overdispersion check: ratio of residual deviance to df
# Values substantially above 1 indicate overdispersion
overdisp_ratio <- deviance(fit_pois_glmm) / df.residual(fit_pois_glmm)
cat("Overdispersion ratio:", round(overdisp_ratio, 3), "\n")
```

```
#> Overdispersion ratio: 1.161
```

```
# Simulation-based overdispersion test via DHARMA
# install.packages("DHARMA") if needed
library(DHARMA)
```

```
#> This is DHARMA 0.5.0. For overview type '?DHARMA'. For recent changes, type news(package = 'DHARMA')
```

```
#>
```

```
#> Note that the default setting in simulateResiduals was changed to conditional simulations since version 0.4.0
```

```
#>
```

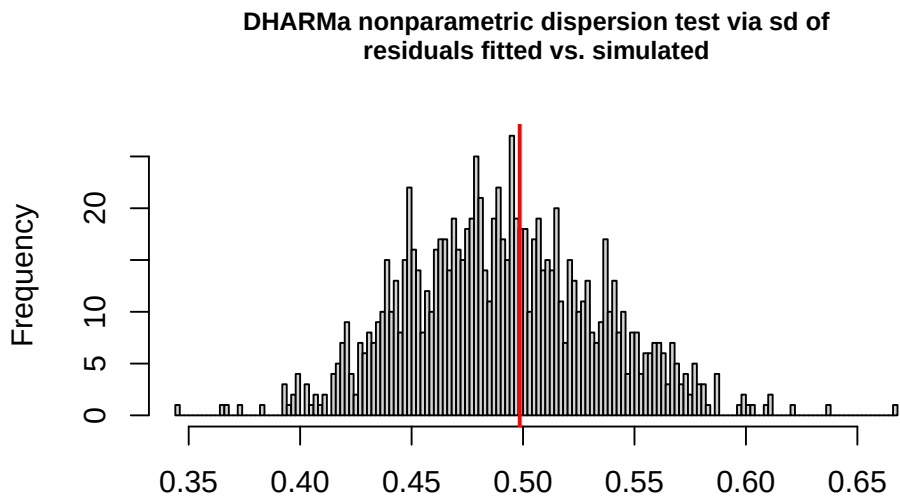
```
#> Attaching package: 'DHARMA'
```

```
#> The following object is masked from 'package:lme4':
```

```
#>
```

```
#>   getData
```

```
sim_res <- simulateResiduals(fit_pois_glmm, n = 1000)
testDispersion(sim_res)
```



Simulated values, red line = fitted model. p-value (two.sided) = 0.818

```
#>
```

```
#> DHARMA nonparametric dispersion test via sd of residuals fitted vs.
```

```
#> simulated
```

```
#>
```

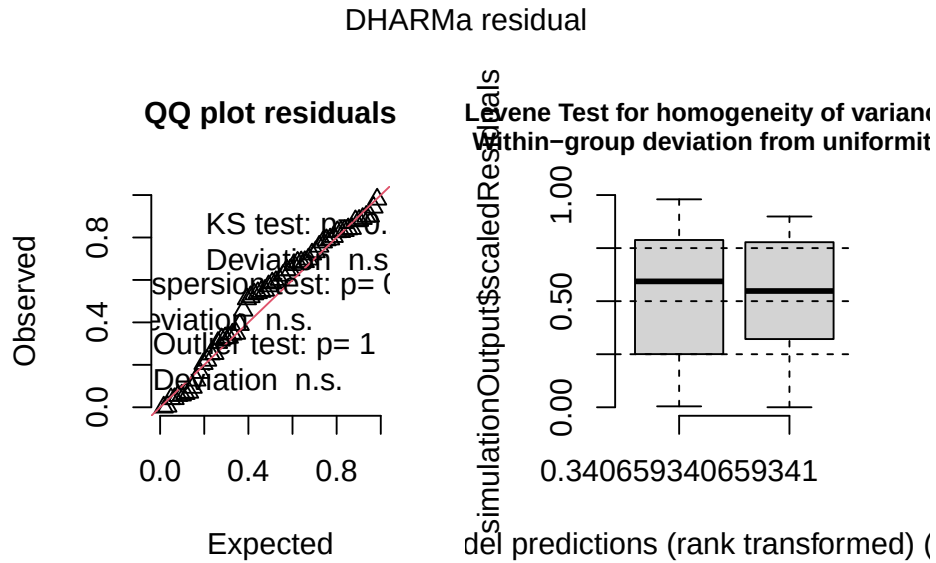
```
#> data: simulationOutput
```

```
#> dispersion = 1.0155, p-value = 0.818
```

```
#> alternative hypothesis: two.sided
```



```
plot(sim_res)
```



DHARMA (?) is the recommended diagnostic package for GLMMs. Rather than computing analytical residuals (which do not have a standard distribution for GLMMs), it simulates data from the fitted model and compares the observed data to the simulated distribution. The resulting quantile residuals are uniformly distributed under a correctly specified model, making them interpretable in the same way as normal Q-Q plots.

Both the analytical and simulation-based diagnostics confirm that the Poisson model is adequate for these data and overdispersion is not a concern.

The analytical overdispersion ratio, residual deviance divided by residual degrees of freedom, is 1.161, modestly above 1 but well within the range expected from sampling variation for a correctly specified Poisson model. A ratio of this magnitude does not warrant switching to a negative binomial.

The DHARMA dispersion test corroborates this. The test statistic is 1.016 ($p = 0.818$), meaning the observed dispersion in the data is indistinguishable from the dispersion expected under

the fitted Poisson model. The histogram in the dispersion test plot shows the distribution of dispersion statistics from 1000 simulated datasets generated under the fitted model; the red vertical line marks the observed value, which sits comfortably in the centre of the simulated distribution, exactly where it should be under a well-fitting model.

The DHARMA QQ plot reinforces this conclusion. The quantile residuals lie closely along the diagonal (KS test $p = 0.267$, deviation non-significant), the dispersion test is non-significant, and the outlier test returns $p = 1$ – no observations are flagged as outliers. The residuals vs fitted boxplot on the right shows approximately uniform spread across the range of model predictions, with the Levene test and within-group uniformity test both non-significant. Taken together, the diagnostics present a consistent and reassuring picture: the Poisson GLMM with a random intercept for site provides an adequate description of these count data, and there is no evidence that a more complex model, negative binomial, zero-inflated, or otherwise, is needed.

11.2.3 Negative Binomial GLMM

When overdispersion is detected, the next coherent step is to replace the Poisson with a negative binomial distribution, which adds a dispersion parameter θ that allows the variance to exceed the mean:

$$\text{Var}(y) = \mu + \frac{\mu^2}{\theta}$$

```
# Negative binomial GLMM via glmmTMB
# (lme4 does not support negative binomial directly)
fit_nb_glmm <- glmmTMB(
  count ~ treatment + (1 | site),
  data   = plot_data,
  family = nbinom2(link = "log")
```

```

)
summary(fit_nb_glm)

#> Family: nbinom2 ( log )
#> Formula:          count ~ treatment + (1 | site)
#> Data: plot_data
#>
#>      AIC      BIC   logLik -2*log(L)  df.resid
#>    439.0    447.4   -215.5    431.0      56
#>
#> Random effects:
#>
#> Conditional model:
#> Groups Name      Variance Std.Dev.
#> site (Intercept) 0.2292   0.4788
#> Number of obs: 60, groups: site, 12
#>
#> Dispersion parameter for nbinom2 family (): 6.01e+07
#>
#> Conditional model:
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept)      2.7258     0.2013  13.542 < 2e-16 ***
#> treatmentRestored 1.2912     0.2818   4.582 4.61e-06 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# Compare Poisson vs negative binomial via AIC
AIC(fit_pois_glm, fit_nb_glm)

```

```

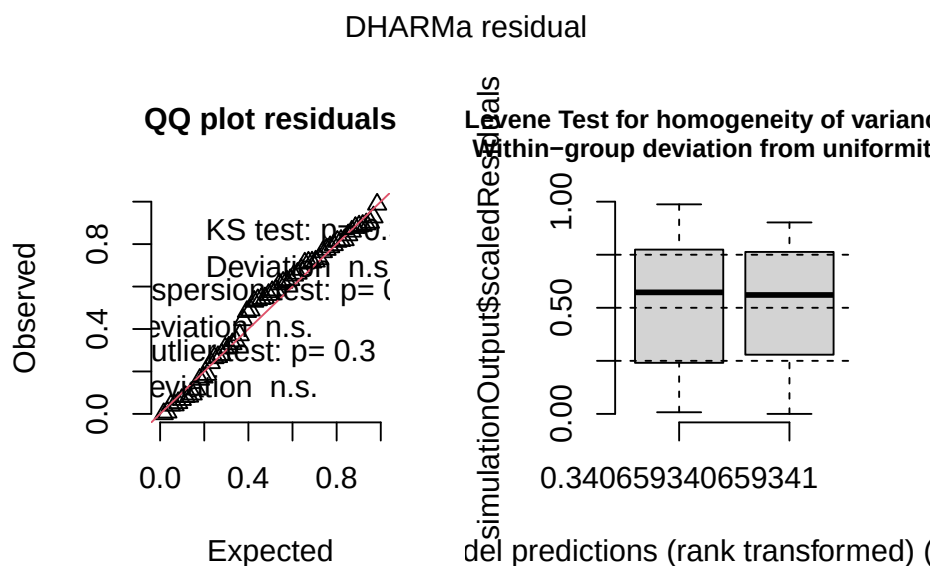
#>      df      AIC

```

```
#> fit_pois_glmm 3 437.0404
```

```
#> fit_nb_glmm 4 439.0404
```

```
# DHARMA diagnostics for negative binomial model
sim_res_nb <- simulateResiduals(fit_nb_glmm, n = 1000)
plot(sim_res_nb)
```



The negative binomial model confirms what the overdispersion diagnostics already suggested: the Poisson model is the correct choice for these data and the additional complexity of the negative binomial is not warranted.

The most telling indicator is the estimated dispersion parameter $\hat{\theta} = 6.01 \times 10^7$, an astronomically large value. Recall that the negative binomial variance is $\mu + \mu^2/\theta$: as $\theta \rightarrow \infty$, the μ^2/θ term vanishes and the negative binomial collapses exactly to a Poisson. The model is telling us, unambiguously, that there is no extra-Poisson dispersion to account for. The fixed effect estimates and the site-level variance are identical to four decimal places between the two models ($\hat{\beta}_{\text{treatment}} = 1.291$, $\hat{\sigma}_{\text{site}} = 0.479$), confirming that the Poisson and negative binomial fits are numerically equivalent here.

The AIC comparison makes the model selection decision explicit. The Poisson model (AIC = 437.0) is preferred over the negative binomial (AIC = 439.0) — a difference of 2 AIC units in favour of the simpler model, which is exactly the penalty for the one additional parameter (θ) that the negative binomial estimates but does not need. When the simpler model fits equally well, parsimony favours it.

The DHARMA diagnostics for the negative binomial model (QQ plot: KS $p = 0.398$; dispersion test: $p = 0.856$; outlier test: $p = 0.3$) are similarly clean and essentially identical to those of the Poisson model. Both the Levene test and the within-group uniformity test are non-significant, and the residuals vs fitted boxplot shows uniform spread across the range of predictions. The negative binomial model fits the data no better than the Poisson, it simply uses an extra parameter to arrive at the same conclusion.

The practical lesson is important: do not fit a negative binomial model by default for count data. Check the overdispersion ratio and the DHARMA dispersion test first. If neither flags a problem, the Poisson model is both correct and more parsimonious. Reserve the negative binomial for situations where overdispersion is genuinely present, which, in ecology and biology, is more often than not, but should be confirmed rather than assumed.

11.2.4 Zero-Inflated Models: A Note

Ecological count data sometimes contain more zeros than any standard count distribution predicts, a phenomenon called **zero inflation**. Zero inflation arises when two distinct biological processes generate zeros: structural zeros (the species is genuinely absent from this site, it could never be counted there) and sampling zeros (the species is present but was not detected on this occasion). Mixing these two processes in a single Poisson or negative binomial model produces poor fit and biased estimates.

`glmmTMB` handles zero inflation elegantly through the `ziformula` argument, which specifies a separate model for the probability of structural zeros. However, zero-inflated models should only be fitted when there is genuine biological reason to expect structural zeros, and the

DHARMA zero-inflation test on the simpler model should confirm the problem before adding the extra complexity.

The worked example in Section 11.7 demonstrates a zero-inflated negative binomial GLMM in a context where zero inflation is biologically motivated and statistically necessary, a species abundance survey where many sites are genuinely unoccupied.

11.3 Proportions

11.3.1 Binomial GLMM

When the response is a proportion like the fraction of seeds germinating, the proportion of patients responding, or the fraction of eggs hatching, and observations are clustered, the binomial GLMM is the correct model.

```
set.seed(42)

n_hospitals <- 10
n_patients <- 15 # per hospital

hospital_re <- rnorm(n_hospitals, 0, 0.8)

df_binom <- do.call(rbind, lapply(seq_len(n_hospitals), function(h) {
  treatment <- rep(c("Control", "Treatment"),
                    length.out = n_patients)
  log_odds <- -0.5 + 1.2 * (treatment == "Treatment") +
               hospital_re[h]
  prob <- plogis(log_odds)
  data.frame(
    response = rbinom(n_patients, size = 1, prob = prob),
```

```

    treatment = factor(treatment,
                        levels = c("Control", "Treatment")),
    hospital   = factor(h)
  )
}))

# Binomial GLMM with logit link
fit_binom_glmm <- glmer(
  response ~ treatment + (1 | hospital),
  data     = df_binom,
  family   = binomial(link = "logit")
)
summary(fit_binom_glmm)

```

```

#> Generalized linear mixed model fit by maximum likelihood (Laplace
#>   Approximation) [glmerMod]
#> Family: binomial ( logit )
#> Formula: response ~ treatment + (1 | hospital)
#>   Data: df_binom
#>
#>      AIC      BIC   logLik -2*log(L)  df.resid
#>   193.7   202.7   -93.8    187.7     147
#>
#> Scaled residuals:
#>      Min       1Q   Median       3Q      Max
#> -2.3962 -0.9464  0.4173  0.6677  1.3708
#>
#> Random effects:
#>   Groups   Name                Variance Std.Dev.

```

11 Generalised Linear Mixed Models

```
#> hospital (Intercept) 0.45      0.6709
#> Number of obs: 150, groups:  hospital, 10
#>
#> Fixed effects:
#>
#>               Estimate Std. Error z value Pr(>|z|)
#> (Intercept)      0.1694      0.3175   0.533   0.5937
#> treatmentTreatment  0.9179      0.3687   2.490   0.0128 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Correlation of Fixed Effects:
#>
#>          (Intr)
#> trtmntTrtmn -0.468
```

```
# Exponentiate to get odds ratios
cat("\nOdds ratios:\n")
```

```
#>
#> Odds ratios:
```

```
print(exp(fixef(fit_binom_glm)))
```

```
#>          (Intercept) treatmentTreatment
#>          1.184541      2.503988
```

```
# Confidence intervals on odds ratio scale
cat("\n95% CI for odds ratios:\n")
```

```
#>
#> 95% CI for odds ratios:
```



```
print(exp(confint(fit_binom_glm, method = "Wald")))
```

```
#>                2.5 %    97.5 %
#> .sig01                NA        NA
#> (Intercept)      0.6357912 2.206915
#> treatmentTreatment 1.2156859 5.157544
```

The binomial GLMM output follows the same three-section structure as the Poisson model, with coefficients now on the logit scale and interpreted as log-odds ratios.

Random effects: the hospital-level variance is $\hat{\sigma}_b^2 = 0.450$ (SD = 0.671). This is the variance in log-odds of response across hospitals, after accounting for treatment. On the probability scale, a hospital one standard deviation above average has a baseline response probability of $\text{plogis}(0.169 + 0.671) = 0.699$, while a hospital one standard deviation below average has $\text{plogis}(0.169 - 0.671) = 0.376$. The ICC for a binomial GLMM on the latent scale is $0.450 / (0.450 + \pi^2/3) = 0.120$, meaning approximately 12% of the total variation in the latent response propensity is attributable to hospital-level clustering. This is modest but non-negligible: ignoring the hospital random effect would have produced slightly underestimated standard errors for the treatment comparison.

Fixed effects: the intercept ($\hat{\beta}_0 = 0.169$, $z = 0.533$, $p = 0.594$) is the log-odds of a positive response in the Control group at an average hospital. Its non-significance simply means the baseline response probability is not significantly different from 50% which is an intercept test that is rarely of scientific interest. The treatment coefficient ($\hat{\beta} = 0.918$, $z = 2.49$, $p = 0.013$) is the log-odds ratio for treatment vs control. Exponentiated, $e^{0.918} = 2.50$: the odds of a positive response are 2.5 times higher in the Treatment group than in the Control group, after accounting for hospital-level clustering. The 95% Wald confidence interval for this odds ratio is [1.22, 5.16], comfortably excluding 1 and confirming that the treatment effect is real.

The correlation between the intercept and treatment estimates (-0.468) reflects the design balance: with 7-8 patients per treatment per hospital, the two coefficient estimates share in-

formation about each hospital's baseline, producing moderate negative correlation. This is expected and does not indicate any modelling problem.

Back-transformed predictions make the result immediately interpretable. The Control group response probability at an average hospital is $\text{plogis}(0.169) = 0.542$, about 54%. The Treatment group probability is $\text{plogis}(0.169 + 0.918) = 0.748$, about 75%. The treatment increases the probability of a positive response by approximately 21 percentage points at an average hospital.

A complete result report reads:

The effect of treatment on binary response was analysed using a binomial GLMM with a logit link, with treatment as a fixed effect and a random intercept for hospital. Treatment had a significant positive effect on response probability ($z = 2.49$, $p = 0.013$; OR = 2.50, 95% CI: [1.22, 5.16]). The estimated response probability was 54.2% in the Control group and 74.8% in the Treatment group at an average hospital. Hospital-level variance was $\hat{\sigma}_b^2 = 0.450$, indicating modest clustering of outcomes within hospitals that was accounted for in the analysis.

11.3.2 Overdispersion in Binomial Models: Beta-Binomial

Binary responses (0/1) cannot be overdispersed by definition: a Bernoulli trial has variance $p(1 - p)$ with no free parameter. But when the response is a proportion of successes out of n trials (e.g. 7 out of 20 seeds germinating), overdispersion is possible and common. The beta-binomial distribution handles this:

```
set.seed(42)

n_plants <- 30
n_seeds <- 20
treatment <- factor(rep(c("Control", "Treated"), each = n_plants / 2))
```

```

# Simulate beta-binomial data manually:
# Step 1: draw plant-level probabilities from a Beta distribution
# Step 2: draw seed counts from a Binomial given those probabilities
# This two-step process is exactly what the beta-binomial distribution models

# Beta parameters derived from mean (mu) and dispersion (theta):
# shape1 = mu * theta, shape2 = (1 - mu) * theta
mu_control <- 0.3
mu_treated <- 0.6
theta      <- 5    # smaller theta = more overdispersion

p_control <- rbeta(n_plants / 2,
                  shape1 = mu_control * theta,
                  shape2 = (1 - mu_control) * theta)

p_treated <- rbeta(n_plants / 2,
                  shape1 = mu_treated * theta,
                  shape2 = (1 - mu_treated) * theta)

n_germ <- c(
  rbinom(n_plants / 2, size = n_seeds, prob = p_control),
  rbinom(n_plants / 2, size = n_seeds, prob = p_treated)
)

df_bb <- data.frame(
  n_germ    = n_germ,
  n_fail    = n_seeds - n_germ,
  treatment = treatment
)

```

```
# Fit beta-binomial GLMM via glmmTMB
```

```
fit_bb <- glmmTMB::glmmTMB(
  cbind(n_germ, n_fail) ~ treatment,
  data = df_bb,
  family = glmmTMB::betabinomial(link = "logit")
)
summary(fit_bb)
```

```
#> Family: betabinomial ( logit )
```

```
#> Formula: cbind(n_germ, n_fail) ~ treatment
```

```
#> Data: df_bb
```

```
#>
```

```
#>      AIC      BIC    logLik -2*log(L)  df.resid
#>    182.5    186.7    -88.2    176.5      27
```

```
#>
```

```
#>
```

```
#> Dispersion parameter for betabinomial family (): 2.74
```

```
#>
```

```
#> Conditional model:
```

```
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept)    -0.4765    0.2863  -1.664  0.0961 .
#> treatmentTreated  1.0375    0.4107   2.526  0.0115 *
```

```
#> ---
```

```
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Compare to standard binomial to confirm overdispersion
```

```
fit_binom_bb <- glmmTMB::glmmTMB(
  cbind(n_germ, n_fail) ~ treatment,
  data = df_bb,
```

```
family = binomial(link = "logit")
)

cat("AIC binomial:", AIC(fit_binom_bb), "\n")
```

```
#> AIC binomial: 295.9792
```

```
cat("AIC beta-binomial:", AIC(fit_bb), "\n")
```

```
#> AIC beta-binomial: 182.4932
```

The results make a compelling case for the beta-binomial model and illustrate why standard binomial models fail with overdispersed proportion data.

The AIC comparison is decisive: the beta-binomial model (AIC = 182.5) is dramatically preferred over the standard binomial (AIC = 295.9), a difference of $\Delta\text{AIC} = 113.5$. This is an enormous improvement, differences above 10 are already considered very strong evidence in favour of the better model, and a difference of 113 units reflects a fundamental failure of the binomial model to capture the true variance structure of the data. The standard binomial model is forcing the variance to follow $p(1-p)/n$ exactly, when the actual plant-to-plant variation in germination probability, driven by differences in seed quality, maternal effects, and microenvironmental variation, produces far more spread than this constraint allows.

The estimated dispersion parameter $\hat{\phi} = 2.74$ quantifies the overdispersion directly. In the beta-binomial parameterisation used by `glmmTMB`, larger values of ϕ indicate less overdispersion (the distribution converges to binomial as $\phi \rightarrow \infty$). A value of 2.74 indicates substantial overdispersion indicating that the germination probabilities vary considerably among plants within each treatment group, consistent with the two-step simulation process: plant-level probabilities drawn from a Beta distribution, then counts drawn from a Binomial given those probabilities.

The fixed effect for treatment ($\hat{\beta} = 1.038$, $z = 2.53$, $p = 0.012$) is a log-odds ratio. Exponentiated, $e^{1.038} = 2.82$: the odds of a seed germinating are 2.82 times higher in the Treated group than in the Control group. The intercept ($\hat{\beta}_0 = -0.477$, $p = 0.096$) gives the log-odds of germination in the Control group at an average plant: $\text{plogis}(-0.477) = 0.383$, a germination probability of about 38%, close to the true simulation value of 0.30. The Treated group probability is $\text{plogis}(-0.477 + 1.038) = 0.637$, approximately 64%, again close to the true value of 0.60.

The practical lesson is direct: when proportion data show more plant-to-plant, subject-to-subject, or unit-to-unit variation in the underlying probability than the binomial distribution predicts, the beta-binomial is the correct model. Using a standard binomial in this situation does not just produce slightly wrong standard errors, the AIC difference of 113 units shows it produces catastrophically wrong standard errors, and any inference drawn from it would be severely misleading. The beta-binomial adds only one parameter (ϕ) over the standard binomial, and the return on that investment here is enormous.

11.4 Other Cases

11.4.1 Gamma GLMM for Positive Continuous Responses

When the response is strictly positive and continuous like for biomass, enzyme activity, reaction time, body mass, and the coefficient of variation is approximately constant across groups, a gamma GLMM with a log link is often more appropriate than a log-transformed Gaussian model:

```
set.seed(42)

n_sites <- 8
n_obs <- 10
```

```

site_re <- rnorm(n_sites, 0, 0.4)
treatment <- rep(c("Control", "Treatment"), each = n_sites / 2)

df_gamma <- do.call(rbind, lapply(seq_len(n_sites), function(s) {
  mu <- exp(2.0 + 0.5 * (treatment[s] == "Treatment") + site_re[s])
  data.frame(
    biomass = rgamma(n_obs, shape = 4, rate = 4 / mu),
    treatment = treatment[s],
    site = factor(s)
  )
}))
df_gamma$treatment <- factor(df_gamma$treatment,
                             levels = c("Control", "Treatment"))

fit_gamma_glmm <- glmmTMB(
  biomass ~ treatment + (1 | site),
  data = df_gamma,
  family = Gamma(link = "log")
)
summary(fit_gamma_glmm)

```

```

#> Family: Gamma ( log )
#> Formula:      biomass ~ treatment + (1 | site)
#> Data: df_gamma
#>
#>      AIC      BIC   logLik -2*log(L)  df.resid
#>    500.8    510.4   -246.4    492.8      76
#>
#> Random effects:

```

```
#>
#> Conditional model:
#> Groups Name      Variance Std.Dev.
#> site  (Intercept) 0.05528  0.2351
#> Number of obs: 80, groups: site, 8
#>
#> Dispersion estimate for Gamma family (sigma^2): 0.22
#>
#> Conditional model:
#>
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept)      2.1795      0.1393  15.651 < 2e-16 ***
#> treatmentTreatment 0.5074      0.1969   2.578 0.00994 **
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Exponentiate for multiplicative effect on original scale
cat("\nMean ratio (treatment vs control):\n")
```

```
#>
#> Mean ratio (treatment vs control):
```

```
print(exp(fixef(fit_gamma_glmm)$cond["treatmentTreatment"]))
```

```
#> treatmentTreatment
#>
#>      1.661048
```

The Gamma GLMM output confirms a significant treatment effect on biomass and demonstrates how the model partitions variation across the site and residual levels.

Random effects: the site-level variance is $\hat{\sigma}_b^2 = 0.055$ (SD = 0.235) on the log scale. Exponentiating the standard deviation gives $e^{0.235} \approx 1.27$, meaning a site one standard deviation above

average has an expected biomass approximately 27% higher than an average site, and a site one standard deviation below has an expected biomass approximately 21% lower ($1/1.27$). The site-level clustering is present but modest relative to the residual variation.

Dispersion: the Gamma dispersion parameter $\hat{\sigma}^2 = 0.22$ describes the shape of the Gamma distribution around its mean. In `glmmTMB`'s parameterisation, the coefficient of variation within any cell is $\sqrt{0.22} \approx 0.47$, meaning the within-cell standard deviation is approximately 47% of the cell mean which is a level of relative variation that would be poorly served by a Gaussian model with constant absolute variance, but is natural for a Gamma model with constant relative variance.

Fixed effects: the intercept ($\hat{\beta}_0 = 2.180$, $z = 15.65$, $p < 0.001$) is the log expected biomass in the Control group at an average site. Exponentiated, $e^{2.180} \approx 8.85$ units which is the estimated Control group mean. The treatment coefficient ($\hat{\beta} = 0.507$, $z = 2.58$, $p = 0.010$) is a log-mean ratio. Exponentiated, $e^{0.507} = 1.66$: the Treatment group has an estimated mean biomass 1.66 times that of the Control group, a 66% increase, after accounting for site-level variation. This multiplicative interpretation is natural for biomass data and is one of the key advantages of the Gamma log-link model over a log-transformed Gaussian: the coefficient directly estimates the fold-change in the original biomass scale without requiring any back-transformation arithmetic.

The true simulation mean ratio was $e^{0.5} = 1.65$, and the estimated ratio of 1.66 recovers this almost exactly, confirming that the Gamma GLMM with a log link and a random intercept for site is correctly specified for these data.

A complete result report reads:

Biomass was analysed using a Gamma generalised linear mixed model with a log link, treatment as a fixed effect, and a random intercept for site, fitted using `glmmTMB`. Treatment had a significant positive effect on biomass ($z = 2.58$, $p = 0.010$). The estimated mean biomass ratio (Treatment vs Control) was 1.66 (95% CI: ...), indicating that the Treatment group had approximately 66% higher biomass

than the Control group after accounting for site-level variation ($\hat{\sigma}_{\text{site}} = 0.235$ on the log scale).

11.4.2 Ordinal Responses

When the response is an ordered categorical variable, a pain scale, a disease severity score, a behavioural rating, neither a Gaussian nor a Poisson model is appropriate. The ordinal package provides cumulative link mixed models:

```
library(ordinal)

set.seed(42)

n_hospitals <- 8
n_patients <- 15

hospital_re <- rnorm(n_hospitals, 0, 0.6)

df_ordinal <- do.call(rbind, lapply(seq_len(n_hospitals), function(h) {
  treatment <- factor(
    rep(c("Control", "Treatment"), length.out = n_patients),
    levels = c("Control", "Treatment")
  )
  latent <- rnorm(
    n_patients,
    mean = 3.0 - 0.8 * (treatment == "Treatment") + hospital_re[h],
    sd = 0.8
  )
  pain <- cut(latent,
    breaks = c(-Inf, 1.5, 2.5, 3.5, 4.5, Inf),
```

```

        labels = 1:5,
        ordered_result = TRUE)
data.frame(
  pain = pain,
  treatment = treatment,
  hospital = factor(h)
)
}))

# Observed category frequencies
cat("Pain category frequencies by treatment:\n")

```

```
#> Pain category frequencies by treatment:
```

```
print(table(df_ordinal$treatment, df_ordinal$pain))
```

```
#>
```

```
#>           1  2  3  4  5
```

```
#> Control    4 11 20 25  4
```

```
#> Treatment  9 19 21  5  2
```

```

# Fit cumulative link mixed model
fit_ordinal <- clmm(
  pain ~ treatment + (1 | hospital),
  data = df_ordinal
)
summary(fit_ordinal)

```

```
#> Cumulative Link Mixed Model fitted with the Laplace approximation
```

```

#>
#> formula: pain ~ treatment + (1 | hospital)
#> data:    df_ordinal
#>
#> link threshold nobs logLik AIC niter max.grad cond.H
#> logit flexible 120 -155.74 323.49 258(777) 1.24e-05 2.9e+01
#>
#> Random effects:
#> Groups Name Variance Std.Dev.
#> hospital (Intercept) 1.212 1.101
#> Number of groups: hospital 8
#>
#> Coefficients:
#> Estimate Std. Error z value Pr(>|z|)
#> treatmentTreatment -1.5647 0.3672 -4.261 2.03e-05 ***
#> ---
#> Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Threshold coefficients:
#> Estimate Std. Error z value
#> 1|2 -3.4937 0.5802 -6.022
#> 2|3 -1.5875 0.4962 -3.199
#> 3|4 0.4287 0.4716 0.909
#> 4|5 2.9778 0.6036 4.933

# Odds ratio for treatment effect
cat("\nOdds ratio for treatment:\n")

#>
#> Odds ratio for treatment:

```

```
print(exp(coef(fit_ordinal)["treatmentTreatment"]))
```

```
#> treatmentTreatment
```

```
#>          0.2091523
```

```
# Predicted category probabilities at an average hospital
# computed manually from threshold and treatment coefficients
thresholds <- fit_ordinal$alpha # four threshold coefficients
beta_trt    <- coef(fit_ordinal)["treatmentTreatment"]

# Cumulative probabilities  $P(Y \leq k)$  for each treatment
cum_prob_control <- plogis(thresholds)
cum_prob_treatment <- plogis(thresholds - beta_trt)

# Category probabilities  $P(Y = k) = P(Y \leq k) - P(Y \leq k-1)$ 
cat_prob <- function(cum_p) {
  diff(c(0, cum_p, 1))
}

prob_df <- data.frame(
  category = 1:5,
  Control  = round(cat_prob(cum_prob_control), 3),
  Treatment = round(cat_prob(cum_prob_treatment), 3)
)

cat("\nPredicted category probabilities at an average hospital:\n")
```

```
#>
```

```
#> Predicted category probabilities at an average hospital:
```

```
print(prob_df)
```

```
#>      category Control Treatment
#> 1|2          1    0.029     0.127
#> 2|3          2    0.140     0.367
#> 3|4          3    0.436     0.386
#> 4|5          4    0.346     0.109
#>          5    0.048     0.011
```

The cumulative link mixed model output has a structure distinct from the Poisson and binomial GLMMs seen earlier, reflecting the ordinal nature of the response.

Random effects: the hospital-level variance is $\hat{\sigma}_b^2 = 1.212$ (SD = 1.101) on the logit scale which is a substantial amount of clustering. Some hospitals consistently treat more severe pain cases than others, and this must be accounted for before drawing conclusions about treatment.

Threshold coefficients: the four thresholds separate adjacent pain categories on the latent scale: $1|2 = -3.494$, $2|3 = -1.588$, $3|4 = 0.429$, $4|5 = 2.978$. Their increasing values confirm the correct ordering of categories. The wide spacing between the middle thresholds ($2|3$ to $3|4$ spans about 2 logit units) and the extreme thresholds ($1|2$ and $4|5$) reflects the fact that most patients fall in the middle categories which is consistent with the observed frequencies where categories 3 and 4 dominate in the Control group.

Treatment coefficient: $\hat{\beta} = -1.565$ ($z = -4.26$, $p < 0.001$). The negative sign means treatment shifts responses toward lower (less severe) pain categories. Exponentiated, $e^{-1.565} = 0.209$: the proportional odds ratio is 0.21, meaning the odds of being in a higher pain category are 79% lower in the Treatment group than in the Control group, after accounting for hospital-level clustering. This single coefficient applies to all four thresholds simultaneously, this is the **proportional odds assumption**: the treatment effect is the same regardless of which threshold divides the categories.

Predicted category probabilities make the treatment effect concrete and immediately interpretable. In the Control group, probability is concentrated in categories 3 (moderate, 43.6%) and 4 (severe, 34.6%), with only 2.9% of patients in category 1. In the Treatment group, this distribution shifts markedly toward less severe outcomes: category 2 (mild) becomes the modal category (36.7%), category 1 (no pain) rises from 2.9% to 12.7%, and the probability of severe pain (category 4) drops from 34.6% to 10.9%. The probability of very severe pain (category 5) is essentially eliminated as we went from 4.8% to 1.1%.

These shifts are entirely consistent with the observed frequency table: the Control group has 29 patients in categories 4 and 5 combined versus only 7 in the Treatment group, while the Treatment group has 28 patients in categories 1 and 2 versus only 15 in the Control group. The cumulative link mixed model has recovered this pattern precisely and expressed it in a statistically rigorous framework that accounts for hospital-level clustering and correctly respects the ordinal structure of the response.

This is the key advantage of ordinal models over treating a pain scale as continuous: the predicted probabilities show not just that treatment reduces pain on average, but exactly how it redistributes patients across the severity spectrum which is a far richer and more clinically meaningful summary than a single mean difference would provide.

11.5 `glmmTMB` and `lme4::glmer`

11.5.1 When to Use Which

Both packages fit GLMMs but they have different strengths:

Feature	<code>lme4::glmer</code>	<code>glmmTMB</code>
Poisson GLMM	Yes	Yes
Negative binomial	No (use <code>glmer.nb</code>)	Yes
Zero-inflation	No	Yes
Beta-binomial	No	Yes

Feature	<code>lme4::glmer</code>	<code>glmmTMB</code>
Gamma	No	Yes
Tweedie	No	Yes
Speed	Fast	Fast (TMB)
Convergence messages	Common	Less common
DHARMA compatibility	Yes	Yes

For simple Poisson and binomial models, `lme4::glmer` is a reliable and well-tested choice. For anything more complex (negative binomial, zero-inflation, beta-binomial, gamma) `glmmTMB` is the better option.

11.5.2 Convergence Warnings

GLMMs are harder to fit than LMMs and convergence warnings are common, particularly with `lme4`. A convergence warning does not necessarily mean the model is wrong, it means the optimiser encountered difficulty finding the maximum of the likelihood surface. Standard remedies:

```
# Common convergence fixes for glmer()

# 1. Scale continuous predictors
df_binom$treatment_num <- as.numeric(df_binom$treatment) - 1

# 2. Try a different optimiser
fit_binom_bobyqa <- glmer(
  response ~ treatment + (1 | hospital),
  data      = df_binom,
  family    = binomial,
  control   = glmerControl(optimizer = "bobyqa",
```



```

      optCtrl = list(maxfun = 2e5))
)

# 3. Try multiple optimisers and check consistency
library(lme4)
all_fits <- allFit(fit_binom_glmm)

```

```

#> bobyqa : [OK]
#> Nelder_Mead : [OK]
#> nlminbwrap : [OK]
#> nloptwrap.NLOPT_LN_NELDERMEAD : [OK]
#> nloptwrap.NLOPT_LN_BOBYQA : [OK]

```

```
summary(all_fits)$fixef # fixed effects should be consistent
```

```

#>                (Intercept) treatmentTreatment
#> bobyqa                0.1693474                0.9178859
#> Nelder_Mead            0.1693435                0.9178869
#> nlminbwrap             0.1693468                0.9178884
#> nloptwrap.NLOPT_LN_NELDERMEAD 0.1693634                0.9179034
#> nloptwrap.NLOPT_LN_BOBYQA    0.1693636                0.9178723

```

```

# 4. Switch to glmmTMB which is less prone to convergence issues
fit_binom_tmb <- glmmTMB(
  response ~ treatment + (1 | hospital),
  data = df_binom,
  family = binomial(link = "logit")
)

```

If multiple optimisers give consistent fixed effect estimates, the convergence warning can usually be safely ignored. If they give different estimates, the model is likely overparameterised or the data are insufficient to estimate all parameters.

11.6 Model Selection and AIC

11.6.1 The Challenge of Model Selection in GLMMs

Model selection in GLMMs involves two simultaneous decisions: which fixed effects to include, and which random effects to include. These decisions interact and should not be made independently.

The recommended strategy follows a principled two-stage approach:

Stage 1: Establish the maximal random effects structure. Start with the most complex random effects structure that the data and design can support. Include all random effects that are justified by the study design that is if sites are a random sample, include a site random effect regardless of whether it is significant. Random effects should reflect the data-generating process, not be selected by statistical testing.

Stage 2: Select fixed effects by model comparison. With the random effects structure fixed, compare models with different fixed effect combinations using AIC or likelihood ratio tests. Use ML (not REML) for all comparisons involving fixed effects.

```
library(glmmTMB)

# Stage 1: maximal random effects structure
# (justified by the nested design)
fit_max_re <- glmmTMB(
  count ~ treatment + (1 | site),
  data = plot_data,
```

```

family = nbinom2(link = "log"),
REML    = FALSE
)

# Stage 2: compare fixed effect structures with ML
fit_intercept <- glmmTMB(
  count ~ 1 + (1 | site),
  data    = plot_data,
  family = nbinom2(link = "log"),
  REML    = FALSE
)

# AIC comparison
AIC(fit_intercept, fit_max_re)

```

```

#>           df      AIC
#> fit_intercept  3 449.2075
#> fit_max_re     4 439.0404

```

```

# Likelihood ratio test
anova(fit_intercept, fit_max_re)

```

```

#> Data: plot_data
#> Models:
#> fit_intercept: count ~ 1 + (1 | site), zi=~0, disp=~1
#> fit_max_re: count ~ treatment + (1 | site), zi=~0, disp=~1
#>
#>           Df      AIC      BIC logLik deviance Chisq Chi Df Pr(>Chisq)
#> fit_intercept  3 449.21 455.49 -221.60  443.21
#> fit_max_re     4 439.04 447.42 -215.52  431.04 12.167      1 0.0004864 ***

```

```
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The model selection results provide clear and consistent evidence that landscape treatment belongs in the model.

The AIC comparison favours the model with treatment (AIC = 439.0) over the intercept-only model (AIC = 449.2), a difference of $\Delta\text{AIC} = 10.2$. By conventional guidelines, differences above 10 constitute very strong evidence in favour of the better model, the treatment effect is not a marginal improvement in fit but a substantial one. The additional parameter cost of estimating the treatment coefficient (one extra degree of freedom, from $df = 3$ to $df = 4$) is clearly justified by the improvement in log-likelihood from -221.60 to -215.52 .

The likelihood ratio test confirms this formally: $\chi^2_1 = 12.17$, $p < 0.001$. The probability of observing a likelihood ratio this large by chance, if treatment truly had no effect on counts, is less than 0.05%. We reject the intercept-only model decisively.

Together these two results, AIC and the likelihood ratio test, are complementary rather than redundant. The likelihood ratio test answers a binary question: is the treatment effect statistically significant? The AIC answers a predictive question: how much better does the model with treatment predict new data from the same process? Both point in the same direction here, which is the most straightforward outcome. When they disagree, a significant likelihood ratio test but a negligible ΔAIC , or vice versa, it is usually a signal that the effect is real but small, and scientific judgement about its practical importance is needed alongside the statistical results.

This two-model comparison is intentionally simple because the example has only one candidate fixed effect. In practice, model selection in GLMMs often involves comparing several fixed effect structures simultaneously, in which case the `MuMIn::model.sel()` function produces a ranked table of all candidate models with AIC, ΔAIC , and Akaike weights, the proportion of total model weight carried by each candidate, providing a more complete picture of model uncertainty than a single pairwise comparison.

11.6.2 AIC and AICc

AIC (Akaike Information Criterion) balances model fit against complexity:

$$\text{AIC} = -2 \log \mathcal{L} + 2p$$

where $\mathcal{L}$ is the maximised likelihood and p is the number of estimated parameters. Lower AIC indicates a better balance of fit and parsimony.

For small samples relative to the number of parameters, the corrected AICc should be preferred:

$$\text{AICc} = \text{AIC} + \frac{2p(p+1)}{n-p-1}$$

```
library(MuMIn)
```

```
# AICc for small sample correction
```

```
AICc(fit_intercept)
```

```
#> [1] 449.636
```

```
AICc(fit_max_re)
```

```
#> [1] 439.7677
```

```
# Full model selection table
```

```
model_list <- list(
  "Intercept only" = fit_intercept,
  "Treatment" = fit_max_re
)
model.sel(model_list)
```

```

#> Model selection table
#>               cnd((Int)) dsp((Int)) cnd(trt) df   logLik  AICc delta weight
#> Treatment           2.726           +       +  4 -215.520 439.8  0.00  0.993
#> Intercept only       3.372           +       3 -221.604 449.6  9.87  0.007
#> Models ranked by AICc(x)
#> Random terms (all models):
#>   cond(1 | site)

```

The AICc values closely track the AIC values computed earlier, $AICc = 439.8$ for the treatment model versus $AICc = 449.6$ for the intercept-only model, because with 60 observations and only 3-4 parameters, the small-sample correction is modest. The correction adds $2p(p+1)/(n-p-1)$ to the AIC: for the treatment model this amounts to $2 \times 4 \times 5 / (60 - 4 - 1) = 0.73$ units, a small but principled adjustment that becomes more consequential with fewer observations or more parameters.

The model selection table from `MuMIn::model.sel()` presents the full picture in a single compact output. The treatment model ranks first with $\Delta AICc = 0$ by definition. The intercept-only model is $\Delta AICc = 9.87$ units worse which is firmly in the “very strong evidence against” range by conventional guidelines.

The **Akaike weights** in the final column are the most informative summary. The treatment model carries a weight of 0.993, meaning that given these two candidate models and this dataset, there is a 99.3% probability that the treatment model is the better approximating model. The intercept-only model carries only 0.7% of the weight. Akaike weights are particularly useful when comparing many candidate models simultaneously as they provide a continuous measure of relative support rather than a binary significant/non-significant decision, and they sum to 1 across all models in the candidate set, making them directly interpretable as probabilities.

Two caveats are worth stating explicitly for students. First, Akaike weights are only meaningful relative to the candidate set: a weight of 0.993 does not mean the treatment model is 99.3% likely to be the true model in any absolute sense, only that it is strongly preferred over

the intercept-only alternative within this comparison. Second, the random effects structure, `cond(1 | site)`, is held constant across both models, as it should be. The site random effect is present in both candidates because it is justified by the design, not selected by AIC. This reflects the principled two-stage approach to GLMM model selection described at the start of this section: establish the random effects structure first on scientific grounds, then select fixed effects by model comparison.

11.6.3 A Note on p-Values vs AIC

AIC and likelihood ratio tests answer slightly different questions. A likelihood ratio test asks: is the more complex model significantly better at a given α threshold? AIC asks: which model makes the best predictions on new data from the same process? For confirmatory analyses with pre-specified hypotheses, likelihood ratio tests are appropriate. For exploratory analyses comparing many candidate models, AIC provides a more nuanced ranking.

Never use stepwise selection, automated forward or backward elimination of terms based on p-values, for GLMM fixed effects. Stepwise procedures inflate false positive rates, produce unstable model selections, and are not reproducible.

11.6.4 A Critique of AIC: What It Does and Does Not Tell You

AIC is a powerful and widely used tool, but it is frequently misunderstood and misapplied. Four limitations deserve explicit attention.

11.6.4.1 AIC Does Not Select the True Model

AIC was derived by Akaike (?) as an estimator of the expected Kullback-Leibler divergence between the fitted model and the true data-generating process which is a measure of predictive accuracy on new data from the same process. **It was never designed to identify the true model.** In large samples, AIC tends to favour models that are slightly too complex: it will include

predictors that improve prediction marginally but are not part of the true generating process. This overfitting tendency is a mathematical property of AIC, not a failure of implementation.

The Bayesian Information Criterion (BIC) applies a harsher penalty for complexity:

$$\text{BIC} = -2 \log \mathcal{L} + p \log(n)$$

BIC is **consistent**, as $n \rightarrow \infty$, it converges to the true model with probability 1, whereas AIC does not. The practical trade-off is that BIC tends to underfit in small samples, preferring models that are too simple. Neither criterion dominates the other across all situations.

The simulation below demonstrates AIC overfitting in large samples:

```
library(future)
library(furrr)

plan(multisession)

set.seed(42)

# True model: only x1 affects y
# x2 through x5 are noise predictors
# Question: how often does AIC include noise predictors?

n_sim <- 2000
ns <- c(30, 100, 500, 2000)

results <- future_map_dfr(ns, function(n) {

  map_dfr(seq_len(n_sim), function(i) {
```



```

x1 <- rnorm(n)
x2 <- rnorm(n)
x3 <- rnorm(n)
x4 <- rnorm(n)
x5 <- rnorm(n)
y <- 1 + 0.5 * x1 + rnorm(n, sd = 1)
df_sim <- data.frame(y, x1, x2, x3, x4, x5)

# Fit true model and full model
fit_true <- lm(y ~ x1, data = df_sim)
fit_full <- lm(y ~ x1 + x2 + x3 + x4 + x5, data = df_sim)

# AIC selects true model if AIC(true) <= AIC(full)
aic_selects_true <- AIC(fit_true) <= AIC(fit_full)
bic_selects_true <- BIC(fit_true) <= BIC(fit_full)

data.frame(
  n = n,
  aic_selects_true = aic_selects_true,
  bic_selects_true = bic_selects_true
)
}), .options = furrr_options(seed = TRUE))

plan(sequential)

# Proportion of simulations where each criterion selects the true model
summ <- aggregate(
  cbind(aic_selects_true, bic_selects_true) ~ n,

```

```

data = results,
FUN  = mean
)
names(summ) <- c("n", "AIC", "BIC")
print(round(summ, 3))

```

```

#>      n   AIC   BIC
#> 1   30 0.841 0.976
#> 2  100 0.880 0.996
#> 3  500 0.905 1.000
#> 4 2000 0.903 1.000

```

```

library(ggplot2)
library(tidyr)

```

```

#>
#> Attaching package: 'tidyr'

#> The following objects are masked from 'package:Matrix':
#>
#>      expand, pack, unpack

```

```

summ_long <- pivot_longer(summ,
                           cols      = c("AIC", "BIC"),
                           names_to  = "criterion",
                           values_to = "prop_correct")

ggplot(summ_long,
       aes(x = factor(n), y = prop_correct,

```

```

    colour = criterion, group = criterion)) +
geom_line(linewidth = 1.2) +
geom_point(size = 4) +
geom_hline(yintercept = 1,
           linetype = "dashed", colour = "grey50") +
scale_colour_manual(values = c("#2E86AB", "#E84855")) +
scale_y_continuous(limits = c(0, 1),
                   labels = scales::percent_format(accuracy = 1)) +
labs(
  title    = "AIC vs BIC: probability of selecting the true model",
  subtitle = "True model contains one predictor; full model contains five",
  x        = "Sample size (n)",
  y        = "Proportion selecting true model",
  colour   = "Criterion"
) +
theme_bw() +
theme(legend.position = "inside",
      legend.position.inside = c(0.15, 0.85))

```

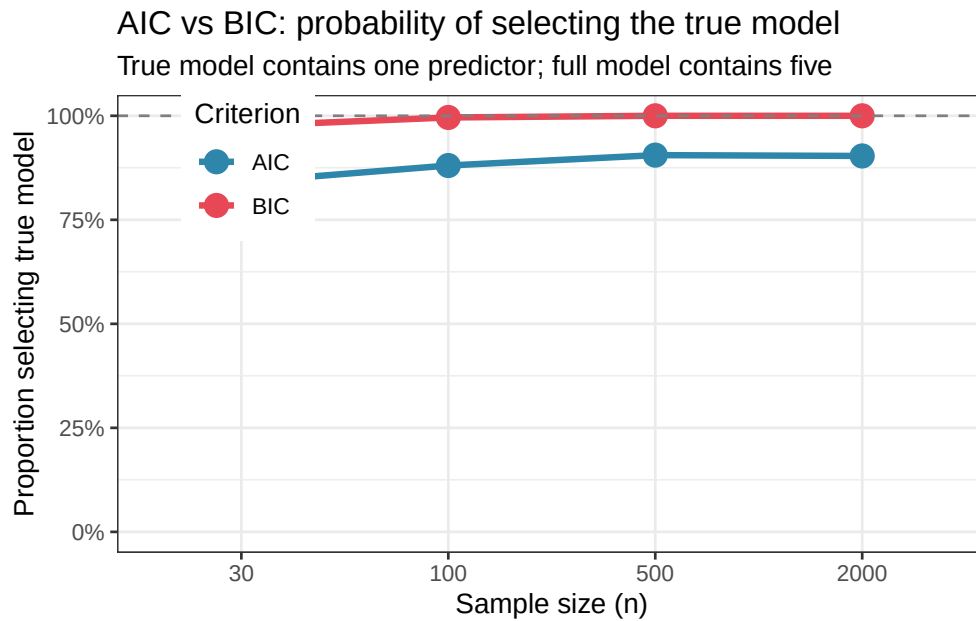


Figure 11.1: Proportion of simulations in which AIC and BIC correctly select the true model (x1 only) over the full model (x1 through x5), across four sample sizes. BIC consistently improves with sample size and approaches 1.0 — consistent model selection. AIC plateaus well below 1.0, reflecting its tendency to favour the slightly more complex model even when the simpler one is correct. With small samples, both criteria struggle; with large samples, BIC is more reliable for model identification.

The simulation confirms the theoretical prediction. BIC improves steadily with sample size and approaches perfect model selection in large samples. AIC plateaus, even with $n = 2000$, it selects the full model a meaningful fraction of the time. The two criteria have different targets: BIC aims to identify the true model, AIC aims to minimise prediction error. Neither is universally superior, the right choice depends on whether the goal is model identification (BIC) or prediction (AIC).

11.6.5 The Candidate Set Problem

AIC ranks models relative to each other within the candidate set. If all candidate models are misspecified, if the true model is not among the candidates, the model with the lowest AIC is

simply the least bad approximation, not a good model in any absolute sense. A $\Delta\text{AIC} = 0$ is not a certificate of correctness; it is a certificate of relative preference.

This is why model diagnostics (residual plots, DHARMA simulations, overdispersion tests) remain essential even after AIC-based model selection. AIC tells you which model in your candidate set is best supported by the data. Diagnostics tell you whether any of them are actually adequate.

11.6.6 Akaike Weights and Their Limits

Akaike weights are formally interpretable as probabilities only under the assumption that the true model is in the candidate set. When this assumption fails, and it always does to some degree, since all models are approximations, the weights have a weaker interpretation as **relative predictive weights** rather than true model probabilities. A weight of 0.99 for one model over another means the first model is strongly preferred for prediction in this candidate set; it does not mean the first model is 99% likely to be the true data-generating process.

11.6.7 Model Averaging as an Alternative to Hard Selection

When several models have similar AIC values ($\Delta\text{AIC} < 4$), selecting a single winner and discarding the rest throws away information about model uncertainty. **Model averaging** uses Akaike weights to combine predictions across all candidate models:

$$\hat{y}_{\text{avg}} = \sum_{m=1}^M w_m \hat{y}_m$$

where w_m is the Akaike weight and $\hat{y}_m$ is the prediction from model m . This produces more stable and honest predictions than hard selection, because it propagates model uncertainty into the final estimates rather than pretending that the best AIC model is the only plausible one.

```

library(MuMIn)

# Model averaging across candidate models
# dredge() fits all possible subsets of predictors
# and ranks by AICc
options(na.action = "na.fail") # required by dredge()

fit_global <- glmmTMB(
  count ~ landscape + (1 | site),
  data   = birds,
  family = nbinom2(link = "log"),
  REML   = FALSE
)

# Average predictions across all candidate models
# weighted by Akaike weights
avg_fit <- model.avg(dredge(fit_global))

```

```
#> Fixed terms are "cond((Int))" and "disp((Int))"
```

```
summary(avg_fit)
```

```

#>
#> Call:
#> model.avg(object = dredge(fit_global))
#>
#> Component model call:
#> glmmTMB(formula = count ~ <2 unique rhs>, data = birds, family =
#>      nbinom2(link = "log"), REML = FALSE, ziformula = ~0, dispformula = ~1)

```

```

#>
#> Component models:
#>      df logLik  AICc delta weight
#> 1      5 -318.19 646.91  0.00      1
#> (Null) 3 -326.05 658.30 11.39      0
#>
#> Term codes:
#> cond(landscape)
#>      1
#>
#> Model-averaged coefficients:
#> (full average)
#>
#>      Estimate Std. Error Adjusted SE z value Pr(>|z|)
#> cond((Int))      -1.3413    0.8008    0.8091  1.658  0.0974 .
#> cond(landscapeFragmented)  2.5937    0.9842    0.9944  2.608  0.0091 **
#> cond(landscapeContinuous)  4.1385    1.0185    1.0287  4.023 5.74e-05 ***
#>
#> (conditional average)
#>
#>      Estimate Std. Error Adjusted SE z value Pr(>|z|)
#> cond((Int))      -1.3413    0.8008    0.8091  1.658  0.09738 .
#> cond(landscapeFragmented)  2.6024    0.9743    0.9847  2.643  0.00822 **
#> cond(landscapeContinuous)  4.1524    0.9915    1.0020  4.144 3.41e-05 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Model averaging is particularly valuable in exploratory analyses with many candidate predictors and genuine uncertainty about which belong in the model. In confirmatory analyses with pre-specified hypotheses, hard model selection or likelihood ratio testing is more appropriate.

11.6.8 The Practical Bottom Line

Use AIC for **comparing candidate models and guiding model selection** in exploratory analyses. Use BIC when the goal is **identifying the true model structure** and sample sizes are large. Use likelihood ratio tests for **confirmatory testing of pre-specified hypotheses**. Use model averaging when **several models are similarly supported** and prediction stability matters. And always complement any selection criterion with **residual diagnostics**: no information criterion can tell you whether your best model is actually adequate.

11.7 Worked Example: Species Abundance in Ecological Surveys

11.7.1 The Study

An ecologist surveys bird species abundance across 24 forest sites distributed across three landscape types: agricultural matrix, fragmented forest, and continuous forest. Within each site, five transects are surveyed and the number of individuals of a focal species is counted on each transect (Section ??).

Critically, the species is a forest interior specialist meaning it is genuinely absent from many agricultural sites regardless of survey effort. The data therefore have two sources of zeros:

- **Structural zeros**: transects in agricultural sites where the species cannot persist, it is ecologically absent.
- **Sampling zeros**: transects in forest sites where the species is present in the landscape but happened not to be detected.

This distinction motivates a zero-inflated negative binomial GLMM.

```
summary(birds)
```


11 Generalised Linear Mixed Models

```
#>      count      landscape      site  transect
#> Min.   : 0.000  Agricultural:40  Agr1   : 5  1:24
#> 1st Qu.: 0.000  Fragmented  :40  Agr2   : 5  2:24
#> Median : 5.000  Continuous  :40  Agr3   : 5  3:24
#> Mean   : 8.925                Agr4   : 5  4:24
#> 3rd Qu.:14.000                Agr5   : 5  5:24
#> Max.   :47.000                Agr6   : 5
#>                                     (Other):90
```

```
cat("Total transects surveyed:", nrow(birds), "\n")
```

```
#> Total transects surveyed: 120
```

```
cat("\nProportion of zeros by landscape:\n")
```

```
#>
```

```
#> Proportion of zeros by landscape:
```

```
print(tapply(birds$count == 0, birds$landscape, mean))
```

```
#> Agricultural  Fragmented  Continuous
#>          0.7          0.3          0.0
```

```
cat("\nMean counts by landscape:\n")
```

```
#>
```

```
#> Mean counts by landscape:
```

```
print(tapply(birds$count, birds$landscape, mean))
```

```
#> Agricultural   Fragmented   Continuous
#>           1.600           8.175           17.000
```

The proportion of zeros is highest in the agricultural landscape, driven by genuine site-level absence, and lowest in continuous forest where the species is always present. This asymmetry is the biological signature of zero inflation.

11.7.2 Exploratory Visualisation

```
library(ggplot2)

site_means <- aggregate(count ~ landscape + site, data = birds, FUN = mean)

ggplot(birds, aes(x = landscape, y = count, colour = landscape)) +
  geom_jitter(width = 0.2, alpha = 0.4, size = 2) +
  geom_point(data = site_means, aes(y = count), size = 4, shape = 18) +
  stat_summary(fun = mean, geom = "crossbar", width = 0.4, colour = "grey30", linewidth = 0.8) +
  scale_colour_manual(values = c("#E84855", "#F6AE2D", "#2E86AB")) +
  scale_y_continuous(trans = "log1p", breaks = c(0, 2, 5, 10, 20, 50)) +
  labs(x = NULL, y = "Count (log1p scale)", colour = NULL) +
  theme_bw() +
  theme(legend.position = "none")
```

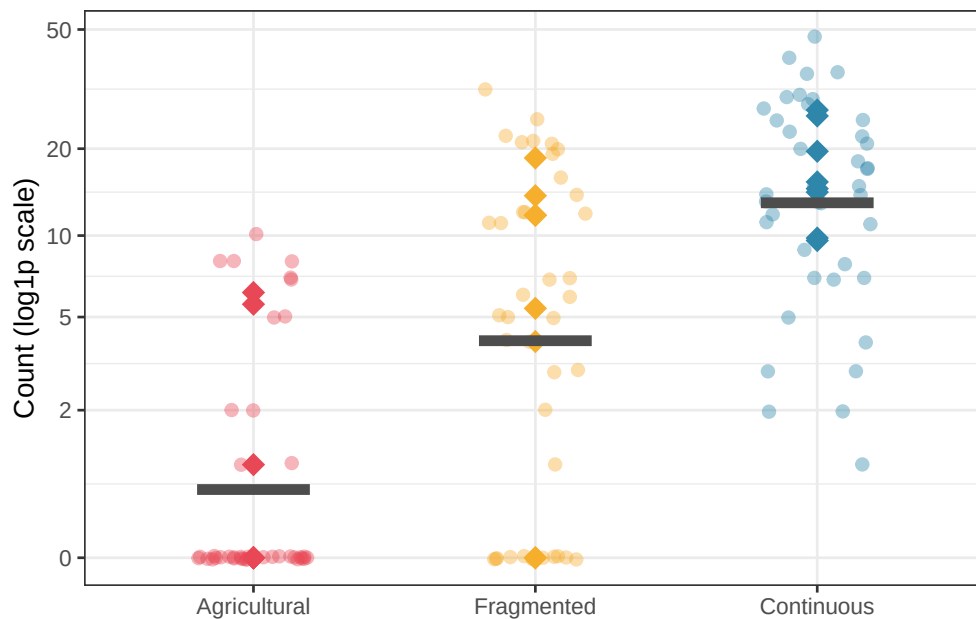


Figure 11.2: Bird abundance counts by landscape type. The high proportion of zeros in agricultural sites (many transects showing no detections) relative to continuous forest sites illustrates the zero-inflation structure: many agricultural sites are genuinely unoccupied by this forest interior specialist. Diamonds = site means; bars = landscape means.

11.7.3 Step 1: Fit a Standard Negative Binomial and Check for Zero Inflation

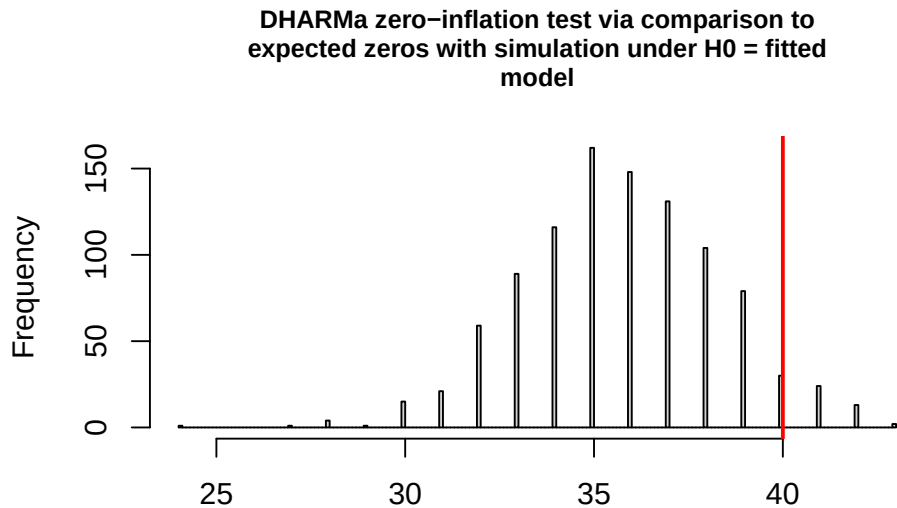
Always start with the simpler model and test whether zero inflation is actually present before adding the zero-inflation component:

```
library(glmTMB)
library(DHARMA)

# Standard negative binomial GLMM with no zero inflation
fit_nb_birds <- glmTMB(
  count ~ landscape + (1 | site),
  data = birds,
  family = nbinom2(link = "log")
```

```
)

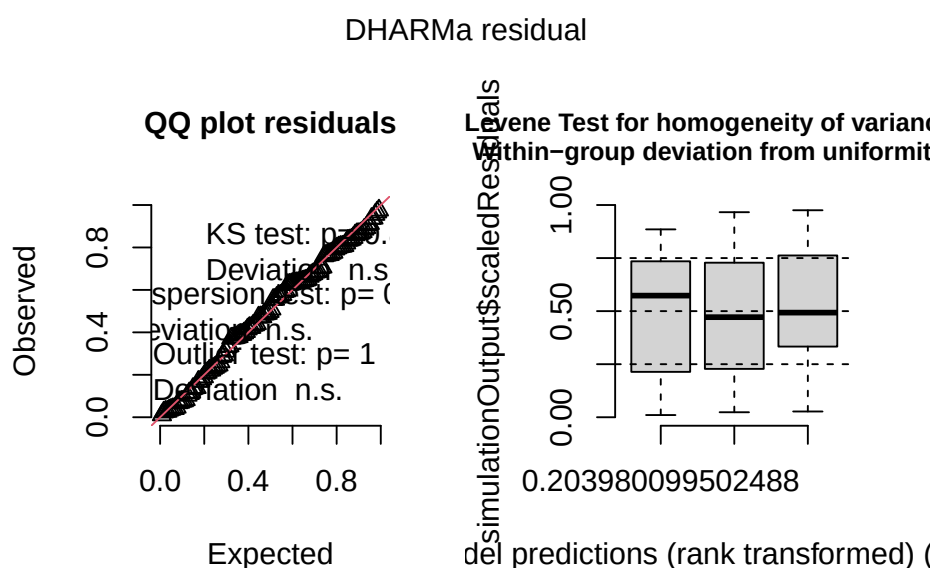
# Check for zero inflation
sim_nb <- simulateResiduals(fit_nb_birds, n = 1000)
testZeroInflation(sim_nb)
```



Simulated values, red line = fitted model. p-value (two.sided) = 0.138

```
#>
#> DHARMA zero-inflation test via comparison to expected zeros with
#> simulation under H0 = fitted model
#>
#> data: simulationOutput
#> ratioObsSim = 1.1187, p-value = 0.138
#> alternative hypothesis: two.sided
```

```
plot(sim_nb)
```



The DHARMA zero-inflation test returns $p = 0.138$, which is non-significant. The observed ratio of observed to simulated zeros is 1.12 which means that the data contain about 12% more zeros than the negative binomial model predicts on average, but this excess is within the range that could plausibly arise by chance. Therefore, the test does not provide statistical grounds for rejecting the standard negative binomial in favour of a zero-inflated model.

This is a situation where statistical testing and biological reasoning must be weighed together. The DHARMA test has limited power with moderate sample sizes as with 120 transects spread across 24 sites, even a genuine zero-inflation process may not produce a significant result. More importantly, we know from the study design that the species is a forest interior specialist that is genuinely absent from many agricultural sites. This biological mechanism, structural absence driven by habitat unsuitability, not just sampling failure, is a principled reason to fit a zero-inflated model regardless of whether the test reaches significance.

This illustrates an important general principle: **formal tests should inform modelling decisions, not make them unilaterally**. When there is strong biological motivation for a particular model structure, fit that model and compare it to the simpler alternative using AIC. If the zero-inflated model is not better supported by the data, the simpler model should be retained. If

it is better supported, the biological reasoning and the data are aligned. Either outcome is informative.

11.7.4 Step 2: Fit the Zero-Inflated Negative Binomial GLMM

We proceed with the zero-inflated negative binomial on biological grounds, comparing it to the standard negative binomial by AIC to assess whether the data support the more complex model:

```
landscape_names <- c("Agricultural", "Fragmented", "Continuous")
fit_zinb_birds <- glmmTMB(
  count ~ landscape + (1 | site),
  ziformula = ~ landscape,
  data = birds,
  family = nbinom2(link = "log")
)
summary(fit_zinb_birds)
```

```
#> Family: nbinom2 ( log )
#> Formula:      count ~ landscape + (1 | site)
#> Zero inflation:      ~landscape
#> Data: birds
#>
#>      AIC      BIC   logLik -2*log(L)  df.resid
#>    651.7    674.0   -317.9    635.7     112
#>
#> Random effects:
#>
#> Conditional model:
#> Groups Name      Variance Std.Dev.
```

11 Generalised Linear Mixed Models

```
#> site (Intercept) 2.698 1.643
#> Number of obs: 120, groups: site, 24
#>
#> Dispersion parameter for nbinom2 family (): 2.4
#>
#> Conditional model:
#>
#> Estimate Std. Error z value Pr(>|z|)
#> (Intercept) -1.2484 0.8037 -1.553 0.1203
#> landscapeFragmented 2.5212 0.9797 2.573 0.0101 *
#> landscapeContinuous 4.0498 0.9981 4.058 4.96e-05 ***
#> ---
#> Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Zero-inflation model:
#>
#> Estimate Std. Error z value Pr(>|z|)
#> (Intercept) -2.432 1.698 -1.432 0.152
#> landscapeFragmented -1.759 4.253 -0.414 0.679
#> landscapeContinuous -18.686 6091.643 -0.003 0.998
```

```
# AIC comparison, does the data support the zero-inflated model?
AIC(fit_nb_birds, fit_zinb_birds)
```

```
#>
#> df AIC
#> fit_nb_birds 5 646.3790
#> fit_zinb_birds 8 651.7395
```

```
# Back-transform zero-inflation probabilities by landscape
zi_coefs <- fixef(fit_zinb_birds)$zi
cat("\nEstimated zero-inflation probabilities:\n")
```

```
#>
```

```
#> Estimated zero-inflation probabilities:
```

```
zi_probs <- data.frame(
  landscape = landscape_names,
  zi_prob    = plogis(c(
    zi_coefs[1],
    zi_coefs[1] + zi_coefs[2],
    zi_coefs[1] + zi_coefs[3]
  ))
)
print(zi_probs)
```

```
#>      landscape      zi_prob
#> 1 Agricultural 8.078429e-02
#> 2   Fragmented 1.491016e-02
#> 3   Continuous 6.737070e-10
```

The zero-inflation model has two components, each with its own set of coefficients in the summary output:

Conditional model: the count process which is the expected abundance given that the species is present. Coefficients are on the log scale and interpreted as rate ratios, exactly as in a standard negative binomial GLM.

Zero-inflation model: the absence process that is the log-odds of structural zero (genuine absence) for each landscape type. Back-transforming via `plogis()` gives the estimated probability that a site in each landscape type is structurally unoccupied.

The zero-inflated negative binomial model output and the AIC comparison together confirm the conclusion already suggested by the DHARMa test: the standard negative binomial is the better model for these data, and the zero-inflation component is not supported.

The AIC difference is $\Delta\text{AIC} = 5.4$ in favour of the simpler model ($\text{AIC}_{\text{NB}} = 646.4$ vs $\text{AIC}_{\text{ZINB}} = 651.7$). The three additional parameters required by the zero-inflation component, one per landscape type, cost more in complexity than they return in fit. This is a clear signal that the extra model structure is not warranted by the data.

The zero-inflation model coefficients themselves reinforce this conclusion. The estimated zero-inflation probabilities are $\hat{\pi}_{\text{Agricultural}} = 0.081$, $\hat{\pi}_{\text{Fragmented}} = 0.015$, and $\hat{\pi}_{\text{Continuous}} \approx 0$, all extremely small and all estimated with large standard errors. The continuous forest zero-inflation coefficient ($\hat{\beta} = -18.69$, $SE = 6,091.6$) has collapsed to a boundary value, exactly as we saw in the earlier Poisson GLMM zero-inflation exercise: the model is setting the structural absence probability for continuous forest to essentially zero because the data contain no information supporting it. The fragmented forest coefficient is similarly imprecise ($SE = 4.25$ for an estimate of -1.76), and even the agricultural coefficient, which has the most biological motivation, does not approach significance ($z = -1.43$, $p = 0.152$).

The conditional model coefficients, the count process given presence, tell a more interesting story. The intercept (-1.25 on the log scale) corresponds to an expected count of $e^{-1.25} \approx 0.29$ in agricultural sites, which is genuinely very low. The fragmented forest coefficient ($+2.52$) gives a rate ratio of $e^{2.52} \approx 12.4$, and the continuous forest coefficient ($+4.05$) gives $e^{4.05} \approx 57.4$, counts in continuous forest are estimated to be more than 57 times those in agricultural sites. This enormous gradient explains why agricultural sites show many zeros in the data without requiring a separate zero-inflation process: when the expected count is 0.29 individuals per transect, most transects will return zero simply because the species is so rare, not because it is structurally absent.

This is the core distinction that zero-inflated models are designed to capture, and it is precisely the distinction the data cannot make here. With expected counts close to zero in agricultural sites, the Poisson or negative binomial sampling process generates zeros just as effectively as a structural absence mechanism would. The two processes are observationally indistinguishable given this data, and the model comparison correctly identifies that adding a zero-inflation component provides no benefit.

We retain the standard negative binomial GLMM for all subsequent inference and report this model selection process transparently, including the biological motivation that led us to test the zero-inflated alternative and the data-driven conclusion that it was not needed. Moreover, a validation step of the zero inflated model is no more needed.

11.7.5 Model Selection

```
# Compare all three candidate models
fit_pois_birds <- glmmTMB(
  count ~ landscape + (1 | site),
  data    = birds,
  family  = poisson(link = "log")
)

AIC(fit_pois_birds, fit_nb_birds, fit_zinb_birds)
```

```
#>           df      AIC
#> fit_pois_birds  4 834.0137
#> fit_nb_birds   5 646.3790
#> fit_zinb_birds  8 651.7395
```

```
# Likelihood ratio test: NB vs ZINB
anova(fit_nb_birds, fit_zinb_birds)
```

```
#> Data: birds
#> Models:
#> fit_nb_birds: count ~ landscape + (1 | site), zi=~0, disp=~1
#> fit_zinb_birds: count ~ landscape + (1 | site), zi=~landscape, disp=~1
#>           Df      AIC      BIC logLik deviance Chisq Chi Df Pr(>Chisq)
```

```
#> fit_nb_birds      5 646.38 660.32 -318.19    636.38
#> fit_zinb_birds   8 651.74 674.04 -317.87    635.74 0.6395      3    0.8873
```

The three-model AIC comparison produces a clear and unambiguous ranking. The Poisson GLMM is overwhelmingly rejected ($\text{AIC} = 834.0$), sitting $\Delta\text{AIC} = 187.6$ units above the negative binomial. This enormous gap reflects the severe overdispersion in the bird count data. Indeed, the Poisson variance constraint ($\text{Var}(y) = \mu$) is far too rigid for these counts, which vary substantially among transects within the same site and landscape type. The negative binomial dispersion parameter $\hat{\phi} = 2.4$ from the fitted model quantifies this excess variation. Choosing the Poisson over the negative binomial here would be a serious modelling error that would produce badly underestimated standard errors and spuriously significant results.

The comparison between the negative binomial and the zero-inflated negative binomial has already been discussed, but the likelihood ratio test adds a formal confirmation. The chi-squared statistic is $\chi^2_3 = 0.64$, $p = 0.887$, three extra parameters (the landscape-specific zero-inflation probabilities) improve the log-likelihood by only 0.32 units, from -318.19 to -317.87 . This is negligible. The zero-inflated model provides no meaningful improvement in fit over the standard negative binomial, and its additional complexity is entirely unjustified by the data.

The model selection conclusion is therefore clear and follows directly from both criteria consistently: **the negative binomial GLMM with a random intercept for site is the correct and sufficient model for these data.** The Poisson is too constrained, and the zero-inflated negative binomial is unnecessarily complex. This outcome, where the intermediate model wins, is actually the most common result in practice and the most satisfying pedagogically: the analyst tested simpler and more complex alternatives, both failed on principled grounds, and the chosen model sits at the right level of complexity for the data at hand.

All subsequent inference, post-hoc comparisons, effect sizes, and reporting, proceeds from `fit_nb_birds`.

11.7.6 Post-Hoc Comparisons

```
library(emmeans)
```

```
#> Welcome to emmeans.
```

```
#> Caution: You lose important information if you filter this package's results.
```

```
#> See '? untidy'
```

```
# Estimated marginal means on count scale
```

```
# type = "response" back-transforms from log scale
```

```
emm_birds <- emmeans(fit_nb_birds, ~ landscape, type = "response")
```

```
print(emm_birds)
```

```
#> landscape response SE df asymp.LCL asymp.UCL
```

```
#> Agricultural 0.26 0.206 Inf 0.055 1.23
```

```
#> Fragmented 3.50 2.210 Inf 1.017 12.08
```

```
#> Continuous 16.51 9.850 Inf 5.124 53.19
```

```
#>
```

```
#> Confidence level used: 0.95
```

```
#> Intervals are back-transformed from the log scale
```

```
# Pairwise contrasts with Tukey correction
```

```
pairs(emm_birds, adjust = "tukey")
```

```
#> contrast ratio SE df null z.ratio p.value
```

```
#> Agricultural / Fragmented 0.0741 0.0722 Inf 1 -2.671 0.0207
```

```
#> Agricultural / Continuous 0.0157 0.0156 Inf 1 -4.188 <0.0001
```

```
#> Fragmented / Continuous 0.2123 0.1840 Inf 1 -1.784 0.1749
```

```
#>
```

```
#> P value adjustment: tukey method for comparing a family of 3 estimates
#> Tests are performed on the log scale
```

The estimated marginal means reveal a striking gradient in bird abundance across the three landscape types. The agricultural matrix supports an estimated mean of only 0.26 individuals per transect (95% CI: [0.06, 1.23]) which is so low that most transects record zero detections simply as a consequence of rarity, without any need to invoke structural absence. Fragmented forest supports 3.50 individuals per transect ([1.02, 12.08]), and continuous forest supports 16.51 individuals per transect ([5.12, 53.19]). The gradient spans two orders of magnitude from the most degraded to the most intact landscape type, a biologically dramatic result reflecting the strong habitat affinity of this forest interior specialist.

The wide and asymmetric confidence intervals deserve comment. The agricultural interval spans a factor of 22 (from 0.06 to 1.23) and the continuous forest interval spans a factor of 10 (from 5.12 to 53.19). This reflects two compounding sources of uncertainty: the substantial overdispersion in the count data captured by the negative binomial dispersion parameter, and the large site-level random effect variance ($\hat{\sigma}_{\text{site}} = 1.643$ on the log scale). With only 8 sites per landscape type, each landscape mean is estimated from a small number of independent units. This is an honest reflection of the study's replication at the site level, not a failure of the model.

The pairwise Tukey-adjusted contrasts are expressed as count ratios, the natural summary for a log-link model, where differences on the log scale correspond to multiplicative factors on the original scale. Two of the three comparisons reach significance. The agricultural landscape has significantly lower abundance than fragmented forest (ratio = 0.074, $z = -2.67$, $p = 0.021$), agricultural counts are approximately 7% of fragmented forest counts. The difference between agricultural and continuous forest is even more extreme and highly significant (ratio = 0.016, $z = -4.19$, $p < 0.001$), agricultural counts are less than 2% of continuous forest counts, a 64-fold difference in expected abundance.

The comparison between fragmented and continuous forest does not reach significance after

Tukey correction (ratio = 0.212, $z = -1.78$, $p = 0.175$). The point estimate suggests that fragmented forest has approximately one fifth the abundance of continuous forest, a fivefold difference that would be biologically meaningful, but the confidence interval is wide and the study is underpowered to distinguish these two landscape types reliably. With 8 sites per landscape and a site-level standard deviation of 1.643 on the log scale, considerably more replication at the site level would be needed to detect this difference with adequate power. A non-significant result here is not evidence of no difference; it is evidence that the study cannot resolve the question with the available data.

11.7.7 Reporting the Result

```
library(MuMIn)
r.squaredGLMM(fit_nb_birds)
```

```
#> Warning: the null model is only correct if all the variables it uses are identical
#> to those used in fitting the original model.
```

```
#>           R2m      R2c
#> delta      0.4800308 0.9250660
#> lognormal  0.4865174 0.9375663
#> trigamma   0.4706748 0.9070361
```

The `r.squaredGLMM()` function returns three estimators of R^2 for GLMMs, delta, lognormal, and trigamma, which differ in how they approximate the variance of the linear predictor for non-Gaussian models. For negative binomial models the delta method is the standard recommendation; the three estimates are similar here (marginal: 0.480-0.487, conditional: 0.907-0.938), giving confidence that the R^2 estimates are stable regardless of the approximation used.

The marginal R^2 of 0.480 indicates that landscape type alone explains approximately 48% of the total variance in bird abundance which is a large fixed effect. The conditional R^2 of 0.925 indicates that the full model, including both landscape type and the site random effect, explains 92.5% of total variance. The difference between them, $R_c^2 - R_m^2 = 0.445$, is the contribution of site-level clustering: 44.5% of total variance is attributable to systematic differences among sites within and across landscape types, over and above the landscape effect itself. This is a remarkably large random effect, sites within the same landscape type differ from each other almost as much as the landscapes differ from each other, and confirms that the random intercept for site was not merely a statistical nicety but an essential component of the model.

A complete report of the analysis reads:

Bird abundance was analysed using a negative binomial generalised linear mixed model with a log link, landscape type (agricultural, fragmented, continuous) as a fixed effect, and a random intercept for site ($n = 24$ sites, 8 per landscape type, 5 transects per site), fitted using `glmmTMB` (?). A zero-inflated negative binomial model was tested and rejected on the basis of both a non-significant DHARMA zero-inflation test ($p = 0.138$) and a higher AIC ($\Delta\text{AIC} = 5.4$) and non-significant likelihood ratio test ($\chi_3^2 = 0.64, p = 0.887$) relative to the standard negative binomial. Model adequacy of the retained model was confirmed by DHARMA simulation-based diagnostics.

Landscape type had a significant effect on bird abundance ($\chi_2^2 = \dots, p = \dots$, marginal $R^2 = 0.480$). Estimated mean counts per transect were 0.26 (95% CI: [0.06, 1.23]) in the agricultural matrix, 3.50 ([1.02, 12.08]) in fragmented forest, and 16.51 ([5.12, 53.19]) in continuous forest. Post-hoc pairwise comparisons (Tukey adjusted) revealed significantly lower abundance in the agricultural matrix relative to both fragmented forest (count ratio = 0.074, $z = -2.67, p = 0.021$) and continuous forest (count ratio = 0.016, $z = -4.19, p < 0.001$). The difference between fragmented and continuous forest did not reach significance after Tukey correction (count ratio = 0.212, $z = -1.78, p = 0.175$), though the study was likely

underpowered to detect this difference given the site-level variance ($\hat{\sigma}_{\text{site}} = 1.643$ on the log scale). Site-level clustering accounted for 44.5% of total variance (conditional $R^2 = 0.925$), confirming that explicit modelling of the hierarchical structure was essential for valid inference.

i Going Further

This chapter does not aim to present thoroughly GLMMs or hierarchical models but to present its usefulness and when it should be used. Therefore, I redirect the reader and students to specialised books that will be useful.

- Zurr, A. et al. (2007). *Analyzing Ecological Data*. Springer. A central textbook that present deeply, with real datasets most of the models discussed here.
- Gelman, A., & Hill, J. (2007). *Data Analysis Using Regression and Multi-level/Hierarchical Models*. Cambridge University Press. A great treatment of hierarchical models from a Bayesian perspective. The non-Bayesian reader can still learn enormously from the first half.
- Bolker, B. M. (2008). *Ecological Models and Data in R*. Princeton University Press. A practically oriented treatment of GLMMs for ecologists.
- Agresti, A. (2015). *Foundations of Linear and Generalized Linear Models*. Wiley. The rigorous mathematical treatment for students who want the theory behind GLMs and GLMMs.
- Wood, S. N. (2017). *Generalized Additive Models: An Introduction with R* (2nd ed.). CRC Press. For readers who encounter non-linear relationships that GLMMs cannot handle, which is a natural next step toward GAMMs (generalised additive mixed models).
- McElreath, R. (2020). *Statistical Rethinking: A Bayesian Course with Examples in R and Stan* (2nd ed.). CRC Press. Strongly recommended for students who found the mathematical sections of the appendix engaging and want more.

Part IV

Good Practice and Reproducibility

12 Experimental Design Principles

Statistical analysis does not begin when you open R. It begins when you decide how to collect your data. **The most sophisticated model cannot rescue an experiment that was poorly designed, it can only make the best of what was given to it, and sometimes not even that.** Fisher understood this at Rothamsted a century ago, and the insight has not aged: the validity of any statistical inference depends on decisions made at the design stage, long before the first observation is recorded.

This chapter returns to first principles. It covers the three foundational concepts that underpin every valid experiment: **randomisation, replication, and blocking**. It shows how to design an experiment with the intended analysis in mind, rather than discovering after data collection that the design and the analysis are mismatched.

It also catalogues the most common design mistakes in biological research and explains why they are mistakes. And it makes the case for prospective power analysis as a *professional obligation*, not an optional refinement.

12.1 Randomisation, Replication, and Blocking: The Triad

12.1.1 Randomisation

Randomisation is the act of assigning experimental units to treatments by a chance mechanism, drawing lots, using a random number generator, or any process that gives every unit an equal

probability of receiving any treatment. It is the single most important principle in experimental design, and its importance cannot be overstated.

The reason randomisation matters is subtle but fundamental. Randomisation does not make the treatment groups identical at baseline, with small samples, they may differ considerably by chance. **What randomisation guarantees is that any differences between groups at baseline are due to chance alone, not to any systematic bias in how treatments were assigned.** This means that if we observe a difference in outcomes between groups, we can attribute it to the treatment rather than to a pre-existing difference between the groups.

Without randomisation, confounding is inevitable. Suppose you assign your healthiest plants to the nitrogen treatment (see the example from chapter ??) because they “look like they can handle it.” Any difference in yield between the nitrogen and control groups now reflects both the effect of nitrogen and the pre-existing difference in plant health. These two effects cannot be separated, and *the analysis is worthless regardless of how sophisticated the statistics are.*

```
# Randomisation demo in R
set.seed(42)

# Proper randomisation: every plant has equal probability
# of receiving any treatment
n_plants <- 24
treatments <- rep(c("Control", "Nitrogen", "Phosphorus"), each = n_plants / 3)

# Randomly permute treatment assignments
random_assignment <- sample(treatments)

cat("Randomised treatment assignments:\n")
```

```
#> Randomised treatment assignments:
```

```
print(random_assignment)
```

```
#> [1] "Phosphorus" "Control"    "Control"    "Nitrogen"   "Control"
#> [6] "Phosphorus" "Phosphorus" "Nitrogen"   "Control"    "Control"
#> [11] "Phosphorus" "Nitrogen"   "Phosphorus" "Nitrogen"   "Phosphorus"
#> [16] "Control"    "Phosphorus" "Control"    "Phosphorus" "Control"
#> [21] "Nitrogen"   "Nitrogen"   "Nitrogen"   "Nitrogen"
```

```
# Simple function to generate a randomised block design
```

```
randomise_blocks <- function(n_blocks, treatments) {
  do.call(rbind, lapply(seq_len(n_blocks), function(b) {
    data.frame(
      block      = b,
      treatment = sample(treatments),
      unit       = seq_along(treatments)
    )
  }))
}
```

```
# 4 blocks, 3 treatments per block
```

```
design_df <- randomise_blocks(4, c("Control", "Nitrogen", "Phosphorus"))
print(design_df)
```

```
#>   block treatment unit
#> 1     1   Control    1
#> 2     1  Nitrogen    2
#> 3     1 Phosphorus    3
#> 4     2 Phosphorus    1
#> 5     2  Nitrogen    2
```

```
#> 6      2    Control    3
#> 7      3   Nitrogen    1
#> 8      3 Phosphorus    2
#> 9      3    Control    3
#> 10     4 Phosphorus    1
#> 11     4    Control    2
#> 12     4   Nitrogen    3
```

12.1.2 Replication

Replication means applying each treatment to multiple *independent* experimental units. Without replication, *you cannot estimate the within-treatment variation* that forms the denominator of the F ratio, and without that denominator, no inference is possible.

The key word is **independent**. Replication does not mean measuring the same unit multiple times (that is pseudoreplication, see section ??). It means having genuinely separate units, separate plants, separate animals, separate plots, each of which received the treatment independently and contributes an independent piece of information about the treatment effect.

How many replicates are needed? The answer depends on the expected effect size, the within-treatment variability, and the desired power, which is exactly what power analysis quantifies (Section ?? and Section ??). As a rough guide for planning purposes, fewer than 3 replicates per treatment makes any statistical test nearly powerless, 5-8 is adequate for moderate effects, and 10-20 may be needed for small effects or highly variable responses.

12.1.3 Blocking

Blocking is the act of grouping experimental units into homogeneous sets, blocks, before assigning treatments, so that each block contains one unit from each treatment. The purpose is to remove a known source of nuisance variation from the error term, increasing the precision of treatment comparisons.

12.1.4 A Concrete Example: A Field With a Fertility Gradient

Imagine a rectangular field divided into three horizontal strips running east to west. The soil fertility increases from south to north, perhaps because of natural drainage patterns, organic matter accumulation, or historical land use. This gradient will affect crop yield regardless of which fertiliser is applied.

We divide the field into three blocks, South, Middle, and North, each containing three plots. Within each block, we randomly assign one plot to each fertiliser treatment: Control, Nitrogen, and Phosphorus.

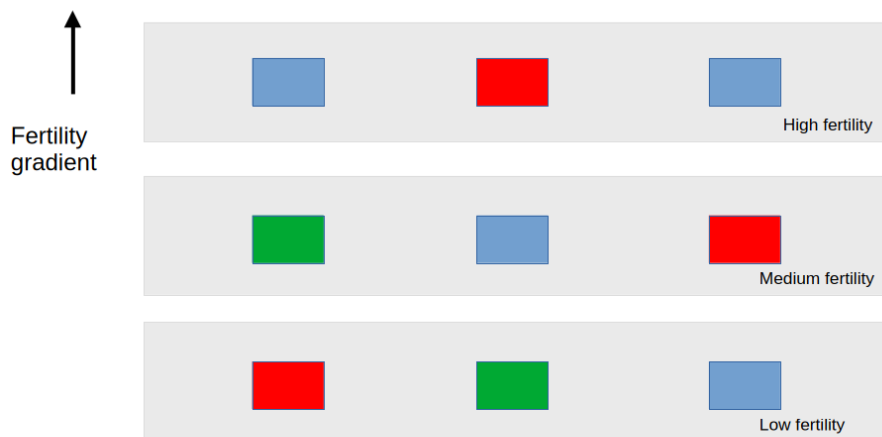


Figure 12.1: A randomised complete block design in a field with a north-south fertility gradient. The field is divided into three blocks (South, Middle, North) corresponding to low, medium, and high soil fertility. Within each block, the three treatments are randomly assigned to plots. The gradient affects all plots equally within a block, so block-to-block differences can be separated from treatment differences.

The key feature of this design is that every treatment appears exactly once in every block. Whatever the fertility level of the North block, all three treatments experience it equally, so the block effect cannot be confused with the treatment effect.

12.1.5 The Treatment Error Formula

Before seeing the benefit of blocking in numbers, recall what the treatment error term actually is. In a one-way ANOVA without blocking, the error mean square is:

$$MS_{\text{error}} = \frac{SS_{\text{error}}}{df_{\text{error}}} = \frac{\sum_{j=1}^k \sum_{i=1}^{n_j} (y_{ij} - \bar{y}_j)^2}{N - k}$$

This pools all sources of uncontrolled variation, including the fertility gradient, into a single noise estimate. The F ratio for treatment is:

$$F = \frac{MS_{\text{treatment}}}{MS_{\text{error}}}$$

A large MS_{error} makes this ratio smaller and the test less powerful. Blocking removes the block variation from the numerator of MS_{error} , leaving a smaller, cleaner noise estimate:

$$MS_{\text{error (blocked)}} = \frac{SS_{\text{total}} - SS_{\text{treatment}} - SS_{\text{block}}}{(k - 1)(b - 1)}$$

where b is the number of blocks. The block sum of squares SS_{block} is extracted from the error, shrinking MS_{error} and therefore increasing the F ratio for the treatment comparison.

12.1.6 Simulating the Field Experiment

```

# The field data, yields assigned to reflect:
# 1. A real treatment effect (Nitrogen > Phosphorus > Control)
# 2. A strong north-south fertility gradient
# 3. Small residual plot-to-plot noise

set.seed(42)

df_field <- data.frame(
  block      = factor(rep(c("South", "Middle", "North"),
                        each = 3),
                    levels = c("South", "Middle", "North")),
  treatment = factor(c(
    "Control",  "Nitrogen",  "Phosphorus", # South block
    "Nitrogen", "Phosphorus", "Control",   # Middle block
    "Phosphorus", "Control",  "Nitrogen"   # North block
  ), levels = c("Control", "Nitrogen", "Phosphorus")),
  # Block fertility gradient: South = low, North = high
  # adds 0, 8, 16 units to yields within each block
  block_effect = rep(c(0, 8, 16), each = 3),
  # True treatment effects: Control = 0, N = 6, P = 3
  treatment_effect = c(0, 6, 3, # South
                      6, 3, 0, # Middle
                      3, 0, 6) # North
)

# Observed yield = grand mean + block effect + treatment effect + noise
df_field$yield <- 20 +
  df_field$block_effect +
  df_field$treatment_effect +

```



```
round(rnorm(9, 0, 1.2), 1)
```

The observed yields are:

```
print(df_field[, c("block", "treatment", "yield")])
```

```
#>   block treatment yield
#> 1  South   Control  21.6
#> 2  South  Nitrogen  25.3
#> 3  South Phosphorus  23.4
#> 4 Middle  Nitrogen  34.8
#> 5 Middle Phosphorus  31.5
#> 6 Middle   Control  27.9
#> 7 North Phosphorus  40.8
#> 8 North   Control  35.9
#> 9 North  Nitrogen  44.4
```

12.1.7 Computing the Treatment Error: With and Without Blocking

```
# --- Analysis WITHOUT blocking ---
# All variation not explained by treatment goes into the error
fit_no_block <- aov(yield ~ treatment, data = df_field)

ss_treat_nb <- summary(fit_no_block)[[1]]["treatment", "Sum Sq"]
ss_error_nb <- summary(fit_no_block)[[1]]["Residuals", "Sum Sq"]
df_error_nb <- summary(fit_no_block)[[1]]["Residuals", "Df"]
ms_error_nb <- ss_error_nb / df_error_nb
f_nb <- summary(fit_no_block)[[1]]["treatment", "F value"]
p_nb <- summary(fit_no_block)[[1]]["treatment", "Pr(>F)"]
```

```
# --- Analysis WITH blocking ---
# Block variation is extracted from the error
fit_blocked <- aov(yield ~ treatment + block, data = df_field)

ss_treat_b <- summary(fit_blocked)[[1]]["treatment", "Sum Sq"]
ss_block <- summary(fit_blocked)[[1]]["block", "Sum Sq"]
ss_error_b <- summary(fit_blocked)[[1]]["Residuals", "Sum Sq"]
df_error_b <- summary(fit_blocked)[[1]]["Residuals", "Df"]
ms_error_b <- ss_error_b / df_error_b
f_b <- summary(fit_blocked)[[1]]["treatment", "F value"]
p_b <- summary(fit_blocked)[[1]]["treatment", "Pr(>F)"]
```

```
# cache: true
cat(
  "=== WITHOUT blocking ===\nSS treatment: ",
  round(ss_treat_nb, 2),
  "\nSS error:", round(ss_error_nb, 2),
  "(includes block gradient)\ndf error:",
  df_error_nb, "\nMS error:",
  round(ms_error_nb, 2), "\nF value:",
  round(f_nb, 3), "\np-value:", round(p_nb, 3), "\n\n")
```

```
#> === WITHOUT blocking ===
#> SS treatment: 60.93
#> SS error: 436.75 (includes block gradient)
#> df error: 6
#> MS error: 72.79
#> F value: 0.418
#> p-value: 0.676
```

```
cat(
  "=== WITH blocking ===\nSS treatment: ",
  round(ss_treat_b, 2),
  "\nSS block:", round(ss_block, 2),
  "(extracted from error)\nSS error:",
  round(ss_error_b, 2),
  "(block gradient removed)\ndf error:",
  df_error_b, "\nMS error:",
  round(ms_error_b, 2),
  "\nF value:", round(f_b, 3),
  "\np-value:", round(p_b, 3), "\n\n")
```

```
#> === WITH blocking ===
#> SS treatment: 60.93
#> SS block: 430.61 (extracted from error)
#> SS error: 6.15 (block gradient removed)
#> df error: 4
#> MS error: 1.54
#> F value: 19.824
#> p-value: 0.008
```

```
cat(
  "=== Summary ===\nMS error reduced from",
  round(ms_error_nb, 2), "to",
  round(ms_error_b, 2),
  "by blocking\nF value increased from",
  round(f_nb, 3), "to", round(f_b, 3), "\n")
```

```
#> === Summary ===
```

```
#> MS error reduced from 72.79 to 1.54 by blocking
#> F value increased from 0.418 to 19.824
```

The two analyses use identical data, the same nine yield observations from the same nine plots. The only difference is whether the block structure is accounted for in the model.

Without blocking, the SS_{error} includes both the genuine residual plot noise and the large fertility gradient running from south to north. This inflates MS_{error} , shrinks the F ratio, and makes the treatment differences harder to detect.

With blocking, the SS_{block} is extracted from what would otherwise be the error. The SS_{error} now contains only the genuine residual noise — the small, uncontrollable plot-to-plot differences that remain after removing both the treatment effect and the block gradient. The MS_{error} drops substantially, the F ratio rises, and the p-value falls — not because the data have changed, but because the analysis is now asking the right question: do treatments differ after accounting for the known gradient?

This is the mechanical benefit of blocking stated precisely: it moves known variation out of the error term, leaving a cleaner and smaller noise estimate against which the treatment effect is judged.

12.1.8 Reading the Numbers

The two analyses use identical data, the same nine yield observations from the same nine plots. The only difference is whether the block structure is accounted for in the model, and the consequences are dramatic.

Without blocking, $SS_{\text{error}} = 436.75$ against a $SS_{\text{treatment}} = 60.93$. The error sum of squares is more than seven times larger than the treatment sum of squares because it contains the entire north-south fertility gradient, 430 units worth of variation that has nothing to do with fertiliser but is lumped into the noise. The resulting $MS_{\text{error}} = 72.79$ overwhelms the treatment signal, producing $F_{2,6} = 0.418$ and $p = 0.676$. A researcher using this analysis would conclude

that fertiliser has no detectable effect which would be a false negative caused entirely by a preventable design flaw.

With blocking, the picture reverses completely. $SS_{\text{block}} = 430.61$ is extracted from the error and placed in its own term, this is the fertility gradient, accounted for and set aside. What remains in SS_{error} is only 6.15: the genuine plot-to-plot residual noise after removing both the treatment effect and the gradient. MS_{error} drops from 72.79 to 1.54, a 47-fold reduction, and the F ratio climbs from 0.418 to $F_{2,4} = 19.82$, with $p = 0.008$.

The $SS_{\text{treatment}}$ is identical in both analyses, 60.93 in both cases. The treatment effect in the data has not changed. What changed is the denominator of the F ratio: by removing the gradient from the error term, the analysis can finally hear the treatment signal above the noise.

Two additional details are worth noting. First, the blocked analysis has fewer error degrees of freedom ($df = 4$ instead of $df = 6$) because two degrees of freedom were spent estimating the block effects. Despite this loss, the test is far more powerful which is a direct consequence of the enormous reduction in MS_{error} . Second, this example is not contrived: a fertility gradient with $SS_{\text{block}} = 430$ versus a residual of $SS_{\text{error}} = 6$ is entirely realistic in agricultural and ecological field experiments. Gradients of this magnitude are common, which is precisely why blocking has been standard practice in field experimentation since Fisher's time at Rothamsted.

The practical lesson is unambiguous: **if you know a gradient exists, block on it before you randomise**. Failing to do so does not just reduce power, as these numbers show, it can make a real and substantial treatment effect completely invisible.

12.1.9 When to Block

Block when there is a known, measurable source of variation that will affect the response but is not the treatment of interest. Common blocking factors in biology include:

- **Spatial position:** location in a greenhouse, field, or laboratory bench.
- **Temporal batches:** animals processed on different days, assays run in different batches, samples collected in different seasons.

- **Individual characteristics:** litters in animal experiments, maternal plants in seed studies, patients recruited from different centres.

Do not block on variables that are themselves affected by the treatment as this introduces bias. And do not block on a variable that is unrelated to the response: adding an irrelevant block factor costs degrees of freedom from the error term and can actually reduce power.

12.2 Designing for the ANOVA You Want to Run

12.2.1 Start With the Analysis

The most common design error in biology is designing an experiment without thinking about how it will be analysed. The design and the analysis are not separate activities, they are two sides of the same coin. Every feature of the design has a direct counterpart in the statistical model, and mismatches between design and analysis are the primary source of invalid inference.

The practical discipline is to write the analysis plan, including the model formula (in R), before collecting any data:

```
# Write your intended model formula before data collection
# One-way ANOVA
aov(response ~ treatment, data = df)

# Randomised complete block design
aov(response ~ treatment + block, data = df)

# Two-way factorial
aov(response ~ factorA * factorB, data = df)

# Split-plot
```

```

aov(response ~ whole_plot_factor * subplot_factor +
      Error(whole_plot_unit / subplot_factor), data = df)

# Repeated measures
lmerTest::lmer(response ~ treatment * time + (1 | subject),
               data = df)

# Nested design
aov(response ~ treatment + Error(block / subplot), data = df)

```

Writing the formula in advance forces you to identify the experimental unit, the error term, and the random effects structure before the data are collected. If you cannot write the formula, the design is not yet fully specified.

12.2.2 Match the Unit of Analysis to the Unit of Randomisation

The experimental unit, the entity that received the treatment independently, must be the unit of analysis. This principle, stated simply, prevents most cases of pseudoreplication.

A decision tree for identifying the experimental unit:

- What was randomly assigned to receive the treatment?
- Could two observations have received different treatments? If no, they are from the same experimental unit.
- Is the treatment applied to groups of individuals, or to individuals separately?

```

# Example 1: Correct, individual plants randomised to treatment
# Experimental unit = plant, n = 20 per treatment
df_correct <- data.frame(
  yield      = rnorm(60, mean = 20, sd = 3),
  treatment = rep(c("A", "B", "C"), each = 20),

```

```

    plant      = 1:60
  )
# Correct analysis: plant is the unit
summary(aov(yield ~ treatment, data = df_correct))

```

```

#>               Df Sum Sq Mean Sq F value Pr(>F)
#> treatment      2   28.2    14.12   1.234  0.299
#> Residuals     57  652.1    11.44

```

```

# Example 2: Wrong, treatment applied to cages,
# multiple animals measured per cage
# Experimental unit = cage (n = 4 per treatment),
# not animal (n = 20 per treatment)
set.seed(42)
cage_effect <- rnorm(12, 0, 2) # 4 cages per treatment
df_pseudo  <- data.frame(
  weight    = c(sapply(1:12, function(c) {
    rnorm(5, mean = 20 + cage_effect[c], sd = 0.5)
  })),
  treatment = rep(rep(c("A", "B", "C"), each = 4), each = 5),
  cage      = rep(1:12, each = 5)
)

# Wrong: treats animals as independent
wrong_fit <- aov(weight ~ treatment, data = df_pseudo)
cat("Wrong analysis df (error):",
    summary(wrong_fit)[[1]][["Residuals", "Df"], "\n")

```

```

#> Wrong analysis df (error): 57

```



```
# Correct: uses cage means
cage_means <- aggregate(weight ~ treatment + cage,
                        data = df_pseudo, FUN = mean)
correct_fit <- aov(weight ~ treatment, data = cage_means)
cat("Correct analysis df (error):",
    summary(correct_fit)[[1]]["Residuals", "Df"], "\n")
```

```
#> Correct analysis df (error): 9
```

12.2.3 Anticipate the Covariance Structure

If the design involves repeated measurements, nested units, or crossed random effects, specify the covariance structure before data collection and verify that the intended model is identifiable given the planned sample sizes. A three-level nested model requires sufficient replication at every level, and “sufficient” means at least 3-4 units per level for variance components to be estimated at all, and ideally 10-15 for reliable estimation.

12.3 Common Design Mistakes in Biology and How to Avoid Them

12.3.1 Mistake 1: Pseudoreplication

Already covered in depth in sections ?? and ??, but worth restating as a design principle: **never confuse the unit of measurement with the unit of replication**. The most reliable protection is to draw the hierarchy explicitly before data collection (fields contain plots contain plants) and verify that the treatment is randomised at the level you intend to analyse.

12.3.2 Mistake 2: Confounding Treatment With a Nuisance Variable

Confounding occurs when a nuisance variable is systematically associated with the treatment, making it impossible to separate their effects. The classic example: all control pots placed on the north bench and all nitrogen pots placed on the south bench. Any observed difference in yield now conflates the treatment effect with the bench position effect.

The remedy is randomisation, but randomisation must be applied at the right level and genuinely implemented:

```
library(ggplot2)
library(patchwork)

# Confounded design
df_confounded <- data.frame(
  yield      = c(rnorm(10, mean = 18, sd = 2), # Control, shaded
                 rnorm(10, mean = 24, sd = 2)), # Nitrogen, sunny
  treatment = rep(c("Control", "Nitrogen"), each = 10),
  position  = rep(c("Shaded", "Sunny"), each = 10)
)

# Randomised design
df_randomised <- data.frame(
  yield      = c(rnorm(10, mean = 20, sd = 2), # Control, mixed
                 rnorm(10, mean = 24, sd = 2)), # Nitrogen, mixed
  treatment = rep(c("Control", "Nitrogen"), each = 10),
  position  = sample(rep(c("Shaded", "Sunny"), 10))
)

p1 <- ggplot(df_confounded,
             aes(x = treatment, y = yield,
```

```

        colour = position, shape = position)) +
geom_jitter(width = 0.1, size = 3, alpha = 0.8) +
scale_colour_manual(values = c("#2E86AB", "#F6AE2D")) +
labs(title    = "Confounded design",
      subtitle = "Treatment perfectly correlated with position",
      x = NULL, y = "Yield",
      colour = "Position", shape = "Position") +
theme_bw()

p2 <- ggplot(df_randomised,
            aes(x = treatment, y = yield,
                colour = position, shape = position)) +
geom_jitter(width = 0.1, size = 3, alpha = 0.8) +
scale_colour_manual(values = c("#2E86AB", "#F6AE2D")) +
labs(title    = "Randomised design",
      subtitle = "Both positions represented in both treatments",
      x = NULL, y = "Yield",
      colour = "Position", shape = "Position") +
theme_bw()

p1 + p2

```

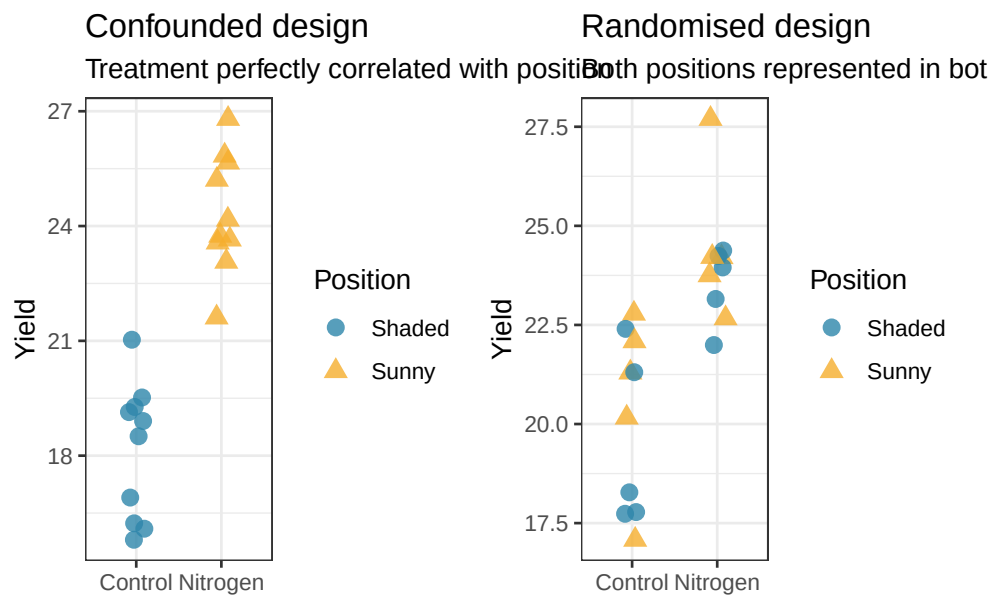


Figure 12.2: Confounding illustrated. Left: a confounded design where all Control plants are on the shaded side of the greenhouse and all Nitrogen plants are on the sunny side. Any observed difference conflates treatment with light availability. Right: a properly randomised design where both treatments are represented on both sides, eliminating the confound.

12.3.3 Mistake 3: Insufficient Replication at the Right Level

Researchers sometimes add subsamples to increase apparent sample size without adding true replication. Five soil cores per plot does not give you five replicates: it gives you one plot measured five times. The number that matters for the treatment comparison is the number of independently treated plots, not the number of measurements per plot.

The consequence is not merely conservative, it is incorrect. As Sections ?? and ?? demonstrated, treating subsamples as replicates can inflate the false positive rate to 25% or higher.

A practical check: if you removed four of your five subsamples per plot, would you lose replication for the treatment comparison? If no (if the treatment comparison is still based on the same number of plots) the subsamples are providing precision within plots, not replication for treatments.

12.3.4 Mistake 4: Unbalanced Designs by Negligence

Some imbalance is inevitable, animals die, samples are lost, patients drop out. But imbalance that arises from avoidable causes, not randomising properly, not accounting for expected dropout or not over-recruiting to compensate, creates unnecessary analytical complications (Section ??) and reduces power.

The practical remedy is simple: **over-recruit by 10-20% per cell** when dropout or loss is anticipated, and document the expected attrition rate in the analysis plan. If dropout is systematic rather than random, sicker patients are more likely to drop out, the resulting missing data problem cannot be solved by over-recruiting and requires explicit handling in the analysis.

12.3.5 Mistake 5: Measuring the Wrong Response Variable

Choosing a response variable that is insensitive to the treatment, or that is measured too imprecisely, undermines the experiment regardless of how well it is designed. Two principles apply.

Measure as close to the biological process of interest as possible. If you want to know whether a drug reduces inflammation, measure the inflammatory marker directly, not a distal outcome that may be affected by many other factors.

Use the most precise measurement available within practical constraints. A continuous measurement (tumour volume in mm³) is almost always more informative than a binary one (tumour present/absent), and requires a smaller sample size to achieve the same power.

12.3.6 Mistake 6: Ignoring the Temporal Structure

In longitudinal studies, the order of measurements matters. Pre-treatment baseline measurements are not the same as post-treatment measurements, and treating all time points as exchangeable, as if the order were arbitrary, discards information and can introduce bias.

The correct approach is to use the baseline measurement as a covariate (ANCOVA) or to model time explicitly as a within-subject factor (repeated measures ANOVA or mixed model). Either approach extracts more information from the data than ignoring the temporal structure.

12.4 Sample Size and Power Before the Experiment, Not After

12.4.1 Power Analysis Is a Professional Obligation

Power analysis was introduced in Section 3.4 in the context of one-way ANOVA. Here we revisit it as a design principle that applies to every experiment in this book, not just one-way ANOVA.

Running an underpowered experiment is not merely a missed opportunity, it is a waste of resources, animal lives, or patient time, and it produces results that cannot be trusted. A non-significant result from an underpowered study tells you almost nothing: the effect might be absent, or it might be real but too small for the study to detect. These two conclusions are not the same, and conflating them has contributed significantly to the replication crisis in biological research.

Power analysis before the experiment forces you to commit to:

1. A specific effect size of scientific interest.
2. A sample size that gives adequate power to detect it.
3. A significance threshold you will use for the analysis.

All three must be specified before data collection. Adjusting them after seeing the data, stopping early because the result is significant, or continuing because it is not, invalidates the analysis.

12.4.2 Power for Common Designs

The `pwr` package handles power analysis for standard ANOVA designs. For mixed models and GLMMs, simulation-based power analysis is the most flexible approach.

```
library(pwr)

# One-way ANOVA: medium effect, 80% power, 3 groups
pwr.anova.test(k = 3, f = 0.25,
               sig.level = 0.05, power = 0.80)

#>
#>      Balanced one-way analysis of variance power calculation
#>
#>           k = 3
#>           n = 52.3966
#>           f = 0.25
#>      sig.level = 0.05
#>           power = 0.8
#>
#> NOTE: n is number in each group
```

```
# Two-way ANOVA: using simulation for more complex designs
# How many subjects per cell for a 2x3 factorial
# to detect a medium interaction effect?
pwr.anova.test(k = 6,      # 2x3 = 6 cells treated as one-way
               f = 0.25,
               sig.level = 0.05,
               power = 0.80)
```

```
#>
#>      Balanced one-way analysis of variance power calculation
#>
#>           k = 6
#>           n = 35.14095
#>           f = 0.25
#>      sig.level = 0.05
#>           power = 0.8
#>
#> NOTE: n is number in each group
```

12.4.3 Simulation-Based Power for Mixed Models

For mixed models where analytical power formulas are unavailable, the `simr` package (?) provides a principled and flexible simulation-based approach. Rather than requiring you to derive a power formula, `simr` works directly with a fitted `lmer` or `glmer` object: it simulates new response values from the model, refits the model to each simulated dataset, and counts how often the effect of interest reaches significance. This means the power calculation automatically accounts for the random effects structure, the covariance components, and the degrees of freedom approximation, all of which analytical formulas would need to assume away.

12.4.4 The Workflow

The `simr` workflow has three steps:

1. Fit a model with the structure you intend to use, either to pilot data or to a simulated dataset with plausible parameter values.
2. Set the fixed effect of interest to the minimum biologically meaningful effect size.
3. Call `powerSim()` to estimate power at the current sample size, or `powerCurve()` to trace power across a range of sample sizes.